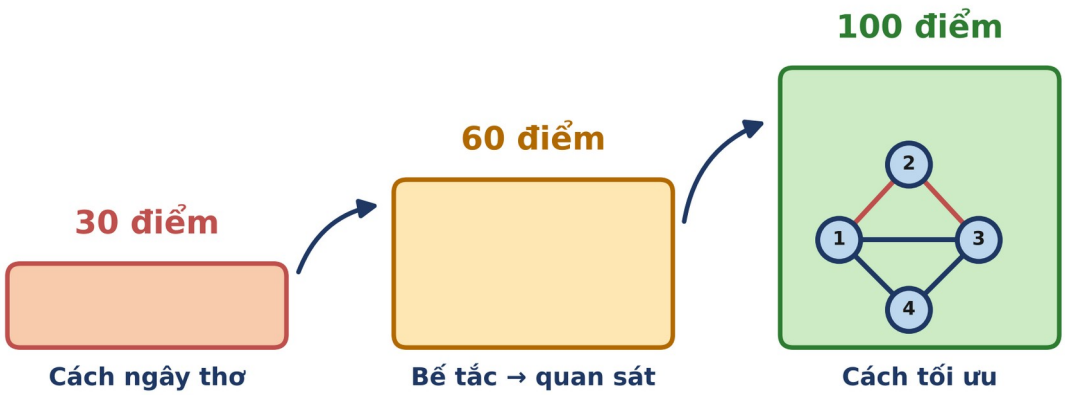
Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

LÝ THUYẾT ĐỒ THỊ

Từ cơ bản đến nâng cao

TẬP 3 · HƯỚNG DẪN GIẢI



Chương 1. LÀM QUEN VỚI ĐỒ THỊ
Chương 2. ĐƯỜNG ĐI NGẮN NHẤT
Chương 3. CÂY VÀ CÂY KHUNG
Chương 4. CẤU TRÚC CỦA ĐỒ THỊ
Chương 5. GHÉP CẶP VÀ LUỒNG

Phân tích từ cách ngây thơ đến cách tối ưu · 51 bài · bảng điểm chấm thật
Dành cho học sinh THPT ôn luyện học sinh giỏi cấp tỉnh và tiếp cận cấp quốc gia

LỜI NÓI ĐẦU

Tập 3 trình bày lời giải cho cả 51 bài tập của Tập 2. Mỗi lời giải không chỉ đưa ra chương trình đạt 100 điểm, mà đi lại đúng con đường một học sinh giỏi suy nghĩ trong phòng thi: bắt đầu từ cách ngây thơ, đếm số phép tính để thấy bế tắc, rồi phá bế tắc bằng một quan sát hay một công cụ phù hợp của lí thuyết đồ thị.

Với các bài đồ thị, điểm thường không mất vì thuật toán mà vì những chi tiết cài đặt: đệ quy quá sâu làm tràn ngăn xếp, tổng khoảng cách vượt kiểu `int`, dữ liệu có cạnh lặp hoặc khuyên, đồ thị không liên thông... Vì vậy bên cạnh cách ngây thơ, nhiều bài còn phân tích những lời giải **"trông như đúng"**: đúng thuật toán nhưng sai một chi tiết, và cho thấy chi tiết đó làm mất bao nhiêu điểm.

Mọi chương trình trong sách đều được kiểm tra hai lần: lời giải mẫu được đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu nhỏ ngẫu nhiên, và **mọi** cách làm (kể cả cách ngây thơ, cách sai) đều được chấm trên bộ test của sách với giới hạn thời gian và bộ nhớ của đề, ngăn xếp 8 MB. Các con số trong **bảng điểm** cuối mỗi bài là điểm thực tế mà từng chương trình nhận được, không phải ước lượng.

Lê Thành Nam

CẤU TRÚC MỘT LỜI GIẢI

Mục	Nội dung
1. Hiểu bài toán	Diễn đạt lại đề bằng ngôn ngữ đồ thị: đỉnh là gì, cạnh là gì, đề bài thực chất hỏi gì; nhận ra mô hình quen thuộc ẩn sau câu chuyện.
2. Cách ngây thơ	Sơ đồ khối, chương trình và độ phức tạp của cách làm đầu tiên ai cũng nghĩ ra.
3. Bế tắc	Vì sao cách hiện tại không vượt qua được subtask lớn; đôi khi kèm một lời giải trung gian hoặc một lời giải sai "trông như đúng".
4. Phá bế tắc	Quan sát then chốt, thuật toán, chạy tay trên ví dụ, lập luận vì sao đúng.
5. Cách tối ưu	Chương trình hoàn chỉnh và độ phức tạp.
6. Bẫy thường gặp	Những lỗi làm mất điểm hay gặp nhất.
7. Bảng điểm	Điểm thực tế của từng cách làm trên bộ test. Dòng tô xanh là lời giải mẫu.

Cách chấm điểm trong sách

- Mỗi bài có một bộ test chia theo đúng các subtask của Tập 2 (thường 2 – 6 test cho mỗi subtask, gồm cả test ngẫu nhiên lẫn test được thiết kế để "bẫy" những lỗi hay gặp). Đáp án của test được sinh bằng lời giải mẫu đã kiểm chứng.
- Mỗi chương trình chạy với giới hạn thời gian và bộ nhớ của đề, **ngăn xếp 8 MB** (như phần lớn hệ thống chấm chạy Linux; trên Windows, ngăn xếp mặc định chỉ 1 MB nên đệ quy sâu còn nguy hiểm hơn).
- Một subtask được **trọn điểm** khi chương trình qua **mọi** test của subtask đó, ngược lại được 0 điểm.
- Bài có nhiều đáp án được chấm bằng chương trình kiểm tra (checker).

> **LỜI KHUYÊN**
>
> Đừng đọc lời giải như đọc truyện. Sau mục 3, hãy gấp sách và tự tìm cách phá bế tắc trong ít nhất mười lăm phút. Sau mục 5, hãy tự viết lại chương trình mà không nhìn sách, rồi tự sinh test để đối chiếu với cách ngây thơ của chính em (Tập 2, Phụ lục B).

MỤC LỤC

LỜI NÓI ĐẦU **2**

CẤU TRÚC MỘT LỜI GIẢI **3**

CHƯƠNG 1. LÀM QUEN VỚI ĐỒ THỊ **6**

Bài 1.1. BẮT TAY 7
Bài 1.2. NGƯỜI NỔI TIẾNG 12
Bài 1.3. TAM GIÁC BẠN BÈ 16
Bài 2.1. CHUYỂN ĐỔI 22
Bài 2.2. BẠN CHUNG 26
Bài 2.3. BẢN ĐỒ KHO BÁU 30
Bài 3.1. ĐẾM ĐẢO 35
Bài 3.2. CÂY HAY KHÔNG 40
Bài 3.3. MẠNG LƯỚI DỰ PHÒNG 44
Bài 4.1. MÃ ĐI TUẦN 50
Bài 4.2. CHẠY TRỐN ĐÁM CHÁY 55
Bài 4.3. CHIA ĐỘI 61

CHƯƠNG 2. ĐƯỜNG ĐI NGẮN NHẤT **66**

Bài 5.1. ĐƯỜNG TỚI TRƯỜNG 67
Bài 5.2. NÚT GIAO QUAN TRỌNG 72
Bài 5.3. XE BUÝT ĐÊM 78
Bài 6.1. ĐỔI TIỀN 83
Bài 6.2. CHUYẾN BAY GIÁ RẺ 87
Bài 6.3. LỊCH THI ĐẤU 91
Bài 7.1. THỦ ĐÔ MỚI 96
Bài 7.2. XẾP HẠNG KÌ THỦ 100
Bài 7.3. XE TẢI QUÁ KHỔ 104
Bài 8.1. ĐÈN GIAO THÔNG 109
Bài 8.2. PHÁ TƯỜNG 114
Bài 8.3. ROBOT XOAY 119

CHƯƠNG 3. CÂY VÀ CÂY KHUNG **124**

Bài 9.1. MẠNG BẠN BÈ 125
Bài 9.2. PHÁ CẦU 131
Bài 9.3. HAI PHE 137
Bài 10.1. NỐI MẠNG TRƯỜNG HỌC 142
Bài 10.2. ĐƯỜNG ÍT DỐC 146
Bài 10.3. GIẾNG VÀ ỐNG 151
Bài 11.1. HỌ HÀNG 155
Bài 11.2. CÁP QUANG YẾU NHẤT 161
Bài 11.3. NHÂN VIÊN CẤP K 166

CHƯƠNG 4. CẤU TRÚC CỦA ĐỒ THỊ **171**

Bài 12.1. XẾP HẠNG GIẢI ĐẤU 172
Bài 12.2. DỰ ÁN XÂY NHÀ 178
Bài 12.3. ĐẾM LỘ TRÌNH 185
Bài 13.1. ĐƯỜNG HUYẾT MẠCH 190
Bài 13.2. TRẠM BIẾN ÁP 195
Bài 13.3. CÁP DỰ PHÒNG 200
Bài 14.1. LAN TRUYỀN TIN ĐỒN 205

Bài 14.2. KHO BÁU THỊ TRẤN.....210
Bài 14.3. HỘI ĐỒNG TUYỂN CHỌN.....216
Bài 15.1. NÉT VẼ.....221
Bài 15.2. GHÉP DOMINO.....225
Bài 15.3. MẬT KHẨU KÉT SẮT.....230

CHƯƠNG 5. GHÉP CẶP VÀ LUỒNG.....235

Bài 16.1. XẾP PHÒNG THI.....236
Bài 16.2. ĐẶT DOMINO.....241
Bài 16.3. CAMERA GIÁM SÁT.....246
Bài 17.1. ĐƯỜNG ỐNG.....252
Bài 17.2. LỘ TRÌNH RỜI NHAU.....259
Bài 17.3. PHÂN CÔNG TRỰC NHẬT.....265

PHỤ LỤC A. TỔNG HỢP LỜI GIẢI VÀ ĐIỂM SỐ.....271

PHỤ LỤC B. THƯ VIỆN MÃ NGUỒN HAY DÙNG.....274

CHƯƠNG 1. LÀM QUEN VỚI ĐỒ THỊ

Các bài tập của chương 1 kiểm tra những kĩ năng nền tảng: đếm bậc, chọn cách biểu diễn đồ thị, duyệt đồ thị bằng DFS và BFS. Thuật toán không khó, nên điểm số thường bị mất ở những chỗ rất "đời thường": tràn số, tràn ngăn xếp, dữ liệu bị lặp, hoặc làm lại một việc nhiều lần trong khi chỉ cần làm một lần. Bảng dưới đây tóm tắt con đường từ cách ngây thơ tới lời giải mẫu của từng bài.

Bài	Tên bài	Mức	Cách ngây thơ (điểm)	Chìa khóa của lời giải
1.1	**BẮT TAY**	★	Mô phỏng từng cái bắt tay (60)	Định lí bắt tay + điều kiện "người nhiều nhất".
1.2	**NGƯỜI NỔI TIẾNG**	★★	Thử từng người, duyệt lại m dòng (30)	Bán bậc vào/ra sau khi bỏ cung lặp.
1.3	**TAM GIÁC BẠN BÈ**	★★★	Ba vòng lặp trên ma trận kề (50)	Hướng cạnh từ đỉnh bậc nhỏ sang bậc lớn: $O(m\sqrt{m})$.
2.1	**CHUYỂN ĐỔI**	★	In mọi ô bằng 1 thành cạnh (0)	Duyệt ma trận đúng thứ tự, chỉ lấy $i < j$.
2.2	**BẠN CHUNG**	★★	Ma trận kề, duyệt mọi w (30)	Duyệt danh sách ngắn hơn + ghi nhớ cặp đã hỏi.
2.3	**BẢN ĐỒ KHO BÁU**	★★★	Đếm lại toàn bộ sau mỗi sự kiện (60)	Mỗi sự kiện chỉ ảnh hưởng tới 5 ô.
3.1	**ĐẾM ĐẢO**	★	DFS đệ quy (40)	Loang bằng ngăn xếp tự quản lí.
3.2	**CÂY HAY KHÔNG**	★★	Chỉ sửa số cạnh: $\lvert m - (n - 1)\rvert$ (30)	Đáp số = $(m - (n - k)) + (k - 1)$.
3.3	**MẠNG LƯỚI DỰ PHÒNG**	★★★	BFS riêng từ từng máy (30)	Kích thước thành phần; nối hai nhóm lớn nhất.
4.1	**MÃ ĐI TUẦN**	★	BFS lại cho mỗi câu hỏi (60)	BFS một lần từ ô xuất phát.
4.2	**CHẠY TRỐN ĐÁM CHÁY**	★★	BFS cho An, bỏ qua lửa (30)	BFS nhiều nguồn cho lửa, BFS cho An với điều kiện "<".
4.3	**CHIA ĐỘI**	★★★	Thử 2^n cách chia (20)	Tô hai màu + tổng tập con bằng bitset.

BÀI 1.1 BẮT TAY

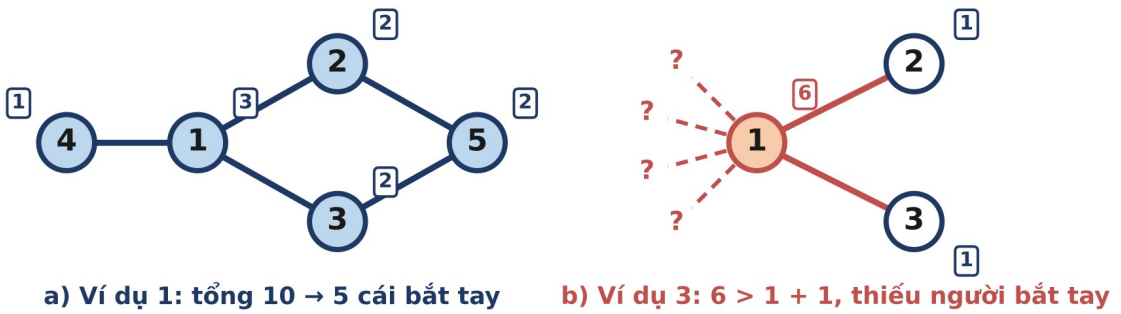
Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
BATTAY.INP	BATTAY.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Tập 1, Bài 1 · Đề bài: Tập 2, trang 12

1. Hiểu bài toán

Mỗi bạn là một **đỉnh**, mỗi lần bắt tay là một **cạnh** nối hai đỉnh khác nhau; vì hai bạn được bắt tay nhiều lần nên đồ thị có thể có **cạnh lặp**, nhưng không có **khuyên**. Câu hỏi trở thành: dãy $d_1, ..., d_n$ có phải là **dãy bậc** của một đa đồ thị không khuyên hay không, và nếu có thì đồ thị ấy có bao nhiêu cạnh.

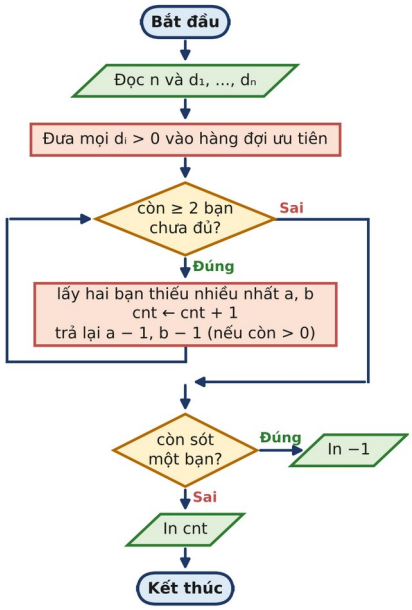
Theo định lí bắt tay, nếu tồn tại thì số cạnh **bắt buộc** bằng $(d_1 + ... + d_n) / 2$, nên phần khó duy nhất là quyết định "có tồn tại hay không".



Hình 1.1.1. *Ví dụ 1 có một cách bắt tay hợp lệ; ví dụ 3 thì bạn 1 "thiếu người" để bắt tay.*

2. Cách ngây thơ: Mô phỏng từng cái bắt tay

Nếu chưa nghĩ ra điều kiện, ta có thể **thử xây** đồ thị: chừng nào còn ít nhất hai bạn chưa đủ số lần bắt tay, cho **hai bạn còn thiếu nhiều nhất** bắt tay nhau. Nếu cuối cùng còn sót đúng một bạn chưa đủ thì không thể xếp được. (Chọn hai bạn thiếu nhiều nhất là quan trọng: nếu chọn tùy tiện, ta có thể để lại một bạn thiếu rất nhiều mà không còn ai bắt tay cùng.)



Hình 1.1.2. *Sơ đồ khối của cách mô phỏng.*

Cách ngây thơ: mô phỏng bằng hàng đợi ưu tiên

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("BATTAY.INP", "r")) {
        freopen("BATTAY.INP", "r", stdin);
        freopen("BATTAY.OUT", "w", stdout);
    }
    int n;
    scanf("%d", &n);
    priority_queue<long long> pq;          // số lần bắt tay còn thiếu của từng bạn
    long long cnt = 0;
    for (int i = 0; i < n; i++) {
        long long d;
        scanf("%lld", &d);
        if (d > 0) pq.push(d);
    }
    // Mỗi bước: hai bạn còn thiếu nhiều nhất bắt tay nhau một lần
    while (pq.size() >= 2) {
        long long a = pq.top(); pq.pop();
        long long b = pq.top(); pq.pop();
        cnt++;
        if (a > 1) pq.push(a - 1);
        if (b > 1) pq.push(b - 1);
    }
    if (!pq.empty()) printf("-1\n");       // còn đúng một bạn chưa đủ
    else printf("%lld\n", cnt);
    return 0;
}
```

Chạy tay ví dụ 1 (d = 3 2 2 1 2): lần lượt bắt tay các cặp (1, 2), (1, 3), (5, 1), (2, 3), (4, 5); sau 5 bước mọi bạn đều đủ: in **5**.

Độ phức tạp: mỗi vòng lặp tạo đúng một cái bắt tay, nên có S/2 vòng, mỗi vòng O(log n). Tổng O(S log n) với $S = d_1 + ... + d_n$.

3. Bế tắc

- **Subtask 1, 2** ($d_i \le 100$, $n \le 100$): $S \le 10^4$, chạy tức thì.
- **Subtask 3** ($d_i \le 10^9$, $n \le 10^5$): S tới 10^{14}, mô phỏng không bao giờ xong.

Cách mô phỏng được **60 điểm**. Hai cách "rút gọn" hay gặp cũng chỉ được điểm một phần:

- **Chỉ kiểm tra tổng chẵn** (đúng định lí bắt tay nhưng quên điều kiện thứ hai): sai ở ví dụ 3, được **30 điểm** (chỉ Subtask 1, nơi đề bài đảm bảo điều kiện thứ hai luôn đúng).
- **Đủ hai điều kiện nhưng cộng bằng `int`**: tổng 10^{14} bị tràn số, được **60 điểm**.

> **ĐIỂM BẾ TẮC**
>
> Mô phỏng làm **từng cái bắt tay một**, trong khi câu hỏi chỉ cần biết **có** làm được hay không. Ta cần một điều kiện kiểm tra được bằng một lần duyệt dãy.

4. Phá bế tắc

Gọi S là tổng và M là số lớn nhất trong dãy. Hai điều kiện **cần** rất tự nhiên:

1) **S chẵn** (định lí bắt tay: tổng bậc bằng 2 lần số cạnh).
2) **$M \le S - M$**: mỗi cái bắt tay của bạn "nhiều nhất" cần một bạn **khác**, và tổng số lần bắt tay của những bạn khác là $S - M$.

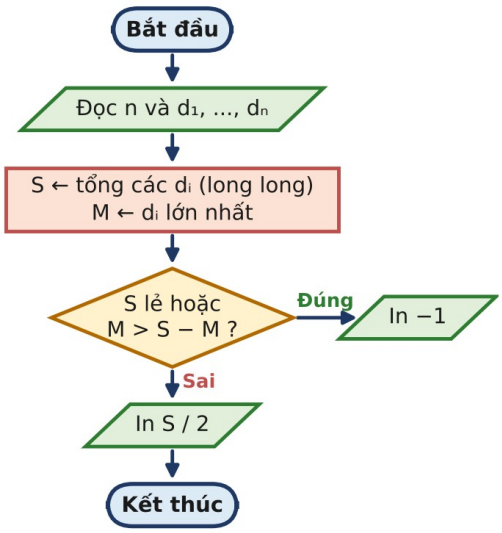
> **VÌ SAO HAI ĐIỀU KIỆN LÀ ĐỦ?**
>
> Quy nạp theo S. Nếu $S = 0$ thì không cần bắt tay. Nếu $S > 0$, cho hai bạn có số lớn nhất $M \ge M_2$ bắt tay một lần ($M_2 > 0$ vì $M \le S - M$). Dãy mới có tổng $S - 2$, vẫn chẵn; chỉ cần kiểm tra điều kiện 2 cho số lớn nhất mới M'.
>
> - Nếu $M' = M - 1$: điều kiện $M - 1 \le (S - 2) - (M - 1)$ tương đương $2M \le S$, đúng theo giả thiết.
> - Nếu không, $M' = M$ là một số chưa bị giảm, tức dãy có ít nhất **ba** số bằng M, nên $S \ge 3M$. Cần $2M + 2 \le S$: đúng khi $M \ge 2$; khi $M = 1$ thì $S \ge 3$ và S chẵn nên $S \ge 4 = 2M + 2$.
>
> Vậy dãy mới vẫn thỏa hai điều kiện, theo quy nạp xếp được. Chính thuật toán mô phỏng ở mục 2 là cách xây dựng theo lập luận này

Chạy tay ba ví dụ:

Dãy d	S	M	S chẵn? $M \le S - M$?	Đáp số
3 2 2 1 2	10	3	Đúng; $3 \le 7$	**5**
3 3 1 2	9	3	Sai	**−1**

Dãy d	S	M	S chẵn? M ≤ S − M?	Đáp số
6 1 1	8	6	Đúng; 6 > 2	**−1**

5. Cách tối ưu



Hình 1.1.3. *Sơ đồ khối của lời giải mẫu.*

BATTAY.CPP: kiểm tra hai điều kiện, O(n)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("BATTAY.INP", "r")) {
        freopen("BATTAY.INP", "r", stdin);
        freopen("BATTAY.OUT", "w", stdout);
    }
    int n;
    scanf("%d", &n);
    long long s = 0, mx = 0;              // tổng tới 1e14: phải dùng long long
    for (int i = 0; i < n; i++) {
        long long d;
        scanf("%lld", &d);
        s += d;
        mx = max(mx, d);
    }
    // Điều kiện cần và đủ: tổng chẵn, và người nhiều nhất không vượt tổng của
    // những người còn lại
    if (s % 2 != 0 || mx > s - mx) printf("-1\n");
    else printf("%lld\n", s / 2);
    return 0;
}
```

Độ phức tạp: một lần duyệt dãy: O(n) thời gian, O(1) bộ nhớ.

6. Bẫy thường gặp

- **Tràn số:** S có thể tới $10^5 \cdot 10^9 = 10^{14}$; cả S lẫn M phải là `long long`.
- **Quên điều kiện M ≤ S − M:** tổng chẵn là chưa đủ (ví dụ 3).
- **n = 1:** dãy một số d_1 chỉ hợp lệ khi $d_1 = 0$; công thức M ≤ S − M tự xử lí đúng trường hợp này.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
Mô phỏng từng cái bắt tay	30	30	0	**60**
Chỉ kiểm tra tổng chẵn	30	0	0	**30**
Đủ hai điều kiện, dùng int	30	30	0	**60**
Đủ hai điều kiện, long long	30	30	40	**100**

Chấm trên 13 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

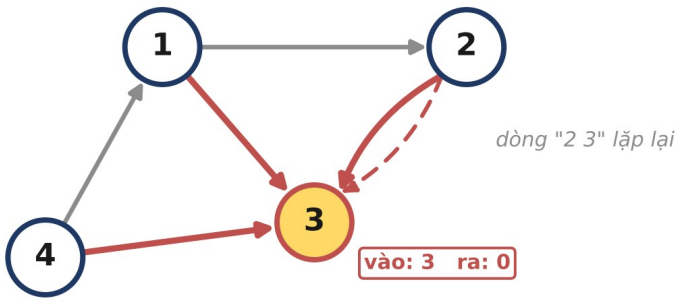
BÀI 1.2 NGƯỜI NỔI TIẾNG

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
NOITIENG.INP	NOITIENG.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Tập 1, Bài 1 · Đề bài: Tập 2, trang 15

1. Hiểu bài toán

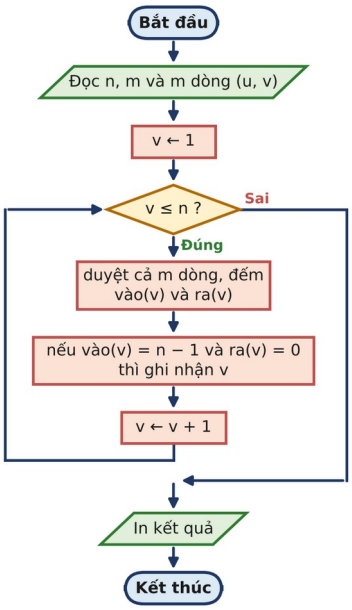
Mỗi tài khoản là một đỉnh, quan hệ "u theo dõi v" là một **cung** u → v. Người nổi tiếng là đỉnh c có **bán bậc vào** bằng n − 1 (mọi đỉnh khác đều có cung tới c) và **bán bậc ra** bằng 0. Chi tiết làm bài khác bài "đếm bậc" thông thường: **một cung có thể xuất hiện nhiều lần** trong dữ liệu.



Hình 1.2.1. *Ví dụ 1: dòng "2 3" bị ghi hai lần nhưng chỉ là một cung.*

2. Cách ngây thơ: Thử từng người, đếm lại trên toàn bộ dữ liệu

Với mỗi người v, duyệt lại cả m dòng dữ liệu để đếm số người theo dõi v và số người v theo dõi.



Hình 1.2.2. *Sơ đồ khối của cách thử từng người.*

Cách ngây thơ: O(n · m)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("NOITIENG.INP", "r")) {
        freopen("NOITIENG.INP", "r", stdin);
        freopen("NOITIENG.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<int> a(m), b(m);
    for (int i = 0; i < m; i++) scanf("%d %d", &a[i], &b[i]);
    int ans = -1;
    for (int v = 1; v <= n; v++) {          // thử từng người, đếm lại trên toàn bộ danh sách
        int in = 0, out = 0;
        for (int i = 0; i < m; i++) {
            if (b[i] == v) in++;
            if (a[i] == v) out++;
        }
        if (in == n - 1 && out == 0) ans = v;
    }
    printf("%d\n", ans);
    return 0;
}
```

Độ phức tạp: n lần duyệt m dòng: O(n · m). Với n = m = $2 \cdot 10^5$ là $4 \cdot 10^{10}$ phép tính.

3. Bế tắc

- **Subtask 1** ($n \le 1000$): $n \cdot m \le 2 \cdot 10^8$ phép so sánh đơn giản, chạy kịp: **30 điểm**.
- **Subtask 2, 3**: quá thời gian.

Tổng cộng **30 điểm**. Cách sửa tự nhiên là đếm bậc **một lần** cho mọi đỉnh (như Bài 1 của Tập 1): mỗi dòng u v cộng 1 vào ra[u] và vào[v]. Cách này đúng với Subtask 2 nhưng **sai** khi có dòng lặp: ở ví dụ 1, vào[3] = 4 ≠ n − 1. Được **60 điểm**.

ĐIỂM BẾ TẮC

Đếm bậc theo từng dòng dữ liệu là đếm **số dòng**, không phải đếm **số cung khác nhau**. Phải biến danh sách dòng thành danh sách cung trước khi đếm.

4. Phá bế tắc

Hai dòng giống nhau là hai cặp (u, v) bằng nhau. Sau khi **sắp xếp** danh sách cặp, các cặp bằng nhau đứng liền nhau; hàm `unique` giữ lại mỗi cặp một lần. Khi đó đếm bậc như bình thường.

Bước	Danh sách cặp (u, v)	Ghi chú
Dữ liệu vào	(1,3) (2,3) (4,3) (1,2) (4,1) (2,3)	6 dòng
Sau khi sắp xếp	(1,2) (1,3) (2,3) (2,3) (4,1) (4,3)	hai cặp (2,3) đứng cạnh nhau
Sau `unique`	(1,2) (1,3) (2,3) (4,1) (4,3)	5 cung khác nhau
Đếm bậc	vào[3] = 3 = n − 1, ra[3] = 0	đáp số **3**

VÌ SAO CÓ NHIỀU NHẤT MỘT NGƯỜI NỔI TIẾNG?

Giả sử a và b đều nổi tiếng. Vì a nổi tiếng nên b phải theo dõi a, nhưng b nổi tiếng thì không theo dõi ai: mâu thuẫn. Vì vậy chỉ cần duyệt v và in đỉnh thỏa mãn (nếu có).

5. Cách tối ưu

NOITIENG.CPP: bỏ cung lặp rồi đếm bán bậc, O(m log m)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("NOITIENG.INP", "r")) {
        freopen("NOITIENG.INP", "r", stdin);
        freopen("NOITIENG.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<pair<int,int>> e(m);
    for (auto &p : e) scanf("%d %d", &p.first, &p.second);
    // Bỏ các dòng lặp: sắp xếp rồi giữ lại mỗi cặp một lần
    sort(e.begin(), e.end());
    e.erase(unique(e.begin(), e.end()), e.end());
    vector<int> in(n + 1, 0), out(n + 1, 0);
    for (auto &p : e) { out[p.first]++; in[p.second]++; }
    int ans = -1;
    for (int v = 1; v <= n; v++)
        if (in[v] == n - 1 && out[v] == 0) ans = v;
    printf("%d\n", ans);
    return 0;
}
```

Độ phức tạp: sắp xếp O(m log m), đếm O(n + m).

> **MỞ RỘNG: BÀI TOÁN NGƯỜI NỔI TIẾNG KINH ĐIỂN**
>
> Nếu thay vì danh sách cung, ta chỉ được **hỏi** "a có theo dõi b không?", thì vẫn tìm được người nổi tiếng với chỉ khoảng 3n câu hỏi: giữ một ứng viên c; với mỗi người i, nếu c theo dõi i thì c không thể nổi tiếng, đổi c = i. Cuối cùng kiểm tra lại ứng viên duy nhất.

6. Bẫy thường gặp

- **Quên dòng lặp:** lỗi phổ biến nhất của bài, mất trọn Subtask 3.
- **Dùng ma trận kề** để khử lặp: $n = 2 \cdot 10^5$ thì ma trận có $4 \cdot 10^{10}$ ô, không đủ bộ nhớ.
- **Quên điều kiện bán bậc ra bằng 0**: một người được mọi người theo dõi nhưng lại theo dõi người khác thì không nổi tiếng.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
Thử từng người, duyệt lại m dòng	30	0	0	**30**
Đếm bậc, không bỏ dòng lặp	30	30	0	**60**
Bỏ cung lặp rồi đếm bậc	30	30	40	**100**

Chấm trên 10 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

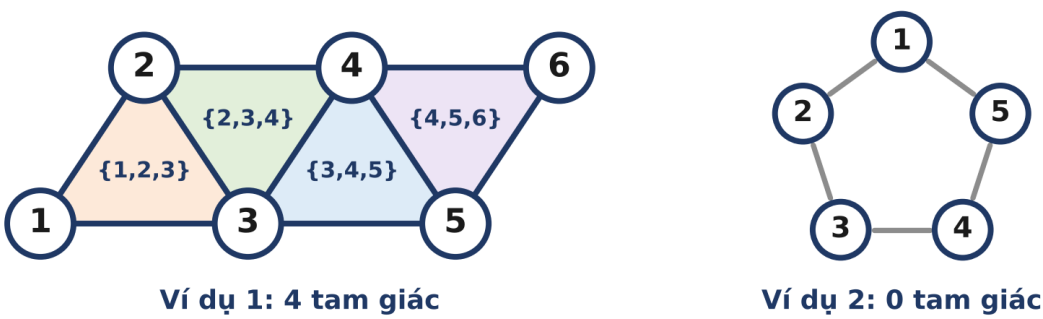
BÀI 1.3 TAM GIÁC BẠN BÈ

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
TAMGIAC.INP	TAMGIAC.OUT	2 giây	256 MB	★★★ NÂNG CAO

Liên hệ: Tập 1, Bài 1 · Đề bài: Tập 2, trang 17

1. Hiểu bài toán

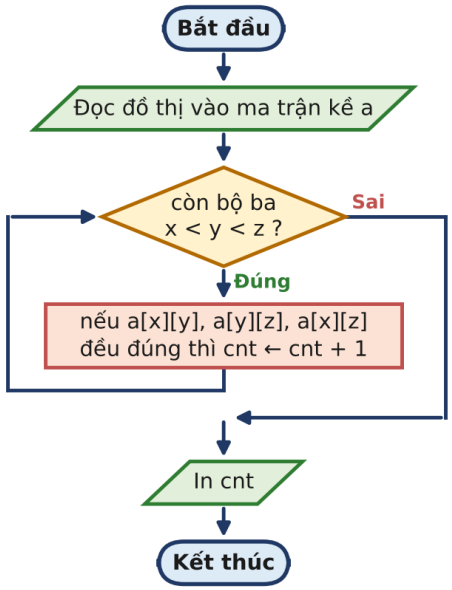
Ta cần đếm số **chu trình độ dài 3** (tam giác) trong một đơn đồ thị vô hướng tới 10^5 đỉnh và $2 \cdot 10^5$ cạnh. Mỗi tam giác {x, y, z} có ba cạnh, ba đỉnh; cách đếm nào cũng phải cẩn thận để mỗi tam giác chỉ được tính **một lần**.



Hình 1.3.1. *Hai ví dụ của đề.*

2. Cách ngây thơ: Thử mọi bộ ba đỉnh

Lưu ma trận kề a và thử mọi bộ ba x < y < z.



Hình 1.3.2. Sơ đồ khối của cách thử mọi bộ ba.

Cách ngây thơ: ba vòng lặp

```cpp
#include <bits/stdc++.h>
using namespace std;

const int N = 2005;
bool a[N][N];

int main() {
    if (fopen("TAMGIAC.INP", "r")) {
        freopen("TAMGIAC.INP", "r", stdin);
        freopen("TAMGIAC.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    if (n >= N) { printf("0\n"); return 0; }   // không đủ bộ nhớ cho ma trận
    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        a[u][v] = a[v][u] = true;
    }
    long long cnt = 0;
    for (int x = 1; x <= n; x++)
        for (int y = x + 1; y <= n; y++)
            for (int z = y + 1; z <= n; z++)
                if (a[x][y] && a[y][z] && a[x][z]) cnt++;
    printf("%lld\n", cnt);
    return 0;
}
```

Độ phức tạp: trên giấy là $O(n^3)$. Thực tế điều kiện `a[x][y]` được kiểm tra trước nên vòng z chỉ thật sự chạy khi (x, y) là cạnh: $O(n \cdot m)$. Bộ nhớ $O(n^2)$.

3. Bế tắc

- **Subtask 1** ($n \le 100$) và **Subtask 2** ($n \le 2000$): chạy kịp, **50 điểm**.
- **Subtask 3** ($n \le 10^5$): ma trận 10^{10} ô không thể khai báo.

Hai cải tiến "nửa vời" dễ nghĩ tới đều dừng ở cùng mức điểm:

- **Ma trận `bitset`, đếm theo cạnh:** với mỗi cạnh (u, v), số tam giác chứa nó là `(a[u] & a[v]).count()`; cộng lại rồi chia 3. Rất nhanh, nhưng vẫn cần ma trận $n \times n$: **50 điểm**.
- **Danh sách kề, đếm theo cạnh:** với mỗi cạnh (u, v), đánh dấu các bạn của u rồi duyệt các bạn của v. Không cần ma trận, nhưng chi phí là Σ (deg u + deg v) trên mọi cạnh = Σ $\deg(v)^2$. Một "ngôi sao" có 10^5 cạnh làm con số này lên 10^{10}: **50 điểm**.

ĐIỂM BẾ TẮC

Các đỉnh có bậc rất lớn bị duyệt lại rất nhiều lần (mỗi cạnh chạm vào chúng lại duyệt toàn bộ danh sách kề của chúng). Cần một cách đếm mà mỗi đỉnh chỉ phải "mở rộng" các hàng xóm ít ỏi của mình.

4. Phá bế tắc

Đặt một thứ tự toàn phần trên các đỉnh: u **thấp hơn** v nếu $\deg(u) < \deg(v)$, hoặc bậc bằng nhau và $u < v$. **Hướng** mỗi cạnh từ đầu thấp sang đầu cao. Mỗi tam giác có đúng một đỉnh thấp nhất u, đỉnh giữa v và đỉnh cao nhất w, nên nó xuất hiện **đúng một lần** dưới dạng $u \to v \to w$ cùng với $u \to w$.

VÌ SAO MỖI ĐỈNH CHỈ CÓ ÍT CUNG ĐI RA?

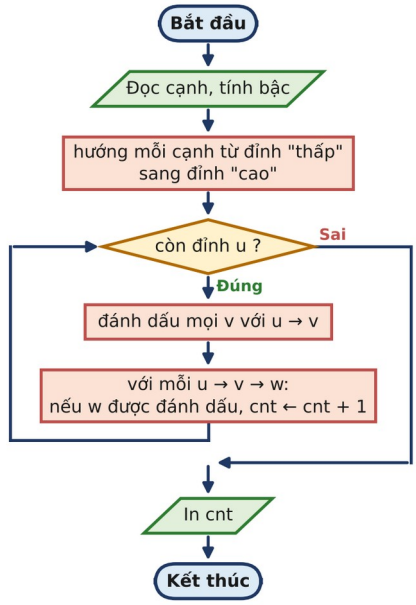
Giả sử u có k cung đi ra. Các đỉnh đích đều "cao hơn" u nên có bậc ít nhất $\deg(u) \ge k$. Vậy có k đỉnh bậc $\ge k$, tổng bậc ít nhất $k^2 \le 2m$, suy ra $k \le \sqrt{(2m)} \approx 632$. Mỗi cạnh $u \to v$ chỉ phải duyệt các cung ra của v (không quá $\sqrt{(2m)}$), nên tổng chi phí là $O(m\sqrt{m}) \approx 1{,}3 \cdot 10^8$ phép tính đơn giản trong trường hợp xấu nhất.

Chạy tay ví dụ 1. Bậc: deg(1) = 2, deg(2) = 3, deg(3) = 4, deg(4) = 4, deg(5) = 3, deg(6) = 2.

Đỉnh u	Cung đi ra ($u \to v$)	Tam giác tìm được ($u \to v \to w$ và $u \to w$)
1	$1 \to 2$, $1 \to 3$	$1 \to 2 \to 3$: **{1, 2, 3}**
2	$2 \to 3$, $2 \to 4$	$2 \to 3 \to 4$: **{2, 3, 4}**
3	$3 \to 4$	(không có)
4	(không có)	(không có)
5	$5 \to 3$, $5 \to 4$	$5 \to 3 \to 4$: **{3, 4, 5}**
6	$6 \to 4$, $6 \to 5$	$6 \to 5 \to 4$: **{4, 5, 6}**

Đếm được 4 tam giác, mỗi tam giác đúng một lần.

5. Cách tối ưu



Hình 1.3.3. *Sơ đồ khối của lời giải mẫu.*

TAMGIAC.CPP: định hướng cạnh theo bậc, O(m√m)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("TAMGIAC.INP", "r")) {
        freopen("TAMGIAC.INP", "r", stdin);
        freopen("TAMGIAC.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<int> eu(m), ev(m), deg(n + 1, 0);
    for (int i = 0; i < m; i++) {
        scanf("%d %d", &eu[i], &ev[i]);
        deg[eu[i]]++; deg[ev[i]]++;
    }
    // "u thấp hơn v" nếu bậc nhỏ hơn, bằng bậc thì số hiệu nhỏ hơn
    auto low = [&](int u, int v) { return deg[u] < deg[v] || (deg[u] ==
deg[v]
        && u < v); };
    vector<vector<int>> out(n + 1);        // mỗi cạnh chỉ giữ chiều từ đỉnh
        thấp sang đỉnh cao
    for (int i = 0; i < m; i++) {
        int u = eu[i], v = ev[i];
        if (low(u, v)) out[u].push_back(v); else out[v].push_back(u);
    }
    vector<int> mark(n + 1, 0);
    long long cnt = 0;
    for (int u = 1; u <= n; u++) {
        for (int v : out[u]) mark[v] = u;  // đánh dấu các đỉnh cao hơn kề u
        for (int v : out[u])
            for (int w : out[v])
                if (mark[w] == u) cnt++;   // tam giác u < v < w được đếm
đúng
                    một lần
    }
    printf("%lld\n", cnt);
    return 0;
}
```

Độ phức tạp: O(m√m) thời gian, O(n + m) bộ nhớ. Mảng `mark[w] = u` thay cho việc xóa dấu sau mỗi đỉnh.

6. Bẫy thường gặp

- **Đếm ba lần mỗi tam giác** rồi quên chia 3 (ở các cách đếm theo cạnh).
- **So sánh chỉ theo bậc** mà không phá hòa bằng số hiệu: hai đỉnh cùng bậc sẽ không có thứ tự, cạnh giữa chúng bị hướng cả hai chiều hoặc không chiều nào.
- **Khai báo ma trận kề cho n = 10^5:** 10^{10} byte, chương trình không chạy được.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (20)	ST2 (30)	ST3 (50)	Tổng
Ba vòng lặp trên ma trận kề	20	30	0	**50**
Ma trận bitset, đếm theo cạnh	20	30	0	**50**

Thuật toán	ST1 (20)	ST2 (30)	ST3 (50)	Tổng
Danh sách kề, đếm theo cạnh	20	30	0	**50**
Định hướng cạnh theo bậc	20	30	50	**100**

Chấm trên 10 test, giới hạn 2 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

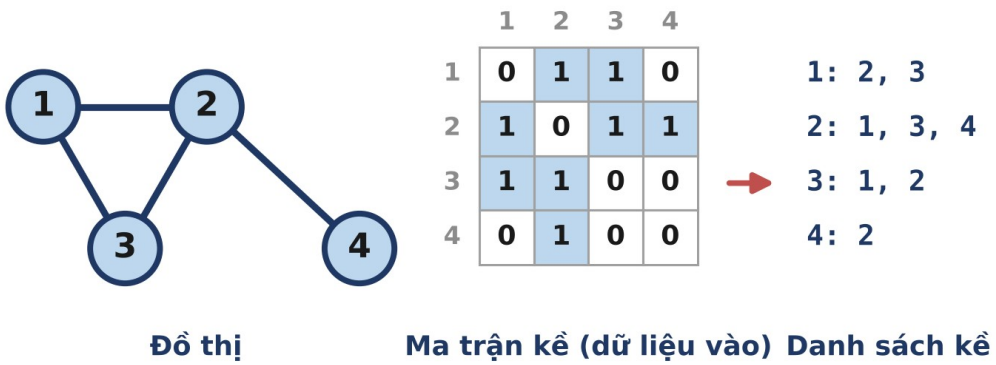
BÀI 2.1 CHUYỂN ĐỔI

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
CHUYENDOI.INP	CHUYENDOI.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Tập 1, Bài 2 · Đề bài: Tập 2, trang 19

1. Hiểu bài toán

Bài luyện chuyển đổi giữa ba cách biểu diễn của Tập 1, Bài 2. Ma trận kề đối xứng, nên mỗi cạnh (u, v) xuất hiện **hai lần**: ở ô (u, v) và ô (v, u). Danh sách cạnh chỉ được in mỗi cạnh một lần, còn danh sách kề của u thì phải có v và danh sách kề của v phải có u.



Hình 2.1.1. *Ba cách biểu diễn cùng một đồ thị.*

2. Cách ngây thơ: In mọi ô bằng 1 thành một cạnh

Cách viết vội vàng nhất là gặp ô bằng 1 nào thì thêm cạnh đó vào danh sách cạnh. Danh sách kề thì đúng, nhưng danh sách cạnh bị gấp đôi.

Cách sai: thiếu điều kiện i < j

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("CHUYENDOI.INP", "r")) {
        freopen("CHUYENDOI.INP", "r", stdin);
        freopen("CHUYENDOI.OUT", "w", stdout);
    }
    int n;
    scanf("%d", &n);
    vector<vector<int>> g(n + 1);
    vector<pair<int,int>> e;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++) {
            int x;
            scanf("%d", &x);
            if (x) { g[i].push_back(j); e.push_back({i, j}); }   // LỖI: thiếu
                điều kiện i < j
        }
    printf("%d\n", (int)e.size());
    for (auto [u, v] : e) printf("%d %d\n", u, v);
    for (int i = 1; i <= n; i++) {
        printf("%d", (int)g[i].size());
        for (int v : g[i]) printf(" %d", v);
        printf("\n");
    }
    return 0;
}
```

Với ví dụ, chương trình in "8" ở dòng đầu thay vì "4", kèm cả (2, 1), (3, 1), ... Mọi test có ít nhất một cạnh đều sai: **0 điểm**.

3. Bế tắc

Ngoài lỗi đếm đôi, có hai nỗi lo khác: phải **sắp xếp** danh sách cạnh và các danh sách kề hay không, và đọc 10^6 số có kịp không.

> **ĐIỂM BẾ TẮC**
>
> Ma trận đối xứng: thông tin của nửa trên đường chéo đã đủ để dựng danh sách cạnh. Câu hỏi là làm sao để kết quả **tự** đúng thứ tự mà không phải sắp xếp.

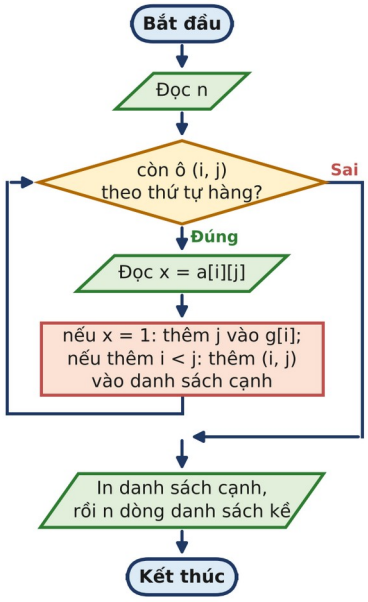
4. Phá bế tắc

Ta đọc ma trận **theo thứ tự hàng**: i = 1..n, trong mỗi hàng j = 1..n. Hai quan sát:

1) Các cạnh (i, j) với i < j được gặp theo đúng thứ tự "tăng dần theo i, cùng i thì tăng dần theo j" mà đề yêu cầu.

2) Trong hàng i, các j được gặp theo thứ tự tăng dần, nên đẩy j vào g[i] ngay khi gặp thì g[i] tự được sắp xếp.

Vậy chỉ cần một lần đọc, thêm điều kiện i < j cho danh sách cạnh. Đọc 10^6 số bằng `scanf` mất khoảng 0,1 – 0,2 giây.



Hình 2.1.2. *Sơ đồ khối của lời giải mẫu.*

5. Cách tối ưu

CHUYENDOI.CPP: một lần đọc ma trận, O(n²)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("CHUYENDOI.INP", "r")) {
        freopen("CHUYENDOI.INP", "r", stdin);
        freopen("CHUYENDOI.OUT", "w", stdout);
    }
    int n;
    scanf("%d", &n);
    vector<vector<int>> g(n + 1);          // danh sách kề
    vector<pair<int,int>> e;               // danh sách cạnh
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++) {
            int x;
            scanf("%d", &x);
            if (!x) continue;
            g[i].push_back(j);             // j tăng dần nên g[i] tự sắp xếp
            if (i < j) e.push_back({i, j});// mỗi cạnh chỉ lấy một lần
        }
    printf("%d\n", (int)e.size());
    for (auto [u, v] : e) printf("%d %d\n", u, v);
    for (int i = 1; i <= n; i++) {
        printf("%d", (int)g[i].size());
        for (int v : g[i]) printf(" %d", v);
        printf("\n");
    }
    return 0;
}
```

Độ phức tạp: $O(n^2)$ để đọc ma trận (không thể ít hơn vì phải đọc hết dữ liệu), bộ nhớ $O(n + m)$.

6. Bẫy thường gặp

- **Đếm đôi cạnh** khi quên điều kiện i < j.
- **Lưu cả ma trận** vào mảng `int a[1000][1000]` vẫn được (4 MB), nhưng không cần thiết.
- **In dòng trống sai:** đỉnh cô lập vẫn phải có một dòng "0".

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (40)	ST2 (60)	Tổng
In mọi ô bằng 1 thành cạnh	0	0	**0**
Duyệt theo hàng, lấy i < j	40	60	**100**

Chấm trên 8 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

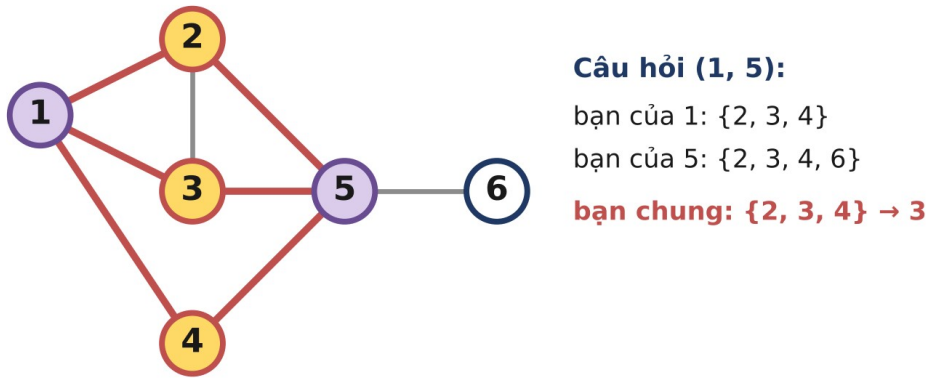
BÀI 2.2 BẠN CHUNG

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
BANCHUNG.INP	BANCHUNG.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Tập 1, Bài 2 · Đề bài: Tập 2, trang 22

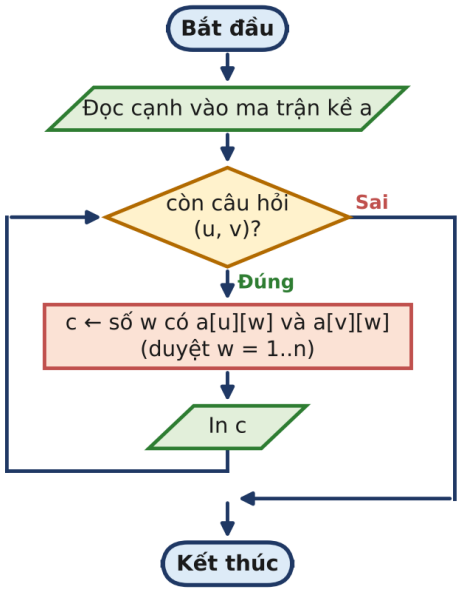
1. Hiểu bài toán

Số bạn chung của u và v là |N(u) ∩ N(v)|, kích thước giao của hai danh sách kề. Bài kiểm tra việc chọn cách biểu diễn: ma trận kề trả lời "w có kề u không" trong O(1) nhưng tốn O(n) cho mỗi câu hỏi và O(n^2) bộ nhớ; danh sách kề thì gọn nhưng phải tính giao.



Hình 2.2.1. *Lượt xem (1, 5): giao của {2, 3, 4} và {2, 3, 4, 6}.*

2. Cách ngây thơ: Ma trận kề, duyệt mọi w



Hình 2.2.2. Sơ đồ khối của cách dùng ma trận kề.

Cách ngây thơ: ma trận kề, O(n) mỗi câu hỏi

```
#include <bits/stdc++.h>
using namespace std;

const int N = 505;
bool a[N][N];

int main() {
    if (fopen("BANCHUNG.INP", "r")) {
        freopen("BANCHUNG.INP", "r", stdin);
        freopen("BANCHUNG.OUT", "w", stdout);
    }
    int n, m, q;
    scanf("%d %d %d", &n, &m, &q);
    if (n >= N) return 0;                    // ma trận không đủ chỗ
    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        a[u][v] = a[v][u] = true;
    }
    while (q--) {
        int u, v;
        scanf("%d %d", &u, &v);
        int c = 0;
        for (int w = 1; w <= n; w++) if (a[u][w] && a[v][w]) c++;
        printf("%d\n", c);
    }
    return 0;
}
```

Độ phức tạp: $O(n^2)$ bộ nhớ, $O(n \cdot q)$ thời gian. Với $n = 500$: $5 \cdot 10^7$ phép tính.

3. Bế tắc

- **Subtask 1** ($n \le 500$): chạy tốt: **30 điểm**.
- **Subtask 2, 3** (n tới 10^5): ma trận quá lớn.

Chuyển sang danh sách kề **đã sắp xếp**: với câu hỏi (u, v), duyệt danh sách **ngắn hơn** và tìm nhị phân trong danh sách dài hơn. Mỗi câu hỏi tốn $O(\min(\deg u, \deg v) \cdot \log n)$. Khi mọi bậc ≤ 300 (Subtask 2), mỗi câu hỏi tốn vài nghìn phép tính: chạy kịp. Nhưng Subtask 3 có hai "người nổi tiếng" cùng $5 \cdot 10^4$ bạn và 10^5 câu hỏi đều hỏi cặp này: $5 \cdot 10^9$ phép tính. Cách này được **60 điểm**.

> **ĐIỂM BẾ TẮC**
>
> Cùng một cặp (u, v) được hỏi lại hàng vạn lần, và lần nào chương trình cũng tính lại từ đầu.

4. Phá bế tắc

Ghi nhớ đáp số của mỗi cặp đã tính (dùng `map` với khóa là cặp đã chuẩn hóa: u là đỉnh có danh sách ngắn hơn). Vấn đề còn lại: có thể có nhiều cặp **khác nhau** cùng tốn kém không?

> **ĐÁNH GIÁ: TỔNG CHI PHÍ TRÊN CÁC CẶP KHÁC NHAU**
>
> Chọn ngưỡng $B = \sqrt{q}$. Đỉnh **lớn** là đỉnh có bậc $> m/B$; có không quá $2m/(m/B) = 2B$ đỉnh lớn.
>
> - Cặp có ít nhất một đỉnh nhỏ: chi phí $\le$ bậc của đỉnh nhỏ $\le m/B$; có $\le q$ cặp: tổng $\le q \cdot m/B = m\sqrt{q}$.
> - Cặp gồm hai đỉnh lớn: mỗi đỉnh lớn u ghép với nhiều nhất $2B$ đỉnh lớn khác, chi phí mỗi lần $\le \deg(u)$; tổng $\le \Sigma \deg(u) \cdot 2B \le 2m \cdot 2B = 4m\sqrt{q}$.
>
> Vậy tổng chi phí là $O(m\sqrt{q} \cdot \log n)$, khoảng $3 \cdot 10^7 \cdot 17$ phép tính trong trường hợp xấu nhất: chạy kịp.

5. Cách tối ưu

BANCHUNG.CPP: danh sách ngắn hơn, tìm nhị phân, ghi nhớ cặp

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("BANCHUNG.INP", "r")) {
        freopen("BANCHUNG.INP", "r", stdin);
        freopen("BANCHUNG.OUT", "w", stdout);
    }
    int n, m, q;
    scanf("%d %d %d", &n, &m, &q);
    vector<vector<int>> g(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        g[u].push_back(v); g[v].push_back(u);
    }
    for (int i = 1; i <= n; i++) sort(g[i].begin(), g[i].end());
    map<pair<int,int>, int> memo;          // kết quả của các cặp đã hỏi
    while (q--) {
        int u, v;
        scanf("%d %d", &u, &v);
        if (g[u].size() > g[v].size() || (g[u].size() == g[v].size() && u > v)) swap(u, v);
        auto it = memo.find({u, v});
        if (it == memo.end()) {
            int c = 0;
            for (int x : g[u]) if (binary_search(g[v].begin(), g[v].end(), x))
                c++;
            it = memo.insert({{u, v}, c}).first;
        }
        printf("%d\n", it->second);
    }
    return 0;
}
```

Độ phức tạp: O((m + q) log n + m√q log n) thời gian, O(n + m + q) bộ nhớ.

6. Bẫy thường gặp

- **Chuẩn hóa khóa của `map`:** (u, v) và (v, u) là cùng một câu hỏi; nếu không chuẩn hóa, kẻ ra đề có thể hỏi xen kẽ hai chiều để làm ghi nhớ mất tác dụng một nửa.
- **Quên sắp xếp danh sách kề** trước khi dùng `binary_search`.
- **Dùng ma trận** cho $n = 10^5$.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
Ma trận kề, duyệt mọi w	30	0	0	**30**
Danh sách ngắn hơn + tìm nhị phân	30	30	0	**60**
Như trên + ghi nhớ cặp đã hỏi	30	30	40	**100**

Chấm trên 7 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

BÀI 2.3 BẢN ĐỒ KHO BÁU

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
BANDO.INP	BANDO.OUT	1 giây	256 MB	★★★ NÂNG CAO

Liên hệ: Tập 1, Bài 2 · Đề bài: Tập 2, trang 24

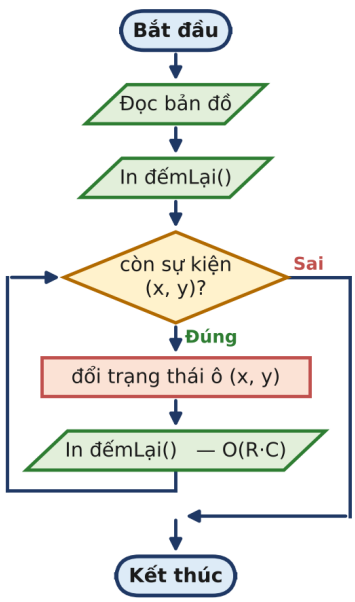
1. Hiểu bài toán

Lưới R × C là một đồ thị không cần dựng: ô (x, y) kề với tối đa 4 ô (x ± 1, y), (x, y ± 1). Một ô trống là **khả nghi** khi số ô trống kề với nó bằng đúng k. Sau mỗi sự kiện, một ô đổi trạng thái và ta cần số ô khả nghi.



Hình 2.3.1. *Ví dụ: số trong ô trống là số ô trống kề nó; ô vàng là ô khả nghi (k = 2).*

2. Cách ngây thơ: Đếm lại toàn bộ sau mỗi sự kiện



Hình 2.3.2. *Sơ đồ khối của cách đếm lại.*

Cách ngây thơ: O(R · C) mỗi sự kiện

```cpp
#include <bits/stdc++.h>
using namespace std;

int R, C, K;
vector<string> a;
const int dx[4] = {-1, 1, 0, 0}, dy[4] = {0, 0, -1, 1};

long long countAll() {                    // đếm lại toàn bộ bản đồ: O(R·C)
    long long c = 0;
    for (int x = 0; x < R; x++)
        for (int y = 0; y < C; y++) {
            if (a[x][y] != '.') continue;
            int k = 0;
            for (int d = 0; d < 4; d++) {
                int u = x + dx[d], v = y + dy[d];
                if (u >= 0 && u < R && v >= 0 && v < C && a[u][v] == '.') k++;
            }
            if (k == K) c++;
        }
    return c;
}

int main() {
    if (fopen("BANDO.INP", "r")) {
        freopen("BANDO.INP", "r", stdin);
        freopen("BANDO.OUT", "w", stdout);
    }
    int q;
    cin >> R >> C >> K >> q;
    a.resize(R);
    for (auto &s : a) cin >> s;
    printf("%lld\n", countAll());
    while (q--) {
        int x, y;
        scanf("%d %d", &x, &y); x--; y--;
        a[x][y] = (a[x][y] == '.' ? '#' : '.');
        printf("%lld\n", countAll());
    }
    return 0;
}
```

Độ phức tạp: $O(R \cdot C \cdot (q + 1))$. Với $R = C = 1000$, $q = 2 \cdot 10^5$ là $2 \cdot 10^{11}$ phép tính.

3. Bế tắc

- **Subtask 1** ($q = 0$) và **Subtask 2** (lưới 100×100, $q \le 100$): chạy tức thì.
- **Subtask 3**: không thể chạy xong.

Được **60 điểm**.

> **ĐIỂM BẾ TẮC**
>
> Mỗi sự kiện chỉ thay đổi **một** ô, vậy mà ta đếm lại cả triệu ô.

4. Phá bế tắc

Ô (x, y) đổi trạng thái thì những giá trị nào có thể thay đổi?

- Trạng thái của chính ô (x, y) (trống hay đá), và số ô trống kề nó **không đổi** (vì các ô kề không đổi), nhưng nó có thể trở thành / thôi là ô khả nghi.
- Số ô trống kề của **bốn ô kề** (x, y) thay đổi đúng 1.
- Mọi ô khác: không có gì thay đổi.

Vì vậy chỉ cần xét 5 ô: **trừ** đóng góp cũ của 5 ô (ô nào đang khả nghi thì trừ 1), đổi trạng thái, rồi **cộng** đóng góp mới của 5 ô. Kĩ thuật "trừ cũ – cộng mới" này rất hay dùng khi dữ liệu thay đổi cục bộ.

Sự kiện	5 ô bị ảnh hưởng	Thay đổi
Đào ô (1, 3)	(1, 3), (1, 2), (1, 4), (2, 3)	trước: chỉ (1, 2) khả nghi, trừ 1; sau: (1, 4) có 2 kề trống, cộng 1; vẫn **6**
Lấp ô (2, 2)	(2, 2), (1, 2), (3, 2), (2, 1), (2, 3)	trước: (2, 1), (3, 2) khả nghi, trừ 2; sau: (1, 2) giảm còn 2 kề trống, cộng 1; 6 − 2 + 1 = **5**

5. Cách tối ưu

BANDO.CPP: trừ cũ – cộng mới trên 5 ô, O(1) mỗi sự kiện

```cpp
#include <bits/stdc++.h>
using namespace std;

int R, C, K;
vector<string> a;
const int dx[4] = {-1, 1, 0, 0}, dy[4] = {0, 0, -1, 1};

bool inside(int x, int y) { return x >= 0 && x < R && y >= 0 && y < C; }

// 1 nếu ô (x, y) là ô khả nghi, 0 nếu không
int good(int x, int y) {
    if (a[x][y] != '.') return 0;
    int k = 0;
    for (int d = 0; d < 4; d++) {
        int u = x + dx[d], v = y + dy[d];
        if (inside(u, v) && a[u][v] == '.') k++;
    }
    return k == K;
}

int main() {
    if (fopen("BANDO.INP", "r")) {
        freopen("BANDO.INP", "r", stdin);
        freopen("BANDO.OUT", "w", stdout);
    }
    int q;
    cin >> R >> C >> K >> q;
    a.resize(R);
    for (auto &s : a) cin >> s;
    long long cnt = 0;
    for (int x = 0; x < R; x++)
        for (int y = 0; y < C; y++) cnt += good(x, y);
    printf("%lld\n", cnt);
    while (q--) {
        int x, y;
        scanf("%d %d", &x, &y); x--; y--;
        // chỉ ô (x, y) và bốn ô kề bị ảnh hưởng
        vector<pair<int,int>> cells = {{x, y}};
        for (int d = 0; d < 4; d++)
            if (inside(x + dx[d], y + dy[d])) cells.push_back({x + dx[d], y
+
                dy[d]});
        for (auto [u, v] : cells) cnt -= good(u, v);   // bỏ đóng góp cũ
        a[x][y] = (a[x][y] == '.' ? '#' : '.');
        for (auto [u, v] : cells) cnt += good(u, v);   // cộng đóng góp mới
        printf("%lld\n", cnt);
    }
    return 0;
}
```

Độ phức tạp: O(R · C + q) thời gian, O(R · C) bộ nhớ.

6. Bẫy thường gặp

- **Quên chính ô (x, y)** trong 5 ô cần cập nhật.
- **Ra ngoài lưới** khi xét ô kề của ô ở biên.

- **Cập nhật trước khi trừ:** phải trừ đóng góp **cũ** trước khi đổi trạng thái, rồi mới cộng đóng góp mới.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
Đếm lại toàn bộ sau mỗi sự kiện	30	30	0	**60**
Trừ cũ – cộng mới trên 5 ô	30	30	40	**100**

Chấm trên 8 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

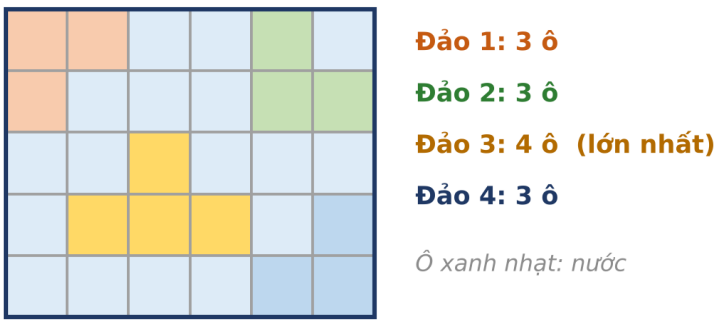
BÀI 3.1 ĐẾM ĐẢO

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
DEMDAO.INP	DEMDAO.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Tập 1, Bài 3 · Đề bài: Tập 2, trang 26

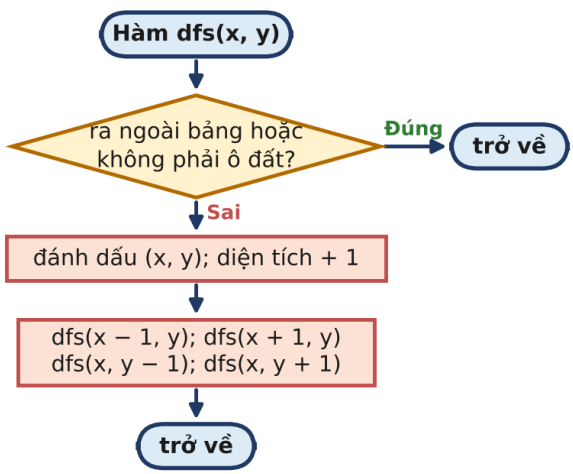
1. Hiểu bài toán

Mỗi ô đất là một đỉnh, hai ô đất chung cạnh được nối bằng một cạnh. Hòn đảo là **thành phần liên thông**, diện tích là số đỉnh của thành phần. Thuật toán là "loang" (DFS hoặc BFS) quen thuộc; chỗ khó duy nhất là kích thước: lưới 1000 × 1000 có thể là **một** hòn đảo 10^6 ô.



Hình 3.1.1. *Bốn hòn đảo của ví dụ.*

2. Cách ngây thơ: Loang bằng DFS đệ quy



Hình 3.1.2. *Sơ đồ khối của hàm loang đệ quy.*

Cách ngây thơ: DFS đệ quy

```cpp
#include <bits/stdc++.h>
using namespace std;

int R, C, area;
vector<string> a;

void dfs(int x, int y) {                    // loang đệ quy
    if (x < 0 || x >= R || y < 0 || y >= C || a[x][y] != '#') return;
    a[x][y] = 'x';
    area++;
    dfs(x - 1, y); dfs(x + 1, y); dfs(x, y - 1); dfs(x, y + 1);
}

int main() {
    if (fopen("DEMDAO.INP", "r")) {
        freopen("DEMDAO.INP", "r", stdin);
        freopen("DEMDAO.OUT", "w", stdout);
    }
    cin >> R >> C;
    a.resize(R);
    for (auto &s : a) cin >> s;
    int cnt = 0, best = 0;
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++)
            if (a[i][j] == '#') {
                cnt++; area = 0;
                dfs(i, j);
                best = max(best, area);
            }
    printf("%d %d\n", cnt, best);
    return 0;
}
```

Độ phức tạp: O(R · C) thời gian, nhưng độ sâu đệ quy có thể tới R · C = 10^6.

3. Bế tắc

- **Subtask 1** (R, C ≤ 50): độ sâu ≤ 2500, chạy tốt.
- **Subtask 2**: một hòn đảo phủ kín bảng 1000 × 1000 (hoặc hình "con rắn" dài) làm đệ quy sâu 10^6 tầng. Mỗi tầng tốn vài chục byte ngăn xếp, vượt giới hạn 8 MB: chương trình bị dừng với lỗi **Runtime Error**.

Được **40 điểm**, dù thuật toán hoàn toàn đúng.

ĐIỂM BẾ TẮC

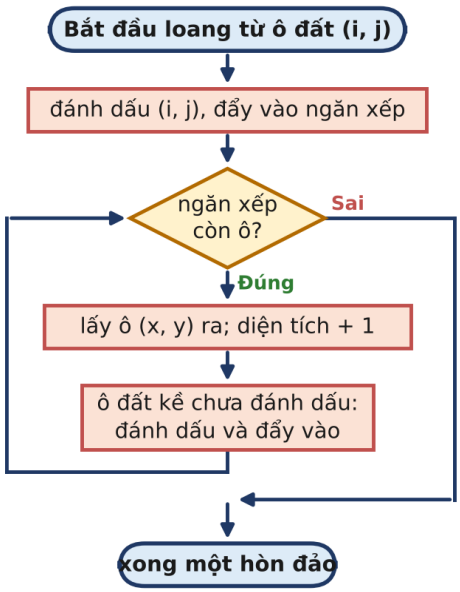
Ngăn xếp của chương trình (stack) nhỏ hơn nhiều so với bộ nhớ chung (heap): thường chỉ 1 – 8 MB. Đệ quy sâu hàng triệu tầng là cách chắc chắn nhất để mất điểm.

4. Phá bế tắc

Thay ngăn xếp của hệ thống bằng một `vector` **tự quản lí** (nằm trên heap, giới hạn bởi 256 MB): đẩy ô xuất phát vào, rồi lặp "lấy một ô ra, đẩy các ô đất kề chưa thăm vào".

ĐÁNH DẤU KHI ĐẨY, KHÔNG PHẢI KHI LẤY

Nếu chỉ đánh dấu khi lấy ô ra, một ô có thể bị đẩy vào ngăn xếp nhiều lần (bởi nhiều ô kề) và được đếm vào diện tích nhiều lần. Đánh dấu **ngay khi đẩy** đảm bảo mỗi ô vào ngăn xếp đúng một lần, kích thước ngăn xếp không quá R · C.



Hình 3.1.3. *Sơ đồ khối của bước loang không đệ quy.*

5. Cách tối ưu

DEMDAO.CPP: loang bằng ngăn xếp tự quản lí, O(R · C)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DEMDAO.INP", "r")) {
        freopen("DEMDAO.INP", "r", stdin);
        freopen("DEMDAO.OUT", "w", stdout);
    }
    int R, C;
    cin >> R >> C;
    vector<string> a(R);
    for (auto &s : a) cin >> s;
    const int dx[4] = {-1, 1, 0, 0}, dy[4] = {0, 0, -1, 1};
    int cnt = 0, best = 0;
    vector<pair<int,int>> st;              // ngăn xếp tự quản lí
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++) {
            if (a[i][j] != '#') continue;
            cnt++;
            int area = 0;
            a[i][j] = 'x';                 // đánh dấu ngay khi đẩy vào
            st.push_back({i, j});
            while (!st.empty()) {
                auto [x, y] = st.back(); st.pop_back();
                area++;
                for (int d = 0; d < 4; d++) {
                    int u = x + dx[d], v = y + dy[d];
                    if (u >= 0 && u < R && v >= 0 && v < C && a[u][v] ==
'#')
                        {
                        a[u][v] = 'x';
                        st.push_back({u, v});
                    }
                }
            }
            best = max(best, area);
        }
    printf("%d %d\n", cnt, best);
    return 0;
}
```

Độ phức tạp: O(R · C) thời gian và bộ nhớ. Dùng `queue` thay `vector` (BFS) cũng được.

6. Bẫy thường gặp

- **DFS đệ quy** trên lưới lớn: tràn ngăn xếp.
- **Đánh dấu muộn:** đếm trùng diện tích, ngăn xếp phình to.
- **Coi hai ô chung góc là kề nhau:** đề chỉ cho phép 4 hướng.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (40)	ST2 (60)	Tổng
DFS đệ quy	40	0	**40**
Loang bằng ngăn xếp tự quản lí	40	60	**100**

Chấm trên 7 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

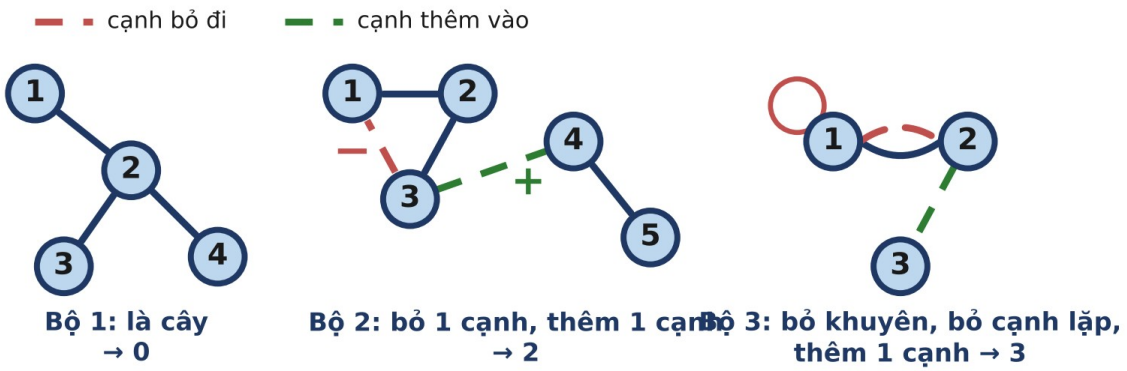
BÀI 3.2 CÂY HAY KHÔNG

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
CAYKHONG.INP	CAYKHONG.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Tập 1, Bài 3, Bài 9 · Đề bài: Tập 2, trang 28

1. Hiểu bài toán

Ta được xóa cạnh hoặc thêm cạnh, cần biến đồ thị (có thể có khuyên, cạnh lặp) thành một cây với ít thao tác nhất. Một cây có hai tính chất: **liên thông** và có **đúng n − 1 cạnh**. Đáp số phụ thuộc vào hai đại lượng: số cạnh m và số thành phần liên thông k.



Hình 3.2.1. *Ba bộ dữ liệu của ví dụ.*

2. Cách ngây thơ: Chỉ sửa số cạnh cho bằng n − 1

Một lời giải "trông như đúng": cây có n − 1 cạnh, vậy cần |m − (n − 1)| thao tác thêm hoặc bớt.

Cách sai: chỉ quan tâm số cạnh

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("CAYKHONG.INP", "r")) {
        freopen("CAYKHONG.INP", "r", stdin);
        freopen("CAYKHONG.OUT", "w", stdout);
    }
    int T;
    scanf("%d", &T);
    while (T--) {
        int n, m;
        scanf("%d %d", &n, &m);
        for (int i = 0; i < m; i++) { int u, v; scanf("%d %d", &u, &v); }
        // Chỉ "sửa" cho đủ n - 1 cạnh, không quan tâm tới liên thông
        printf("%d\n", abs(m - (n - 1)));
    }
    return 0;
}
```

Ở bộ 2 của ví dụ: $m = 4 = n - 1$ nên chương trình in 0, nhưng đồ thị gồm một tam giác và một cạnh rời, không phải cây.

3. Bế tắc

Khi đồ thị liên thông và không có khuyên, cạnh lặp (Subtask 1), mỗi cạnh thừa đều nằm trên chu trình và xóa nó không làm mất liên thông, nên công thức $|m - (n - 1)| = m - n + 1$ lại đúng: **30 điểm**.

> **ĐIỂM BẾ TẮC**
>
> Số cạnh đúng chưa đủ: các cạnh phải "rải" đều để nối mọi đỉnh. Cần đếm số thành phần liên thông, và chứng minh được một cận dưới chặt.

4. Phá bế tắc

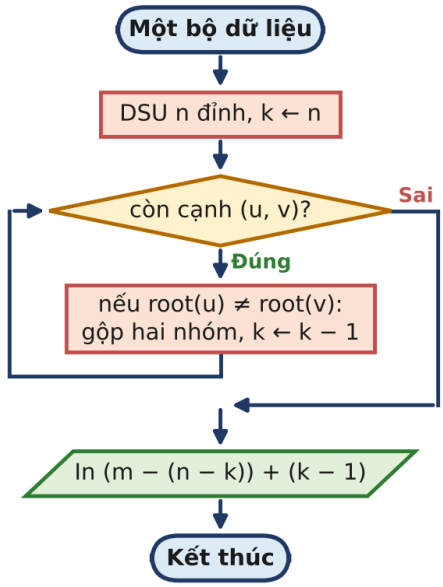
Gọi k là số thành phần liên thông, a là số lần thêm và r là số lần xóa.

1) **Cận dưới cho a:** xóa cạnh không bao giờ làm giảm k; mỗi lần thêm cạnh giảm k nhiều nhất 1. Cuối cùng $k = 1$, nên $\mathbf{a \geq k - 1}$.
2) **Cận dưới cho r:** số cạnh cuối cùng $m - r + a = n - 1$, nên $r = m - n + 1 + a \geq \mathbf{m - n + k}$.
3) **Đạt được:** trong mỗi thành phần giữ lại một cây khung và xóa các cạnh còn lại: đúng $m - (n - k)$ lần xóa (khuyên và cạnh lặp nằm trong số này). Sau đó nối k cây bằng $k - 1$ cạnh.

Vậy đáp số là $\mathbf{(m - (n - k)) + (k - 1) = m - n + 2k - 1}$.

Bộ	n	m	k	m − n + 2k − 1
1	4	3	1	3 − 4 + 2 − 1 = **0**
2	5	4	2	4 − 5 + 4 − 1 = **2**
3	3	3	2	3 − 3 + 4 − 1 = **3**

Đếm k bằng DSU (Tập 1, Bài 9): mỗi lần gộp thành công hai nhóm, k giảm 1. Khuyên (u, u) và cạnh lặp tự động bị bỏ qua vì hai đầu đã cùng nhóm.



Hình 3.2.2. *Sơ đồ khối xử lí một bộ dữ liệu.*

5. Cách tối ưu

CAYKHONG.CPP: DSU đếm số thành phần

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> par;
int root(int x) {                          // DSU, nén đường đi không đệ quy
    while (par[x] != x) { par[x] = par[par[x]]; x = par[x]; }
    return x;
}

int main() {
    if (fopen("CAYKHONG.INP", "r")) {
        freopen("CAYKHONG.INP", "r", stdin);
        freopen("CAYKHONG.OUT", "w", stdout);
    }
    int T;
    scanf("%d", &T);
    while (T--) {
        int n, m;
        scanf("%d %d", &n, &m);
        par.assign(n + 1, 0);
        iota(par.begin(), par.end(), 0);
        int k = n;                         // số thành phần liên thông
        for (int i = 0; i < m; i++) {
            int u, v;
            scanf("%d %d", &u, &v);
            u = root(u); v = root(v);
            if (u != v) { par[u] = v; k--; }
        }
        // xóa m - (n - k) cạnh thừa, thêm k - 1 cạnh nối các thành phần
        printf("%lld\n", (long long)(m - (n - k)) + (k - 1));
    }
    return 0;
}
```

Độ phức tạp: $O((n + m) \cdot \alpha(n))$ cho mỗi bộ dữ liệu; tổng trên cả tệp $O(\Sigma n + \Sigma m)$.

6. Bẫy thường gặp

- **Không khởi tạo lại DSU** giữa các bộ dữ liệu.
- **Khai báo mảng toàn cục cỡ cố định rồi "xóa" bằng `memset` cả mảng** cho mỗi bộ: với $T = 10^4$ bộ, tốn $10^4 \cdot 2 \cdot 10^5$ phép gán. Chỉ khởi tạo $n + 1$ phần tử đầu.
- **Quên khuyên:** đỉnh có khuyên vẫn có thể là đỉnh cô lập (thành phần riêng).

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
Chỉ sửa số cạnh: $\lvert m - (n - 1)\rvert$	30	0	0	**30**
Công thức $m - n + 2k - 1$ với DSU	30	30	40	**100**

Chấm trên 9 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

BÀI 3.3 MẠNG LƯỚI DỰ PHÒNG

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
DUPHONG.INP	DUPHONG.OUT	1 giây	256 MB	★★★ NÂNG CAO

Liên hệ: Tập 1, Bài 3 · Đề bài: Tập 2, trang 30

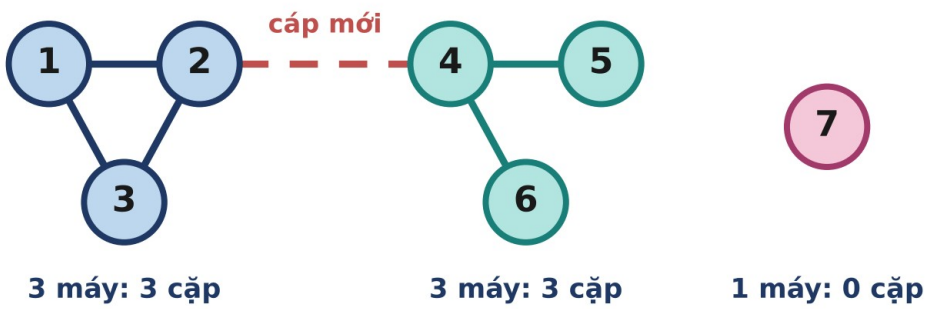
1. Hiểu bài toán

Ba câu hỏi đều quy về **kích thước các thành phần liên thông** $s_1, s_2, \ldots, s_k$:

1) Máy v liên lạc được với $s_{comp(v)}$ máy (tính cả nó).

2) Số cặp liên lạc được: $\Sigma\, s_i(s_i - 1)/2$.

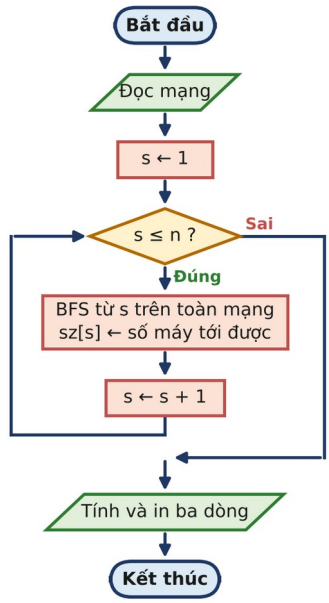
3) Một sợi cáp nối hai nhóm khác nhau kích thước a, b tạo thêm a · b cặp mới; nối trong cùng một nhóm không tạo cặp nào. Vậy chọn hai nhóm **lớn nhất**.



Hình 3.3.1. *Ví dụ: hai nhóm 3 máy và một máy lẻ.*

2. Cách ngây thơ: BFS riêng từ từng máy

Câu hỏi 1 có thể trả lời trực tiếp: với mỗi máy s, BFS để đếm số máy tới được.



Hình 3.3.2. *Sơ đồ khối của cách BFS từ từng máy.*

Cách ngây thơ: n lần BFS

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DUPHONG.INP", "r")) {
        freopen("DUPHONG.INP", "r", stdin);
        freopen("DUPHONG.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<vector<int>> g(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        g[u].push_back(v); g[v].push_back(u);
    }
    vector<long long> sz(n + 1);
    vector<int> seen(n + 1, 0);
    for (int s = 1; s <= n; s++) {          // BFS riêng từ từng máy: O(n·(n +
        m))
        vector<int> q = {s};
        seen[s] = s;
        for (size_t h = 0; h < q.size(); h++)
            for (int v : g[q[h]]) if (seen[v] != s) { seen[v] = s;
                q.push_back(v); }
        sz[s] = q.size();
    }
    long long pairs = 0;
    for (int v = 1; v <= n; v++) { printf("%lld%c", sz[v], v == n ? '\n' : '
        '); pairs += sz[v] - 1; }
    pairs /= 2;
    // hai nhóm lớn nhất: duyệt các kích thước nhóm (mỗi nhóm s máy xuất hiện
        s lần)
    map<long long, long long> times;
    for (int v = 1; v <= n; v++) times[sz[v]]++;
    vector<long long> comp;
    for (auto [s, t] : times) for (long long k = 0; k < t / s; k++)
        comp.push_back(s);
    sort(comp.rbegin(), comp.rend());
    long long best = pairs + (comp.size() >= 2 ? comp[0] * comp[1] : 0);
    printf("%lld\n%lld\n", pairs, best);
    return 0;
}
```

Độ phức tạp: $O(n \cdot (n + m))$. Với $n = 1000$ là $2 \cdot 10^6$; với $n = 10^5$ là $2 \cdot 10^{10}$.

3. Bế tắc

- **Subtask 1** ($n \le 1000$): chạy kịp: **30 điểm**.
- **Subtask 2, 3**: quá thời gian.

Cải tiến tự nhiên: các máy cùng nhóm có cùng đáp số, nên chỉ cần duyệt **mỗi nhóm một lần** (DFS từ đỉnh chưa thăm, ghi số hiệu nhóm). Viết DFS bằng **đệ quy** thì đúng và nhanh ở Subtask 2, nhưng Subtask 3 có mạng hình đường thẳng $5 \cdot 10^5$ máy: đệ quy sâu $5 \cdot 10^5$ tầng, tràn ngăn xếp. Cách này được **60 điểm**.

ĐIỂM BẾ TẮC

Thuật toán đã đúng, độ phức tạp đã tốt; chỉ còn "chiếc áo" đệ quy là không vừa với dữ liệu.

4. Phá bế tắc

Duyệt từng nhóm bằng ngăn xếp tự quản lí (như bài 3.1), ghi comp[v] và sz[k]. Sau đó:

- Dòng 1: in sz[comp[v]] cho mọi v.
- Dòng 2: Σ sz[k](sz[k] − 1)/2, dùng `long long` (có thể tới $1{,}25 \cdot 10^{11}$).
- Dòng 3: sắp xếp sz giảm dần, cộng thêm sz[1] · sz[2] nếu có ít nhất hai nhóm.

VÌ SAO NỐI HAI NHÓM LỚN NHẤT LÀ TỐT NHẤT?

Số cặp mới là a · b với a, b là kích thước hai nhóm khác nhau được nối. Tích này lớn nhất khi a, b là hai số lớn nhất trong dãy sz (mọi số đều dương).

Ví dụ: sz = (3, 3, 1). Dòng 2: 3 + 3 + 0 = 6. Dòng 3: 6 + 3 · 3 = **15**.

5. Cách tối ưu

DUPHONG.CPP: duyệt từng thành phần một lần, không đệ quy

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DUPHONG.INP", "r")) {
        freopen("DUPHONG.INP", "r", stdin);
        freopen("DUPHONG.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<vector<int>> g(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        g[u].push_back(v); g[v].push_back(u);
    }
    vector<int> comp(n + 1, 0);            // comp[v]: số hiệu nhóm của máy v
    vector<long long> sz(1, 0);            // sz[k]: số máy của nhóm k
    for (int s = 1; s <= n; s++) {
        if (comp[s]) continue;
        int k = sz.size();
        sz.push_back(0);
        vector<int> st = {s};              // DFS bằng ngăn xếp tự quản lí
        comp[s] = k;
        while (!st.empty()) {
            int u = st.back(); st.pop_back();
            sz[k]++;
            for (int v : g[u]) if (!comp[v]) { comp[v] = k; st.push_back(v); }
        }
    }
    for (int v = 1; v <= n; v++) printf("%lld%c", sz[comp[v]], v == n ? '\n' : ' ');
    long long pairs = 0;                   // có thể tới 1,25e11
    for (size_t k = 1; k < sz.size(); k++) pairs += sz[k] * (sz[k] - 1) / 2;
    // cập nhật nối hai nhóm lớn nhất a, b tạo thêm a·b cặp
    vector<long long> s(sz.begin() + 1, sz.end());
    sort(s.rbegin(), s.rend());
    long long best = pairs + (s.size() >= 2 ? s[0] * s[1] : 0);
    printf("%lld\n%lld\n", pairs, best);
    return 0;
}
```

Độ phức tạp: O(n + m + k log k).

6. Bẫy thường gặp

- **Tràn số** ở dòng 2 và 3 khi dùng `int`.
- **Chỉ có một nhóm:** không có cặp nhóm nào để nối, dòng 3 bằng dòng 2.
- **DFS đệ quy** trên đồ thị đường thẳng.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
BFS riêng từ từng máy	30	0	0	**30**
Duyệt mỗi nhóm một lần, DFS đệ quy	30	30	0	**60**
Duyệt mỗi nhóm một lần, ngăn xếp	30	30	40	**100**

Chấm trên 9 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

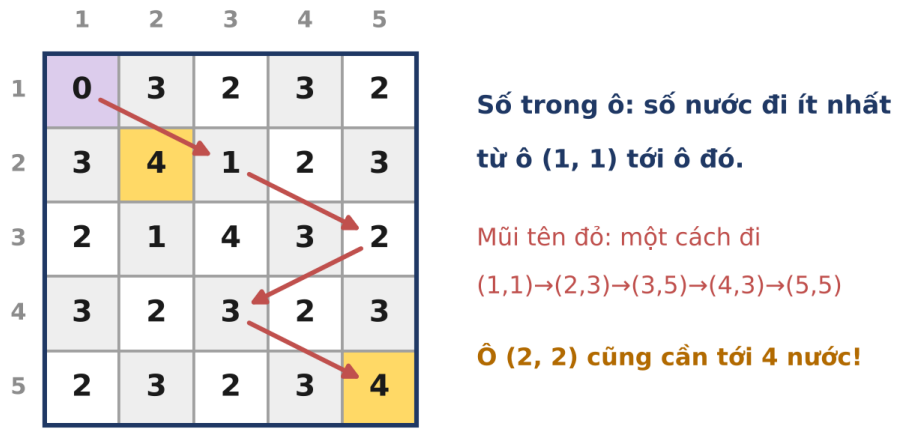
BÀI 4.1 MÃ ĐI TUẦN

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
`MADITUAN.INP`	`MADITUAN.OUT`	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Tập 1, Bài 4 · Đề bài: Tập 2, trang 32

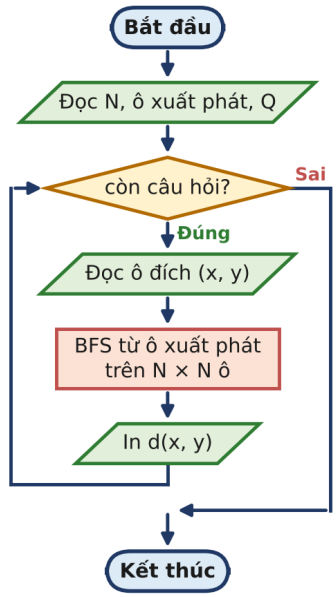
1. Hiểu bài toán

Mỗi ô là một đỉnh; hai ô có cạnh nếu quân mã nhảy được từ ô này sang ô kia. Số nước ít nhất là **khoảng cách** trên đồ thị không trọng số: BFS. Điểm đặc biệt: **mọi câu hỏi có chung ô xuất phát**.



Hình 4.1.1. *Kết quả BFS từ ô (1, 1) trên bàn cờ 5 × 5.*

2. Cách ngây thơ: BFS lại cho mỗi câu hỏi



Hình 4.1.2. *Sơ đồ khối của cách BFS cho từng câu hỏi.*

Cách ngây thơ: một lần BFS cho mỗi câu hỏi

```cpp
#include <bits/stdc++.h>
using namespace std;

int N;
const int dx[8] = {1, 2, 2, 1, -1, -2, -2, -1}, dy[8] = {2, 1, -1, -2, -2, -1,
    1, 2};

int bfs(int sx, int sy, int tx, int ty) {  // một lần BFS cho một câu hỏi
    vector<int> d(N * N, -1);
    queue<int> q;
    d[(sx - 1) * N + sy - 1] = 0; q.push((sx - 1) * N + sy - 1);
    while (!q.empty()) {
        int c = q.front(); q.pop();
        int x = c / N, y = c % N;
        if (x == tx - 1 && y == ty - 1) return d[c];
        for (int k = 0; k < 8; k++) {
            int u = x + dx[k], v = y + dy[k];
            if (u >= 0 && u < N && v >= 0 && v < N && d[u * N + v] < 0) {
                d[u * N + v] = d[c] + 1; q.push(u * N + v);
            }
        }
    }
    return -1;
}

int main() {
    if (fopen("MADITUAN.INP", "r")) {
        freopen("MADITUAN.INP", "r", stdin);
        freopen("MADITUAN.OUT", "w", stdout);
    }
    int sx, sy, Q;
    scanf("%d %d %d %d", &N, &sx, &sy, &Q);
    while (Q--) {
        int x, y;
        scanf("%d %d", &x, &y);
        printf("%d\n", bfs(sx, sy, x, y));
    }
    return 0;
}
```

Độ phức tạp: $O(Q \cdot N^2)$ trong trường hợp xấu nhất.

3. Bế tắc

- **Subtask 1** ($N \le 8$): mỗi lần BFS chỉ 64 ô: chạy tốt.
- **Subtask 2** ($Q = 1$): một lần BFS trên 10^6 ô: chạy tốt.
- **Subtask 3**: 10^5 lần BFS trên 10^6 ô là 10^{11} bước.

Được **60 điểm**.

ĐIỂM BẾ TẮC

Mỗi lần BFS đều bắt đầu từ cùng một ô, và đều tính lại khoảng cách tới gần như mọi ô. Kết quả của lần BFS đầu tiên đã trả lời được tất cả các câu hỏi.

4. Phá bế tắc

BFS **một lần** từ (x_0, y_0) trên toàn bàn cờ, lưu d[x][y] cho mọi ô (−1 nếu không tới được), rồi trả lời mỗi câu hỏi bằng một phép tra bảng.

Chạy tay với N = 5, xuất phát (1, 1), các ô được lấy ra khỏi hàng đợi theo từng lớp:

Lớp d	Các ô	Số ô
0	(1, 1)	1
1	(2, 3), (3, 2)	2
2	(1, 3), (1, 5), (2, 4), (3, 1), (3, 5), (4, 2), (4, 4), (5, 1), (5, 3)	9
3	(1, 2), (1, 4), (2, 1), (2, 5), (3, 4), (4, 1), (4, 3), (4, 5), (5, 2), (5, 4)	10
4	**(2, 2)**, (3, 3), **(5, 5)**	3

Ô (5, 5) và cả ô (2, 2) "ngay bên cạnh" đều cần 4 nước, khớp với ví dụ.

> **MỞ RỘNG: KHI MỖI CÂU HỎI CÓ Ô XUẤT PHÁT RIÊNG**
>
> Nếu ô xuất phát khác nhau, không thể BFS một lần. Với bàn cờ **vô hạn** người ta có công thức đóng cho số nước của quân mã; với bàn cờ hữu hạn, có thể BFS ngược từ ô đích nếu các câu hỏi có chung ô đích.

5. Cách tối ưu

MADITUAN.CPP: BFS một lần, trả lời O(1)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("MADITUAN.INP", "r")) {
        freopen("MADITUAN.INP", "r", stdin);
        freopen("MADITUAN.OUT", "w", stdout);
    }
    int N, sx, sy, Q;
    scanf("%d %d %d %d", &N, &sx, &sy, &Q);
    const int dx[8] = {1, 2, 2, 1, -1, -2, -2, -1}, dy[8] = {2, 1, -1, -2, -2,
        -1, 1, 2};
    // d[x][y]: số nước ít nhất từ ô xuất phát (đánh số từ 0), -1 nếu chưa tới
    vector<vector<int>> d(N, vector<int>(N, -1));
    queue<pair<int,int>> q;
    d[sx - 1][sy - 1] = 0;
    q.push({sx - 1, sy - 1});
    while (!q.empty()) {                    // BFS một lần duy nhất
        auto [x, y] = q.front(); q.pop();
        for (int k = 0; k < 8; k++) {
            int u = x + dx[k], v = y + dy[k];
            if (u >= 0 && u < N && v >= 0 && v < N && d[u][v] < 0) {
                d[u][v] = d[x][y] + 1;
                q.push({u, v});
            }
        }
    }
    while (Q--) {                           // mỗi câu hỏi O(1)
        int x, y;
        scanf("%d %d", &x, &y);
        printf("%d\n", d[x - 1][y - 1]);
    }
    return 0;
}
```

Độ phức tạp: $O(N^2 + Q)$.

6. Bẫy thường gặp

- **Bàn cờ nhỏ (N ≤ 3):** có ô không tới được, phải in −1 (ví dụ ô giữa bàn 3 × 3).
- **Khai báo `int d[1000][1000]` trong hàm `main`:** 4 MB trên ngăn xếp có thể gây lỗi; dùng `vector` hoặc biến toàn cục.
- **Nhầm hàng và cột** khi đánh số từ 1.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
BFS lại cho mỗi câu hỏi	30	30	0	**60**
BFS một lần, trả lời bằng tra bảng	30	30	40	**100**

Chấm trên 8 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

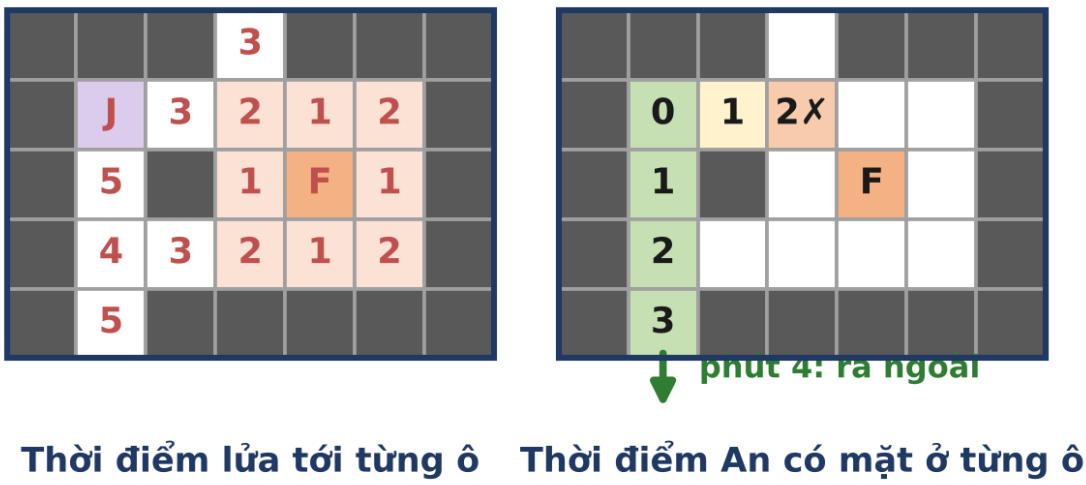
BÀI 4.2 CHẠY TRỐN ĐÁM CHÁY

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
CHAYTRON.INP	CHAYTRON.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Tập 1, Bài 4 · Đề bài: Tập 2, trang 34

1. Hiểu bài toán

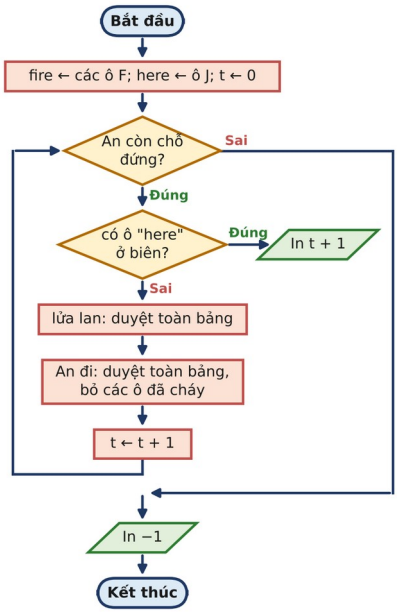
Hai "đối tượng" cùng di chuyển: lửa lan theo mọi hướng từ **nhiều** ô, còn An chọn đường đi. Điểm mấu chốt: **lửa không phụ thuộc vào An**. Vì vậy có thể tính trước, cho mỗi ô, phút lửa tới ô đó, rồi coi đó là "hạn chót" để An bước vào.



Hình 4.2.1. *Ví dụ 1: phút lửa tới (trái) và phút An có mặt (phải).*

2. Cách ngây thơ: Mô phỏng từng phút trên toàn bảng

Mô phỏng đúng như đề bài: giữ hai bảng "ô đang cháy" và "ô An có thể đang đứng". Mỗi phút, cho lửa lan, rồi cho An đi sang các ô kề chưa cháy.



Hình 4.2.2. *Sơ đồ khối của cách mô phỏng từng phút.*

Cách ngây thơ: mô phỏng từng phút

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("CHAYTRON.INP", "r")) {
        freopen("CHAYTRON.INP", "r", stdin);
        freopen("CHAYTRON.OUT", "w", stdout);
    }
    int R, C;
    cin >> R >> C;
    vector<string> a(R);
    for (auto &s : a) cin >> s;
    const int dx[4] = {-1, 1, 0, 0}, dy[4] = {0, 0, -1, 1};
    // fire[x][y], here[x][y]: ô đang cháy / An có thể đang đứng ở ô đó
    vector<vector<char>> fire(R, vector<char>(C, 0)), here(R,
vector<char>(C,
        0));
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++) { fire[i][j] = a[i][j] == 'F'; here[i]
[j]
            = a[i][j] == 'J'; }
    for (int t = 0; ; t++) {                // mô phỏng từng phút trên toàn
        bảng
        bool any = false;
        for (int i = 0; i < R; i++)
            for (int j = 0; j < C; j++) if (here[i][j]) {
                any = true;
                if (i == 0 || j == 0 || i == R - 1 || j == C - 1) {
                    printf("%d\n", t + 1); return 0; }
            }
        if (!any) break;
        auto nf = fire, nh = here;
        for (int i = 0; i < R; i++)
            for (int j = 0; j < C; j++) if (fire[i][j])
                for (int d = 0; d < 4; d++) {
                    int u = i + dx[d], v = j + dy[d];
                    if (u >= 0 && u < R && v >= 0 && v < C && a[u][v] !=
'#')
                        nf[u][v] = 1;
                }
        for (int i = 0; i < R; i++)
            for (int j = 0; j < C; j++) {
                nh[i][j] = 0;
                if (a[i][j] == '#' || nf[i][j]) continue;
                for (int d = 0; d < 4; d++) {
                    int u = i + dx[d], v = j + dy[d];
                    if (u >= 0 && u < R && v >= 0 && v < C && here[u][v])
                        nh[i][j] = 1;
                }
            }
        fire = nf; here = nh;
    }
    printf("-1\n");
    return 0;
}
```

Độ phức tạp: mỗi phút $O(R \cdot C)$; số phút có thể tới cỡ $R \cdot C$. Tổng $O((R \cdot C)^2)$.

3. Bế tắc

- **Subtask 2** (lưới ≤ 100 × 100): chạy kịp.

- **Subtask 1, 3** (lưới 1000 × 1000, đường chạy dài): quá thời gian.

Mô phỏng được **30 điểm**. Một bạn khác nhận ra Subtask 1 không có lửa nên viết BFS cho An mà **bỏ qua lửa**: đúng Subtask 1 nhưng sai khi có lửa: **30 điểm**.

> **ĐIỂM BẾ TẮC**
>
> Mỗi phút, mô phỏng lại tính lại trạng thái lửa của cả bảng, trong khi đường đi của lửa là **cố định**, có thể tính một lần là xong.

4. Phá bế tắc

Lần 1: BFS nhiều nguồn cho lửa. Đẩy mọi ô `F` vào hàng đợi với f = 0. BFS cho f[x][y] = phút lửa tới ô (x, y) (vô cùng nếu không bao giờ tới).

Lần 2: BFS cho An. Từ ô `J`, An được bước vào ô (u, v) ở phút t + 1 khi và chỉ khi ô đó không phải tường và **t + 1 < f[u][v]** (lửa tới **sau** An). Ô biên đầu tiên được lấy ra khỏi hàng đợi, ở phút t, cho đáp số t + 1.

> **VÌ SAO BFS VẪN ĐÚNG KHI CÓ "HẠN CHÓT"?**
>
> Nếu An tới được ô X ở phút t mà lửa chưa tới, thì tới X sớm hơn càng an toàn hơn. Vì vậy đến mỗi ô **sớm nhất có thể** luôn là tốt nhất, và BFS (cho thời điểm sớm nhất) không bỏ sót cách đi nào.

Chi tiết quan trọng nhất là dấu <. Viết ≤ (cho An vào ô mà lửa tới cùng phút) là sai: đề bài nói An không được bước vào ô mà lửa tới vào cùng phút đó. Cách viết sai này trông gần như giống hệt lời giải mẫu, vượt qua cả hai ví dụ, nhưng trượt những test có "cuộc đua sát nút": chỉ được **30 điểm**.

Ô (ví dụ 1)	Phút lửa tới	Phút An tới (đi lên)	Được vào?
(2, 3)	2	1	1 < 2: được
(2, 4)	2	2	2 < 2: **không**
(3, 2)	4	1	được
(5, 2)	5	3	được, ra ngoài ở phút **4**

5. Cách tối ưu

CHAYTRON.CPP: hai lần BFS, O(R · C)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("CHAYTRON.INP", "r")) {
        freopen("CHAYTRON.INP", "r", stdin);
        freopen("CHAYTRON.OUT", "w", stdout);
    }
    int R, C;
    cin >> R >> C;
    vector<string> a(R);
    for (auto &s : a) cin >> s;
    const int INF = INT_MAX, dx[4] = {-1, 1, 0, 0}, dy[4] = {0, 0, -1, 1};
    // Lần 1: BFS nhiều nguồn từ mọi ô 'F', f[x][y] = phút lửa tới ô (x, y)
    vector<vector<int>> f(R, vector<int>(C, INF));
    queue<pair<int,int>> q;
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++) if (a[i][j] == 'F') { f[i][j] = 0;
            q.push({i, j}); }
    while (!q.empty()) {
        auto [x, y] = q.front(); q.pop();
        for (int d = 0; d < 4; d++) {
            int u = x + dx[d], v = y + dy[d];
            if (u >= 0 && u < R && v >= 0 && v < C && a[u][v] != '#' &&
                f[u][v] == INF) {
                f[u][v] = f[x][y] + 1;
                q.push({u, v});
            }
        }
    }
    // Lần 2: BFS cho An, chỉ bước vào ô mà lửa tới SAU An
    vector<vector<int>> p(R, vector<int>(C, -1));
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++) if (a[i][j] == 'J') { p[i][j] = 0;
            q.push({i, j}); }
    while (!q.empty()) {
        auto [x, y] = q.front(); q.pop();
        if (x == 0 || y == 0 || x == R - 1 || y == C - 1) {   // ô biên:
phút
            sau ra ngoài
            printf("%d\n", p[x][y] + 1);
            return 0;
        }
        for (int d = 0; d < 4; d++) {
            int u = x + dx[d], v = y + dy[d];  // (x, y) không ở biên nên
(u,
                v) trong bảng
            if (a[u][v] != '#' && p[u][v] < 0 && p[x][y] + 1 < f[u][v]) {
                p[u][v] = p[x][y] + 1;
                q.push({u, v});
            }
        }
    }
    printf("-1\n");
    return 0;
}
```

Độ phức tạp: mỗi ô vào mỗi hàng đợi nhiều nhất một lần: O(R · C).

6. Bẫy thường gặp

- **Dấu ≤ thay vì <** (lỗi kinh điển của bài này).
- **An đứng sẵn ở ô biên:** đáp số là 1.
- **Ô không có lửa tới:** f = vô cùng, nhớ dùng giá trị đủ lớn.
- **BFS từng nguồn lửa riêng lẻ** rồi lấy min: đúng nhưng chậm gấp số ô lửa lần.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
BFS cho An, bỏ qua lửa	30	0	0	**30**
Mô phỏng từng phút	0	30	0	**30**
Hai lần BFS, điều kiện ≤ (sai)	30	0	0	**30**
Hai lần BFS, điều kiện <	30	30	40	**100**

Chấm trên 14 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

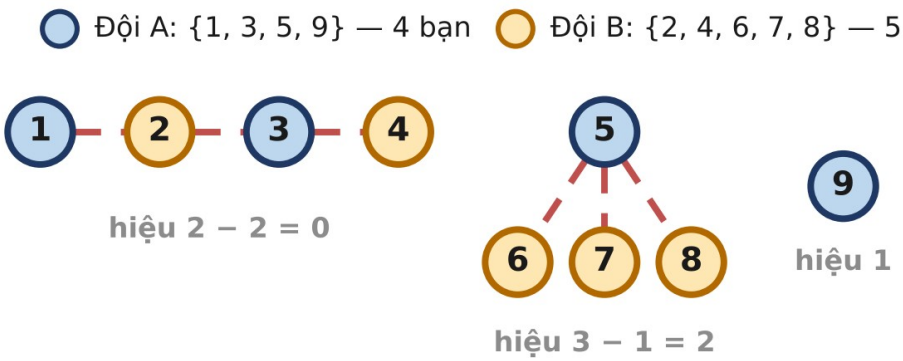
BÀI 4.3 CHIA ĐỘI

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
CHIADOI.INP	CHIADOI.OUT	1 giây	256 MB	★★★ NÂNG CAO

Liên hệ: Tập 1, Bài 4 · Đề bài: Tập 2, trang 36

1. Hiểu bài toán

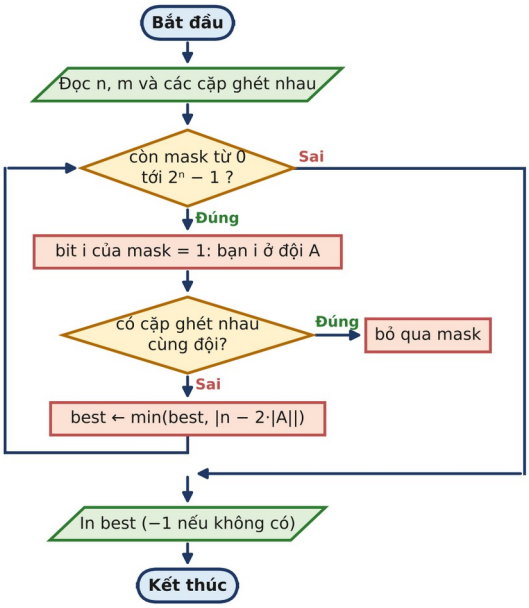
Hai bạn ghét nhau phải khác đội: đây là bài **tô hai màu** đồ thị (Tập 1, Bài 4). Trong mỗi thành phần liên thông, cách tô hai màu là **duy nhất** (sai khác một lần đổi màu toàn bộ). Gọi a_i, b_i là số đỉnh hai màu của thành phần i; ta chỉ được chọn, cho mỗi thành phần, màu nào vào đội A.



Hình 4.3.1. *Ví dụ 1: ba thành phần có hiệu 0, 2 và 1.*

Độ chênh lệch cuối cùng là $|\pm c_1 \pm c_2 \pm \ldots \pm c_k|$ với $c_i = |a_i - b_i|$. Bài toán trở thành: **chia các số c_i thành hai nhóm có tổng gần nhau nhất**.

2. Cách ngây thơ: Thử mọi cách chia



Hình 4.3.2. *Sơ đồ khối của cách thử 2^n cách chia.*

Cách ngây thơ: thử 2^n tập con

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("CHIADOI.INP", "r")) {
        freopen("CHIADOI.INP", "r", stdin);
        freopen("CHIADOI.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<int> u(m), v(m);
    for (int i = 0; i < m; i++) { scanf("%d %d", &u[i], &v[i]); u[i]--;
        v[i]--; }
    if (n > 25) return 0;                  // 2^n quá lớn
    int best = -1;
    for (int mask = 0; mask < (1 << n); mask++) {   // bit i = 1: bạn i ở
đội
        A
        bool ok = true;
        for (int i = 0; i < m && ok; i++)
            if (((mask >> u[i]) & 1) == ((mask >> v[i]) & 1)) ok = false;
        if (!ok) continue;
        int d = abs(n - 2 * __builtin_popcount(mask));
        if (best < 0 || d < best) best = d;
    }
    printf("%d\n", best);
    return 0;
}
```

Độ phức tạp: $O(2^n \cdot m)$: chỉ dùng được khi $n \le 20$: **20 điểm**.

3. Bế tắc

Sau khi tô màu (O(n + m)), còn lại bài toán chia các c_i. Hai hướng hay gặp:

- **Tham lam:** xét c_i từ lớn tới nhỏ, đặt mỗi số vào phía đang có tổng nhỏ hơn. Đúng khi đồ thị liên thông (chỉ có một số), nhưng sai với dãy 3, 3, 2, 2, 2: tham lam cho 3 + 2 + 2 = 7 và 3 + 2 = 5, chênh 2, trong khi 3 + 3 = 2 + 2 + 2 chênh 0. Được **50 điểm**.
- **Quy hoạch động cái túi:** can[x] = có chọn được nhóm có tổng x không; với mỗi c_i cập nhật can từ cao xuống thấp. Đúng, nhưng tốn O(k · S) với $S = \Sigma c_i \le n$. Khi có 10^5 đỉnh cô lập ($k = S = 10^5$), tốn 10^{10} phép tính: **70 điểm**.

> **ĐIỂM BẾ TẮC**
>
> Quy hoạch động tổng tập con đã đúng; vấn đề là hằng số và số lượng phần tử. Cần làm mỗi bước cập nhật nhanh hơn, và giảm số bước.

4. Phá bế tắc

Hai cải tiến độc lập:

1) **`bitset`:** lưu can dưới dạng `bitset<100001>`; bước "thêm số d" chỉ là `can |= can << d`, xử lí 64 giá trị một lúc: mỗi bước tốn S / 64 phép tính.
2) **Gộp các số bằng nhau:** vì $\Sigma c_i \le n$, số giá trị **khác nhau** của c_i không quá khoảng $\sqrt{(2n)} \approx 450$. Một giá trị d xuất hiện t lần được tách thành các "gói" d, 2d, 4d, ..., phần dư; mọi số lượng từ 0 tới t đều ghép được từ các gói. Số gói chỉ khoảng log t cho mỗi giá trị.

Tổng số bước cập nhật $O(\sqrt{n} \cdot \log n)$, mỗi bước O(n / 64): rất nhanh. Cuối cùng tìm x có can[x] = 1 làm $|S - 2x|$ nhỏ nhất.

Ví dụ 1	Giá trị	Ghi chú
Hiệu c_i	0, 2, 1	S = 3
Tổng tập con đạt được	0, 1, 2, 3	chọn {1} hoặc {2}
Đáp số	min $\lvert 3 - 2x \rvert$ = **1**	x = 1 hoặc x = 2

5. Cách tối ưu

CHIADOI.CPP: tô màu + tổng tập con bằng bitset

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MX = 100001;

int main() {
    if (fopen("CHIADOI.INP", "r")) {
        freopen("CHIADOI.INP", "r", stdin);
        freopen("CHIADOI.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<vector<int>> g(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        g[u].push_back(v); g[v].push_back(u);
    }
    // Bước 1: tô hai màu từng thành phần bằng BFS, ghi lại hiệu |a - b|
    vector<int> col(n + 1, -1), diffs;
    for (int s = 1; s <= n; s++) {
        if (col[s] >= 0) continue;
        int c[2] = {0, 0};
        col[s] = 0;
        vector<int> q = {s};
        for (size_t h = 0; h < q.size(); h++) {
            int u = q[h];
            c[col[u]]++;
            for (int v : g[u]) {
                if (col[v] < 0) { col[v] = col[u] ^ 1; q.push_back(v); }
                else if (col[v] == col[u]) { printf("-1\n"); return 0; }
//
                    chu trình lẻ
            }
        }
        diffs.push_back(abs(c[0] - c[1]));
    }
    // Bước 2: chia các hiệu thành hai nhóm có tổng gần nhau nhất (tổng tập con)
    int S = 0;
    for (int d : diffs) S += d;
    map<int,int> cnt;                       // gộp các hiệu bằng nhau
    for (int d : diffs) if (d) cnt[d]++;
    bitset<MX> can;
    can[0] = 1;
    for (auto [d, c] : cnt)                 // tách c bản sao thành 1, 2, 4, ... bản
        for (int k = 1; c > 0; k *= 2) {
            int t = min(k, c);
            can |= can << (d * t);
            c -= t;
        }
    int best = S;
    for (int x = 0; x <= S; x++) if (can[x]) best = min(best, abs(S - 2 * x));
    printf("%d\n", best);
    return 0;
}
```

Độ phức tạp: $O(n + m + (n / 64) \cdot \sqrt{n} \cdot \log n)$.

6. Bẫy thường gặp

- **Quên các đỉnh cô lập:** mỗi đỉnh cô lập là một thành phần có hiệu 1.
- **Phát hiện chu trình lẻ** phải kiểm tra cả cạnh nối tới đỉnh đã tô (không chỉ cạnh cây BFS).
- **Tham lam** cho bài chia tập: chỉ đúng trong một số trường hợp đặc biệt.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (20)	ST2 (30)	ST3 (20)	ST4 (30)	Tổng
Thử 2^n cách chia	20	0	0	0	**20**
Tô màu + tham lam	20	30	0	0	**50**
Tô màu + QHĐ cái túi $O(k \cdot S)$	20	30	20	0	**70**
Tô màu + bitset, gộp số bằng nhau	20	30	20	30	**100**

Chấm trên 12 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

CHƯƠNG 2. ĐƯỜNG ĐI NGẮN NHẤT

Ở chương này, cách ngây thơ thường không sai mà chỉ chậm: chạy lại cùng một thuật toán quá nhiều lần, hoặc dùng thuật toán "quá mạnh" (Bellman-Ford) khi chỉ cần Dijkstra. Mặt khác, rất nhiều lời giải "trông như đúng" mất điểm vì tràn số, vì sai một dấu so sánh, hoặc vì bỏ quên một chi tiết của mô hình (giới hạn số cạnh, thời điểm xuất bến, đèn đỏ...).

Bài	Tên bài	Mức	Cách ngây thơ (điểm)	Chìa khóa của lời giải
5.1	**ĐƯỜNG TỚI TRƯỜNG**	★	Dijkstra $O(n^2)$ (40)	Dijkstra với hàng đợi ưu tiên và long long.
5.2	**NÚT GIAO QUAN TRỌNG**	★★	Xóa thử từng đỉnh (30)	Dijkstra hai đầu + quét các "mốc" d_1.
5.3	**XE BUÝT ĐÊM**	★★★	Nới lỏng lặp (kiểu Bellman-Ford) (30)	Dijkstra theo thời điểm tới; tính chuyến kế tiếp bằng công thức.
6.1	**ĐỔI TIỀN**	★	Thử các vòng 2 và 3 quầy (30)	Trọng số $-\log r$, chu trình âm với mọi đỉnh là nguồn.
6.2	**CHUYẾN BAY GIÁ RẺ**	★★	Dijkstra, bỏ qua giới hạn k (30)	Bellman-Ford đúng k lượt, mỗi lượt từ bản sao.
6.3	**LỊCH THI ĐẤU**	★★★	Đường dài nhất trên DAG (bỏ qua loại 2) (40)	Hệ ràng buộc hiệu: đường đi dài nhất, phát hiện chu trình dương.
7.1	**THỦ ĐÔ MỚI**	★	Bellman-Ford từ từng thành phố (30)	Floyd với k ở vòng ngoài cùng.
7.2	**XẾP HẠNG KÌ THỦ**	★★	Floyd trên ma trận bool (80)	Bao đóng bắc cầu bằng bitset.
7.3	**XE TẢI QUÁ KHỔ**	★★★	Tìm nhị phân + DFS mỗi đơn hàng (30)	Floyd với (max, min).
8.1	**ĐÈN GIAO THÔNG**	★	BFS trên nút, đánh dấu theo nút (60)	BFS trên trạng thái (nút, t mod P).
8.2	**PHÁ TƯỜNG**	★★	BFS thường, đánh dấu theo ô (30)	BFS trên trạng thái (ô, số tường đã phá).
8.3	**ROBOT XOAY**	★★★	BFS thường (mọi thao tác như nhau) (30)	BFS 0-1 trên trạng thái (ô, hướng).

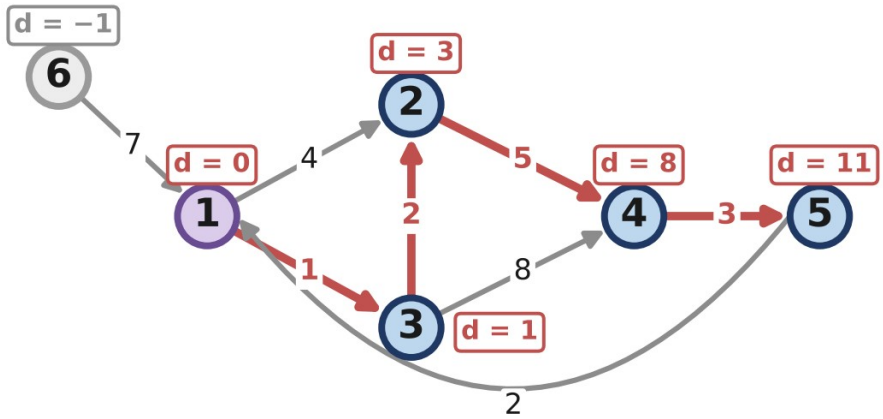
BÀI 5.1 ĐƯỜNG TỚI TRƯỜNG

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
DUONGTRUONG.INP	DUONGTRUONG.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Tập 1, Bài 5 · Đề bài: Tập 2, trang 39

1. Hiểu bài toán

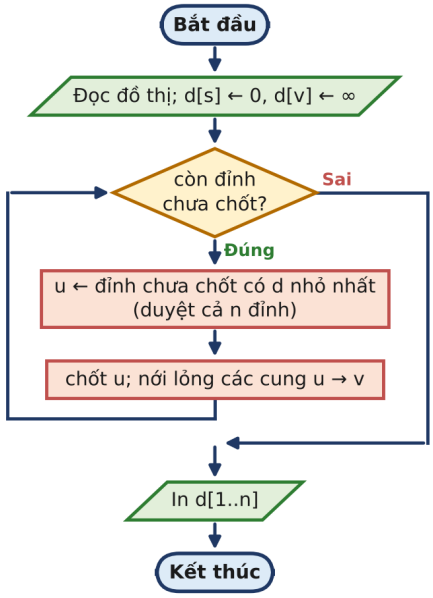
Đồ thị có hướng, trọng số không âm, một nguồn: đây là bài toán gốc của thuật toán Dijkstra (Tập 1, Bài 5). Hai chi tiết: in −1 cho đỉnh không tới được, và khoảng cách có thể tới $(n-1)\cdot 10^9 \approx 10^{14}$.



Hình 5.1.1. *Cây đường đi ngắn nhất từ nút 1 của ví dụ.*

2. Cách ngây thơ: Dijkstra $O(n^2)$

Cài đặt Dijkstra nguyên bản: mỗi lượt duyệt cả n đỉnh để tìm đỉnh chưa chốt có nhãn nhỏ nhất.



Hình 5.1.2. *Sơ đồ khối của Dijkstra $O(n^2)$.*

Cách ngây thơ: Dijkstra tìm đỉnh nhỏ nhất bằng vòng lặp

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DUONGTRUONG.INP", "r")) {
        freopen("DUONGTRUONG.INP", "r", stdin);
        freopen("DUONGTRUONG.OUT", "w", stdout);
    }
    int n, m, s;
    scanf("%d %d %d", &n, &m, &s);
    vector<vector<pair<int,int>>> g(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        g[u].push_back({v, w});
    }
    const long long INF = LLONG_MAX;
    vector<long long> d(n + 1, INF);
    vector<bool> done(n + 1, false);
    d[s] = 0;
    for (int it = 0; it < n; it++) {
        int u = -1;                          // tìm đỉnh chưa chốt có nhãn
nhỏ
            nhất: O(n)
        for (int v = 1; v <= n; v++)
            if (!done[v] && d[v] != INF && (u < 0 || d[v] < d[u])) u = v;
        if (u < 0) break;
        done[u] = true;
        for (auto [v, w] : g[u]) d[v] = min(d[v], d[u] + w);
    }
    for (int v = 1; v <= n; v++) printf("%lld%c", d[v] == INF ? -1 : d[v], v
        == n ? '\n' : ' ');
    return 0;
}
```

Độ phức tạp: n lượt, mỗi lượt O(n): $O(n^2 + m)$. Với n = 1000 là 10^6; với $n = 10^5$ là 10^{10}.

3. Bế tắc

- **Subtask 1** ($n \le 1000$): **40 điểm**.
- **Subtask 2** ($n \le 10^5$): quá thời gian.

Một lỗi khác "đắt giá" hơn: cài Dijkstra với hàng đợi ưu tiên nhưng lưu khoảng cách bằng `int`. Ở các test có trọng số tới 10^9, chỉ cần đường đi qua 3 cạnh là tràn số; chương trình được **0 điểm**, kém cả cách ngây thơ.

> **ĐIỂM BẾ TẮC**
>
> Việc tìm "đỉnh chưa chốt có nhãn nhỏ nhất" tốn O(n) mỗi lượt. Cần một cấu trúc dữ liệu lấy ra phần tử nhỏ nhất nhanh hơn.

4. Phá bế tắc

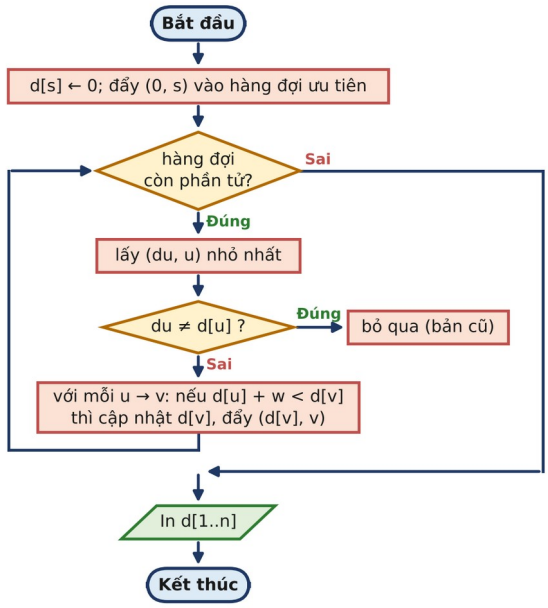
Dùng `priority_queue` chứa các cặp (nhãn, đỉnh). Mỗi lần giảm d[v] thì đẩy thêm một cặp mới (không xóa cặp cũ). Khi lấy ra cặp (du, u) mà du ≠ d[u] thì đó là **bản cũ**, bỏ qua.

Chạy tay ví dụ (nguồn s = 1):

Lấy ra	Nới lỏng	d[1..6] sau bước
(0, 1)	1→2: 4; 1→3: 1	0 4 1 ∞ ∞ ∞
(1, 3)	3→2: 1 + 2 = 3; 3→4: 9	0 3 1 9 ∞ ∞
(3, 2)	2→4: 3 + 5 = 8	0 3 1 8 ∞ ∞
(4, 2)	bản cũ, bỏ qua	0 3 1 8 ∞ ∞
(8, 4)	4→5: 11	0 3 1 8 11 ∞
(9, 4)	bản cũ, bỏ qua	
(11, 5)	5→1: không cải thiện	0 3 1 8 11 ∞

Nút 6 vẫn là ∞: in −1.

5. Cách tối ưu



Hình 5.1.3. *Sơ đồ khối của Dijkstra với hàng đợi ưu tiên.*

DUONGTRUONG.CPP: Dijkstra O(m log n)

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

int main() {
    if (fopen("DUONGTRUONG.INP", "r")) {
        freopen("DUONGTRUONG.INP", "r", stdin);
        freopen("DUONGTRUONG.OUT", "w", stdout);
    }
    int n, m, s;
    scanf("%d %d %d", &n, &m, &s);
    vector<vector<pair<int,int>>> g(n + 1);     // g[u]: các cặp (v, w)
    for (int i = 0; i < m; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        g[u].push_back({v, w});
    }
    const ll INF = LLONG_MAX;
    vector<ll> d(n + 1, INF);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
    d[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du != d[u]) continue;               // bản cũ: bỏ qua
        for (auto [v, w] : g[u])
            if (d[u] + w < d[v]) {
                d[v] = d[u] + w;
                pq.push({d[v], v});
            }
    }
    for (int v = 1; v <= n; v++)
        printf("%lld%c", d[v] == INF ? -1 : d[v], v == n ? '\n' : ' ');
    return 0;
}
```

Độ phức tạp: mỗi cung đẩy nhiều nhất một cặp: O(m log m) = O(m log n).

6. Bẫy thường gặp

- **`int` cho khoảng cách** (mất trọn điểm).
- **In "vô cùng"** thay vì −1 cho đỉnh không tới được.
- **Quên bỏ qua bản cũ:** vẫn đúng nhưng có thể chậm đi nhiều lần.
- **Trọng số 0:** Dijkstra vẫn đúng; không cần xử lí riêng.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (40)	ST2 (60)	Tổng
Dijkstra $O(n^2)$	40	0	**40**
Dijkstra + hàng đợi, khoảng cách int	0	0	**0**
Dijkstra + hàng đợi, long long	40	60	**100**

Chấm trên 8 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

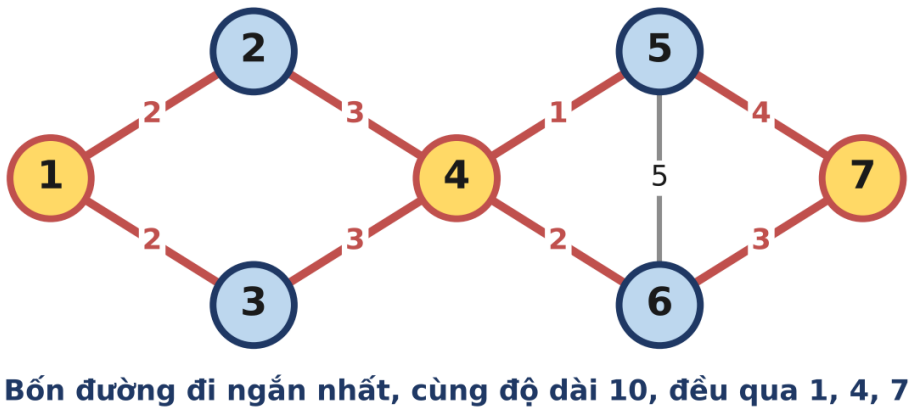
BÀI 5.2 NÚT GIAO QUAN TRỌNG

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
NUTGIAO.INP	NUTGIAO.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Tập 1, Bài 5 · Đề bài: Tập 2, trang 42

1. Hiểu bài toán

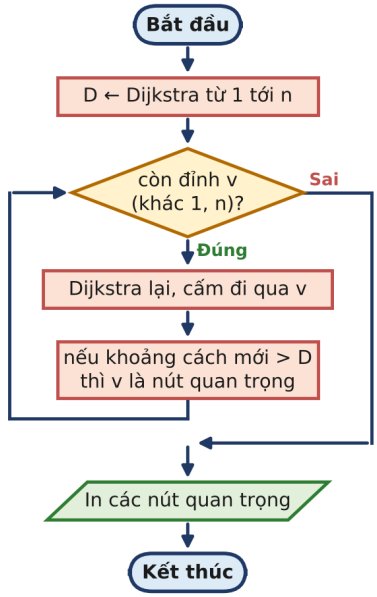
Đỉnh v quan trọng khi **mọi** đường đi ngắn nhất 1 → n đều qua v. Từ Tập 1, ta đã biết kiểm tra v nằm trên **một** đường ngắn nhất: $d_1[v] + d_n[v] = D$, với d_1, d_n là khoảng cách từ 1 và từ n, $D = d_1[n]$. Điều kiện "mọi" mạnh hơn nhiều.



Hình 5.2.1. *Mọi đỉnh của ví dụ đều nằm trên một đường ngắn nhất, nhưng chỉ 1, 4, 7 nằm trên tất cả.*

2. Cách ngây thơ: Xóa thử từng đỉnh

v quan trọng khi và chỉ khi xóa v đi thì khoảng cách từ 1 tới n tăng lên (hoặc không còn đường). Vậy thử xóa từng đỉnh và chạy lại Dijkstra.



Hình 5.2.2. *Sơ đồ khối của cách xóa thử từng đỉnh.*

Cách ngây thơ: n lần Dijkstra

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
int n, m;
vector<vector<pair<int,int>>> g;

ll dijkstra(int banned) {                       // khoảng cách 1 -> n khi cấm
    đỉnh banned
    vector<ll> d(n + 1, LLONG_MAX);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
    d[1] = 0; pq.push({0, 1});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du != d[u]) continue;
        for (auto [v, w] : g[u])
            if (v != banned && d[u] + w < d[v]) { d[v] = d[u] + w;
                pq.push({d[v], v}); }
    }
    return d[n];
}

int main() {
    if (fopen("NUTGIAO.INP", "r")) {
        freopen("NUTGIAO.INP", "r", stdin);
        freopen("NUTGIAO.OUT", "w", stdout);
    }
    scanf("%d %d", &n, &m);
    g.assign(n + 1, {});
    for (int i = 0; i < m; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        g[u].push_back({v, w}); g[v].push_back({u, w});
    }
    ll D = dijkstra(0);
    vector<int> ans = {1};
    for (int v = 2; v < n; v++)                 // xóa thử từng đỉnh: n lần Dijkstra
        if (dijkstra(v) > D) ans.push_back(v);
    ans.push_back(n);
    printf("%d\n", (int)ans.size());
    for (size_t i = 0; i < ans.size(); i++) printf("%d%c", ans[i], i + 1 ==
        ans.size() ? '\n' : ' ');
    return 0;
}
```

Độ phức tạp: $O(n \cdot m \log n)$: với $n = 1000$, $m = 2000$ khoảng $2 \cdot 10^7$; với $n = 10^5$ thì quá chậm.

3. Bế tắc

- **Subtask 1**: **30 điểm**.
- **Subtask 2**: quá thời gian.

Một lời giải sai hay gặp: in mọi đỉnh có $d_1[v] + d_n[v] = D$ (nằm trên **một** đường ngắn nhất). Ở ví dụ, chương trình in cả 7 đỉnh: **0 điểm**.

ĐIỂM BẾ TẮC

Ta biết **tập** các đỉnh và cạnh nằm trên đường ngắn nhất (một DAG), nhưng chưa biết đỉnh nào là "cổ chai" mà mọi đường đều phải qua. Số đường đi có thể lên tới $2^{n/2}$: không thể liệt kê.

4. Phá bế tắc

Mọi trọng số đều dương nên dọc theo **bất kì** đường ngắn nhất nào, giá trị d_1 tăng ngặt từ 0 tới D. Hãy coi mỗi giá trị $d_1[v]$ của các đỉnh "trên đường" là một **mốc** trên trục số. Một đường ngắn nhất đi qua mốc x theo một trong hai cách: dừng ở một đỉnh có $d_1 = x$, hoặc đi qua một cạnh (a, b) với $d_1[a] < x < d_1[b]$ ("nhảy qua" mốc x).

ĐIỀU KIỆN CẦN VÀ ĐỦ

Đỉnh v (trên đường) là quan trọng khi và chỉ khi: **không** có đỉnh nào khác trên đường có cùng mốc $d_1[v]$, và **không** có cạnh-đường-ngắn-nhất nào nhảy qua mốc $d_1[v]$.

Nếu có đỉnh $u \neq v$ cùng mốc thì đường ngắn nhất qua u không qua v (vì các mốc trên một đường tăng ngặt). Nếu có cạnh (a, b) nhảy qua mốc thì đường $1 \to a \to b \to n$ là đường ngắn nhất không qua v. Ngược lại, mọi đường ngắn nhất buộc phải "đáp" xuống mốc $d_1[v]$, và chỉ có v ở đó.

Cài đặt: nén các mốc, với mỗi cạnh đường ngắn nhất cộng 1 vào khoảng mốc (l, r) mở bằng **mảng hiệu**, rồi quét.

Mốc d_1	Các đỉnh	Bị nhảy qua bởi	Quan trọng?
0	1	—	**1**
2	2, 3	—	không (hai đỉnh)
5	4	—	**4**
6	5	cạnh 4 – 6 ($5 \to 7$)	không
7	6	cạnh 5 – 7 ($6 \to 10$)	không
10	7	—	**7**

CÁCH KHÁC: ĐẾM SỐ ĐƯỜNG ĐI

Đếm số đường ngắn nhất $cnt_1[v]$ (từ 1) và $cnt_n[v]$ (từ n) trong lúc chạy Dijkstra; v quan trọng khi $cnt_1[v] \cdot cnt_n[v] = cnt_1[n]$. Số đường quá lớn nên phải lấy theo mô-đun một số nguyên tố, và khi đó có (rất nhỏ) khả năng hai số khác nhau trùng mô-đun. Trên bộ test của sách, cách này cũng được **100 điểm**.

5. Cách tối ưu

NUTGIAO.CPP: hai lần Dijkstra + quét mốc bằng mảng hiệu

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
int n, m;
vector<vector<pair<int,int>>> g;

vector<ll> dijkstra(int s) {
    vector<ll> d(n + 1, LLONG_MAX);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
    d[s] = 0; pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du != d[u]) continue;
        for (auto [v, w] : g[u])
            if (d[u] + w < d[v]) { d[v] = d[u] + w; pq.push({d[v], v}); }
    }
    return d;
}

int main() {
    if (fopen("NUTGIAO.INP", "r")) {
        freopen("NUTGIAO.INP", "r", stdin);
        freopen("NUTGIAO.OUT", "w", stdout);
    }
    scanf("%d %d", &n, &m);
    g.assign(n + 1, {});
    vector<array<int,3>> e(m);
    for (auto &x : e) {
        scanf("%d %d %d", &x[0], &x[1], &x[2]);
        g[x[0]].push_back({x[1], x[2]}); g[x[1]].push_back({x[0], x[2]});
    }
    auto d1 = dijkstra(1), dn = dijkstra(n);
    ll D = d1[n];
    auto on = [&](int v) { return d1[v] != LLONG_MAX && dn[v] != LLONG_MAX
&&
        d1[v] + dn[v] == D; };
    // Các "mốc" d1 của những đỉnh nằm trên ít nhất một đường ngắn nhất
    vector<ll> val;
    for (int v = 1; v <= n; v++) if (on(v)) val.push_back(d1[v]);
    sort(val.begin(), val.end());
    map<ll,int> same;                          // số đỉnh có cùng mốc
    for (ll x : val) same[x]++;
    val.erase(unique(val.begin(), val.end()), val.end());
    int K = val.size();
    // Cạnh a -> b trên đường ngắn nhất "nhảy qua" mọi mốc nằm hẳn giữa
d1[a]
        và d1[b]
    vector<int> cover(K + 1, 0);               // mảng hiệu
    for (auto &x : e)
        for (int t = 0; t < 2; t++) {
            int a = t ? x[1] : x[0], b = t ? x[0] : x[1];
            if (!on(a) || !on(b) || d1[a] + x[2] + dn[b] != D) continue;
            int l = upper_bound(val.begin(), val.end(), d1[a]) -
val.begin();
            int r = lower_bound(val.begin(), val.end(), d1[b]) -
val.begin();
            if (l < r) { cover[l]++; cover[r]--; }
        }
    vector<char> key(K);
    for (int i = 0, c = 0; i < K; i++) {
```

```
56          c += cover[i];
57          key[i] = (c == 0 && same[val[i]] == 1);  // không bị nhảy qua, và
chỉ
                có một đỉnh
58      }
59      vector<int> ans;
60      for (int v = 1; v <= n; v++)
61          if (on(v) && key[lower_bound(val.begin(), val.end(), d1[v]) -
                val.begin()]) ans.push_back(v);
62      printf("%d\n", (int)ans.size());
63      for (size_t i = 0; i < ans.size(); i++) printf("%d%c", ans[i], i + 1 ==
            ans.size() ? '\n' : ' ');
64      return 0;
65  }
```

Độ phức tạp: hai lần Dijkstra O(m log n), sắp xếp mốc O(n log n), quét O(n + m log n).

6. Bẫy thường gặp

- **"Trên một đường" khác "trên mọi đường".**
- **Quên kiểm tra cạnh theo cả hai chiều** (đồ thị vô hướng: cạnh (u, v) có thể được dùng theo chiều v → u).
- **So sánh khoảng cách khi một trong hai là vô cùng:** cộng LLONG_MAX sẽ tràn số.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (70)	Tổng
Xóa thử từng đỉnh	30	0	**30**
Chỉ kiểm tra $d_1[v] + d_n[v] = D$	0	0	**0**
Đếm số đường đi theo mô-đun	30	70	**100**
Quét mốc bằng mảng hiệu	30	70	**100**

Chấm trên 8 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

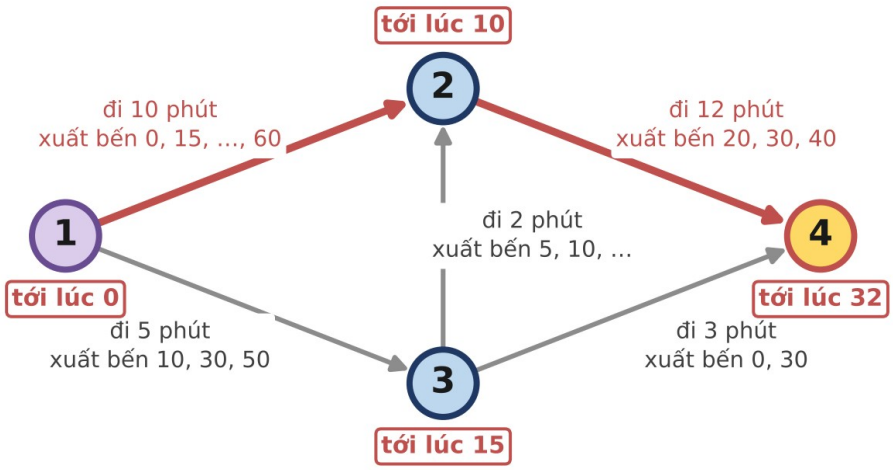
BÀI 5.3 XE BUÝT ĐÊM

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
XEBUYT.INP	XEBUYT.OUT	1 giây	256 MB	★★★ NÂNG CAO

Liên hệ: Tập 1, Bài 5 · Đề bài: Tập 2, trang 44

1. Hiểu bài toán

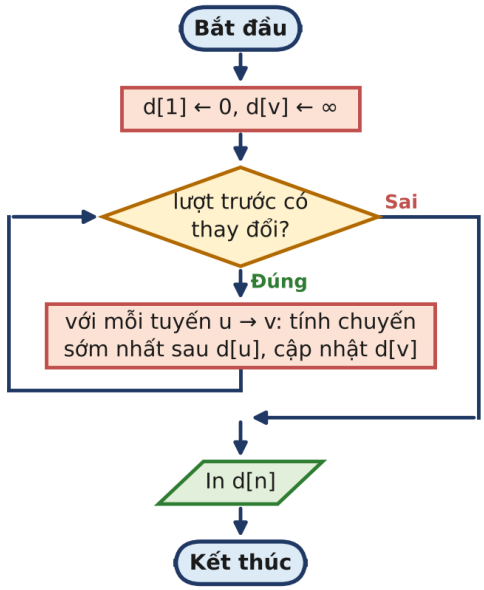
Mỗi tuyến là một cung có "trọng số" **phụ thuộc thời điểm**: nếu tới bến u lúc t, ta phải chờ chuyến kế tiếp rồi mới đi w phút. Đỉnh là bến, nhãn d[v] là **thời điểm sớm nhất** có mặt ở v.



Hình 5.3.1. *Ví dụ: chờ chuyến lúc 20 ở bến 2, về tới bến 4 lúc 32.*

2. Cách ngây thơ: Nới lỏng mọi tuyến tới khi ổn định

Giống Bellman-Ford: lặp đi lặp lại việc duyệt mọi tuyến, cập nhật thời điểm tới, cho tới khi không còn gì thay đổi.



Hình 5.3.2. *Sơ đồ khối của cách nới lỏng lặp.*

Cách ngây thơ: nới lỏng lặp

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

int main() {
    if (fopen("XEBUYT.INP", "r")) {
        freopen("XEBUYT.INP", "r", stdin);
        freopen("XEBUYT.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<array<ll,6>> e(m);
    for (auto &x : e) for (int k = 0; k < 6; k++) scanf("%lld", &x[k]);
    const ll INF = LLONG_MAX;
    vector<ll> d(n + 1, INF);
    d[1] = 0;
    bool changed = true;
    while (changed) {                              // nới lỏng mọi tuyến tới khi
        không đổi
        changed = false;
        for (auto &x : e) {
            int u = x[0], v = x[1]; ll w = x[2], a = x[3], p = x[4], b = x[5];
            if (d[u] == INF) continue;
            ll dep = d[u] <= a ? a : a + (d[u] - a + p - 1) / p * p;
            if (dep <= b && dep + w < d[v]) { d[v] = dep + w; changed = true;
                }
        }
    }
    printf("%lld\n", d[n] == INF ? -1 : d[n]);
    return 0;
}
```

Độ phức tạp: mỗi lượt O(m); số lượt có thể tới n. Tổng $O(n \cdot m) = 2 \cdot 10^{10}$ trong trường hợp xấu nhất.

3. Bế tắc

Nếu các tuyến được liệt kê "ngược chiều" so với đường đi tốt nhất, mỗi lượt chỉ tiến thêm được một bến. Cách này chỉ được **30 điểm** (Subtask 2, $n \le 1000$).

Ở chiều ngược lại, nhiều bạn dùng Dijkstra ngay nhưng quên việc **chờ xe**: coi như tới bến lúc t thì xe chạy ngay lúc t (chỉ kiểm tra $t \le b$). Đúng với Subtask 1 ($a = 0$, $p = 1$ nghĩa là xe chạy bất cứ lúc nào) nhưng sai khi có lịch chạy: **30 điểm**.

> **ĐIỂM BẾ TẮC**
>
> Dijkstra được chứng minh cho trọng số cố định không âm. Ở đây "trọng số" thay đổi theo thời điểm tới. Dijkstra còn đúng không?

4. Phá bế tắc

> **TỚI SỚM KHÔNG BAO GIỜ THIỆT**
>
> Gọi arr(t) là thời điểm tới v nếu rời u lúc t (đi tuyến $u \to v$). Vì có thể chờ ở bến, arr(t) **không giảm** theo t: tới u sớm hơn thì luôn có thể chờ để lên đúng chuyến mà người tới muộn lên. Với tính chất này (gọi là FIFO), lập luận của Dijkstra vẫn đúng nguyên vẹn: khi chốt u với thời điểm nhỏ nhất, không cách nào khác tới u sớm hơn.

Chuyến sớm nhất xuất bến không trước t được tính bằng công thức, không lặp từng chuyến:

- nếu $t \le a$: xuất bến lúc a;
- ngược lại: xuất bến lúc $a + \lceil (t - a) / p \rceil \cdot p$;
- nếu thời điểm xuất bến > b: không đi được tuyến này nữa.

Chốt bến	Thời điểm	Tuyến xét	Tới
1	0	1→2: chuyến 0, tới 10; 1→3: chuyến 10, tới 15	2: 10, 3: 15
2	10	2→4: chuyến 20, tới 32	4: 32
3	15	3→4: chuyến 30, tới 33 (không tốt hơn); 3→2: chuyến 15, tới 17 (không tốt hơn)	
4	**32**		đích

5. Cách tối ưu

XEBUYT.CPP: Dijkstra theo thời điểm tới

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
struct Route { int v; ll w, a, p, b; };

int main() {
    if (fopen("XEBUYT.INP", "r")) {
        freopen("XEBUYT.INP", "r", stdin);
        freopen("XEBUYT.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<vector<Route>> g(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v; ll w, a, p, b;
        scanf("%d %d %lld %lld %lld %lld", &u, &v, &w, &a, &p, &b);
        g[u].push_back({v, w, a, p, b});
    }
    // d[v]: thời điểm sớm nhất có mặt ở bến v
    vector<ll> d(n + 1, LLONG_MAX);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
    d[1] = 0;
    pq.push({0, 1});
    while (!pq.empty()) {
        auto [t, u] = pq.top(); pq.pop();
        if (t != d[u]) continue;
        for (auto &r : g[u]) {
            // chuyến sớm nhất xuất bến không trước thời điểm t
            ll dep = t <= r.a ? r.a : r.a + (t - r.a + r.p - 1) / r.p * r.p;
            if (dep > r.b) continue;            // đã hết chuyến
            if (dep + r.w < d[r.v]) {
                d[r.v] = dep + r.w;
                pq.push({d[r.v], r.v});
            }
        }
    }
    printf("%lld\n", d[n] == LLONG_MAX ? -1 : d[n]);
    return 0;
}
```

Độ phức tạp: O(m log n).

6. Bẫy thường gặp

- **Lặp từng chuyến** để tìm chuyến kế tiếp: p = 1, b − a = 10^9 thì một lần lặp đã tốn 10^9 bước.
- **Tràn số** ở biểu thức a + ⌈(t − a)/p⌉ · p khi dùng `int`.
- **Quên điều kiện chuyến cuối b.**

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
Nới lỏng lặp (kiểu Bellman-Ford)	0	30	0	**30**

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
Dijkstra, quên chờ xe	30	0	0	**30**
Dijkstra theo thời điểm tới	30	30	40	**100**

Chấm trên 9 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

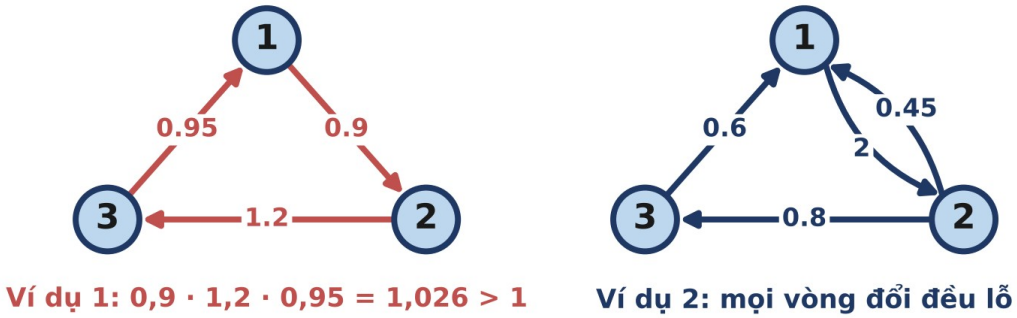
BÀI 6.1 ĐỔI TIỀN

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
DOITIEN.INP	DOITIEN.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Tập 1, Bài 6 · Đề bài: Tập 2, trang 46

1. Hiểu bài toán

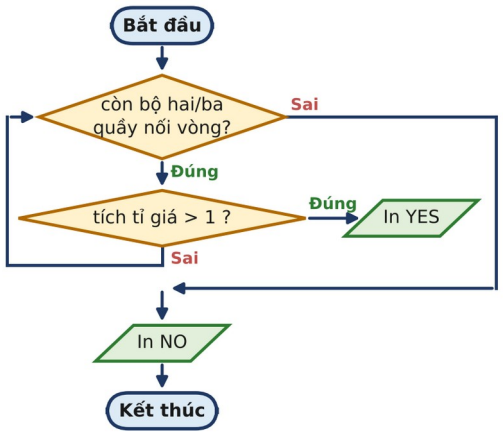
Mỗi loại tiền là một đỉnh, mỗi quầy là một cung u → v có hệ số r. Đổi qua một vòng nhân số tiền với **tích** các hệ số; vòng đổi có lãi khi tích > 1. Bellman-Ford làm việc với **tổng**, nên cần một phép biến đổi.



Hình 6.1.1. *Hai ví dụ của đề.*

2. Cách ngây thơ: Thử các vòng ngắn

Với n ≤ 3, mọi vòng đổi đơn chỉ gồm 2 hoặc 3 quầy: thử hết các bộ quầy nối thành vòng.



Hình 6.1.2. *Sơ đồ khối của cách thử các vòng ngắn.*

Cách ngây thơ: thử mọi vòng 2 và 3 quầy

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DOITIEN.INP", "r")) {
        freopen("DOITIEN.INP", "r", stdin);
        freopen("DOITIEN.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<int> u(m), v(m); vector<double> r(m);
    for (int i = 0; i < m; i++) scanf("%d %d %lf", &u[i], &v[i], &r[i]);
    bool ok = false;
    for (int i = 0; i < m; i++)                  // vòng 2 quầy và vòng 3 quầy
        for (int j = 0; j < m; j++) {
            if (v[i] != u[j]) continue;
            if (v[j] == u[i] && r[i] * r[j] > 1 + 1e-9) ok = true;
            for (int k = 0; k < m && !ok; k++)
                if (v[j] == u[k] && v[k] == u[i] && r[i] * r[j] * r[k] > 1 + 1e-9) ok = true;
        }
    puts(ok ? "YES" : "NO");
    return 0;
}
```

Độ phức tạp: $O(m^3)$. Với $m = 10^4$ là 10^{12}.

3. Bế tắc

- **Subtask 1** ($n \le 3$): **30 điểm**.
- **Subtask 2**: vòng có thể dài tới $n = 100$ quầy, và m lớn: không thể thử hết.

ĐIỂM BẾ TẮC

Tích các hệ số không phải là thứ Bellman-Ford biết xử lí. Cần chuyển "tích > 1" thành một điều kiện về **tổng**.

4. Phá bế tắc

Lấy logarit: $r_1 \cdot r_2 \cdot \ldots \cdot r_k > 1 \iff \ln r_1 + \ldots + \ln r_k > 0 \iff (-\ln r_1) + \ldots + (-\ln r_k) < 0$. Đặt trọng số cung là **−ln r**, vòng có lãi chính là **chu trình âm**.

Quầy (ví dụ 1)	r	−ln r	
1 → 2	0,9	+0,1054	
2 → 3	1,2	−0,1823	
3 → 1	0,95	+0,0513	
Tổng		**−0,0256 < 0**	chu trình âm: YES

VÒNG LÃI CÓ THỂ KHÔNG TỚI ĐƯỢC TỪ LOẠI TIỀN 1

Bellman-Ford từ một nguồn chỉ phát hiện chu trình âm **tới được** từ nguồn. Để không có "nguồn": khởi tạo d[v] = 0 cho **mọi** v (tương đương thêm một đỉnh ảo nối tới mọi đỉnh với trọng số 0). Chạy Bellman-Ford chỉ từ loại tiền 1 là một lỗi thật sự: trên bộ test của sách, cách đó chỉ được **30 điểm**.

5. Cách tối ưu

DOITIEN.CPP: Bellman-Ford với trọng số −ln r

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DOITIEN.INP", "r")) {
        freopen("DOITIEN.INP", "r", stdin);
        freopen("DOITIEN.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<int> U(m), V(m);
    vector<double> W(m);
    for (int i = 0; i < m; i++) {
        double r;
        scanf("%d %d %lf", &U[i], &V[i], &r);
        W[i] = -log(r);                          // tích > 1  <=>  tổng -log
<                                                    0
    }
    // Mọi đỉnh đều là nguồn: tương đương thêm đỉnh ảo nối tới mọi đỉnh với
        trọng số 0
    vector<double> d(n + 1, 0);
    bool neg = false;
    for (int it = 0; it < n; it++) {
        bool changed = false;
        for (int i = 0; i < m; i++)
            if (d[U[i]] + W[i] < d[V[i]] - 1e-9) {
                d[V[i]] = d[U[i]] + W[i];
                changed = true;
            }
        if (!changed) break;
        if (it == n - 1) neg = true;             // lượt thứ n vẫn còn giảm:
có                                                   chu trình âm
    }
    puts(neg ? "YES" : "NO");
    return 0;
}
```

Độ phức tạp: $O(n \cdot m) = 10^6$.

6. Bẫy thường gặp

- **So sánh số thực không có sai số:** dùng $d[u] + w < d[v] - 10^{-9}$ để tránh vòng lặp do sai số làm tròn.
- **Chỉ xuất phát từ loại tiền 1.**

- **Nhân trực tiếp các tỉ giá** trong n lượt: vẫn làm được (tìm "đường đi có tích lớn nhất"), nhưng phải cẩn thận tràn số thực khi tỉ giá lớn.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (70)	Tổng
Thử các vòng 2 và 3 quầy	30	0	**30**
Bellman-Ford chỉ từ loại tiền 1	30	0	**30**
Bellman-Ford, mọi đỉnh là nguồn	30	70	**100**

Chấm trên 9 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

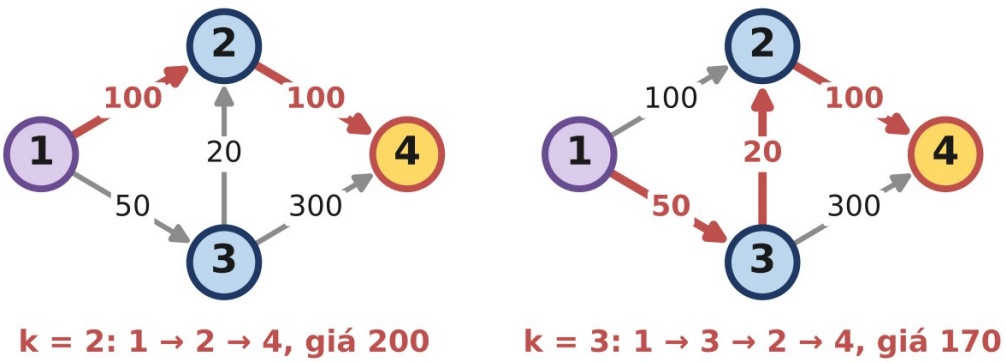
BÀI 6.2 CHUYẾN BAY GIÁ RẺ

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
BAYRE.INP	BAYRE.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Tập 1, Bài 6 · Đề bài: Tập 2, trang 49

1. Hiểu bài toán

Đường đi rẻ nhất với **không quá k cạnh**. Dijkstra không biết đếm số cạnh; Bellman-Ford thì có: sau lượt thứ i, nếu cài đặt đúng, d[v] chính là giá rẻ nhất dùng không quá i cạnh.



Hình 6.2.1. *Cùng một mạng bay, k = 2 và k = 3 cho hai đáp số khác nhau.*

2. Cách ngây thơ: Dijkstra, bỏ qua giới hạn

Cách đầu tiên nhiều bạn viết là Dijkstra thông thường, "hi vọng" đường rẻ nhất không quá dài.

Cách ngây thơ: Dijkstra không quan tâm tới k

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

int main() {
    if (fopen("BAYRE.INP", "r")) {
        freopen("BAYRE.INP", "r", stdin);
        freopen("BAYRE.OUT", "w", stdout);
    }
    int n, m, k;
    scanf("%d %d %d", &n, &m, &k);
    vector<vector<pair<int,int>>> g(n + 1);
    for (int i = 0; i < m; i++) { int u, v, c; scanf("%d %d %d", &u, &v,
&c);
        g[u].push_back({v, c}); }
    vector<ll> d(n + 1, LLONG_MAX);             // Dijkstra thường: bỏ qua
        giới hạn k chuyến
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
    d[1] = 0; pq.push({0, 1});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du != d[u]) continue;
        for (auto [v, c] : g[u]) if (d[u] + c < d[v]) { d[v] = d[u] + c;
            pq.push({d[v], v}); }
    }
    printf("%lld\n", d[n] == LLONG_MAX ? -1 : d[n]);
    return 0;
}
```

Ở ví dụ 1 (k = 2), Dijkstra tìm ra đường 1 → 3 → 2 → 4 giá 170 nhưng dùng 3 chuyến: sai.

3. Bế tắc

Dijkstra chỉ đúng ở Subtask 1, nơi $k \geq n - 1$ khiến giới hạn không có tác dụng: **30 điểm**.

Chuyển sang Bellman-Ford chạy đúng k lượt là hướng đúng, nhưng nếu nới lỏng **ngay trên mảng d** thì trong cùng một lượt, một giá trị vừa được cập nhật có thể lập tức được dùng để cập nhật tiếp: một lượt "đi" được nhiều cạnh. Ở ví dụ 1, lượt đầu đã có d[2] = 70 (qua 3) và nếu chuyến 2 → 4 được xét sau đó thì d[4] = 170 ngay sau 2 lượt. Cách này cũng chỉ được **30 điểm**.

ĐIỂM BẾ TẮC

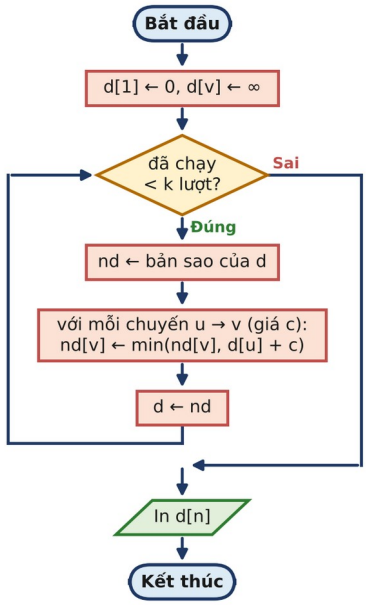
Ý nghĩa "sau lượt i, d dùng không quá i cạnh" chỉ đúng khi mọi phép nới lỏng của lượt i đều xuất phát từ kết quả của lượt $i - 1$.

4. Phá bế tắc

Mỗi lượt, sao chép d thành nd, chỉ đọc từ d và ghi vào nd, cuối lượt gán d = nd. Có thể dừng sớm khi một lượt không thay đổi gì.

Lượt	d[1..4]	Ý nghĩa
0	0, ∞, ∞, ∞	chưa bay

Lượt	d[1..4]	Ý nghĩa
1	0, 100, 50, ∞	≤ 1 chuyến
2	0, 70, 50, **200**	≤ 2 chuyến: đáp số ví dụ 1
3	0, 70, 50, **170**	≤ 3 chuyến: đáp số ví dụ 2



Hình 6.2.2. *Sơ đồ khối của Bellman-Ford k lượt.*

5. Cách tối ưu

BAYRE.CPP: Bellman-Ford k lượt có sao chép

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

int main() {
    if (fopen("BAYRE.INP", "r")) {
        freopen("BAYRE.INP", "r", stdin);
        freopen("BAYRE.OUT", "w", stdout);
    }
    int n, m, k;
    scanf("%d %d %d", &n, &m, &k);
    vector<int> U(m), V(m);
    vector<ll> C(m);
    for (int i = 0; i < m; i++) scanf("%d %d %lld", &U[i], &V[i], &C[i]);
    const ll INF = LLONG_MAX / 4;
    vector<ll> d(n + 1, INF);                    // d sau lượt i: giá rẻ nhất dùng <= i chuyến
    d[1] = 0;
    for (int it = 0; it < k; it++) {
        vector<ll> nd = d;                       // chỉ nới lỏng từ kết quả của lượt trước
        for (int i = 0; i < m; i++)
            if (d[U[i]] < INF && d[U[i]] + C[i] < nd[V[i]]) nd[V[i]] = d[U[i]] + C[i];
        if (nd == d) break;                      // không còn thay đổi: dừng sớm
        d = nd;
    }
    printf("%lld\n", d[n] >= INF ? -1 : d[n]);
    return 0;
}
```

Độ phức tạp: $O(k \cdot (n + m)) = 10^3 \cdot 1{,}1 \cdot 10^4 \approx 10^7$.

6. Bẫy thường gặp

- **Nới lỏng tại chỗ** (lỗi kinh điển).
- **Cộng với vô cùng:** chỉ nới lỏng khi d[u] hữu hạn.
- **Dừng sớm sai:** chỉ dừng khi cả lượt không có thay đổi nào.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
Dijkstra, bỏ qua giới hạn k	30	0	0	**30**
Bellman-Ford k lượt, không sao chép	30	0	0	**30**
Bellman-Ford k lượt, có sao chép	30	30	40	**100**

Chấm trên 10 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

BÀI 6.3 LỊCH THI ĐẤU

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
LICHTHIDAU.INP	LICHTHIDAU.OUT	1 giây	256 MB	★★★ NÂNG CAO

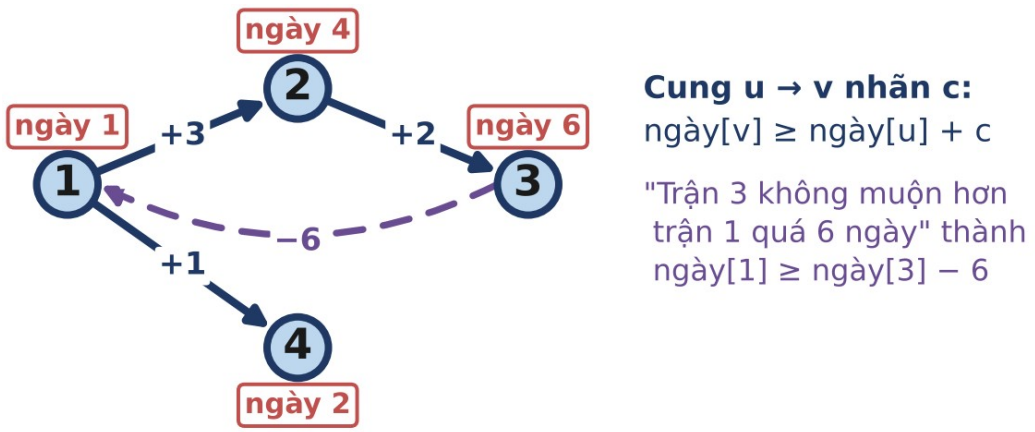
Liên hệ: Tập 1, Bài 6 · Đề bài: Tập 2, trang 51

1. Hiểu bài toán

Mỗi ràng buộc là một bất đẳng thức dạng ngày[y] ≥ ngày[x] + c:

- Loại 1 "A sau B ít nhất c ngày": ngày[A] ≥ ngày[B] + c, cung B → A nhãn c.
- Loại 2 "A không muộn hơn B quá c ngày": ngày[A] ≤ ngày[B] + c ⟺ ngày[B] ≥ ngày[A] − c, cung A → B nhãn −c.

Ngày sớm nhất của mỗi trận chính là 1 + độ dài **đường đi dài nhất** tới nó trong đồ thị ràng buộc. Một **chu trình có tổng dương** cho ra ngày[A] > ngày[A]: vô nghiệm.



Hình 6.3.1. *Đồ thị ràng buộc của ví dụ 1.*

2. Cách ngây thơ: Quy hoạch động trên DAG

Nếu chỉ có ràng buộc loại 1 (cung nhãn không âm, theo một thứ tự ngầm), đồ thị là DAG: sắp xếp tô pô rồi lấy đường đi dài nhất. Cách này bỏ qua ràng buộc loại 2.

Cách ngây thơ: đường đi dài nhất trên DAG (bỏ qua loại 2)

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

int main() {
    if (fopen("LICHTHIDAU.INP", "r")) {
        freopen("LICHTHIDAU.INP", "r", stdin);
        freopen("LICHTHIDAU.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<vector<pair<int,ll>>> g(n + 1);
    vector<int> in(n + 1, 0);
    for (int i = 0; i < m; i++) {
        int t, a, b; ll c;
        scanf("%d %d %d %lld", &t, &a, &b, &c);
        if (t == 1) { g[b].push_back({a, c}); in[a]++; }   // bỏ qua ràng buộc
            loại 2
    }
    // đường đi dài nhất trên DAG theo thứ tự tô pô
    vector<ll> d(n + 1, 1);
    queue<int> q; int cnt = 0;
    for (int v = 1; v <= n; v++) if (!in[v]) q.push(v);
    while (!q.empty()) {
        int u = q.front(); q.pop(); cnt++;
        for (auto [v, c] : g[u]) { d[v] = max(d[v], d[u] + c); if (--in[v] ==
            0) q.push(v); }
    }
    if (cnt < n) { printf("-1\n"); return 0; }
    for (int v = 1; v <= n; v++) printf("%lld%c", d[v], v == n ? '\n' : ' ');
    return 0;
}
```

Độ phức tạp: O(n + m).

3. Bế tắc

Đúng Subtask 1: **40 điểm**. Ràng buộc loại 2 tạo ra cung nhãn âm và chu trình, không còn là DAG.

Bellman-Ford "lặp tới khi hết thay đổi" xử lí được cả hai loại, nhưng khi có chu trình dương thì vòng lặp **không bao giờ dừng**: **40 điểm**.

> **ĐIỂM BẾ TẮC**
>
> Cần một giới hạn số lượt để phân biệt "chưa hội tụ" với "không bao giờ hội tụ".

4. Phá bế tắc

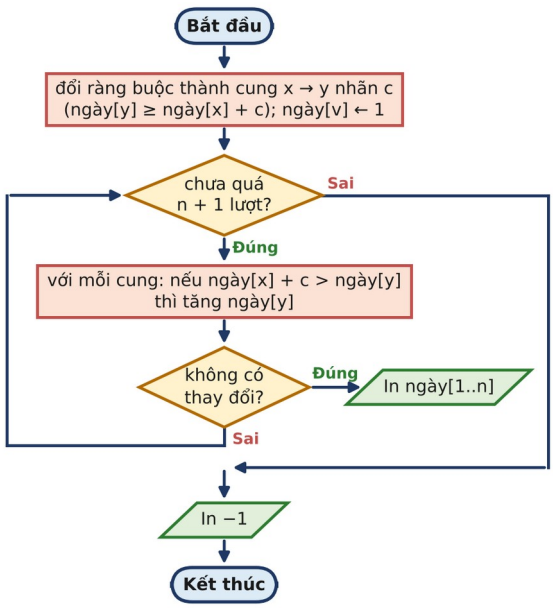
Đây là Bellman-Ford cho đường đi **dài nhất** (đổi dấu so sánh). Khởi tạo ngày[v] = 1 tương đương với một đỉnh ảo nối tới mọi đỉnh. Nếu không có chu trình dương, mọi đường đi dài nhất

có không quá n cạnh, nên sau n lượt các giá trị phải ổn định. Nếu lượt thứ n + 1 vẫn còn tăng: có chu trình dương, in −1.

VÌ SAO KẾT QUẢ LÀ "SỚM NHẤT CÓ THỂ"?

Mỗi lần ngày[y] tăng lên ngày[x] + c là do một ràng buộc **bắt buộc** điều đó. Bằng quy nạp, mọi lịch hợp lệ đều có ngày[v] ít nhất bằng giá trị thuật toán tìm ra; và khi thuật toán dừng thì mọi ràng buộc đều thỏa. Vì vậy kết quả đồng thời là sớm nhất cho mọi trận.

Lượt	ngày[1..4]	Ghi chú
0	1, 1, 1, 1	khởi tạo
1	1, 4, 6, 2	2 ≥ 1 + 3; 3 ≥ 4 + 2; 4 ≥ 1 + 1
2	1, 4, 6, 2	không đổi: dừng, 6 ≤ 1 + 6 thỏa



Hình 6.3.2. *Sơ đồ khối của lời giải mẫu.*

5. Cách tối ưu

LICHTHIDAU.CPP: Bellman-Ford đường đi dài nhất, n + 1 lượt

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

int main() {
    if (fopen("LICHTHIDAU.INP", "r")) {
        freopen("LICHTHIDAU.INP", "r", stdin);
        freopen("LICHTHIDAU.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    // cung (x, y, c) nghĩa là ngày[y] >= ngày[x] + c
    vector<array<ll,3>> e;
    for (int i = 0; i < m; i++) {
        int t, a, b; ll c;
        scanf("%d %d %d %lld", &t, &a, &b, &c);
        if (t == 1) e.push_back({b, a, c});     // ngày[A] >= ngày[B] + c
        else e.push_back({a, b, -c});           // ngày[A] <= ngày[B] + c
<=>
            ngày[B] >= ngày[A] - c
    }
    vector<ll> d(n + 1, 1);                     // ngày sớm nhất, ban đầu là
        ngày 1
    for (int it = 0; it <= n; it++) {           // Bellman-Ford tìm đường đi
        dài nhất
        bool changed = false;
        for (auto &x : e)
            if (d[x[0]] + x[2] > d[x[1]]) { d[x[1]] = d[x[0]] + x[2];
changed
                = true; }
        if (!changed) {
            for (int v = 1; v <= n; v++) printf("%lld%c", d[v], v == n ? '\
n'
                : ' ');
            return 0;
        }
    }
    printf("-1\n");                             // vẫn còn tăng: có chu
trình
        dương
    return 0;
}
```

Độ phức tạp: $O(n \cdot m) = 5 \cdot 10^6$.

6. Bẫy thường gặp

- **Đổi chiều cung sai** khi chuyển ràng buộc loại 2.
- **Lặp vô hạn** khi có chu trình dương.
- **Khởi tạo ngày = 0** (đề yêu cầu ngày là số nguyên dương).

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (40)	ST2 (60)	Tổng
Đường dài nhất trên DAG (bỏ qua loại 2)	40	0	**40**
Bellman-Ford lặp tới khi hết thay đổi	40	0	**40**
Bellman-Ford n + 1 lượt	40	60	**100**

Chấm trên 6 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

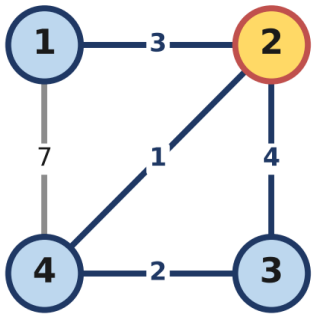
BÀI 7.1 THỦ ĐÔ MỚI

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
THUDO.INP	THUDO.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Tập 1, Bài 7 · Đề bài: Tập 2, trang 53

1. Hiểu bài toán

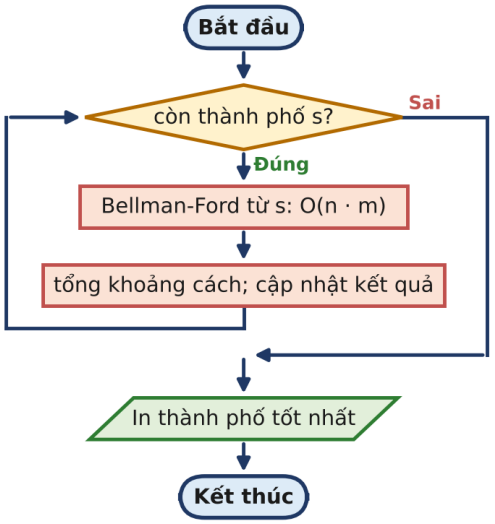
Cần tổng khoảng cách từ mỗi thành phố tới mọi thành phố khác: bài toán đường đi ngắn nhất **giữa mọi cặp**, n ≤ 400: đúng "sân nhà" của thuật toán Floyd (Tập 1, Bài 7).



	1	2	3	4	Tổng
1	0	3	6	4	13
2	3	0	3	1	7
3	6	3	0	2	11
4	4	1	2	0	7

Hình 7.1.1. *Ma trận khoảng cách của ví dụ.*

2. Cách ngây thơ: Bellman-Ford từ từng thành phố



Hình 7.1.2. *Sơ đồ khối của cách chạy Bellman-Ford n lần.*

Cách ngây thơ: n lần Bellman-Ford

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

int main() {
    if (fopen("THUDO.INP", "r")) {
        freopen("THUDO.INP", "r", stdin);
        freopen("THUDO.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<array<int,3>> e(m);
    for (auto &x : e) scanf("%d %d %d", &x[0], &x[1], &x[2]);
    const ll INF = LLONG_MAX / 4;
    int best = 1; ll bs = INF;
    for (int s = 1; s <= n; s++) {              // Bellman-Ford từ từng thành
        phố
        vector<ll> d(n + 1, INF);
        d[s] = 0;
        for (int it = 0; it < n - 1; it++) {
            bool ch = false;
            for (auto &x : e) {
                if (d[x[0]] + x[2] < d[x[1]]) { d[x[1]] = d[x[0]] + x[2]; ch =
                    true; }
                if (d[x[1]] + x[2] < d[x[0]]) { d[x[0]] = d[x[1]] + x[2]; ch =
                    true; }
            }
            if (!ch) break;
        }
        ll sum = 0;
        for (int v = 1; v <= n; v++) sum += d[v];
        if (sum < bs) { bs = sum; best = s; }
    }
    printf("%d %lld\n", best, bs);
    return 0;
}
```

Độ phức tạp: $O(n \cdot n \cdot m) = 400 \cdot 400 \cdot 8 \cdot 10^4 \approx 10^{10}$ trong trường hợp xấu nhất.

3. Bế tắc

- **Subtask 1** ($n \le 50$): chạy kịp: **30 điểm**.
- **Subtask 2**: quá thời gian.

Một lỗi tai hại khi cài Floyd là đặt vòng k vào **trong cùng** (for i, for j, for k). Chương trình vẫn chạy nhanh và cho kết quả "trông hợp lí", nhưng sai: **0 điểm**.

ĐIỂM BẾ TẮC

Bellman-Ford từ mỗi nguồn làm lại rất nhiều việc chung giữa các nguồn. Floyd chia sẻ công việc giữa mọi cặp đỉnh, nhưng thứ tự vòng lặp phải đúng ý nghĩa quy hoạch động.

4. Phá bế tắc

Floyd là quy hoạch động: sau khi xử lí k, d[i][j] là đường ngắn nhất từ i tới j chỉ dùng các đỉnh trung gian thuộc {1, ..., k}. Muốn dùng d[i][k] và d[k][j] "đã hoàn chỉnh với tập {1, ..., k − 1}" thì k phải là vòng **ngoài cùng**.

> **VÌ SAO ĐẶT k TRONG CÙNG LẠI SAI?**
>
> Với thứ tự (i, j, k), khi tính d[i][j] ta dùng d[i][k], d[k][j] của những cặp có thể **chưa** được tối ưu (ví dụ cặp (k, j) với k > i chưa được xét). Đường ngắn nhất cần đi qua hai đỉnh trung gian trở lên có thể bị bỏ sót.

Chạy xong Floyd, tính tổng từng hàng (Hình 7.1.1): hàng 2 và hàng 4 cùng có tổng 7; duyệt i tăng dần, chỉ cập nhật khi **nhỏ hơn hẳn**, nên chọn thành phố **2**.

5. Cách tối ưu

THUDO.CPP: Floyd O(n³)

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

int main() {
    if (fopen("THUDO.INP", "r")) {
        freopen("THUDO.INP", "r", stdin);
        freopen("THUDO.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    const ll INF = LLONG_MAX / 4;                // INF + INF vẫn không tràn
    vector<vector<ll>> d(n + 1, vector<ll>(n + 1, INF));
    for (int i = 1; i <= n; i++) d[i][i] = 0;
    for (int i = 0; i < m; i++) {
        int u, v; ll w;
        scanf("%d %d %lld", &u, &v, &w);
        d[u][v] = min(d[u][v], w);               // giữ con đường ngắn nhất giữa một cặp
        d[v][u] = min(d[v][u], w);
    }
    for (int k = 1; k <= n; k++)                 // k ở ngoài cùng
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= n; j++)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
    int best = 1; ll bs = INF;
    for (int i = 1; i <= n; i++) {
        ll s = 0;
        for (int j = 1; j <= n; j++) s += d[i][j];
        if (s < bs) { bs = s; best = i; }        // "<" giữ số hiệu nhỏ nhất khi bằng nhau
    }
    printf("%d %lld\n", best, bs);
    return 0;
}
```

Độ phức tạp: $O(n^3) = 6{,}4 \cdot 10^7$ phép tính đơn giản; $O(n^2)$ bộ nhớ.

6. Bẫy thường gặp

- **Vòng k không ở ngoài cùng.**
- **INF quá lớn:** INF + INF tràn `long long` nếu INF = LLONG_MAX; dùng LLONG_MAX / 4.
- **Nhiều con đường giữa cùng một cặp:** khởi tạo bằng `min`, không gán đè.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (70)	Tổng
Bellman-Ford từ từng thành phố	30	0	**30**
Floyd, vòng k ở trong cùng	0	0	**0**
Floyd, vòng k ở ngoài cùng	30	70	**100**

Chấm trên 6 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

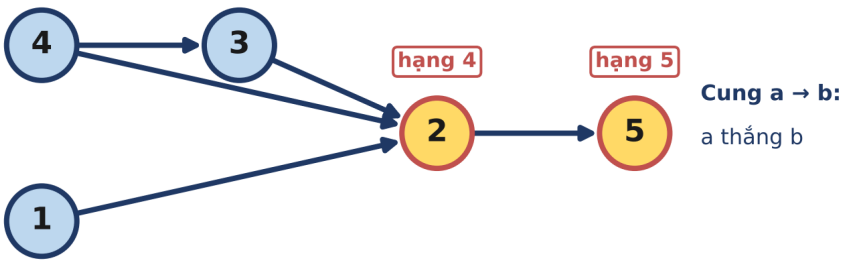
BÀI 7.2 XẾP HẠNG KÌ THỦ

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
XEPHANG.INP	XEPHANG.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Tập 1, Bài 7 · Đề bài: Tập 2, trang 55

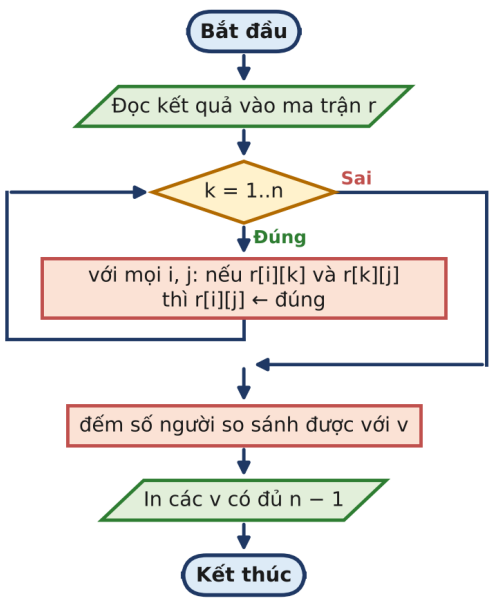
1. Hiểu bài toán

Quan hệ "mạnh hơn" có tính bắc cầu: a thắng b, b thắng c thì chắc chắn a mạnh hơn c. Kì thủ v xác định được thứ hạng khi với **mọi** u ≠ v, ta biết chắc u mạnh hơn hay yếu hơn v: (số người chắc chắn yếu hơn v) + (số người chắc chắn mạnh hơn v) = n − 1.



Hình 7.2.1. *Ví dụ: chỉ kì thủ 2 và 5 xác định được thứ hạng.*

2. Cách ngây thơ: Bao đóng bắc cầu kiểu Floyd



Hình 7.2.2. *Sơ đồ khối của bao đóng bắc cầu kiểu Floyd.*

Cách ngây thơ: Floyd trên ma trận bool

```cpp
#include <bits/stdc++.h>
using namespace std;

const int N = 2005;
bool r[N][N];

int main() {
    if (fopen("XEPHANG.INP", "r")) {
        freopen("XEPHANG.INP", "r", stdin);
        freopen("XEPHANG.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    for (int i = 0; i < m; i++) { int a, b; scanf("%d %d", &a, &b); r[a][b] =
        true; }
    for (int k = 1; k <= n; k++)                // bao đóng bắc cầu kiểu Floyd: O(n^3)
        for (int i = 1; i <= n; i++)
            if (r[i][k])
                for (int j = 1; j <= n; j++)
                    if (r[k][j]) r[i][j] = true;
    vector<int> ans;
    for (int v = 1; v <= n; v++) {
        int c = 0;
        for (int u = 1; u <= n; u++) if (r[u][v] || r[v][u]) c++;
        if (c == n - 1) ans.push_back(v);
    }
    printf("%d\n", (int)ans.size());
    for (size_t i = 0; i < ans.size(); i++) printf("%d%c", ans[i], i + 1 ==
        ans.size() ? '\n' : ' ');
    return 0;
}
```

Độ phức tạp: $O(n^3)$: n = 500 là $1{,}25 \cdot 10^8$, chạy kịp; n = 2000 là $8 \cdot 10^9$.

3. Bế tắc

- **Subtask 1, 2** ($n \le 500$): **40 + 40 điểm**.
- **Subtask 3** ($n \le 2000$): quá thời gian.

Một lời giải sai: chỉ dùng các kết quả **trực tiếp** (v đã đấu với cả n − 1 người). Ở ví dụ, kì thủ 5 chỉ đấu một trận nhưng vẫn xác định được hạng: **0 điểm**.

ĐIỂM BẾ TẮC

Phép "r[i][j] = r[i][j] or (r[i][k] and r[k][j])" xử lí từng ô một, trong khi với cùng i, k, ta đang "or" cả **hàng** k vào hàng i.

4. Phá bế tắc

Lưu mỗi hàng của ma trận là một `bitset<2001>`. Khi r[i][k] = 1, mọi người k mạnh hơn cũng yếu hơn i: `r[i] |= r[k]`, một phép toán trên 2001 bit thực hiện bằng khoảng 32 phép toán 64 bit.

Số người chắc chắn yếu hơn v là `r[v].count()`; số người chắc chắn mạnh hơn v là số u có r[u][v] = 1.

Kì thủ	Chắc chắn yếu hơn	Chắc chắn mạnh hơn	Tổng
1	2, 5	—	2
2	5	1, 3, 4	**4 = n − 1**
3	2, 5	4	3
4	3, 2, 5	—	3
5	—	1, 2, 3, 4	**4 = n − 1**

5. Cách tối ưu

XEPHANG.CPP: bao đóng bắc cầu bằng bitset

```cpp
#include <bits/stdc++.h>
using namespace std;

const int N = 2001;
bitset<N> r[N];                                  // r[a][b] = 1: chắc chắn a mạnh hơn b

int main() {
    if (fopen("XEPHANG.INP", "r")) {
        freopen("XEPHANG.INP", "r", stdin);
        freopen("XEPHANG.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    for (int i = 0; i < m; i++) {
        int a, b;
        scanf("%d %d", &a, &b);
        r[a][b] = 1;
    }
    // bao đóng bắc cầu: nếu i mạnh hơn k thì i mạnh hơn mọi người k mạnh hơn
    for (int k = 1; k <= n; k++)
        for (int i = 1; i <= n; i++)
            if (r[i][k]) r[i] |= r[k];           // n/64 phép tính
    vector<int> ans;
    for (int v = 1; v <= n; v++) {
        int c = r[v].count();                    // số người chắc chắn yếu hơn v
        for (int u = 1; u <= n; u++) if (r[u][v]) c++;   // cộng số người mạnh hơn v
        if (c == n - 1) ans.push_back(v);
    }
    printf("%d\n", (int)ans.size());
    for (size_t i = 0; i < ans.size(); i++) printf("%d%c", ans[i], i + 1 == ans.size() ? '\n' : ' ');
    return 0;
}
```

Độ phức tạp: $O(n^3 / 64) \approx 1{,}25 \cdot 10^8$ phép toán 64 bit; bộ nhớ n^2 bit = 0,5 MB.

CÁCH KHÁC

DFS từ mỗi kì thủ trên đồ thị "thắng" (và trên đồ thị ngược) để đếm số người tới được: $O(n \cdot (n + m)) \approx 2 \cdot 10^8$ cũng chạy kịp với giới hạn của bài.

6. Bẫy thường gặp

- **Chỉ dùng kết quả trực tiếp.**
- **Quên phần "mạnh hơn":** chỉ đếm số người yếu hơn là chưa đủ.
- **Khai báo `bool r[2001][2001]` và dùng ba vòng lặp:** đúng nhưng chậm gấp khoảng 64 lần.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (40)	ST2 (40)	ST3 (20)	Tổng
Floyd trên ma trận bool	40	40	0	**80**
Chỉ dùng kết quả trực tiếp	0	0	0	**0**
Bao đóng bắc cầu bằng bitset	40	40	20	**100**

Chấm trên 9 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

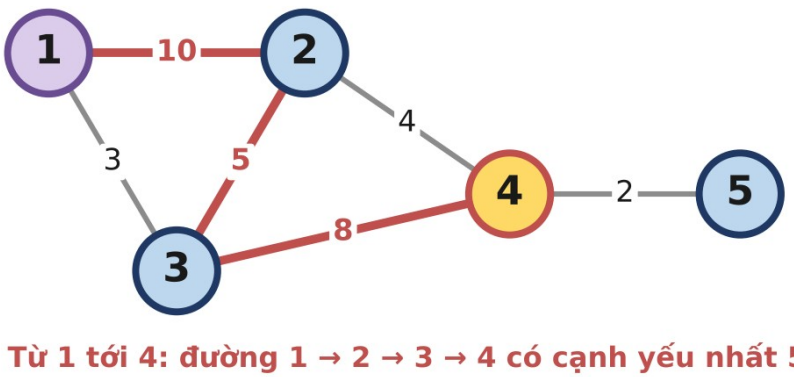
BÀI 7.3 XE TẢI QUÁ KHỔ

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
XETAI.INP	XETAI.OUT	1 giây	256 MB	★★★ NÂNG CAO

Liên hệ: Tập 1, Bài 7 · Đề bài: Tập 2, trang 57

1. Hiểu bài toán

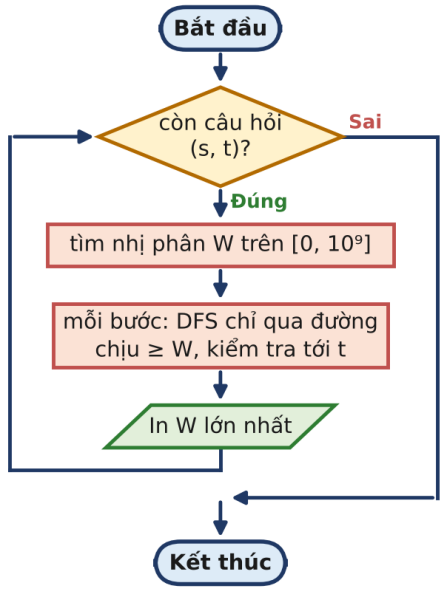
Tải trọng của một đường đi là tải trọng của con đường **yếu nhất** trên đó; cần đường có tải trọng **lớn nhất** giữa hai thị trấn. Đây là bài toán "đường đi có cạnh nhỏ nhất lớn nhất" (bottleneck path), cho rất nhiều cặp (s, t).



Hình 7.3.1. *Từ 1 tới 4, đường 1 → 2 → 3 → 4 chịu được 5 tấn.*

2. Cách ngây thơ: Tìm nhị phân + DFS cho từng đơn hàng

Với một W cố định, "chở được W tấn từ s tới t" khi s, t liên thông trong đồ thị chỉ gồm các con đường chịu ≥ W. Tính chất này đơn điệu theo W, nên tìm nhị phân W.



Hình 7.3.2. *Sơ đồ khối của cách tìm nhị phân cho từng đơn hàng.*

Cách ngây thơ: tìm nhị phân + DFS mỗi câu hỏi

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<vector<pair<int,int>>> g;

bool reach(int s, int t, int W) {               // đi được từ s tới t chỉ dùng
    cạnh chịu >= W tấn?
    vector<char> vis(n + 1, 0);
    vector<int> st = {s}; vis[s] = 1;
    while (!st.empty()) {
        int u = st.back(); st.pop_back();
        if (u == t) return true;
        for (auto [v, c] : g[u]) if (c >= W && !vis[v]) { vis[v] = 1;
            st.push_back(v); }
    }
    return false;
}

int main() {
    if (fopen("XETAI.INP", "r")) {
        freopen("XETAI.INP", "r", stdin);
        freopen("XETAI.OUT", "w", stdout);
    }
    int q;
    scanf("%d %d %d", &n, &m, &q);
    g.assign(n + 1, {});
    for (int i = 0; i < m; i++) { int u, v, c; scanf("%d %d %d", &u, &v, &c);
        g[u].push_back({v, c}); g[v].push_back({u, c}); }
    while (q--) {
        int s, t;
        scanf("%d %d", &s, &t);
        int lo = 0, hi = 1000000000;            // tìm nhị phân tải trọng lớn
            nhất
        while (lo < hi) {
            int mid = lo + (hi - lo + 1) / 2;
            if (reach(s, t, mid)) lo = mid; else hi = mid - 1;
        }
        printf("%d\n", lo);
    }
    return 0;
}
```

Độ phức tạp: $O(q \cdot \log C \cdot (n + m)) \approx 10^5 \cdot 30 \cdot 10^5$: quá lớn.

3. Bế tắc

- **Subtask 1** ($n, q \le 50$): **30 điểm**.
- **Subtask 2**: quá thời gian.

ĐIỂM BẾ TẮC

Có tới 10^5 câu hỏi nhưng chỉ có $n^2 = 2{,}5 \cdot 10^5$ cặp đỉnh khác nhau: tính trước đáp số cho **mọi cặp** rồi trả lời $O(1)$.

4. Phá bế tắc

Floyd không chỉ dành cho "tổng nhỏ nhất". Gọi b[i][j] là tải trọng lớn nhất từ i tới j dùng đỉnh trung gian thuộc {1, ..., k}. Đi qua k thì tải trọng là min(b[i][k], b[k][j]); vì vậy:

- **b[i][j] = max(b[i][j], min(b[i][k], b[k][j]))**, với k ở vòng ngoài cùng.

Lập luận đúng đắn giống hệt Floyd thông thường; chỉ cần thay (min, +) bằng (max, min).

Cặp (ví dụ)	Sau Floyd	Đường tương ứng
(1, 4)	**5**	1 – 2 – 3 – 4: min(10, 5, 8)
(1, 5)	**2**	phải qua 4 – 5 (2 tấn)
(3, 1)	**5**	3 – 2 – 1: min(5, 10)

> **CÁCH KHÁC: CÂY KHUNG LỚN NHẤT**
>
> Đường đi tốt nhất giữa s và t luôn nằm trên **cây khung lớn nhất** (Tập 1, Bài 10). Dựng cây bằng Kruskal (cạnh nặng trước), rồi DFS từ mỗi đỉnh trên cây để có đáp số cho mọi cặp trong $O(n^2)$. Cách này cũng được **100 điểm**.

5. Cách tối ưu

XETAI.CPP: Floyd (max, min)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("XETAI.INP", "r")) {
        freopen("XETAI.INP", "r", stdin);
        freopen("XETAI.OUT", "w", stdout);
    }
    int n, m, q;
    scanf("%d %d %d", &n, &m, &q);
    // b[i][j]: tải trọng lớn nhất từ i tới j (0 = chưa có đường)
    vector<vector<int>> b(n + 1, vector<int>(n + 1, 0));
    for (int i = 0; i < m; i++) {
        int u, v, c;
        scanf("%d %d %d", &u, &v, &c);
        b[u][v] = max(b[u][v], c);              // nhiều đường: giữ đường chịu
            tải tốt nhất
        b[v][u] = max(b[v][u], c);
    }
    // Floyd với phép toán (max, min)
    for (int k = 1; k <= n; k++)
        for (int i = 1; i <= n; i++) {
            if (!b[i][k]) continue;
            for (int j = 1; j <= n; j++)
                b[i][j] = max(b[i][j], min(b[i][k], b[k][j]));
        }
    while (q--) {
        int s, t;
        scanf("%d %d", &s, &t);
        printf("%d\n", b[s][t]);
    }
    return 0;
}
```

Độ phức tạp: $O(n^3 + m + q) \approx 1{,}25 \cdot 10^8$ phép tính đơn giản.

6. Bẫy thường gặp

- **Nhiều con đường giữa một cặp:** giữ con đường chịu tải **lớn nhất** (dùng max khi khởi tạo).
- **Nhầm min và max** khi chuyển từ Floyd thông thường.
- **b[i][i]:** không cần quan tâm vì đề đảm bảo $s \neq t$.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (70)	Tổng
Tìm nhị phân + DFS mỗi đơn hàng	30	0	**30**
Cây khung lớn nhất + DFS từ mỗi đỉnh	30	70	**100**
Floyd (max, min)	30	70	**100**

Chấm trên 6 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

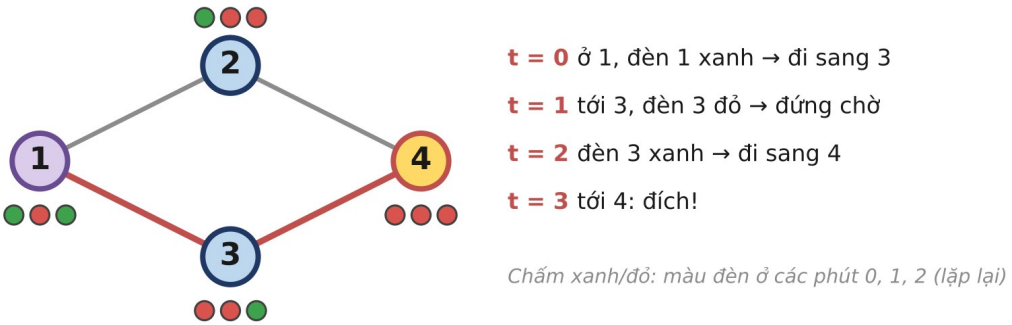
BÀI 8.1 ĐÈN GIAO THÔNG

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
DENGIAOTHONG.INP	DENGIAOTHONG.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Tập 1, Bài 8 · Đề bài: Tập 2, trang 59

1. Hiểu bài toán

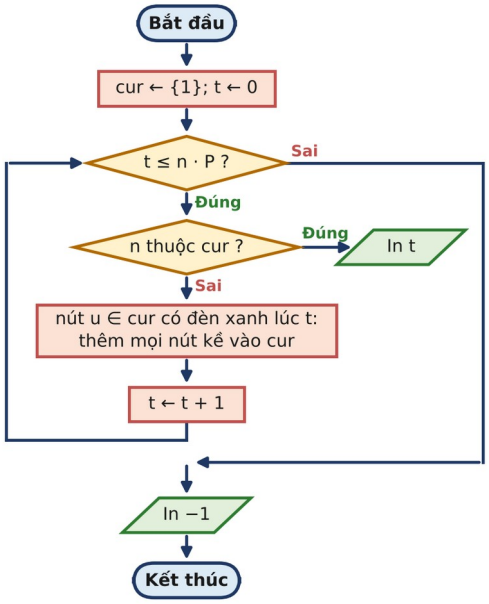
Đèn đổi màu theo thời gian nên **vị trí** thôi chưa đủ mô tả tình huống: tới nút u lúc 3 (đèn xanh) khác với tới u lúc 4 (đèn đỏ). Nhưng đèn lặp lại theo chu kì P, nên chỉ cần biết t **mod P**. Trạng thái là cặp (u, t mod P): có $n \cdot P \le 6 \cdot 10^5$ trạng thái.



Hình 8.1.1. *Ví dụ: xe phải đứng chờ một phút ở nút 3.*

2. Cách ngây thơ: Mô phỏng từng phút

Giữ tập các nút xe có thể đang có mặt ở phút t (vì được chờ, tập này chỉ lớn dần). Mỗi phút, từ các nút đang đèn xanh, thêm các nút kề.



Hình 8.1.2. *Sơ đồ khối của cách mô phỏng từng phút.*

Cách ngây thơ: mô phỏng từng phút

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DENGIAOTHONG.INP", "r")) {
        freopen("DENGIAOTHONG.INP", "r", stdin);
        freopen("DENGIAOTHONG.OUT", "w", stdout);
    }
    int n, m, P;
    cin >> n >> m >> P;
    vector<string> s(n + 1);
    for (int i = 1; i <= n; i++) cin >> s[i];
    vector<vector<int>> g(n + 1);
    for (int i = 0; i < m; i++) { int u, v; cin >> u >> v;
g[u].push_back(v);
        g[v].push_back(u); }
    vector<char> cur(n + 1, 0), nxt;
    cur[1] = 1;
    for (int t = 0; t <= n * P; t++) {          // mô phỏng từng phút: tập
nút
        có thể đang đứng
        if (cur[n]) { printf("%d\n", t); return 0; }
        nxt = cur;
        for (int u = 1; u <= n; u++)
            if (cur[u] && s[u][t % P] == '1') for (int v : g[u]) nxt[v] = 1;
        cur = nxt;
    }
    printf("-1\n");
    return 0;
}
```

Độ phức tạp: mỗi phút O(n + m), số phút tới n · P: O(n · P · (n + m)).

3. Bế tắc

Mô phỏng chạy tốt khi n nhỏ (Subtask 2) hoặc khi đáp số nhỏ, nhưng với $n = 10^4$, P = 60 và đường đi dài thì quá chậm: **60 điểm**.

Một lời giải nhanh nhưng sai: BFS thường trên các **nút**, đánh dấu nút đã thăm theo lần tới đầu tiên, chờ đèn xanh rồi đi tiếp. Sai ở chỗ "lần tới đầu tiên" theo thứ tự BFS chưa chắc là **thời điểm** sớm nhất (vì thời gian chờ khác nhau ở mỗi nút): **60 điểm**.

> **ĐIỂM BẾ TẮC**
>
> Thời gian đã trở thành một phần của "vị trí". Cần một đồ thị mà mỗi bước đi (kể cả bước chờ) đều tốn đúng 1 phút.

4. Phá bế tắc

Đồ thị trạng thái: từ (u, r) có thể

- **chờ**: sang (u, (r + 1) mod P);
- **đi** sang (v, (r + 1) mod P) với mọi v kề u, nếu $s_u[r] = 1$ (đèn xanh).

Mọi bước tốn 1 phút, nên BFS từ (1, 0) cho thời điểm sớm nhất tới mỗi trạng thái. Trạng thái đầu tiên có u = n được lấy ra cho đáp số.

Phút	Trạng thái lấy ra	Đi tới
0	(1, 0)	đèn 1 xanh: (2, 1), (3, 1); chờ: (1, 1)
1	(2, 1), (3, 1), (1, 1)	đèn 2, 3 đỏ, đèn 1 đỏ: chỉ chờ, (2, 2), (3, 2), (1, 2)
2	(3, 2), ...	đèn 3 xanh: (4, 0)
3	(4, 0)	đích: **3**

> **MỞ RỘNG: DIJKSTRA THEO THỜI ĐIỂM TỚI**
>
> Vì được chờ tùy ý, tới sớm không bao giờ thiệt (giống bài 5.3). Có thể chạy Dijkstra trên các nút với nhãn "thời điểm tới sớm nhất", mỗi lần rời u thì chờ tới phút xanh gần nhất. Cách nhìn đồ thị trạng thái tổng quát hơn: nó vẫn dùng được khi **không** được chờ ở một số nút.

5. Cách tối ưu

DENGIAOTHONG.CPP: BFS trên trạng thái (nút, t mod P)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DENGIAOTHONG.INP", "r")) {
        freopen("DENGIAOTHONG.INP", "r", stdin);
        freopen("DENGIAOTHONG.OUT", "w", stdout);
    }
    int n, m, P;
    cin >> n >> m >> P;
    vector<string> s(n + 1);
    for (int i = 1; i <= n; i++) cin >> s[i];
    vector<vector<int>> g(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        g[u].push_back(v); g[v].push_back(u);
    }
    // trạng thái (u, r): đang ở nút u, thời điểm t có t mod P = r; mã hóa thành u * P + r
    vector<int> d((size_t)(n + 1) * P, -1);
    queue<pair<int,int>> q;
    d[1 * P + 0] = 0;
    q.push({1, 0});
    while (!q.empty()) {
        auto [u, r] = q.front(); q.pop();
        int t = d[u * P + r];
        if (u == n) { printf("%d\n", t); return 0; }
        int r2 = (r + 1) % P;
        auto go = [&](int v) {                  // sang trạng thái (v, r2) ở thời điểm t + 1
            if (d[v * P + r2] < 0) { d[v * P + r2] = t + 1; q.push({v, r2}); }
        };
        go(u);                                  // đứng chờ 1 phút
        if (s[u][r] == '1')                     // đèn xanh: được rời nút
            for (int v : g[u]) go(v);
    }
    printf("-1\n");
    return 0;
}
```

Độ phức tạp: $O(P \cdot (n + m)) \approx 60 \cdot 3 \cdot 10^4 = 1{,}8 \cdot 10^6$.

6. Bẫy thường gặp

- **Đánh dấu theo nút** thay vì theo trạng thái.
- **Quên bước "chờ"**: khi mọi nút kề đều đang đỏ thì xe kẹt mãi.
- **n = 1:** đáp số là 0.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
BFS trên nút, đánh dấu theo nút	30	30	0	**60**

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
Mô phỏng từng phút	30	30	0	**60**
BFS trên trạng thái (nút, t mod P)	30	30	40	**100**

Chấm trên 10 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

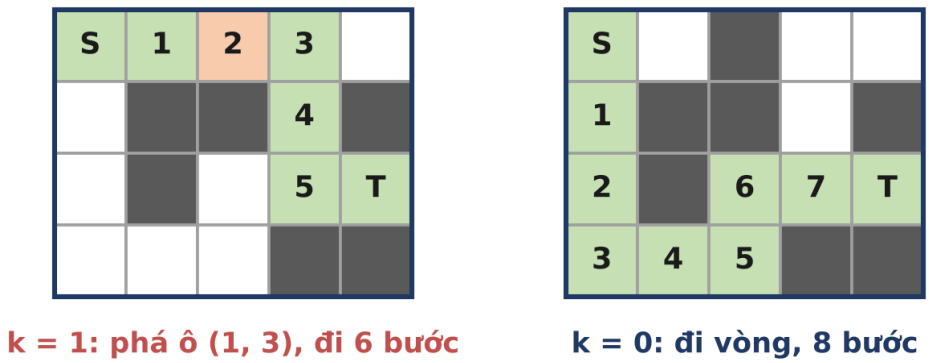
BÀI 8.2 PHÁ TƯỜNG

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
PHATUONG.INP	PHATUONG.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Tập 1, Bài 8 · Đề bài: Tập 2, trang 62

1. Hiểu bài toán

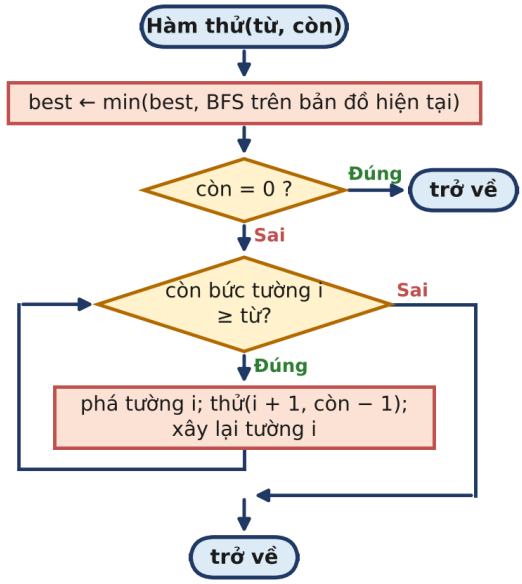
Mê cung có thêm "tài nguyên": tối đa k lần phá tường. Hai lần có mặt ở cùng một ô nhưng còn số búa khác nhau là **hai tình huống khác nhau**: lần tới sau có thể đi được những nơi lần tới trước không đi được.



Hình 8.2.1. *Có búa đi thẳng 6 bước; không có búa phải đi vòng 8 bước.*

2. Cách ngây thơ: Thử mọi tập tường bị phá

Chọn trước tập không quá k bức tường sẽ phá, xóa chúng khỏi bản đồ rồi BFS thường. Thử mọi tập.



Hình 8.2.2. *Sơ đồ khối của cách thử mọi tập tường.*

Cách ngây thơ: thử mọi tập ≤ k tường

```cpp
#include <bits/stdc++.h>
using namespace std;

int R, C, K, best = INT_MAX;
vector<string> a;
vector<pair<int,int>> walls;
const int dx[4] = {-1, 1, 0, 0}, dy[4] = {0, 0, -1, 1};

int bfs() {                                     // BFS thường trên bản đồ hiện tại
    vector<vector<int>> d(R, vector<int>(C, -1));
    queue<pair<int,int>> q;
    for (int i = 0; i < R; i++) for (int j = 0; j < C; j++) if (a[i][j] == 'S') { d[i][j] = 0; q.push({i, j}); }
    while (!q.empty()) {
        auto [x, y] = q.front(); q.pop();
        if (a[x][y] == 'T') return d[x][y];
        for (int t = 0; t < 4; t++) {
            int u = x + dx[t], v = y + dy[t];
            if (u >= 0 && u < R && v >= 0 && v < C && a[u][v] != '#' && d[u][v] < 0) { d[u][v] = d[x][y] + 1; q.push({u, v}); }
        }
    }
    return INT_MAX;
}

void tryBreak(int from, int left) {             // thử mọi tập không quá K bức tường
    best = min(best, bfs());
    if (left == 0) return;
    for (int i = from; i < (int)walls.size(); i++) {
        auto [x, y] = walls[i];
        a[x][y] = '.'; tryBreak(i + 1, left - 1); a[x][y] = '#';
    }
}

int main() {
    if (fopen("PHATUONG.INP", "r")) {
        freopen("PHATUONG.INP", "r", stdin);
        freopen("PHATUONG.OUT", "w", stdout);
    }
    cin >> R >> C >> K;
    a.resize(R);
    for (auto &s : a) cin >> s;
    for (int i = 0; i < R; i++) for (int j = 0; j < C; j++) if (a[i][j] == '#') walls.push_back({i, j});
    tryBreak(0, K);
    printf("%d\n", best == INT_MAX ? -1 : best);
    return 0;
}
```

Độ phức tạp: $O(W^k \cdot R \cdot C)$ với W là số bức tường: k = 0 hoặc 1 thì chạy được, k ≥ 2 thì bùng nổ.

3. Bế tắc

- **Subtask 1** (k = 0) và **Subtask 2** (k = 1, tối đa 10^4 lần BFS 10^4 ô): chạy kịp.
- **Subtask 3** (k tới 10): quá thời gian.

Được **60 điểm**. Một lời giải nhanh nhưng sai: BFS thường, đánh dấu theo **ô**, nhớ số búa đã dùng của lần tới đầu tiên. Khi đường ngắn tới một ô đã dùng hết búa còn đường dài hơn vẫn còn búa, lời giải này bỏ mất đường dài: **30 điểm**.

> **ĐIỂM BẾ TẮC**
>
> "Đã thăm ô (x, y)" là chưa đủ: phải nói rõ đã thăm (x, y) **với bao nhiêu búa đã dùng**.

4. Phá bế tắc

Trạng thái (x, y, j): đang ở ô (x, y), đã phá j bức tường, $0 \le j \le k$. Có $R \cdot C \cdot (k + 1) \le 1{,}1 \cdot 10^5$ trạng thái. Bước vào ô trống: giữ nguyên j; bước vào ô tường: j tăng 1 (chỉ khi $j < k$). Mọi bước tốn 1 nên dùng BFS; trạng thái đầu tiên ở ô T là đáp số.

Ở ví dụ 1 ($k = 1$): (1, 1, 0) → (1, 2, 0) → phá tường (1, 3, 1) → (1, 4, 1) → (2, 4, 1) → (3, 4, 1) → (3, 5, 1) = T sau **6** bước. Với $k = 0$, trạng thái (1, 3, 1) không được phép, BFS phải đi vòng: **8** bước.

5. Cách tối ưu

PHATUONG.CPP: BFS trên trạng thái (ô, số tường đã phá)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("PHATUONG.INP", "r")) {
        freopen("PHATUONG.INP", "r", stdin);
        freopen("PHATUONG.OUT", "w", stdout);
    }
    int R, C, K;
    cin >> R >> C >> K;
    vector<string> a(R);
    for (auto &s : a) cin >> s;
    const int dx[4] = {-1, 1, 0, 0}, dy[4] = {0, 0, -1, 1};
    // trạng thái (x, y, k): ở ô (x, y), đã phá k bức tường
    vector<int> d((size_t)R * C * (K + 1), -1);
    auto id = [&](int x, int y, int k) { return (x * C + y) * (K + 1) +
k; };
    queue<array<int,3>> q;
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++)
            if (a[i][j] == 'S') { d[id(i, j, 0)] = 0; q.push({i, j, 0}); }
    while (!q.empty()) {
        auto [x, y, k] = q.front(); q.pop();
        if (a[x][y] == 'T') { printf("%d\n", d[id(x, y, k)]); return 0; }
        for (int t = 0; t < 4; t++) {
            int u = x + dx[t], v = y + dy[t];
            if (u < 0 || u >= R || v < 0 || v >= C) continue;
            int k2 = k + (a[u][v] == '#');      // bước vào tường: dùng thêm
                một lần búa
            if (k2 > K || d[id(u, v, k2)] >= 0) continue;
            d[id(u, v, k2)] = d[id(x, y, k)] + 1;
            q.push({u, v, k2});
        }
    }
    printf("-1\n");
    return 0;
}
```

Độ phức tạp: $O(R \cdot C \cdot k)$.

6. Bẫy thường gặp

- **Đánh dấu theo ô** (mất điểm ở những test có "búa quý").
- **Coi ô S, T là tường** hoặc quên chúng là ô trống.
- **Phá tường mà không tính bước:** bước vào ô vừa phá cũng tốn 1 bước.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
BFS thường, đánh dấu theo ô	30	0	0	**30**
Thử mọi tập ≤ k tường + BFS	30	30	0	**60**
BFS trên trạng thái (ô, số tường đã phá)	30	30	40	**100**

Chấm trên 12 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

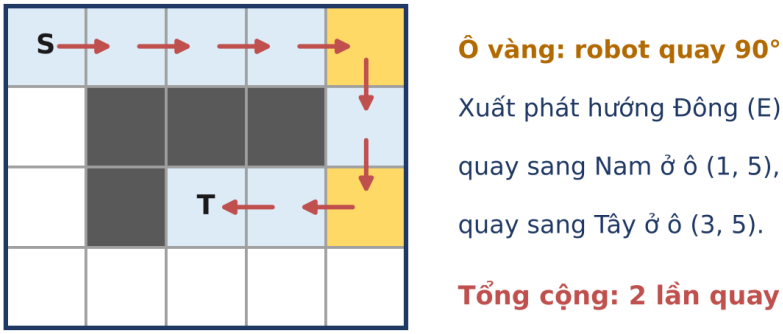
BÀI 8.3 ROBOT XOAY

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
ROBOTXOAY.INP	ROBOTXOAY.OUT	2 giây	256 MB	★★★ NÂNG CAO

Liên hệ: Tập 1, Bài 8 · Đề bài: Tập 2, trang 64

1. Hiểu bài toán

Trạng thái của robot gồm ô đang đứng và **hướng** đang quay mặt: (x, y, k) với k ∈ {N, E, S, W}, tổng cộng $4 \cdot R \cdot C \le 4 \cdot 10^6$ trạng thái. Có hai loại bước: tiến (miễn phí) và xoay (tốn 1). Ta cần đường đi có **tổng chi phí** nhỏ nhất, không phải ít bước nhất.



Hình 8.3.1. *Ví dụ: hai lần xoay ở các ô vàng.*

2. Cách ngây thơ: BFS thường trên trạng thái

BFS coi mọi thao tác (tiến và xoay) đều tốn 1, rồi in số lần xoay trên đường ít **thao tác** nhất tìm được.

Cách sai: BFS coi mọi thao tác như nhau

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("ROBOTXOAY.INP", "r")) {
        freopen("ROBOTXOAY.INP", "r", stdin);
        freopen("ROBOTXOAY.OUT", "w", stdout);
    }
    int R, C; char h;
    cin >> R >> C >> h;
    vector<string> a(R);
    for (auto &s : a) cin >> s;
    const int dx[4] = {-1, 0, 1, 0}, dy[4] = {0, 1, 0, -1};
    string H = "NESW";
    // LỖI: BFS thường coi MỌI thao tác (kể cả tiến) đều tốn 1, rồi in số lần
        xoay của đường ít thao tác nhất
    vector<int> d((size_t)R * C * 4, -1), turns((size_t)R * C * 4, 0);
    queue<array<int,3>> q;
    for (int i = 0; i < R; i++) for (int j = 0; j < C; j++) if (a[i][j] ==
        'S') { int k = H.find(h); d[(i * C + j) * 4 + k] = 0; q.push({i, j,
        k}); }
    while (!q.empty()) {
        auto [x, y, k] = q.front(); q.pop();
        int id = (x * C + y) * 4 + k;
        if (a[x][y] == 'T') { printf("%d\n", turns[id]); return 0; }
        int u = x + dx[k], v = y + dy[k];
        if (u >= 0 && u < R && v >= 0 && v < C && a[u][v] != '#' && d[(u * C +
            v) * 4 + k] < 0) {
            d[(u * C + v) * 4 + k] = d[id] + 1; turns[(u * C + v) * 4 + k] =
                turns[id]; q.push({u, v, k});
        }
        for (int t : {(k + 1) % 4, (k + 3) % 4})
            if (d[(x * C + y) * 4 + t] < 0) { d[(x * C + y) * 4 + t] = d[id] +
                1; turns[(x * C + y) * 4 + t] = turns[id] + 1; q.push({x, y,
                t}); }
    }
    printf("-1\n");
    return 0;
}
```

Độ phức tạp: $O(R \cdot C)$, nhanh nhưng tối ưu sai đại lượng.

3. Bế tắc

Đường ít thao tác nhất thường không phải đường ít lần xoay nhất (một đường thẳng dài chỉ tốn 0 nhưng có nhiều bước tiến): **30 điểm**.

Sửa bằng Dijkstra trên đồ thị trạng thái với trọng số 0/1: đúng, và với giới hạn 2 giây vẫn chạy kịp trên bộ test của sách: **100 điểm**. Tuy nhiên hàng đợi ưu tiên với $4 \cdot 10^6$ trạng thái khá nặng.

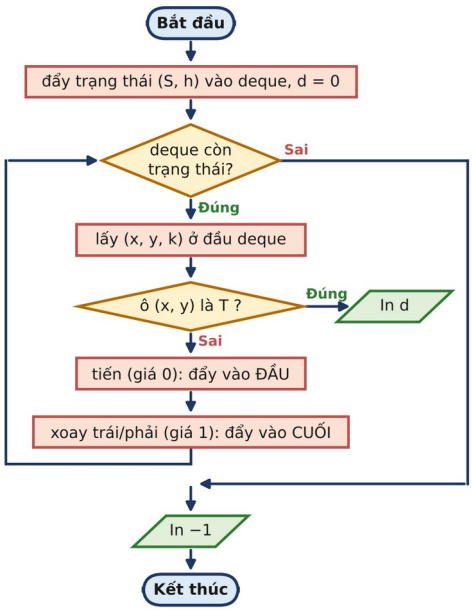
ĐIỂM BẾ TẮC

Trọng số chỉ là 0 hoặc 1: hàng đợi ưu tiên "quá mạnh" cho bài này. Có cấu trúc nào đơn giản như BFS mà vẫn xử lí được cạnh trọng số 0?

4. Phá bế tắc

BFS 0-1 (Tập 1, Bài 8): dùng deque. Cạnh trọng số 0 (tiến) đẩy trạng thái mới vào **đầu**, cạnh trọng số 1 (xoay) đẩy vào **cuối**. Hàng đợi khi đó luôn chứa các nhãn thuộc {d, d + 1}, đóng vai trò hàng đợi ưu tiên với chi phí O(1) mỗi thao tác.

Lấy ra (ví dụ)	Chi phí	Đẩy vào
(1, 1, E)	0	tiến (1, 2, E) vào đầu; xoay (1, 1, N), (1, 1, S) vào cuối
(1, 2, E) ... (1, 5, E)	0	tiến tới cuối hàng; xoay ở (1, 5)
(1, 5, S)	1	tiến (2, 5, S), (3, 5, S) vào đầu
(3, 5, W)	2	tiến (3, 4, W), (3, 3, W) = T: **2**



Hình 8.3.2. *Sơ đồ khối của BFS 0-1.*

5. Cách tối ưu

ROBOTXOAY.CPP: BFS 0-1 trên trạng thái (ô, hướng)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("ROBOTXOAY.INP", "r")) {
        freopen("ROBOTXOAY.INP", "r", stdin);
        freopen("ROBOTXOAY.OUT", "w", stdout);
    }
    int R, C; char h;
    cin >> R >> C >> h;
    vector<string> a(R);
    for (auto &s : a) cin >> s;
    const int dx[4] = {-1, 0, 1, 0}, dy[4] = {0, 1, 0, -1};   // N E S W
    const string H = "NESW";
    // trạng thái (x, y, k): ở ô (x, y), quay mặt hướng k; mã hóa (x * C + y)
        * 4 + k
    vector<int> d((size_t)R * C * 4, INT_MAX);
    deque<int> q;
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++)
            if (a[i][j] == 'S') {
                int id = (i * C + j) * 4 + H.find(h);
                d[id] = 0; q.push_back(id);
            }
    while (!q.empty()) {
        int id = q.front(); q.pop_front();
        int k = id % 4, x = id / 4 / C, y = id / 4 % C, c = d[id];
        if (a[x][y] == 'T') { printf("%d\n", c); return 0; }
        int u = x + dx[k], v = y + dy[k];
        if (u >= 0 && u < R && v >= 0 && v < C && a[u][v] != '#') {
            int id2 = (u * C + v) * 4 + k;      // tiến: trọng số 0, đẩy vào ĐẦU
            if (c < d[id2]) { d[id2] = c; q.push_front(id2); }
        }
        for (int t : {(k + 1) % 4, (k + 3) % 4}) {
            int id2 = (x * C + y) * 4 + t;      // xoay: trọng số 1, đẩy vào CUỐI
            if (c + 1 < d[id2]) { d[id2] = c + 1; q.push_back(id2); }
        }
    }
    printf("-1\n");
    return 0;
}
```

Độ phức tạp: $O(R \cdot C)$, mỗi trạng thái được lấy ra một số ít lần.

6. Bẫy thường gặp

- **BFS thường** tối ưu số thao tác thay vì số lần xoay.
- **Đẩy cạnh 0 vào cuối deque:** thứ tự bị phá vỡ, kết quả sai.
- **Bộ nhớ:** $4 \cdot 10^6$ trạng thái; dùng mảng một chiều, mã hóa (x, y, k) thành một số.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (70)	Tổng
BFS thường (mọi thao tác như nhau)	30	0	**30**
Dijkstra trên trạng thái	30	70	**100**
BFS 0-1 trên trạng thái	30	70	**100**

Chấm trên 7 test, giới hạn 2 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

CHƯƠNG 3. CÂY VÀ CÂY KHUNG

Các bài tập chương 3 xoay quanh ba công cụ: DSU, cây khung nhỏ nhất và nhảy nhị phân trên cây. Cách ngây thơ thường là "làm lại từ đầu sau mỗi thay đổi" hoặc "duyệt lại cây cho mỗi câu hỏi". Những lời giải "trông như đúng" thì hay mất điểm vì DSU không cân bằng, vì quên một loại thông tin (cáp có sẵn, quan hệ khác phe), vì đệ quy quá sâu hoặc vì bảng nhảy nhị phân chiếm quá nhiều bộ nhớ.

Bài	Tên bài	Mức	Cách ngây thơ (điểm)	Chìa khóa của lời giải
9.1	**MẠNG BẠN BÈ**	★	Duyệt lại cả đồ thị sau mỗi sự kiện (40)	DSU nén đường đi + gộp theo kích thước, duy trì cnt và best.
9.2	**PHÁ CẦU**	★★	Dựng lại DSU sau mỗi lần sập (30)	Đi ngược thời gian: phá cầu thành xây cầu.
9.3	**HAI PHE**	★★★	Tô màu lại sau mỗi mẩu tin (30)	DSU có chẵn lẻ (hoặc DSU trên 2n đỉnh).
10.1	**NỐI MẠNG TRƯỜNG HỌC**	★	Prim $O(n^2)$ (30)	Gộp cáp có sẵn vào DSU rồi chạy Kruskal.
10.2	**ĐƯỜNG ÍT DỐC**	★★	Kruskal lại cho mỗi câu hỏi (30)	Cây khung nhỏ nhất + cạnh lớn nhất trên đường đi (nhảy nhị phân).
10.3	**GIẾNG VÀ ỐNG**	★★★	Thử mọi tập giếng và ống (20)	Đỉnh ảo "nguồn nước" + Kruskal.
11.1	**HỌ HÀNG**	★	Leo từng bậc (40)	LCA nhảy nhị phân, dựng bảng bằng BFS.
11.2	**CÁP QUANG YẾU NHẤT**	★★	DFS cho mỗi câu hỏi (30)	Nhảy nhị phân kèm min trên đoạn.
11.3	**NHÂN VIÊN CẤP K**	★★★	Leo từng bậc (20)	Trả lời ngoại tuyến: DFS với ngăn xếp đường đi.

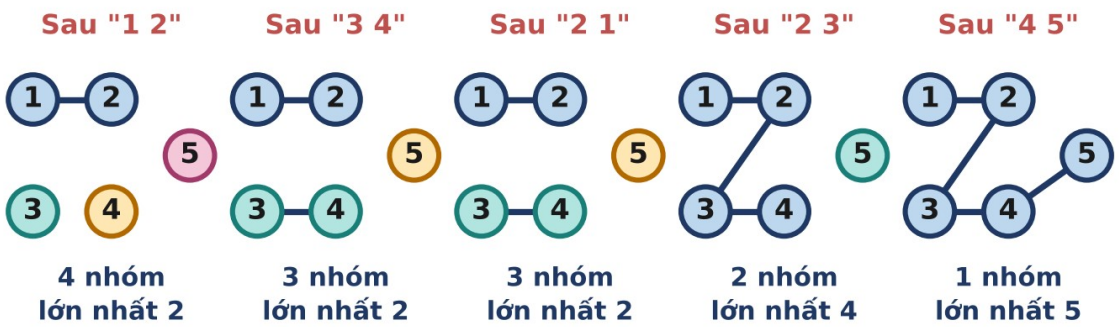
BÀI 9.1 MẠNG BẠN BÈ

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
`MANGBANBE.INP`	`MANGBANBE.OUT`	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Tập 1, Bài 9 · Đề bài: Tập 2, trang 67

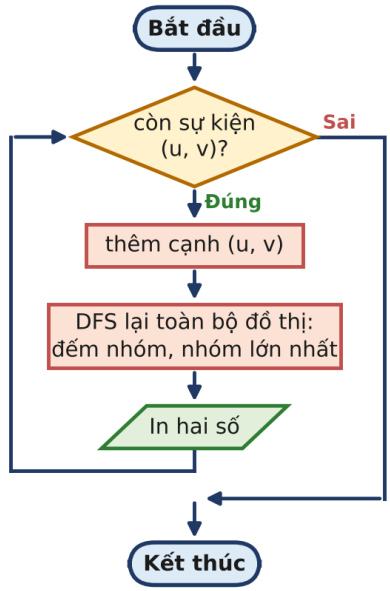
1. Hiểu bài toán

Mỗi lần kết bạn là thêm một cạnh; nhóm bạn là thành phần liên thông. Sau mỗi cạnh mới, cần số thành phần và kích thước thành phần lớn nhất. Các nhóm chỉ **lớn dần**, không bao giờ tách ra: đúng sở trường của DSU (Tập 1, Bài 9).



Hình 9.1.1. *Các nhóm bạn của ví dụ sau từng sự kiện.*

2. Cách ngây thơ: Duyệt lại cả đồ thị sau mỗi sự kiện



Hình 9.1.2. *Sơ đồ khối của cách duyệt lại.*

Cách ngây thơ: DFS lại toàn bộ sau mỗi sự kiện

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("MANGBANBE.INP", "r")) {
        freopen("MANGBANBE.INP", "r", stdin);
        freopen("MANGBANBE.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<vector<int>> g(n + 1);
    while (m--) {
        int u, v;
        scanf("%d %d", &u, &v);
        g[u].push_back(v); g[v].push_back(u);
        vector<char> seen(n + 1, 0);            // đếm lại các nhóm từ đầu: O(n + m)
        int cnt = 0, best = 0;
        for (int s = 1; s <= n; s++) {
            if (seen[s]) continue;
            cnt++; int sz = 0;
            vector<int> st = {s}; seen[s] = 1;
            while (!st.empty()) {
                int x = st.back(); st.pop_back(); sz++;
                for (int y : g[x]) if (!seen[y]) { seen[y] = 1; st.push_back(y); }
            }
            best = max(best, sz);
        }
        printf("%d %d\n", cnt, best);
    }
    return 0;
}
```

Độ phức tạp: $O(m \cdot (n + m))$. Với $n = m = 1000$ khoảng $2 \cdot 10^6$; với $2 \cdot 10^5$ là $8 \cdot 10^{10}$.

3. Bế tắc

- **Subtask 1** ($n, m \le 1000$): **40 điểm**.
- **Subtask 2**: quá thời gian.

Chuyển sang DSU là đúng hướng, nhưng DSU **không cân bằng** (gộp tùy tiện, không nén đường đi) có thể suy biến thành một chuỗi dài. Với dãy sự kiện (1, 2), (1, 3), (1, 4), ..., mỗi lần gộp lại treo cả chuỗi vào một đỉnh mới, và `root(1)` phải đi hết chuỗi: tổng $O(n^2)$. Được **40 điểm**.

ĐIỂM BẾ TẮC

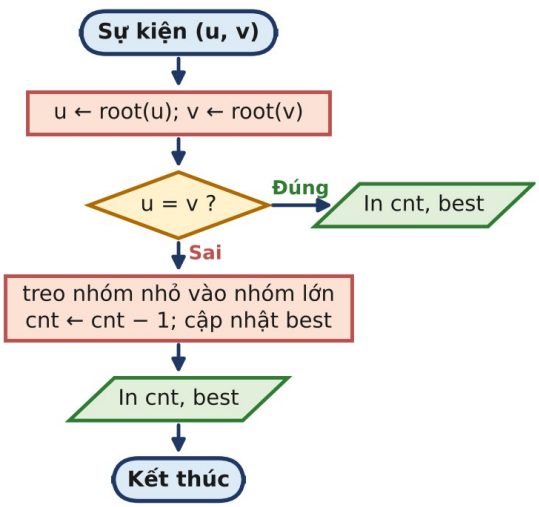
DSU chỉ nhanh khi cây của nó **thấp**. Cần hai kĩ thuật giữ cây thấp: gộp cây nhỏ vào cây lớn, và nén đường đi khi tìm gốc.

4. Phá bế tắc

- **Gộp theo kích thước:** luôn treo gốc của nhóm nhỏ vào gốc của nhóm lớn. Một đỉnh chỉ bị "đẩy sâu" thêm 1 khi nhóm của nó ít nhất gấp đôi, nên độ sâu không quá $\log_2 n$.
- **Nén đường đi:** khi tìm gốc, cho mỗi đỉnh trên đường trỏ thẳng (hoặc nhảy hai bậc) lên trên. Kết hợp hai kĩ thuật, mỗi thao tác gần như O(1).

Giữ thêm hai biến: cnt (số nhóm, ban đầu n, giảm 1 mỗi lần gộp **thành công**) và best (kích thước lớn nhất; vì nhóm không bao giờ nhỏ đi nên chỉ cần max với kích thước nhóm vừa gộp).

Sự kiện	Gộp	cnt	best
1 2	{1} + {2}	4	2
3 4	{3} + {4}	3	2
2 1	đã cùng nhóm	3	2
2 3	{1, 2} + {3, 4}	2	4
4 5	{1, 2, 3, 4} + {5}	1	5



Hình 9.1.3. *Sơ đồ khối xử lí một sự kiện.*

5. Cách tối ưu

MANGBANBE.CPP: DSU cân bằng

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> p, sz;

int root(int x) {                                   // nén đường đi kiểu "nhảy hai bậc"
    while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; }
    return x;
}

int main() {
    if (fopen("MANGBANBE.INP", "r")) {
        freopen("MANGBANBE.INP", "r", stdin);
        freopen("MANGBANBE.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    p.resize(n + 1); sz.assign(n + 1, 1);
    iota(p.begin(), p.end(), 0);
    int cnt = n, best = 1;                          // số nhóm, kích thước nhóm lớn nhất
    while (m--) {
        int u, v;
        scanf("%d %d", &u, &v);
        u = root(u); v = root(v);
        if (u != v) {
            if (sz[u] < sz[v]) swap(u, v);          // gộp nhóm nhỏ vào nhóm lớn
            p[v] = u; sz[u] += sz[v];
            cnt--;
            best = max(best, sz[u]);
        }
        printf("%d %d\n", cnt, best);
    }
    return 0;
}
```

Độ phức tạp: $O((n + m) \cdot \alpha(n))$, gần như tuyến tính.

6. Bẫy thường gặp

- **Giảm cnt cả khi hai bạn đã cùng nhóm** (sự kiện lặp).
- **DSU đệ quy** `root(x) = p[x] = root(p[x])` khi chưa gộp theo kích thước: chuỗi dài làm tràn ngăn xếp.
- **Quên cập nhật best** sau khi gộp.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (40)	ST2 (60)	Tổng
Duyệt lại cả đồ thị sau mỗi sự kiện	40	0	**40**
DSU không cân bằng, không nén	40	0	**40**
DSU nén đường đi + gộp theo kích thước	40	60	**100**

Chấm trên 6 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

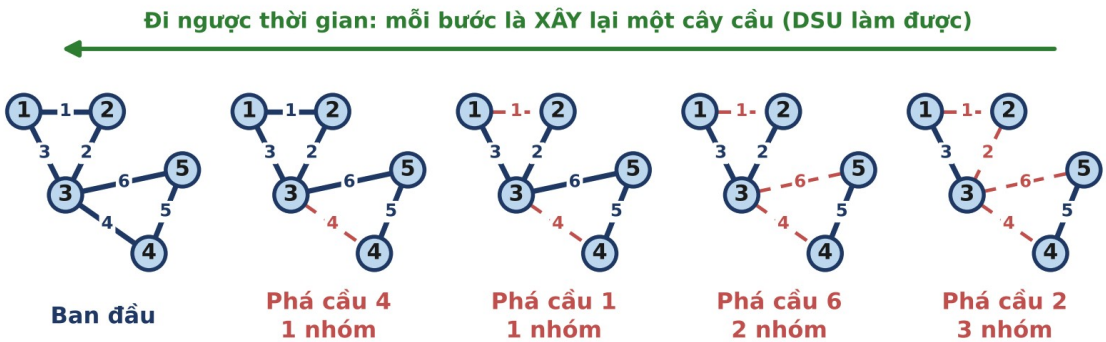
BÀI 9.2 PHÁ CẦU

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
PHACAU.INP	PHACAU.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Tập 1, Bài 9 · Đề bài: Tập 2, trang 70

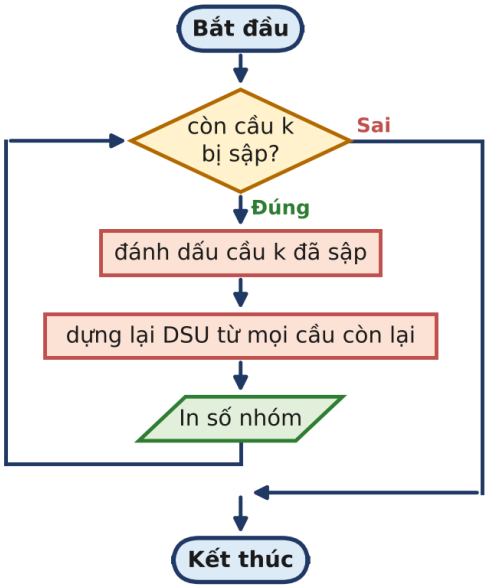
1. Hiểu bài toán

Các cây cầu lần lượt **biến mất**, ta cần số thành phần liên thông sau mỗi lần. DSU chỉ gộp được, không tách được. Nhưng nếu nhìn **ngược thời gian**, các cây cầu lần lượt **xuất hiện**: đúng việc DSU làm tốt.



Hình 9.2.1. *Ví dụ: đi ngược từ phải sang trái là xây lại từng cây cầu.*

2. Cách ngây thơ: Dựng lại DSU sau mỗi lần sập



Hình 9.2.2. *Sơ đồ khối của cách dựng lại.*

Cách ngây thơ: dựng lại DSU từ các cầu còn lại

```
#include <bits/stdc++.h>
using namespace std;

vector<int> p;
int root(int x) { while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; } return
x; }

int main() {
    if (fopen("PHACAU.INP", "r")) {
        freopen("PHACAU.INP", "r", stdin);
        freopen("PHACAU.OUT", "w", stdout);
    }
    int n, m, q;
    scanf("%d %d %d", &n, &m, &q);
    vector<int> U(m + 1), V(m + 1);
    for (int i = 1; i <= m; i++) scanf("%d %d", &U[i], &V[i]);
    vector<char> alive(m + 1, 1);
    while (q--) {
        int k;
        scanf("%d", &k);
        alive[k] = 0;
        p.resize(n + 1); iota(p.begin(), p.end(), 0);   // dựng lại DSU từ
            đầu: O(n + m)
        int cnt = n;
        for (int i = 1; i <= m; i++)
            if (alive[i]) { int a = root(U[i]), b = root(V[i]); if (a != b)
{
                    p[a] = b; cnt--; } }
        printf("%d\n", cnt);
    }
    return 0;
}
```

Độ phức tạp: $O(q \cdot (n + m))$.

3. Bế tắc

- **Subtask 1** ($n, m \le 1000$): **30 điểm**.
- **Subtask 2**: $2 \cdot 10^5$ lần dựng lại, mỗi lần $4 \cdot 10^5$ thao tác: quá thời gian.

ĐIỂM BẾ TẮC

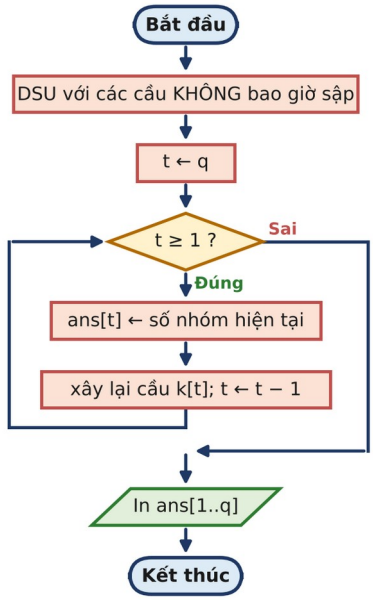
Phải "tách" một nhóm khi cầu sập, mà DSU không làm được điều đó. Muốn dùng DSU, cần đổi bài toán để chỉ còn thao tác **gộp**.

4. Phá bế tắc

Ta biết **trước** toàn bộ danh sách cầu sẽ sập (bài toán **ngoại tuyến**). Bắt đầu từ trạng thái **cuối cùng** (mọi cầu trong danh sách đã sập), rồi đi ngược: với $j = q, q - 1, ..., 1$, ghi lại số nhóm hiện tại làm đáp số thứ j (đó là trạng thái ngay **sau** khi cầu k_j sập), rồi **xây lại** cầu k_j.

Bước ngược	Ghi đáp số	Xây lại	Các nhóm sau đó
khởi đầu		cầu 3, 5 (không bao giờ sập)	{1, 3}, {2}, {4, 5}
$j = 4$	ans[4] = **3**	cầu 2 (2 – 3)	{1, 2, 3}, {4, 5}
$j = 3$	ans[3] = **2**	cầu 6 (5 – 3)	{1, 2, 3, 4, 5}
$j = 2$	ans[2] = **1**	cầu 1	không đổi
$j = 1$	ans[1] = **1**	cầu 4	không đổi

In theo thứ tự $j = 1..q$: 1, 1, 2, 3, khớp với ví dụ.



Hình 9.2.3. *Sơ đồ khối của lời giải ngược thời gian.*

5. Cách tối ưu

PHACAU.CPP: DSU, đi ngược thời gian

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> p;
int cnt;                                        // số nhóm hiện tại

int root(int x) {
    while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; }
    return x;
}
void unite(int a, int b) {
    a = root(a); b = root(b);
    if (a != b) { p[a] = b; cnt--; }
}

int main() {
    if (fopen("PHACAU.INP", "r")) {
        freopen("PHACAU.INP", "r", stdin);
        freopen("PHACAU.OUT", "w", stdout);
    }
    int n, m, q;
    scanf("%d %d %d", &n, &m, &q);
    vector<int> U(m + 1), V(m + 1), k(q);
    vector<char> del(m + 1, 0);
    for (int i = 1; i <= m; i++) scanf("%d %d", &U[i], &V[i]);
    for (auto &x : k) { scanf("%d", &x); del[x] = 1; }
    p.resize(n + 1); iota(p.begin(), p.end(), 0);
    cnt = n;
    // Trạng thái cuối: chỉ còn các cầu không bao giờ sập
    for (int i = 1; i <= m; i++) if (!del[i]) unite(U[i], V[i]);
    // Đi ngược thời gian: ghi đáp số rồi XÂY lại cây cầu
    vector<int> ans(q);
    for (int t = q - 1; t >= 0; t--) {
        ans[t] = cnt;                           // số nhóm ngay sau khi cầu
            k[t] sập
        unite(U[k[t]], V[k[t]]);
    }
    for (int x : ans) printf("%d\n", x);
    return 0;
}
```

Độ phức tạp: $O((n + m + q) \cdot \alpha(n))$.

6. Bẫy thường gặp

- **Ghi đáp số sau khi xây lại** (lệch một bước).
- **Quên các cầu không bao giờ sập** ở trạng thái khởi đầu.
- **Áp dụng cho bài trực tuyến:** nếu câu hỏi phụ thuộc vào đáp số trước (dữ liệu được "mã hóa"), kĩ thuật này không dùng được.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (70)	Tổng
Dựng lại DSU sau mỗi lần sập	30	0	**30**

Thuật toán	ST1 (30)	ST2 (70)	Tổng
Đi ngược thời gian với DSU	30	70	**100**

Chấm trên 6 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

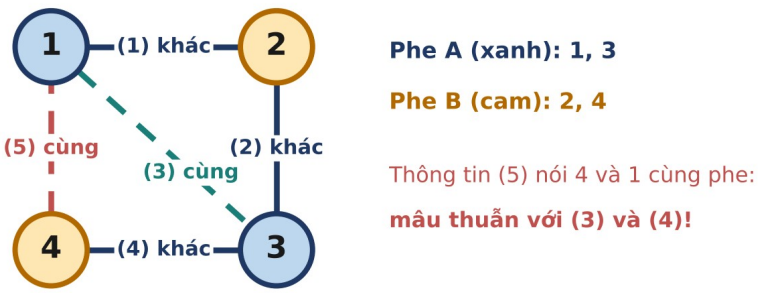
BÀI 9.3 HAI PHE

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
HAIPHE.INP	HAIPHE.OUT	1 giây	256 MB	★★★ NÂNG CAO

Liên hệ: Tập 1, Bài 9 · Đề bài: Tập 2, trang 72

1. Hiểu bài toán

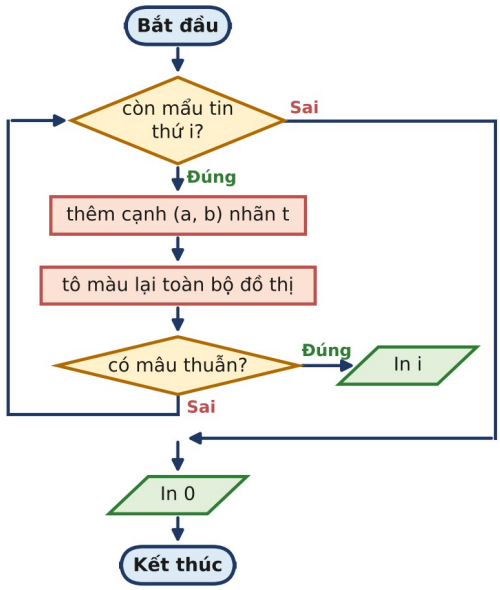
Mỗi mẩu tin cho biết **quan hệ** giữa hai người: cùng phe (0) hay khác phe (1). Quan hệ có thể suy ra theo kiểu "cộng mô-đun 2": a khác b, b khác c thì a cùng c. Thông tin đầu tiên mâu thuẫn là thông tin mà quan hệ nó nói trái với quan hệ đã suy ra được từ các thông tin trước.



Hình 9.3.1. *Ví dụ 1: mẩu tin thứ 5 mâu thuẫn.*

2. Cách ngây thơ: Tô màu lại sau mỗi mẩu tin

Coi mỗi mẩu tin là một cạnh có nhãn. Các thông tin phù hợp khi và chỉ khi tô được hai màu sao cho cạnh nhãn 0 nối hai đỉnh cùng màu, cạnh nhãn 1 nối hai đỉnh khác màu. Sau mỗi mẩu tin, tô màu lại toàn bộ.



Hình 9.3.2. *Sơ đồ khối của cách tô màu lại.*

Cách ngây thơ: tô màu lại sau mỗi mẩu tin

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("HAIPHE.INP", "r")) {
        freopen("HAIPHE.INP", "r", stdin);
        freopen("HAIPHE.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<vector<pair<int,int>>> g(n + 1);
    for (int i = 1; i <= m; i++) {
        int a, b, t;
        scanf("%d %d %d", &a, &b, &t);
        g[a].push_back({b, t}); g[b].push_back({a, t});
        // kiểm tra lại toàn bộ bằng cách tô màu: O(n + i)
        vector<int> col(n + 1, -1);
        for (int s = 1; s <= n; s++) {
            if (col[s] >= 0) continue;
            col[s] = 0; vector<int> st = {s};
            while (!st.empty()) {
                int u = st.back(); st.pop_back();
                for (auto [v, w] : g[u]) {
                    if (col[v] < 0) { col[v] = col[u] ^ w;
st.push_back(v); }
                    else if (col[v] != (col[u] ^ w)) { printf("%d\n", i);
                        return 0; }
                }
            }
        }
    }
    printf("0\n");
    return 0;
}
```

Độ phức tạp: $O(m \cdot (n + m))$.

3. Bế tắc

Chỉ chạy kịp Subtask 2 ($n, m \le 2000$): **30 điểm**.

Một "cải tiến" sai: dùng DSU thường, gộp những người **cùng phe**, và báo mâu thuẫn khi hai người "khác phe" đã cùng nhóm. Cách này bỏ qua hoàn toàn chuỗi suy luận "khác của khác là cùng": ở ví dụ 2 (1 khác 2, 2 khác 3, 1 khác 3) nó không phát hiện được gì: **0 điểm**.

ĐIỂM BẾ TẮC

DSU thường chỉ nhớ "cùng nhóm hay không". Ta cần nhớ thêm: trong cùng nhóm, hai người **cùng phe hay khác phe**.

4. Phá bế tắc

DSU có chẵn lẻ: ngoài p[x], lưu $w[x] \in \{0, 1\}$ là quan hệ giữa x và cha p[x]. Quan hệ giữa x và gốc là XOR các w trên đường đi; gọi là par(x). Hai người cùng gốc thì quan hệ giữa họ là par(a) XOR par(b).

- Mẩu tin (a, b, t), a và b cùng gốc: mâu thuẫn khi par(a) XOR par(b) ≠ t.
- Khác gốc ra, rb: treo ra vào rb với w[ra] = par(a) XOR par(b) XOR t (để quan hệ giữa a và b đúng bằng t).
- Nén đường đi: khi cho x trỏ thẳng lên gốc thì w[x] mới chính là par(x).

Chạy tay ví dụ 1:

Mẩu tin	Kiểm tra	Thao tác
(1, 2, khác)	gốc 1, gốc 2: khác gốc	p[1] = 2, w[1] = 1
(2, 3, khác)	gốc 2, gốc 3	p[2] = 3, w[2] = 1
(1, 3, cùng)	par(1) = 1 XOR 1 = 0, par(3) = 0; 0 XOR 0 = 0 = t	phù hợp
(3, 4, khác)	gốc 3, gốc 4	p[3] = 4, w[3] = 1
(4, 1, cùng)	par(4) = 0, par(1) = 0 XOR 1 = 1; 0 XOR 1 = 1 ≠ 0	**mâu thuẫn: in 5**

CÁCH KHÁC: DSU TRÊN 2n ĐỈNH

Tạo hai đỉnh cho mỗi người: "a ở phe A" và "a ở phe B". Tin "a khác b" gộp (a_A, b_B) và (a_B, b_A); tin "a cùng b" gộp (a_A, b_A) và (a_B, b_B). Có mâu thuẫn khi a_A và a_B rơi vào cùng một nhóm, tức là khi tin "khác" mà a_A, b_A đã cùng nhóm, hoặc tin "cùng" mà a_A, b_B đã cùng nhóm. Dễ cài hơn, cũng được **100 điểm**.

5. Cách tối ưu

HAIPHE.CPP: DSU có chẵn lẻ, nén đường đi không đệ quy

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> p, w;                                   // w[x] = 1 nếu x khác phe với
    p[x]

// trả về (gốc, chẵn lẻ từ x tới gốc); nén đường đi không đệ quy
pair<int,int> root(int x) {
    int r = x, par = 0;
    while (p[r] != r) { par ^= w[r]; r = p[r]; }
    int cur = x, acc = par;                         // acc: chẵn lẻ từ cur tới gốc
    while (p[cur] != cur) {
        int nx = p[cur], wc = w[cur];
        p[cur] = r; w[cur] = acc;                   // treo thẳng vào gốc
        acc ^= wc; cur = nx;
    }
    return {r, par};
}

int main() {
    if (fopen("HAIPHE.INP", "r")) {
        freopen("HAIPHE.INP", "r", stdin);
        freopen("HAIPHE.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    p.resize(n + 1); w.assign(n + 1, 0);
    iota(p.begin(), p.end(), 0);
    for (int i = 1; i <= m; i++) {
        int a, b, t;
        scanf("%d %d %d", &a, &b, &t);
        auto [ra, pa] = root(a);
        auto [rb, pb] = root(b);
        if (ra == rb) {                             // đã biết quan hệ giữa a và b
            if ((pa ^ pb) != t) { printf("%d\n", i); return 0; }
        } else {                                    // nối ra vào rb sao cho quan
            hệ a, b đúng bằng t
            p[ra] = rb;
            w[ra] = pa ^ pb ^ t;
        }
    }
    printf("0\n");
    return 0;
}
```

Độ phức tạp: O(m · log n) trong trường hợp xấu nhất (chỉ nén đường đi), thực tế gần như tuyến tính.

6. Bẫy thường gặp

- **Chỉ gộp các cặp "cùng phe"** (bỏ qua suy luận qua quan hệ "khác phe").
- **Cập nhật w sai khi nén đường đi:** w[x] mới phải là quan hệ giữa x và **gốc**, không phải giữa x và cha cũ.

- **DSU đệ quy** không gộp theo kích thước: chuỗi dài $2 \cdot 10^5$ có thể làm tràn ngăn xếp.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
Tô màu lại sau mỗi mẩu tin	0	30	0	**30**
DSU thường, chỉ gộp "cùng phe"	0	0	0	**0**
DSU trên 2n đỉnh	30	30	40	**100**
DSU có chẵn lẻ	30	30	40	**100**

Chấm trên 9 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

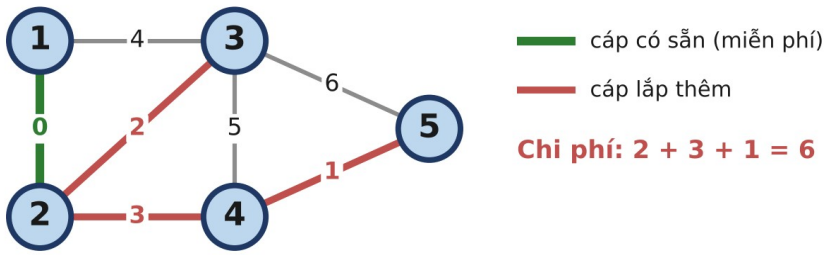
BÀI 10.1 NỐI MẠNG TRƯỜNG HỌC

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
NOIMANG.INP	NOIMANG.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Tập 1, Bài 10 · Đề bài: Tập 2, trang 74

1. Hiểu bài toán

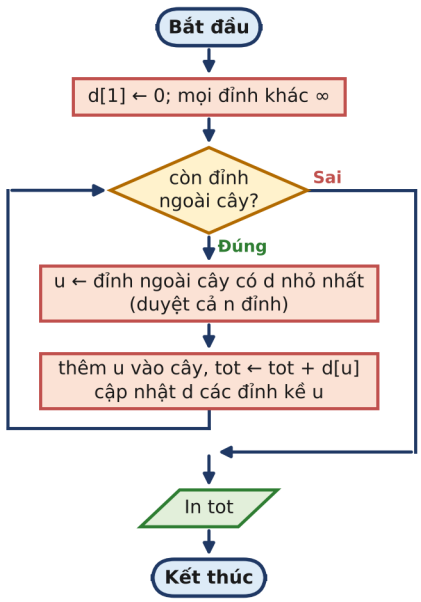
Cần một tập cáp lắp thêm rẻ nhất để toàn mạng liên thông, biết rằng một số cáp đã có sẵn (miễn phí). Đây là cây khung nhỏ nhất (Tập 1, Bài 10) với một số cạnh "bắt buộc" có giá 0.



Hình 10.1.1. *Ví dụ: cáp xanh có sẵn, cáp đỏ được lắp thêm.*

2. Cách ngây thơ: Prim O(n²)

Coi cáp có sẵn là cạnh giá 0 và chạy Prim cài đặt đơn giản: mỗi lượt duyệt cả n đỉnh để tìm đỉnh gần cây nhất.



Hình 10.1.2. *Sơ đồ khối của Prim O(n²).*

Cách ngây thơ: Prim O(n²)

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

int main() {
    if (fopen("NOIMANG.INP", "r")) {
        freopen("NOIMANG.INP", "r", stdin);
        freopen("NOIMANG.OUT", "w", stdout);
    }
    int n, k, m;
    scanf("%d %d %d", &n, &k, &m);
    vector<vector<pair<int,int>>> g(n + 1);
    for (int i = 0; i < k; i++) { int u, v; scanf("%d %d", &u, &v);
        g[u].push_back({v, 0}); g[v].push_back({u, 0}); }
    for (int i = 0; i < m; i++) { int u, v, c; scanf("%d %d %d", &u, &v,
&c);
        g[u].push_back({v, c}); g[v].push_back({u, c}); }
    const ll INF = LLONG_MAX;
    vector<ll> d(n + 1, INF); vector<char> in(n + 1, 0);
    d[1] = 0; ll tot = 0;
    for (int it = 0; it < n; it++) {            // Prim O(n^2): tìm đỉnh gần
        cây nhất bằng vòng lặp
        int u = -1;
        for (int v = 1; v <= n; v++) if (!in[v] && d[v] != INF && (u < 0 ||
            d[v] < d[u])) u = v;
        if (u < 0) { printf("-1\n"); return 0; }
        in[u] = 1; tot += d[u];
        for (auto [v, c] : g[u]) if (!in[v] && c < d[v]) d[v] = c;
    }
    printf("%lld\n", tot);
    return 0;
}
```

Độ phức tạp: O(n^2 + m): n = 1000 là 10^6; n = 10^5 là 10^{10}.

3. Bế tắc

- **Subtask 1** (n ≤ 1000): **30 điểm**.
- **Subtask 2, 3**: quá thời gian.

Kruskal (sắp xếp cạnh + DSU) chạy O(m log m), nhưng nhiều bạn **quên cáp có sẵn**: đọc vào rồi bỏ qua. Đúng ở Subtask 1, 2 (k = 0), sai ở Subtask 3: **60 điểm**.

ĐIỂM BẾ TẮC

Kruskal xử lí cạnh theo chi phí tăng dần. Cáp có sẵn có chi phí 0, nhỏ hơn mọi phương án: chúng phải được "nhận" **trước tiên**.

4. Phá bế tắc

Khởi tạo DSU, gộp hai đầu của mọi cáp có sẵn (không tính tiền). Sau đó chạy Kruskal bình thường trên m phương án. Cuối cùng kiểm tra số nhóm bằng 1; nếu không thì in −1.

Xét cáp (ví dụ)	Giá	Kết quả
1 – 2 (có sẵn)	0	gộp: {1, 2}
4 – 5	1	nhận: {4, 5}
2 – 3	2	nhận: {1, 2, 3}
2 – 4	3	nhận: {1, 2, 3, 4, 5}
1 – 3, 3 – 4, 3 – 5	4, 5, 6	bỏ: hai đầu đã cùng nhóm
Tổng		**6**

5. Cách tối ưu

NOIMANG.CPP: DSU với cáp có sẵn + Kruskal

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> p;
int root(int x) {
    while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; }
    return x;
}

int main() {
    if (fopen("NOIMANG.INP", "r")) {
        freopen("NOIMANG.INP", "r", stdin);
        freopen("NOIMANG.OUT", "w", stdout);
    }
    int n, k, m;
    scanf("%d %d %d", &n, &k, &m);
    p.resize(n + 1); iota(p.begin(), p.end(), 0);
    int cnt = n;                                  // số nhóm trường đang liên lạc được
    for (int i = 0; i < k; i++) {                 // cáp có sẵn: gộp trước, chi phí 0
        int u, v;
        scanf("%d %d", &u, &v);
        u = root(u); v = root(v);
        if (u != v) { p[u] = v; cnt--; }
    }
    vector<array<int,3>> e(m);                    // (chi phí, u, v) để sắp xếp theo chi phí
    for (auto &x : e) scanf("%d %d %d", &x[1], &x[2], &x[0]);
    sort(e.begin(), e.end());
    long long tot = 0;                            // có thể tới 1e14
    for (auto &x : e) {                           // Kruskal
        int a = root(x[1]), b = root(x[2]);
        if (a != b) { p[a] = b; cnt--; tot += x[0]; }
    }
    printf("%lld\n", cnt == 1 ? tot : -1LL);
    return 0;
}
```

Độ phức tạp: O(k + m log m).

6. Bẫy thường gặp

- **Quên cáp có sẵn.**
- **Tổng chi phí bằng `int`:** tới $(n-1)\cdot 10^9 \approx 10^{14}$.
- **Không kiểm tra liên thông** ở cuối.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
Prim $O(n^2)$	30	0	0	**30**
Kruskal, quên cáp có sẵn	30	30	0	**60**
Gộp cáp có sẵn + Kruskal	30	30	40	**100**

Chấm trên 9 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

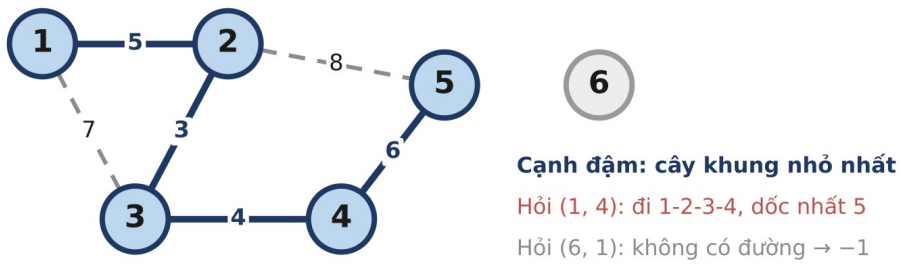
BÀI 10.2 ĐƯỜNG ÍT DỐC

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
ITDOC.INP	ITDOC.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Tập 1, Bài 10 · Đề bài: Tập 2, trang 76

1. Hiểu bài toán

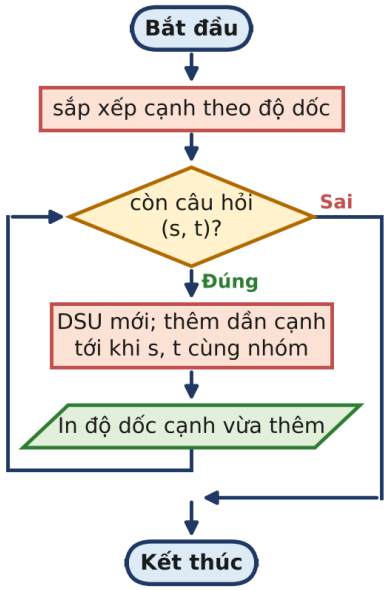
Độ khó của lộ trình là cạnh **lớn nhất** trên đó; cần lộ trình có độ khó **nhỏ nhất**: bài toán minimax path. Tính chất quan trọng của cây khung nhỏ nhất (Tập 1, Bài 10): đường đi giữa s và t trên cây khung nhỏ nhất luôn là một lộ trình minimax.



Hình 10.2.1. *Mọi lộ trình tốt nhất nằm trên cây khung nhỏ nhất.*

2. Cách ngây thơ: Thêm dần cạnh cho từng câu hỏi

Sắp xếp cạnh theo độ dốc. Với mỗi câu hỏi (s, t), dùng DSU mới và thêm dần các cạnh; cạnh làm s, t lần đầu cùng nhóm cho đáp số.



Hình 10.2.2. *Sơ đồ khối của cách thêm dần cạnh cho từng câu hỏi.*

Cách ngây thơ: Kruskal lại cho mỗi câu hỏi

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> p;
int root(int x) { while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; } return
x; }

int main() {
    if (fopen("ITDOC.INP", "r")) {
        freopen("ITDOC.INP", "r", stdin);
        freopen("ITDOC.OUT", "w", stdout);
    }
    int n, m, q;
    scanf("%d %d %d", &n, &m, &q);
    vector<array<int,3>> e(m);
    for (auto &x : e) scanf("%d %d %d", &x[1], &x[2], &x[0]);
    sort(e.begin(), e.end());
    while (q--) {
        int s, t;
        scanf("%d %d", &s, &t);
        if (s == t) { printf("0\n"); continue; }
        p.assign(n + 1, 0); iota(p.begin(), p.end(), 0);
        int ans = -1;
        for (auto &x : e) {                    // thêm dần cạnh theo độ dốc
            tăng dần
            p[root(x[1])] = root(x[2]);
            if (root(s) == root(t)) { ans = x[0]; break; }
        }
        printf("%d\n", ans);
    }
    return 0;
}
```

Độ phức tạp: $O(q \cdot m \cdot \alpha(n))$.

3. Bế tắc

Subtask 1 chạy tức thì; Subtask 2 ($q = 1000$, $m = 2 \cdot 10^5$) là $2 \cdot 10^8$ thao tác DSU. Kết quả thực tế: **30 điểm**.

Dựng cây khung một lần rồi **DFS trên cây** cho mỗi câu hỏi, giữ cạnh lớn nhất trên đường đi: $O(q \cdot n)$. Nghe có vẻ vừa sức với Subtask 2, nhưng $q = 1000$ lần duyệt cây 10^5 đỉnh là 10^8 bước nhảy lung tung trong bộ nhớ (danh sách kề, ngăn xếp), cộng thêm việc xóa mảng mỗi lần: thực tế mất vài giây, chỉ được **30 điểm**. Với $q = 10^5$ thì 10^{10} bước.

> **ĐIỂM BẾ TẮC**
>
> Câu hỏi đã trở thành "cạnh lớn nhất trên đường đi giữa hai đỉnh của một cây". Cần trả lời nhanh hơn $O(n)$ mỗi câu.

4. Phá bế tắc

Dùng **nhảy nhị phân** như bài toán LCA (Tập 1, Bài 11), lưu thêm $mx[v][j]$ = cạnh lớn nhất trên đoạn đường từ v nhảy lên 2^j bước:

- $mx[v][0]$ = độ dốc cạnh $(v, \text{cha } v)$;
- $mx[v][j] = \max(mx[v][j-1], mx[up[v][j-1]][j-1])$.

Đường đi u – v trên cây là $u \to \text{LCA} \to v$. Khi nâng u (hoặc v) lên 2^j bước, gộp mx tương ứng vào đáp số. Đồ thị có thể không liên thông: ta có một **rừng**, mỗi cây treo tại một gốc; hai đỉnh khác cây thì in -1.

> **VÌ SAO ĐƯỜNG TRÊN CÂY KHUNG NHỎ NHẤT LÀ TỐT NHẤT?**
>
> Giả sử cạnh lớn nhất e trên đường cây giữa s và t có độ dốc h, và có lộ trình P giữa s, t với mọi cạnh $< h$. Xóa e khỏi cây chia cây thành hai phần, s và t ở hai phía; P phải có một cạnh f nối hai phía, với độ dốc $< h$. Thay e bằng f được cây khung nhỏ hơn: mâu thuẫn.

5. Cách tối ưu

ITDOC.CPP: Kruskal + nhảy nhị phân với cạnh lớn nhất

```cpp
#include <bits/stdc++.h>
using namespace std;

const int LG = 17;                                  // 2^17 > 1e5
vector<int> p;
int root(int x) {
    while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; }
    return x;
}

int main() {
    if (fopen("ITDOC.INP", "r")) {
        freopen("ITDOC.INP", "r", stdin);
        freopen("ITDOC.OUT", "w", stdout);
    }
    int n, m, q;
    scanf("%d %d %d", &n, &m, &q);
    vector<array<int,3>> e(m);
    for (auto &x : e) scanf("%d %d %d", &x[1], &x[2], &x[0]);
    // 1) Kruskal: rừng khung nhỏ nhất
    sort(e.begin(), e.end());
    p.resize(n + 1); iota(p.begin(), p.end(), 0);
    vector<vector<pair<int,int>>> g(n + 1);
    for (auto &x : e) {
        int a = root(x[1]), b = root(x[2]);
        if (a == b) continue;
        p[a] = b;
        g[x[1]].push_back({x[2], x[0]}); g[x[2]].push_back({x[1], x[0]});
    }
    // 2) Treo từng cây, tính up[v][j] và mx[v][j] = độ dốc lớn nhất khi
nhảy
        2^j bước
    vector<array<int, LG + 1>> up(n + 1), mx(n + 1);
    vector<int> dep(n + 1, -1), tree(n + 1);
    for (int s = 1; s <= n; s++) {
        if (dep[s] >= 0) continue;
        dep[s] = 0; tree[s] = s; up[s].fill(s); mx[s].fill(0);
        vector<int> st = {s};
        while (!st.empty()) {
            int u = st.back(); st.pop_back();
            for (auto [v, h] : g[u]) {
                if (dep[v] >= 0) continue;
                dep[v] = dep[u] + 1; tree[v] = s;
                up[v][0] = u; mx[v][0] = h;
                for (int j = 1; j <= LG; j++) {
                    up[v][j] = up[up[v][j - 1]][j - 1];
                    mx[v][j] = max(mx[v][j - 1], mx[up[v][j - 1]][j - 1]);
                }
                st.push_back(v);
            }
        }
    }
    // 3) Mỗi câu hỏi: cạnh lớn nhất trên đường đi trong cây = nhảy tới LCA
    while (q--) {
        int u, v;
        scanf("%d %d", &u, &v);
        if (tree[u] != tree[v]) { printf("-1\n"); continue; }
        int r = 0;
        if (dep[u] < dep[v]) swap(u, v);
        for (int j = LG; j >= 0; j--)
            if (dep[u] - (1 << j) >= dep[v]) { r = max(r, mx[u][j]); u =
```

```
                    up[u][j]; }
60          if (u != v) {
61              for (int j = LG; j >= 0; j--)
62                  if (up[u][j] != up[v][j]) {
63                      r = max({r, mx[u][j], mx[v][j]});
64                      u = up[u][j]; v = up[v][j];
65                  }
66              r = max({r, mx[u][0], mx[v][0]});
67          }
68          printf("%d\n", r);
69      }
70      return 0;
71  }
```

Độ phức tạp: O(m log m + n log n + q log n).

> **CÁCH KHÁC: TRẢ LỜI NGOẠI TUYẾN**
>
> Đọc hết câu hỏi, rồi chạy Kruskal một lần; mỗi khi gộp hai nhóm, trả lời mọi câu hỏi (s, t) vừa trở nên cùng nhóm (gộp danh sách câu hỏi nhỏ vào lớn). Hoặc dựng "cây Kruskal" (Kruskal reconstruction tree) rồi tìm LCA.

6. Bẫy thường gặp

- **Quên đồ thị không liên thông:** phải in −1.
- **s = t:** đáp số là 0.
- **Bảng nhảy cho đỉnh gốc:** up[gốc][j] = gốc, mx[gốc][j] = 0.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
Kruskal lại cho mỗi câu hỏi	30	0	0	**30**
Cây khung + DFS mỗi câu hỏi	30	0	0	**30**
Cây khung + nhảy nhị phân	30	30	40	**100**

Chấm trên 7 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

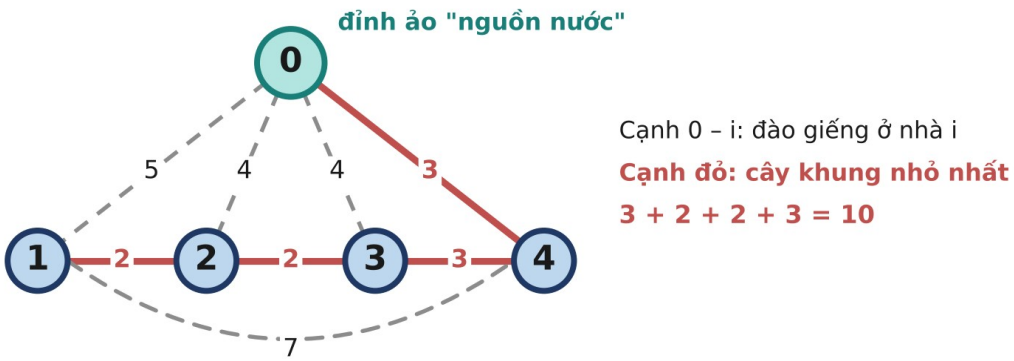
BÀI 10.3 GIẾNG VÀ ỐNG

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
GIENGONG.INP	GIENGONG.OUT	1 giây	256 MB	★★★ NÂNG CAO

Liên hệ: Tập 1, Bài 10 · Đề bài: Tập 2, trang 78

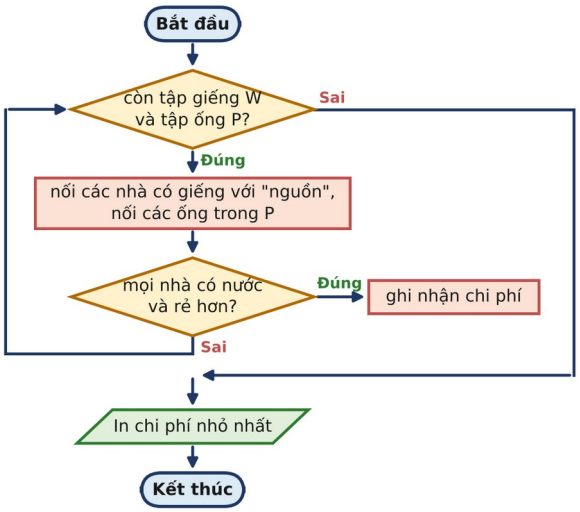
1. Hiểu bài toán

Mỗi nhà phải có nước bằng một trong hai cách: tự đào giếng, hoặc nối ống tới một nhà có nước. Cái khó là không biết trước nên đào giếng ở đâu và bao nhiêu giếng.



Hình 10.3.1. *Đỉnh ảo 0 biến "đào giếng" thành "nối ống tới nguồn nước".*

2. Cách ngây thơ: Thử mọi tập giếng và mọi tập ống



Hình 10.3.2. *Sơ đồ khối của cách vét cạn.*

Cách ngây thơ: thử 2^{n+m} lựa chọn

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<int> p;
int root(int x) { while (p[x] != x) x = p[x]; return x; }

int main() {
    if (fopen("GIENGONG.INP", "r")) {
        freopen("GIENGONG.INP", "r", stdin);
        freopen("GIENGONG.OUT", "w", stdout);
    }
    scanf("%d %d", &n, &m);
    vector<long long> a(n);
    for (auto &x : a) scanf("%lld", &x);
    vector<array<int,3>> e(m);
    for (auto &x : e) scanf("%d %d %d", &x[0], &x[1], &x[2]);
    if (n + m > 24) return 0;                    // 2^(n+m) quá lớn
    long long best = LLONG_MAX;
    for (int wm = 1; wm < (1 << n); wm++)        // tập nhà đào giếng
        for (int em = 0; em < (1 << m); em++) { // tập ống được nối
            p.assign(n + 2, 0); iota(p.begin(), p.end(), 0);
            long long c = 0;
            for (int i = 0; i < n; i++) if (wm >> i & 1) { c += a[i];
p[root(i
                + 1)] = root(0); }
            for (int j = 0; j < m; j++) if (em >> j & 1) { c += e[j][2];
                p[root(e[j][0])] = root(e[j][1]); }
            bool ok = true;
            for (int v = 1; v <= n; v++) if (root(v) != root(0)) ok = false;
            if (ok) best = min(best, c);
        }
    printf("%lld\n", best);
    return 0;
}
```

Độ phức tạp: $O(2^{n+m} \cdot (n + m))$: chỉ dùng được cho $n + m \le 20$: **20 điểm**.

3. Bế tắc

Một ý tưởng "tham lam" hay gặp: đào **một** giếng ở nhà rẻ nhất, rồi nối ống bằng Kruskal (nhóm nào chưa có nước thì đào thêm giếng rẻ nhất trong nhóm). Sai vì có lúc đào thêm giếng rẻ hơn nối một đường ống đắt: **0 điểm**.

ĐIỂM BẾ TẮC

Có hai loại "chi phí" khác nhau (giếng và ống), và số giếng không cố định. Cần đưa cả hai về cùng một loại đối tượng.

4. Phá bế tắc

Thêm **đỉnh ảo 0** là "nguồn nước". Đào giếng ở nhà i tương đương nối ống từ i tới nguồn với giá a_i. Một nhà có nước $\Longleftrightarrow$ nó liên thông với đỉnh 0. Mọi nhà có nước $\Longleftrightarrow$ đồ thị $n + 1$ đỉnh liên thông. Đáp số là **cây khung nhỏ nhất** của đồ thị mới.

Cạnh (theo giá tăng)	Giá	Kruskal
1 – 2	2	nhận
2 – 3	2	nhận
0 – 4 (giếng nhà 4)	3	nhận
3 – 4	3	nhận: mọi đỉnh đã liên thông
0 – 2, 0 – 3, 0 – 1, 1 – 4	4, 4, 5, 7	bỏ
Tổng		**10**

Prim $O(n^2)$ trên đồ thị mới cũng đúng, đủ nhanh cho Subtask 2: **50 điểm**.

5. Cách tối ưu

GIENGONG.CPP: đỉnh ảo + Kruskal

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> p;
int root(int x) {
    while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; }
    return x;
}

int main() {
    if (fopen("GIENGONG.INP", "r")) {
        freopen("GIENGONG.INP", "r", stdin);
        freopen("GIENGONG.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<array<int,3>> e;                    // (chi phí, u, v)
    for (int i = 1; i <= n; i++) {
        int a;
        scanf("%d", &a);
        e.push_back({a, 0, i});                // đào giếng ở nhà i = cạnh nối với đỉnh ảo 0
    }
    for (int i = 0; i < m; i++) {
        int u, v, c;
        scanf("%d %d %d", &u, &v, &c);
        e.push_back({c, u, v});
    }
    sort(e.begin(), e.end());
    p.resize(n + 1); iota(p.begin(), p.end(), 0);
    long long tot = 0;
    for (auto &x : e) {                        // Kruskal trên n + 1 đỉnh
        int a = root(x[1]), b = root(x[2]);
        if (a != b) { p[a] = b; tot += x[0]; }
    }
    printf("%lld\n", tot);
    return 0;
}
```

Độ phức tạp: $O((n + m) \log(n + m))$.

6. Bẫy thường gặp

- **Đào đúng một giếng.**
- **Tổng chi phí bằng `int`.**
- **Quên rằng đỉnh 0 cũng là một đỉnh của DSU** (mảng phải có n + 1 phần tử).

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (20)	ST2 (30)	ST3 (50)	Tổng
Thử mọi tập giếng và ống	20	0	0	**20**
Một giếng rẻ nhất + Kruskal	0	0	0	**0**
Đỉnh ảo + Prim O(n^2)	20	30	0	**50**
Đỉnh ảo + Kruskal	20	30	50	**100**

Chấm trên 9 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

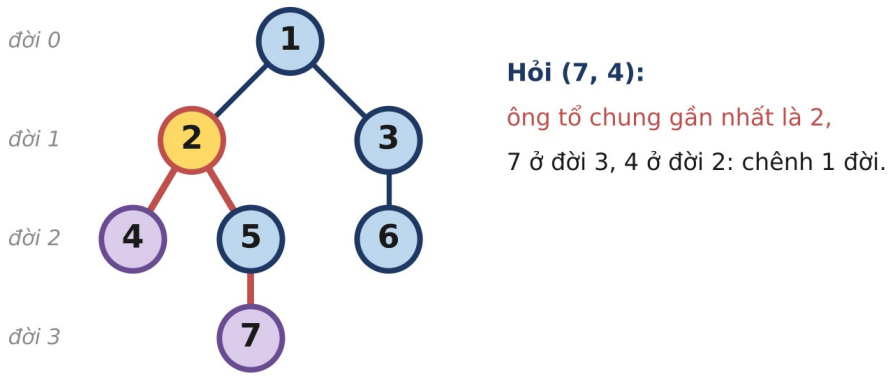
BÀI 11.1 HỌ HÀNG

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
HOHANG.INP	HOHANG.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Tập 1, Bài 11 · Đề bài: Tập 2, trang 80

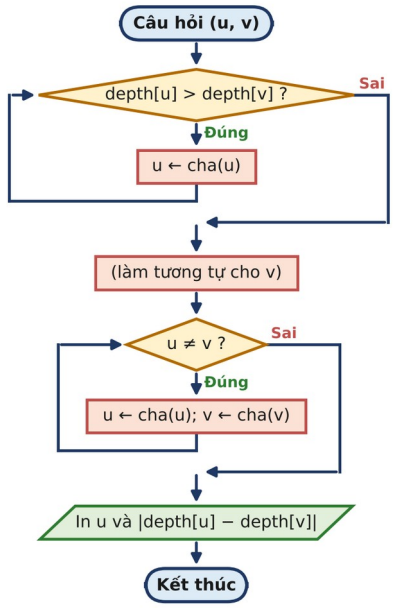
1. Hiểu bài toán

Gia phả là một **cây có gốc** (ông tổ). Ông tổ chung gần nhất là **LCA**, số đời chênh lệch là $|\text{depth}(u) - \text{depth}(v)|$. Chi tiết: ông tổ không nhất thiết là người số 1, và cây có thể rất sâu.



Hình 11.1.1. *Gia phả của ví dụ.*

2. Cách ngây thơ: Leo từng bậc



Hình 11.1.2. *Sơ đồ khối của cách leo từng bậc.*

Cách ngây thơ: leo từng bậc tới tổ tiên chung

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("HOHANG.INP", "r")) {
        freopen("HOHANG.INP", "r", stdin);
        freopen("HOHANG.OUT", "w", stdout);
    }
    int n;
    scanf("%d", &n);
    vector<int> par(n + 1), dep(n + 1, -1);
    for (int i = 1; i <= n; i++) scanf("%d", &par[i]);
    for (int i = 1; i <= n; i++) {              // độ sâu: leo tới khi gặp đỉnh đã biết
        vector<int> path;
        int v = i;
        while (v && dep[v] < 0) { path.push_back(v); v = par[v]; }
        int d = v ? dep[v] : -1;
        for (int k = path.size() - 1; k >= 0; k--) dep[path[k]] = ++d;
    }
    int q;
    scanf("%d", &q);
    while (q--) {
        int u, v;
        scanf("%d %d", &u, &v);
        int diff = abs(dep[u] - dep[v]);
        while (dep[u] > dep[v]) u = par[u];     // leo từng bậc: O(n) mỗi câu hỏi
        while (dep[v] > dep[u]) v = par[v];
        while (u != v) { u = par[u]; v = par[v]; }
        printf("%d %d\n", u, diff);
    }
    return 0;
}
```

Độ phức tạp: O(n) mỗi câu hỏi; O(n · q) tổng cộng.

3. Bế tắc

- **Subtask 1** ($n, q \le 1000$): **40 điểm**.
- **Subtask 2**: gia phả dạng "đường thẳng" $2 \cdot 10^5$ đời và $2 \cdot 10^5$ câu hỏi: $4 \cdot 10^{10}$ bước.

Cài nhảy nhị phân nhưng tính độ sâu bằng **DFS đệ quy**: cây đường thẳng $2 \cdot 10^5$ tầng, mỗi tầng giữ cả một vòng lặp `for` và vài biến: tràn ngăn xếp 8 MB, **40 điểm**.

ĐIỂM BẾ TẮC

Leo từng bậc tốn O(độ sâu). Cần leo nhanh hơn, và cần dựng bảng mà không đệ quy.

4. Phá bế tắc

Bảng **up[v][j]** = tổ tiên thứ 2^j của v, với up[v][j] = up[up[v][j − 1]][j − 1]. Dựng bảng theo thứ tự **BFS** từ ông tổ: khi xử lí v, cha của v đã có bảng đầy đủ.

1) Nâng đỉnh sâu hơn lên cùng độ sâu: phân tích hiệu độ sâu theo bit, nhảy 2^j khi bit j bằng 1.

2) Nếu hai đỉnh đã trùng: đó là LCA. Ngược lại, với j giảm dần, nếu up[u][j] ≠ up[v][j] thì nhảy cả hai. Cuối cùng LCA là cha của u.

Câu hỏi	Diễn biến	Kết quả
(7, 4)	depth 3 và 2: nâng 7 lên 1 bậc → 5; 5 ≠ 4, nhảy cả hai 1 bậc → 2, 2	**2**, chênh 1
(7, 6)	nâng 7 → 5; 5, 6 nhảy tới 2, 3; cha chung **1**	**1**, chênh 1
(5, 7)	nâng 7 → 5: trùng	**5**, chênh 1

5. Cách tối ưu

HOHANG.CPP: LCA nhảy nhị phân, dựng bảng bằng BFS

```cpp
#include <bits/stdc++.h>
using namespace std;

const int LG = 18;                                  // 2^18 > 2e5

int main() {
    if (fopen("HOHANG.INP", "r")) {
        freopen("HOHANG.INP", "r", stdin);
        freopen("HOHANG.OUT", "w", stdout);
    }
    int n;
    scanf("%d", &n);
    vector<vector<int>> ch(n + 1);                  // danh sách con
    int rt = 0;
    for (int i = 1; i <= n; i++) {
        int p;
        scanf("%d", &p);
        if (p) ch[p].push_back(i); else rt = i; // ông tổ có cha = 0
    }
    // BFS từ ông tổ: độ sâu và up[v][j] = tổ tiên thứ 2^j
    vector<array<int, LG + 1>> up(n + 1);
    vector<int> dep(n + 1, 0), q = {rt};
    up[rt].fill(rt);
    for (size_t h = 0; h < q.size(); h++) {
        int u = q[h];
        for (int v : ch[u]) {
            dep[v] = dep[u] + 1;
            up[v][0] = u;
            for (int j = 1; j <= LG; j++) up[v][j] = up[up[v][j - 1]][j -
1];
            q.push_back(v);
        }
    }
    int Q;
    scanf("%d", &Q);
    while (Q--) {
        int u, v;
        scanf("%d %d", &u, &v);
        int diff = abs(dep[u] - dep[v]);
        if (dep[u] < dep[v]) swap(u, v);
        for (int j = LG; j >= 0; j--)           // nâng u lên cùng độ sâu
với
            v
            if (dep[u] - (1 << j) >= dep[v]) u = up[u][j];
        if (u != v) {                           // nhảy cả hai lên khi tổ
tiên
            còn khác nhau
            for (int j = LG; j >= 0; j--)
                if (up[u][j] != up[v][j]) { u = up[u][j]; v = up[v][j]; }
            u = up[u][0];
        }
        printf("%d %d\n", u, diff);
    }
    return 0;
}
```

Độ phức tạp: O(n log n) dựng bảng, O(log n) mỗi câu hỏi.

6. Bẫy thường gặp

- **Giả sử ông tổ là người 1.**
- **DFS đệ quy** trên cây sâu.
- **LG quá nhỏ:** 2^{LG} phải lớn hơn độ sâu lớn nhất ($2^{18} > 2 \cdot 10^5$).

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (40)	ST2 (60)	Tổng
Leo từng bậc	40	0	**40**
Nhảy nhị phân, DFS đệ quy	40	0	**40**
Nhảy nhị phân, BFS	40	60	**100**

Chấm trên 6 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

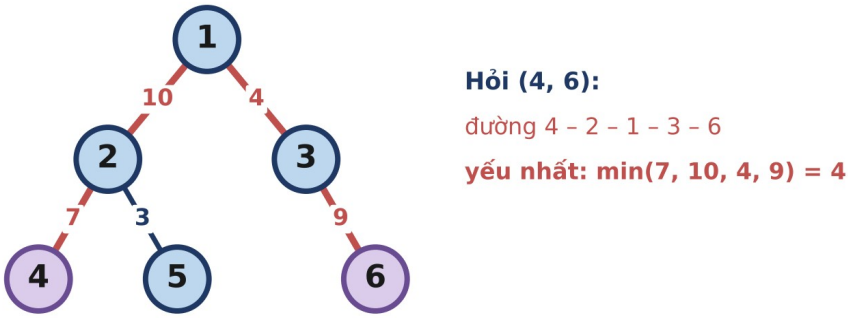
BÀI 11.2 CÁP QUANG YẾU NHẤT

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
CAPQUANG.INP	CAPQUANG.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Tập 1, Bài 11 · Đề bài: Tập 2, trang 82

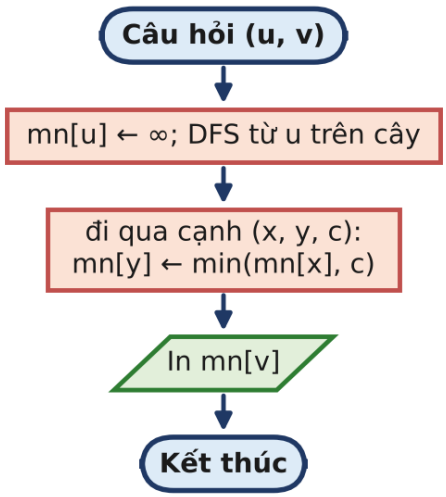
1. Hiểu bài toán

Trên cây, giữa hai trạm có đúng một đường truyền: u → LCA(u, v) → v. Cần cạnh nhỏ nhất trên đường đó, cho 10^5 câu hỏi.



Hình 11.2.1. *Đường truyền 4 – 6 đi qua tổ tiên chung 1.*

2. Cách ngây thơ: DFS cho mỗi câu hỏi



Hình 11.2.2. *Sơ đồ khối của cách DFS cho từng câu hỏi.*

Cách ngây thơ: DFS từ u, giữ cạnh nhỏ nhất

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("CAPQUANG.INP", "r")) {
        freopen("CAPQUANG.INP", "r", stdin);
        freopen("CAPQUANG.OUT", "w", stdout);
    }
    int n, q;
    scanf("%d %d", &n, &q);
    vector<vector<pair<int,int>>> g(n + 1);
    for (int i = 1; i < n; i++) { int u, v, c; scanf("%d %d %d", &u, &v,
&c);
        g[u].push_back({v, c}); g[v].push_back({u, c}); }
    vector<int> mn(n + 1);
    while (q--) {                                  // DFS từ u cho mỗi câu hỏi:
        O(n)
        int u, v;
        scanf("%d %d", &u, &v);
        fill(mn.begin(), mn.end(), -1);
        mn[u] = INT_MAX; vector<int> st = {u};
        while (!st.empty()) {
            int x = st.back(); st.pop_back();
            for (auto [y, c] : g[x]) if (mn[y] < 0) { mn[y] = min(mn[x], c);
                st.push_back(y); }
        }
        printf("%d\n", mn[v]);
    }
    return 0;
}
```

Độ phức tạp: $O(n \cdot q)$.

3. Bế tắc

- **Subtask 1** ($n, q \le 1000$): **30 điểm**.
- **Subtask 2** (cây là đường thẳng 10^5 trạm) và **Subtask 3**: 10^{10} bước.

Subtask 2 gợi ý một bước trung gian: trên đường thẳng, câu hỏi là "giá trị nhỏ nhất trên đoạn [u, v − 1] của dãy băng thông", giải được bằng **bảng thưa** (sparse table): mn[i][j] = min của 2^j phần tử bắt đầu từ i. Ý tưởng "đoạn dài 2^j" ấy chuyển nguyên vẹn lên cây.

ĐIỂM BẾ TẮC

Đường đi trên cây không phải một đoạn liên tiếp của một dãy, nhưng nó gồm hai đoạn "thẳng đứng" (từ u lên LCA và từ v lên LCA).

4. Phá bế tắc

Kết hợp bảng nhảy với min: mn[v][j] = cạnh nhỏ nhất trên đoạn từ v nhảy lên 2^j bước; mn[v][0] là cạnh (v, cha v), mn[v][j] = min(mn[v][j − 1], mn[up[v][j − 1]][j − 1]). Khi tìm LCA, mỗi lần nhảy u hoặc v lên 2^j bước thì lấy min với mn tương ứng; cuối cùng cộng thêm hai cạnh đi vào LCA.

Câu hỏi	Các bước nhảy (treo tại 1)	Kết quả
(4, 5)	cùng độ sâu 2; up[4][0] = up[5][0] = 2: lấy mn[4][0] = 7, mn[5][0] = 3	**3**
(4, 6)	cùng độ sâu; nhảy 1 bậc: 4 → 2 (7), 6 → 3 (9); hai cạnh cuối 10, 4	**4**
(2, 4)	nâng 4 lên 1 bậc (7): trùng 2	**7**

5. Cách tối ưu

CAPQUANG.CPP: nhảy nhị phân kèm min

```cpp
#include <bits/stdc++.h>
using namespace std;

const int LG = 17;

int main() {
    if (fopen("CAPQUANG.INP", "r")) {
        freopen("CAPQUANG.INP", "r", stdin);
        freopen("CAPQUANG.OUT", "w", stdout);
    }
    int n, q;
    scanf("%d %d", &n, &q);
    vector<vector<pair<int,int>>> g(n + 1);
    for (int i = 1; i < n; i++) {
        int u, v, c;
        scanf("%d %d %d", &u, &v, &c);
        g[u].push_back({v, c}); g[v].push_back({u, c});
    }
    // treo cây tại 1; mn[v][j] = băng thông nhỏ nhất trên đoạn nhảy 2^j bước
        từ v
    vector<array<int, LG + 1>> up(n + 1), mn(n + 1);
    vector<int> dep(n + 1, -1), st = {1};
    dep[1] = 0; up[1].fill(1); mn[1].fill(INT_MAX);
    while (!st.empty()) {
        int u = st.back(); st.pop_back();
        for (auto [v, c] : g[u]) {
            if (dep[v] >= 0) continue;
            dep[v] = dep[u] + 1; up[v][0] = u; mn[v][0] = c;
            for (int j = 1; j <= LG; j++) {
                up[v][j] = up[up[v][j - 1]][j - 1];
                mn[v][j] = min(mn[v][j - 1], mn[up[v][j - 1]][j - 1]);
            }
            st.push_back(v);
        }
    }
    while (q--) {
        int u, v;
        scanf("%d %d", &u, &v);
        int r = INT_MAX;
        if (dep[u] < dep[v]) swap(u, v);
        for (int j = LG; j >= 0; j--)
            if (dep[u] - (1 << j) >= dep[v]) { r = min(r, mn[u][j]); u = up[u][j]; }
        if (u != v) {
            for (int j = LG; j >= 0; j--)
                if (up[u][j] != up[v][j]) {
                    r = min({r, mn[u][j], mn[v][j]});
                    u = up[u][j]; v = up[v][j];
                }
            r = min({r, mn[u][0], mn[v][0]});   // hai cạnh cuối đi vào LCA
        }
        printf("%d\n", r);
    }
    return 0;
}
```

Độ phức tạp: O(n log n + q log n).

6. Bẫy thường gặp

- **Quên hai cạnh cuối** đi vào LCA khi $u \neq v$ sau bước nhảy.
- **Khởi tạo mn của gốc bằng 0** (thay vì vô cùng) làm mọi đáp số bằng 0.
- **Duyệt cây đệ quy** trên cây đường thẳng 10^5 đỉnh.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (20)	ST3 (50)	Tổng
DFS cho mỗi câu hỏi	30	0	0	**30**
Nhảy nhị phân kèm min	30	20	50	**100**

Chấm trên 8 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

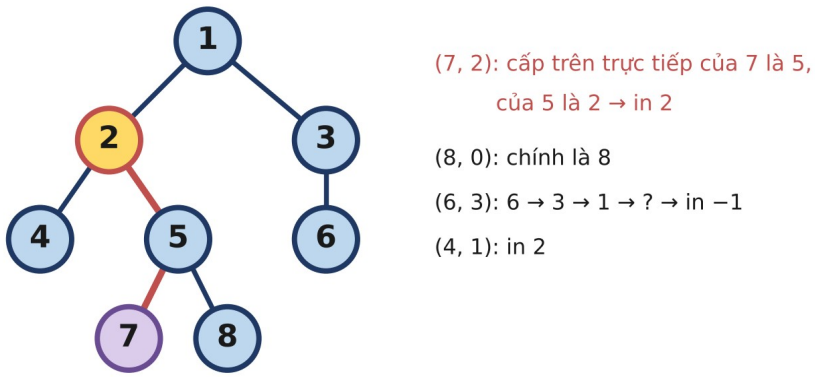
BÀI 11.3 NHÂN VIÊN CẤP K

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
CAPK.INP	CAPK.OUT	2 giây	64 MB	★★★ NÂNG CAO

Liên hệ: Tập 1, Bài 11 · Đề bài: Tập 2, trang 84

1. Hiểu bài toán

Cấp trên thứ k chính là **tổ tiên thứ k**. Với $2 \cdot 10^5$ đỉnh, nhảy nhị phân là đủ. Nhưng với n = q = 10^6 và chỉ **64 MB** bộ nhớ, bảng up[v][j] với 21 cột đã cần 84 MB.



Hình 11.3.1. *Cây quản lí của ví dụ.*

2. Cách ngây thơ: Leo từng bậc

Cách ngây thơ: leo k bậc

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("CAPK.INP", "r")) {
        freopen("CAPK.INP", "r", stdin);
        freopen("CAPK.OUT", "w", stdout);
    }
    int n, q;
    scanf("%d %d", &n, &q);
    vector<int> par(n + 1, 0);
    for (int i = 2; i <= n; i++) scanf("%d", &par[i]);
    while (q--) {
        int v, k;
        scanf("%d %d", &v, &k);
        while (k > 0 && v != 0) { v = par[v]; k--; }   // leo từng bậc
        printf("%d\n", v ? v : -1);
    }
    return 0;
}
```

Độ phức tạp: O(min(k, độ sâu)) mỗi câu hỏi; O(n · q) trong trường hợp xấu nhất.

3. Bế tắc

- **Subtask 1** ($n, q \le 1000$): **20 điểm**.
- **Subtask 2, 3**: cây sâu, quá thời gian.

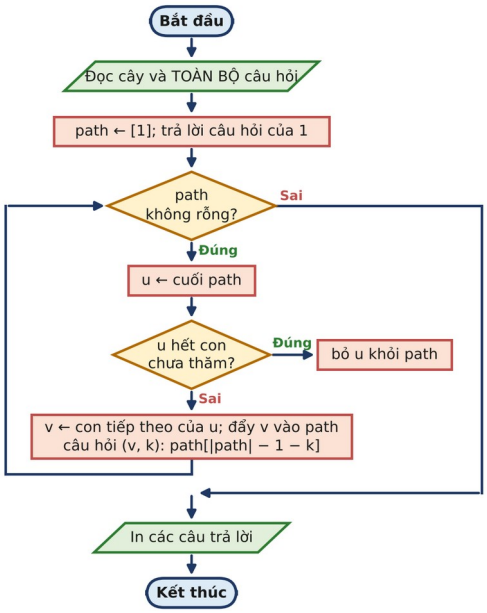
Nhảy nhị phân theo các bit của k giải trọn Subtask 2, nhưng ở Subtask 3 bảng $21 \cdot 10^6$ số nguyên vượt giới hạn 64 MB: **60 điểm**.

> **ĐIỂM BẾ TẮC**
>
> Bảng nhảy nhị phân lưu sẵn câu trả lời cho **mọi** đỉnh và mọi lũy thừa của 2, trong khi ta chỉ cần trả lời q câu hỏi cụ thể. Có cách nào chỉ dùng $O(n + q)$ bộ nhớ?

4. Phá bế tắc

Đọc **toàn bộ** câu hỏi trước và gom theo đỉnh v. DFS (không đệ quy) từ gốc, duy trì mảng **path** là dãy các đỉnh từ gốc tới đỉnh đang đứng. Khi vừa đi xuống v, path chứa đúng các tổ tiên của v theo thứ tự, nên cấp trên thứ k của v là path[|path| − 1 − k] (nếu k < |path|), ngược lại là −1.

Đang ở	path	Trả lời
1	[1]	
2	[1, 2]	
4	[1, 2, 4]	(4, 1): path[1] = **2**
5	[1, 2, 5]	
7	[1, 2, 5, 7]	(7, 2): path[1] = **2**
8	[1, 2, 5, 8]	(8, 0): **8**
6	[1, 3, 6]	(6, 3): 3 ≥ |path| = 3: **−1**



Hình 11.3.2. *Sơ đồ khối của DFS với ngăn xếp đường đi.*

Để tiết kiệm bộ nhớ, danh sách con và danh sách câu hỏi được lưu dạng "đầu – kế tiếp" (mỗi phần tử chỉ tốn một số nguyên), thay cho `vector<vector<int>>`.

5. Cách tối ưu

CAPK.CPP: trả lời ngoại tuyến bằng DFS và ngăn xếp đường đi

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("CAPK.INP", "r")) {
        freopen("CAPK.INP", "r", stdin);
        freopen("CAPK.OUT", "w", stdout);
    }
    int n, q;
    scanf("%d %d", &n, &q);
    // Danh sách con lưu dạng "đầu - kế tiếp" để tiết kiệm bộ nhớ
    vector<int> head(n + 1, -1), nxt(n + 1);
    for (int i = 2; i <= n; i++) {
        int p;
        scanf("%d", &p);
        nxt[i] = head[p]; head[p] = i;
    }
    // Đọc hết câu hỏi, gom theo đỉnh v (cũng dạng đầu - kế tiếp)
    vector<int> qk(q), qhead(n + 1, -1), qnxt(q), ans(q);
    for (int i = 0; i < q; i++) {
        int v;
        scanf("%d %d", &v, &qk[i]);
        qnxt[i] = qhead[v]; qhead[v] = i;
    }
    // DFS không đệ quy; path = các đỉnh từ gốc tới đỉnh đang đứng
    vector<int> path, it(head);                 // it[u]: con kế tiếp cần thăm
        của u
    path.reserve(n);
    auto answer = [&](int v) {                  // trả lời các câu hỏi của v
        for (int j = qhead[v]; j >= 0; j = qnxt[j])
            ans[j] = qk[j] < (int)path.size() ? path[path.size() - 1 - qk[j]]
                   : -1;
    };
    path.push_back(1);
    answer(1);
    while (!path.empty()) {
        int u = path.back();
        if (it[u] < 0) { path.pop_back(); continue; }   // hết con: quay lui
        int v = it[u]; it[u] = nxt[v];
        path.push_back(v);
        answer(v);
    }
    for (int i = 0; i < q; i++) printf("%d\n", ans[i]);
    return 0;
}
```

Độ phức tạp: O(n + q) thời gian và bộ nhớ (khoảng 8 mảng 10^6 số nguyên = 32 MB).

6. Bẫy thường gặp

- **Bảng nhảy nhị phân với n = 10^6:** vượt 64 MB.
- **DFS đệ quy** sâu 10^6 tầng.
- **k lớn hơn độ sâu** (tới 10^9): phải in −1, không truy cập ngoài mảng.

- `vector<vector<int>>` **cho 10^6 đỉnh:** mỗi vector rỗng đã tốn 24 byte, cộng phần dư khi cấp phát.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (20)	ST2 (40)	ST3 (40)	Tổng
Leo từng bậc	20	0	0	**20**
Nhảy nhị phân (bảng 21 · n)	20	40	0	**60**
Ngoại tuyến: DFS + ngăn xếp đường đi	20	40	40	**100**

Chấm trên 8 test, giới hạn 2 giây, 64 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

CHƯƠNG 4. CẤU TRÚC CỦA ĐỒ THỊ

*Chương 4 nhìn vào "bộ xương" của đồ thị: thứ tự tô pô, cầu và khớp, thành phần liên thông mạnh, chu trình Euler. Cách ngây thơ quen thuộc nhất là "thử xóa từng cạnh, từng đỉnh rồi duyệt lại". Lời giải đúng chỉ cần **một** lần DFS (hoặc một lần Kahn) và đọc thêm thông tin từ chính quá trình duyệt đó. Những lời giải "trông như đúng" hay mất điểm vì dừng quá sớm, vì bỏ qua cạnh song song, vì quên gốc của cây DFS, hoặc vì chọn sai chiều khi dựng đáp án.*

Bài	Tên bài	Mức	Cách ngây thơ (điểm)	Chìa khóa của lời giải
12.1	**XẾP HẠNG GIẢI ĐẤU**	★	Duyệt mọi hoán vị (30)	Kahn: thiếu đỉnh thì có vòng; hàng đợi từng có ≥ 2 đỉnh thì nhiều đáp án.
12.2	**DỰ ÁN XÂY NHÀ**	★★	Thử kéo dài từng việc (40)	Hai lượt QHĐ trên thứ tự tô pô: sớm nhất (xuôi) và muộn nhất (ngược).
12.3	**ĐẾM LỘ TRÌNH**	★★★	QHĐ theo số hiệu nút (30)	Chỉ giữ các nút vừa tới được từ 1 vừa tới được n; có vòng trong đó thì vô số.
13.1	**ĐƯỜNG HUYẾT MẠCH**	★	Chặn thử từng con đường (30)	num/low, bỏ qua đúng CẠNH vừa đi tới (không bỏ đỉnh cha).
13.2	**TRẠM BIẾN ÁP**	★★	Thử từng trạm, BFS đếm phần (30)	Khớp: số phần = (số con c có low[c] ≥ num[u]) + 1, riêng gốc thì không cộng 1.
13.3	**CÁP DỰ PHÒNG**	★★★	Đếm lá của đồ thị ban đầu (20)	Nén khối không cầu → cây cầu; đáp số ⌈số lá / 2⌉.
14.1	**LAN TRUYỀN TIN ĐỒN**	★	Đếm học sinh có bán bậc vào 0 (30)	Nén thành phần liên thông mạnh; đếm thành phần có bán bậc vào 0.
14.2	**KHO BÁU THỊ TRẤN**	★★	QHĐ trên đồ thị gốc (Kahn) (30)	Nén SCC (vào thành phần là nhặt hết), QHĐ đường dài nhất trên DAG.
14.3	**HỘI ĐỒNG TUYỂN CHỌN**	★★★	Thử mọi phương án (20)	2-SAT: cung ¬x → y, ¬y → x; Tarjan; chọn khẳng định có số thành phần nhỏ hơn.
15.1	**NÉT VẼ**	★	Chỉ đếm điểm bậc lẻ (30)	Định lí Euler: số điểm bậc lẻ là 0 hoặc 2, và các đoạn liên thông với nhau.
15.2	**GHÉP DOMINO**	★★	Quay lui (30)	7 đỉnh, mỗi quân là một cạnh: đường Euler bằng Hierholzer không đệ quy.
15.3	**MẬT KHẨU KÉT SẮT**	★★★	Quay lui từng phím (50)	Đồ thị de Bruijn: đỉnh là xâu k − 1 chữ số, cạnh là mã; chu trình Euler.

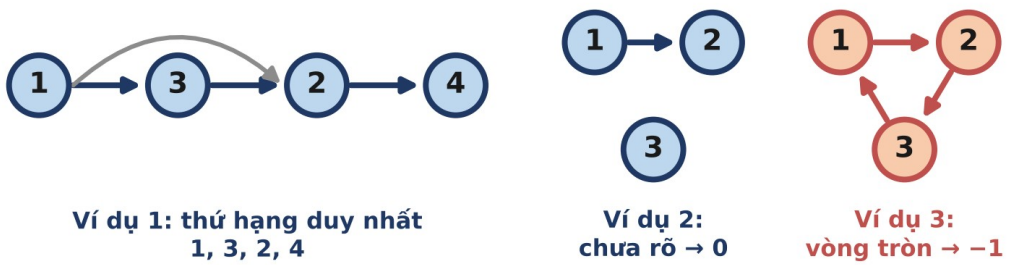
BÀI 12.1 XẾP HẠNG GIẢI ĐẤU

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
XHGIAIDAU.INP	XHGIAIDAU.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Tập 1, Bài 12 · Đề bài: Tập 2, trang 87

1. Hiểu bài toán

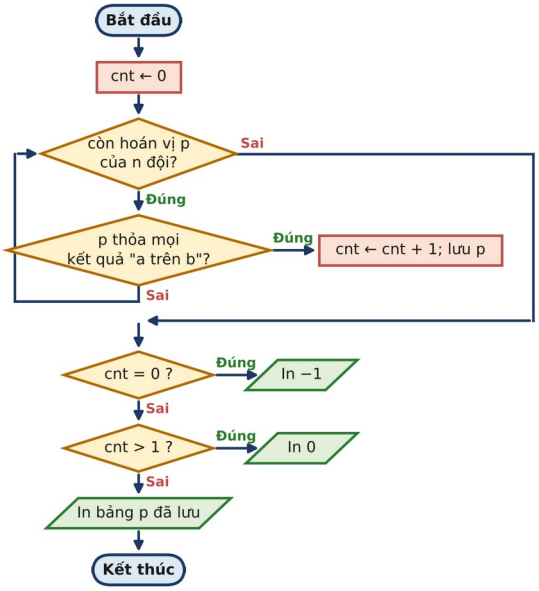
Mỗi kết quả "a b" là một cung a → b. Một bảng xếp hạng thỏa mãn chính là một **thứ tự tô pô** (Tập 1, Bài 12). Có ba khả năng: không có thứ tự nào (đồ thị có chu trình), có đúng một, hoặc có nhiều.



Hình 12.1.1. *Ba ví dụ: duy nhất, nhiều bảng, mâu thuẫn.*

2. Cách ngây thơ: Thử mọi bảng xếp hạng

Với n ≤ 8 có 8! = 40 320 hoán vị. Thử từng hoán vị, kiểm tra cả m kết quả; đếm số hoán vị hợp lệ (đếm tới 2 là đủ).



Hình 12.1.2. *Sơ đồ khối của cách thử mọi hoán vị.*

Cách ngây thơ: duyệt mọi hoán vị

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("XHGIAIDAU.INP", "r")) {
        freopen("XHGIAIDAU.INP", "r", stdin);
        freopen("XHGIAIDAU.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<pair<int,int>> e(m);
    for (auto &x : e) scanf("%d %d", &x.first, &x.second);
    vector<int> p(n), pos(n + 1), found;
    iota(p.begin(), p.end(), 1);
    int cnt = 0;
    do {                                          // thử mọi bảng xếp hạng
        for (int i = 0; i < n; i++) pos[p[i]] = i;
        bool ok = true;
        for (auto &x : e) if (pos[x.first] > pos[x.second]) { ok = false;
            break; }
        if (ok) { cnt++; found = p; if (cnt > 1) break; }
    } while (next_permutation(p.begin(), p.end()));
    if (cnt == 0) printf("-1\n");
    else if (cnt > 1) printf("0\n");
    else for (int i = 0; i < n; i++) printf("%d%c", found[i], i == n - 1 ?
        '\n' : ' ');
    return 0;
}
```

Độ phức tạp: O(n! · m): chỉ dùng được khi n rất nhỏ.

3. Bế tắc

- **Subtask 1** ($n \le 8$): **30 điểm**.

- **Subtask 2**: 10^5! hoán vị, không thể.

Chuyển sang Kahn là đúng hướng. Nhưng một lỗi rất hay gặp là **dừng sớm**: thấy hàng đợi có hai đội là in 0 ngay. Nếu phía sau còn một vòng mâu thuẫn thì đáp số phải là −1 (mâu thuẫn được ưu tiên). Ví dụ n = 4 với hai kết quả "3 4" và "4 3": hàng đợi ban đầu là {1, 2}, chương trình in 0, nhưng đáp án là −1. Cách này chỉ được **0 điểm**: test nào có cả hai hiện tượng đều sai.

> **ĐIỂM BẾ TẮC**
>
> Chạy Kahn thì ra một thứ tự, nhưng làm sao biết thứ tự đó **duy nhất**, và làm sao kiểm tra mâu thuẫn **trước khi** kết luận "nhiều đáp án"?

4. Phá bế tắc

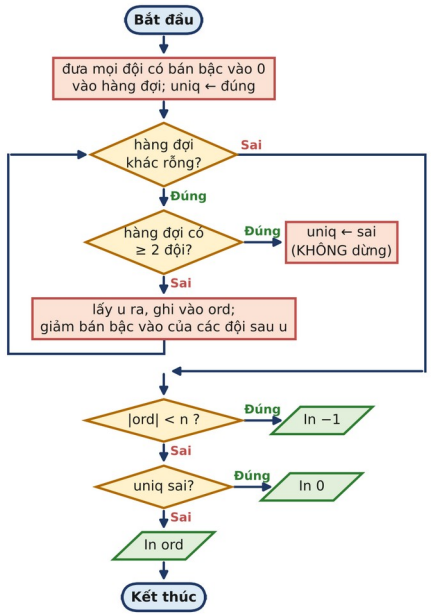
Chạy Kahn **đến hết** và ghi lại hai thông tin:

- **Có vòng?** Số đội lấy ra được ít hơn n thì các đội còn lại đều nằm trên (hoặc phía sau) một vòng: in −1.
- **Duy nhất?** Nếu có lúc hàng đợi chứa từ hai đội trở lên, hai đội đó có thể đổi thứ tự cho nhau (không ai "trên" ai), nên có nhiều đáp án. Ngược lại, mỗi bước chỉ có một lựa chọn, nên thứ tự là duy nhất.

Chạy tay ví dụ 1 (cung 1 → 3, 3 → 2, 2 → 4, 1 → 2):

Bước	Hàng đợi	Lấy ra	Bán bậc vào còn lại
khởi đầu	{1}		2: 2, 3: 1, 4: 1
1	{1}	1	2: 1, 3: **0** → vào hàng đợi
2	{3}	3	2: **0** → vào hàng đợi
3	{2}	2	4: **0** → vào hàng đợi
4	{4}	4	lấy đủ 4 đội

Hàng đợi luôn chỉ có một đội, lấy ra đủ 4 đội: bảng duy nhất **1 3 2 4**. Ở ví dụ 2, ngay từ đầu hàng đợi là {1, 3}: in 0. Ở ví dụ 3, không đội nào có bán bậc vào 0: lấy ra 0 < 3 đội, in −1.



Hình 12.1.3. *Sơ đồ khối của Kahn có kiểm tra tính duy nhất.*

VÌ SAO "HÀNG ĐỢI LUÔN MỘT ĐỘI" LÀ ĐỦ?

Nếu mỗi bước chỉ có đúng một đội u sẵn sàng, thì đội lấy ra liền sau u phải có cung từ u (nếu không, nó đã sẵn sàng cùng lúc với u). Vậy thứ tự tìm được là một đường đi qua cả n đội, và mọi thứ tự tô pô đều phải tôn trọng đường đi đó: chỉ có một thứ tự.

5. Cách tối ưu

XHGIAIDAU.CPP: Kahn, kiểm tra vòng và tính duy nhất

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("XHGIAIDAU.INP", "r")) {
        freopen("XHGIAIDAU.INP", "r", stdin);
        freopen("XHGIAIDAU.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<vector<int>> g(n + 1);
    vector<int> in(n + 1, 0);                    // bán bậc vào
    for (int i = 0; i < m; i++) {
        int a, b;
        scanf("%d %d", &a, &b);
        g[a].push_back(b); in[b]++;
    }
    queue<int> q;
    for (int v = 1; v <= n; v++) if (in[v] == 0) q.push(v);
    vector<int> ord;
    bool uniq = true;
    while (!q.empty()) {
        if (q.size() > 1) uniq = false;          // có hai đội cùng "sẵn sàng":
                                                 //   nhiều bảng
        int u = q.front(); q.pop();              // nhưng KHÔNG dừng: còn phải
                                                 //   kiểm tra mâu thuẫn
        ord.push_back(u);
        for (int v : g[u]) if (--in[v] == 0) q.push(v);
    }
    if ((int)ord.size() < n) printf("-1\n");     // không lấy ra đủ n đội: có vòng
    else if (!uniq) printf("0\n");
    else for (int i = 0; i < n; i++) printf("%d%c", ord[i], i == n - 1 ? '\n'
                                            : ' ');
    return 0;
}
```

Độ phức tạp: $O(n + m)$.

6. Bẫy thường gặp

- **Dừng sớm khi hàng đợi có hai đội** mà chưa kiểm tra vòng.
- **Kiểm tra tính duy nhất chỉ ở bước đầu** (chỉ đếm số đội có bán bậc vào 0 lúc đầu).
- **Dùng hàng đợi ưu tiên** rồi so sánh với một thứ tự khác: tốn công mà không cần thiết.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (70)	Tổng
Duyệt mọi hoán vị	30	0	**30**
Kahn, dừng ngay khi hàng đợi có hai đội	0	0	**0**

Thuật toán	ST1 (30)	ST2 (70)	Tổng
Kahn chạy đến hết	30	70	**100**

Chấm trên 8 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

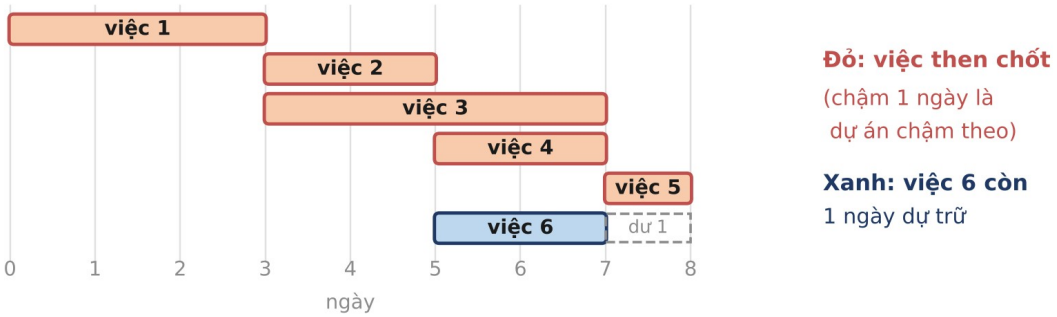
BÀI 12.2 DỰ ÁN XÂY NHÀ

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
XAYNHA.INP	XAYNHA.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Tập 1, Bài 12 · Đề bài: Tập 2, trang 90

1. Hiểu bài toán

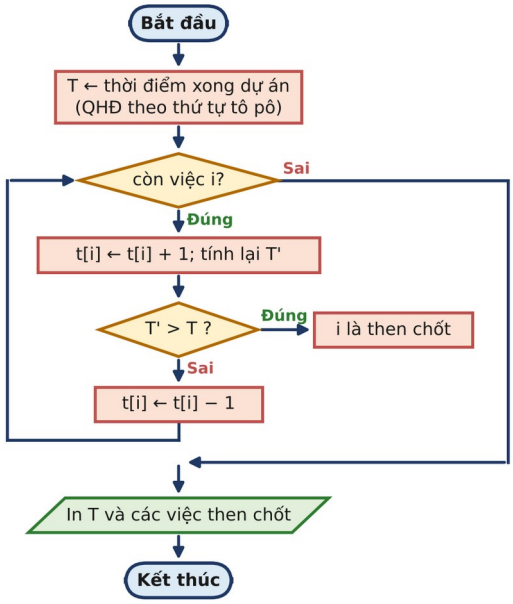
Mỗi công việc là một đỉnh có trọng số t_i; cung p → i nghĩa là p phải xong trước i. Thời điểm hoàn thành dự án là độ dài **đường đi dài nhất** (tính tổng trọng số đỉnh) trên DAG. Việc **then chốt** là việc nằm trên **một** đường đi dài nhất nào đó: kéo dài nó thêm 1 thì đường đi ấy dài thêm 1.



Hình 12.2.1. *Ví dụ: dự án xong lúc 8; có hai đường dài nhất.*

2. Cách ngây thơ: Thử kéo dài từng việc

Làm đúng như định nghĩa: tính T; với mỗi việc i, tăng t_i thêm 1, tính lại T' và so sánh.



Hình 12.2.2. *Sơ đồ khối của cách thử từng việc.*

Cách ngây thơ: thử kéo dài từng việc

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
int n;
vector<ll> t;
vector<vector<int>> g;
vector<int> ord;

ll total() {                                        // thời điểm hoàn thành dự án
    vector<ll> es(n + 1, 0);
    ll T = 0;
    for (int u : ord) {
        T = max(T, es[u] + t[u]);
        for (int v : g[u]) es[v] = max(es[v], es[u] + t[u]);
    }
    return T;
}

int main() {
    if (fopen("XAYNHA.INP", "r")) {
        freopen("XAYNHA.INP", "r", stdin);
        freopen("XAYNHA.OUT", "w", stdout);
    }
    scanf("%d", &n);
    t.resize(n + 1); g.resize(n + 1);
    vector<int> in(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        int k; scanf("%lld %d", &t[i], &k);
        while (k--) { int p; scanf("%d", &p); g[p].push_back(i); in[i]++; }
    }
    queue<int> q;
    for (int v = 1; v <= n; v++) if (!in[v]) q.push(v);
    while (!q.empty()) { int u = q.front(); q.pop(); ord.push_back(u); for (int v : g[u]) if (--in[v] == 0) q.push(v); }
    ll T = total();
    vector<int> c;
    for (int i = 1; i <= n; i++) {                  // thử kéo dài từng việc thêm 1 ngày
        t[i]++;
        if (total() > T) c.push_back(i);
        t[i]--;
    }
    printf("%lld\n%d\n", T, (int)c.size());
    for (size_t i = 0; i < c.size(); i++) printf("%d%c", c[i], i + 1 == c.size() ? '\n' : ' ');
    return 0;
}
```

Độ phức tạp: $O(n \cdot (n + m))$: $n = 1000$ khoảng $3 \cdot 10^6$; $n = 10^5$ khoảng $3 \cdot 10^{10}$.

3. Bế tắc

- **Subtask 1** ($n \le 1000$): **40 điểm**.
- **Subtask 2**: quá thời gian.

Một lời giải nhanh nhưng sai: tìm **một** đường dài nhất (lần ngược từ việc kết thúc muộn nhất) rồi in các việc trên đường đó. Ở ví dụ, đường $1 \to 3 \to 5$ và đường $1 \to 2 \to 4 \to 5$ đều dài 8;

lần ngược chỉ được một trong hai. Kết quả: **0 điểm**, vì khi thời gian các việc nhỏ, có rất nhiều đường dài nhất.

> **ĐIỂM BẾ TẮC**
> Cần biết **mỗi** việc có nằm trên **một** đường dài nhất nào đó hay không, mà không thử từng việc và không liệt kê các đường.

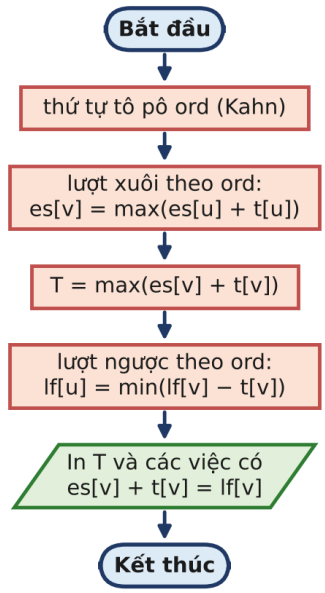
4. Phá bế tắc

Một đường đi qua việc v gồm hai nửa: nửa **trước** v và nửa **sau** v. Tính riêng hai nửa bằng hai lượt QHĐ theo thứ tự tô pô:

- **Lượt xuôi:** es[v] = thời điểm bắt đầu sớm nhất = max(es[p] + t_p) trên các việc p làm trước v. T = max(es[v] + t_v).
- **Lượt ngược** (thứ tự tô pô đảo lại): lf[v] = thời điểm kết thúc muộn nhất mà dự án vẫn xong lúc T = min(lf[w] − t_w) trên các việc w làm sau v (lf[v] = T nếu không có việc nào sau v).
- Việc v **then chốt** khi và chỉ khi nó **không có thời gian dự trữ**: es[v] + t_v = lf[v].

Chạy tay ví dụ (t = 3, 2, 4, 2, 1, 2):

Việc	t	es	es + t	lf	Then chốt?
1	3	0	3	3	**có**
2	2	3	5	min(7 − 2, 8 − 2) = 5	**có**
3	4	3	7	8 − 1 = 7	**có**
4	2	5	7	8 − 1 = 7	**có**
5	1	max(7, 7) = 7	**8**	8	**có**
6	2	5	7	8	không (dư 1 ngày)



Hình 12.2.3. *Sơ đồ khối của hai lượt QHĐ.*

5. Cách tối ưu

XAYNHA.CPP: hai lượt QHĐ trên thứ tự tô pô

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

int main() {
    if (fopen("XAYNHA.INP", "r")) {
        freopen("XAYNHA.INP", "r", stdin);
        freopen("XAYNHA.OUT", "w", stdout);
    }
    int n;
    scanf("%d", &n);
    vector<ll> t(n + 1);
    vector<vector<int>> g(n + 1);               // cung p -> i: p phải làm trước i
    vector<int> in(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        int k;
        scanf("%lld %d", &t[i], &k);
        while (k--) { int p; scanf("%d", &p); g[p].push_back(i); in[i]++; }
    }
    vector<int> ord;                            // thứ tự tô pô (Kahn)
    queue<int> q;
    for (int v = 1; v <= n; v++) if (!in[v]) q.push(v);
    while (!q.empty()) {
        int u = q.front(); q.pop(); ord.push_back(u);
        for (int v : g[u]) if (--in[v] == 0) q.push(v);
    }
    // lượt xuôi: es[v] = thời điểm bắt đầu sớm nhất
    vector<ll> es(n + 1, 0);
    ll T = 0;
    for (int u : ord) {
        T = max(T, es[u] + t[u]);
        for (int v : g[u]) es[v] = max(es[v], es[u] + t[u]);
    }
    // lượt ngược: lf[v] = thời điểm kết thúc muộn nhất mà không làm dự án trễ
    vector<ll> lf(n + 1, T);
    for (int i = n - 1; i >= 0; i--) {
        int u = ord[i];
        for (int v : g[u]) lf[u] = min(lf[u], lf[v] - t[v]);
    }
    vector<int> c;                              // then chốt: không có thời gian dự trữ
    for (int v = 1; v <= n; v++) if (es[v] + t[v] == lf[v]) c.push_back(v);
    printf("%lld\n%d\n", T, (int)c.size());
    for (size_t i = 0; i < c.size(); i++) printf("%d%c", c[i], i + 1 == c.size() ? '\n' : ' ');
    return 0;
}
```

Độ phức tạp: $O(n + \Sigma k_i)$.

6. Bẫy thường gặp

- **Chỉ lần một đường dài nhất.**
- **Dùng int:** T có thể tới $10^5 \cdot 10^9 = 10^{14}$.

- **Duyệt theo số hiệu công việc** thay vì thứ tự tô pô: công việc i có thể phụ thuộc vào công việc có số hiệu lớn hơn.
- **Lượt ngược bắt đầu từ lf =** ∞ thay vì lf = T với các việc cuối cùng.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (40)	ST2 (60)	Tổng
Thử kéo dài từng việc	40	0	**40**
Chỉ lần một đường dài nhất	0	0	**0**
Hai lượt QHĐ (sớm nhất / muộn nhất)	40	60	**100**

Chấm trên 6 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

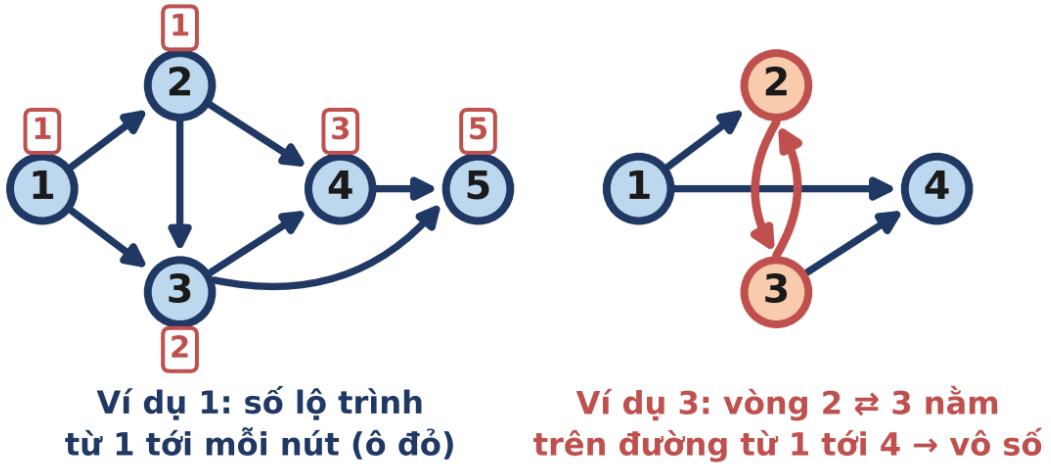
BÀI 12.3 ĐẾM LỘ TRÌNH

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
DEMLOTRINH.INP	DEMLOTRINH.OUT	1 giây	256 MB	★★★ NÂNG CAO

Liên hệ: Tập 1, Bài 12 · Đề bài: Tập 2, trang 92

1. Hiểu bài toán

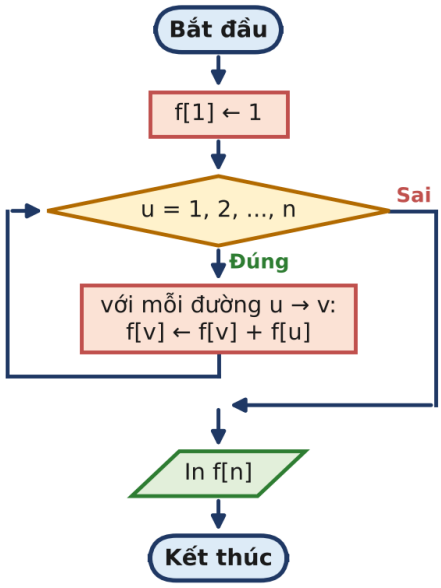
Nếu đồ thị không có chu trình, số lộ trình được đếm bằng QHĐ trên thứ tự tô pô: f[1] = 1, f[v] = tổng f[u] trên các đường u → v. Khó khăn là đồ thị **có thể có chu trình**: một vòng chỉ gây ra vô số lộ trình khi nó vừa **tới được từ 1**, vừa **tới được n**.



Hình 12.3.1. *Ba ví dụ: 5 lộ trình; vòng vô hại; vòng làm vô số lộ trình.*

2. Cách ngây thơ: QHĐ theo số hiệu nút

Ở Subtask 1, mọi con đường có u < v, nên thứ tự 1, 2, ..., n đã là một thứ tự tô pô: cứ duyệt u tăng dần và cộng f[u] vào f[v].



Hình 12.3.2. *Sơ đồ khối của QHĐ theo số hiệu.*

Cách ngây thơ: QHĐ theo số hiệu nút

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    if (fopen("DEMLOTRINH.INP", "r")) {
        freopen("DEMLOTRINH.INP", "r", stdin);
        freopen("DEMLOTRINH.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<vector<int>> g(n + 1);
    for (int i = 0; i < m; i++) { int u, v; scanf("%d %d", &u, &v);
        g[u].push_back(v); }
    vector<long long> f(n + 1, 0);
    f[1] = 1;
    for (int u = 1; u <= n; u++)                // giả sử mọi đường đều có u
<
        v
        for (int v : g[u]) f[v] = (f[v] + f[u]) % MOD;
    printf("%lld\n", f[n]);
    return 0;
}
```

Độ phức tạp: O(n + m), nhưng chỉ đúng khi u < v.

3. Bế tắc

- **Subtask 1** (u < v): **30 điểm**.
- **Subtask 2, 3**: số hiệu không còn là thứ tự tô pô, và có chu trình: sai.

Cải tiến "nửa vời": chạy Kahn trên các nút tới được từ 1, gặp chu trình là in −1. Cách này sai ở ví dụ 2: vòng 3 ⇄ 4 tới được từ 1 nhưng không dẫn tới 5. Chỉ được **30 điểm**.

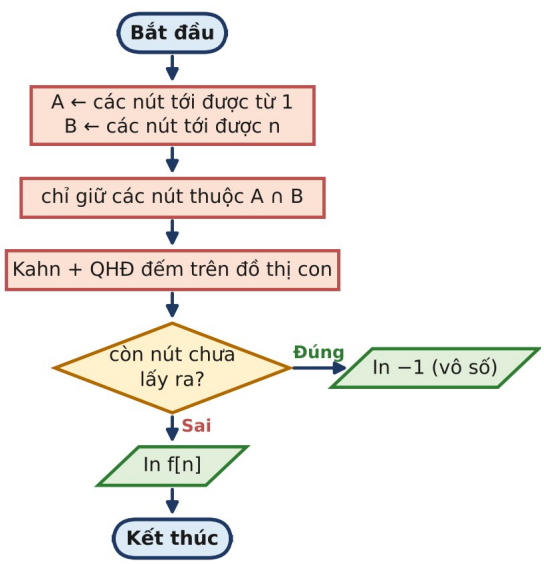
> **ĐIỂM BẾ TẮC**
>
> Phải phân biệt vòng **có ích** (nằm trên một lộ trình 1 → n) với vòng **vô hại** (không tới được từ 1, hoặc không dẫn tới n).

4. Phá bế tắc

Một nút v nằm trên một lộ trình 1 → n khi và chỉ khi v **tới được từ 1** (tập A, duyệt từ 1 trên đồ thị) và **tới được n** (tập B, duyệt từ n trên đồ thị đảo chiều). Chỉ giữ lại các nút thuộc A ∩ B và các đường nối hai nút như vậy:

- Nếu đồ thị con này có chu trình: có thể đi vòng bao nhiêu lần tùy ý rồi mới tới n, nên **vô số** lộ trình (in −1).
- Ngược lại nó là DAG: chạy Kahn kèm QHĐ đếm f, lấy theo mô-đun.

Ví dụ	A (từ 1)	B (tới n)	Kết luận
1	{1, 2, 3, 4, 5}	{1, 2, 3, 4, 5}	DAG: f = 1, 1, 2, 3, **5**
2	{1, 2, 3, 4, 5}	{1, 2, 5}	bỏ 3, 4 (vòng vô hại): **1**
3	{1, 2, 3, 4}	{1, 2, 3, 4}	vòng 2 ⇄ 3 còn lại: **−1**



Hình 12.3.3. *Sơ đồ khối của lời giải.*

5. Cách tối ưu

DEMLOTRINH.CPP: lọc nút có ích, Kahn + QHĐ

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;
int n, m;

vector<char> reach(int s, vector<vector<int>> &G) {   // các đỉnh tới được từ
    s
    vector<char> vis(n + 1, 0);
    vector<int> st = {s}; vis[s] = 1;
    while (!st.empty()) {
        int u = st.back(); st.pop_back();
        for (int v : G[u]) if (!vis[v]) { vis[v] = 1; st.push_back(v); }
    }
    return vis;
}

int main() {
    if (fopen("DEMLOTRINH.INP", "r")) {
        freopen("DEMLOTRINH.INP", "r", stdin);
        freopen("DEMLOTRINH.OUT", "w", stdout);
    }
    scanf("%d %d", &n, &m);
    vector<vector<int>> g(n + 1), rg(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v; scanf("%d %d", &u, &v);
        g[u].push_back(v); rg[v].push_back(u);
    }
    vector<char> A = reach(1, g), B = reach(n, rg);   // A: từ 1 tới được; B:
        tới được n
    if (!A[n]) { printf("0\n"); return 0; }
    // chỉ giữ các nút "có ích": vừa thuộc A vừa thuộc B
    vector<int> in(n + 1, 0);
    int tot = 0;
    for (int u = 1; u <= n; u++) if (A[u] && B[u]) {
        tot++;
        for (int v : g[u]) if (A[v] && B[v]) in[v]++;
    }
    if (in[1]) { printf("-1\n"); return 0; }   // có vòng quay về 1
    vector<long long> f(n + 1, 0);              // f[v]: số lộ trình 1 -> v
        (mod)
    f[1] = 1;
    queue<int> q; q.push(1);
    int done = 0;
    while (!q.empty()) {                        // Kahn trên đồ thị con có ích
        int u = q.front(); q.pop(); done++;
        for (int v : g[u]) if (A[v] && B[v]) {
            f[v] = (f[v] + f[u]) % MOD;
            if (--in[v] == 0) q.push(v);
        }
    }
    if (done < tot) printf("-1\n");             // còn nút chưa lấy ra: có
        vòng trên lộ trình
    else printf("%lld\n", f[n]);
    return 0;
}
```

Độ phức tạp: O(n + m).

6. Bẫy thường gặp

- **Coi mọi chu trình là vô số lộ trình.**
- **Quên khuyên** u → u: đó cũng là một vòng (Kahn tự phát hiện vì bán bậc vào không bao giờ về 0).
- **In 0 khi không tới được n?** Đúng, nhưng nhớ kiểm tra trước khi xét vòng.
- **Kiểm tra "vô số" bằng f lớn:** f đã lấy mô-đun nên không dùng để phát hiện vô hạn được.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
QHĐ theo số hiệu nút	30	0	0	**30**
Mọi chu trình đều là vô số	30	0	0	**30**
Lọc nút có ích + Kahn	30	30	40	**100**

Chấm trên 8 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

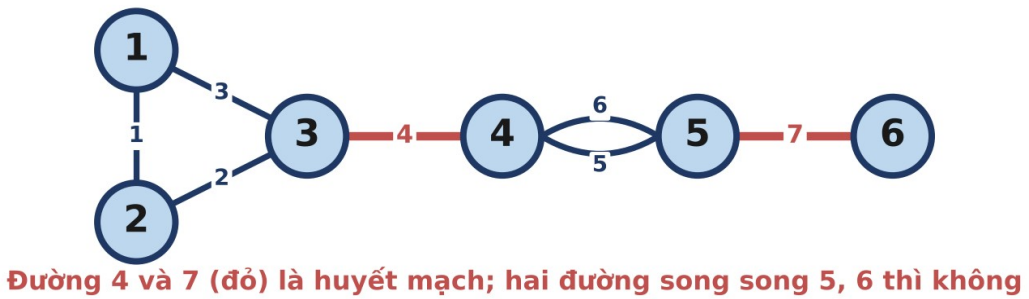
BÀI 13.1 ĐƯỜNG HUYẾT MẠCH

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
HUYETMACH.INP	HUYETMACH.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Tập 1, Bài 13 · Đề bài: Tập 2, trang 94

1. Hiểu bài toán

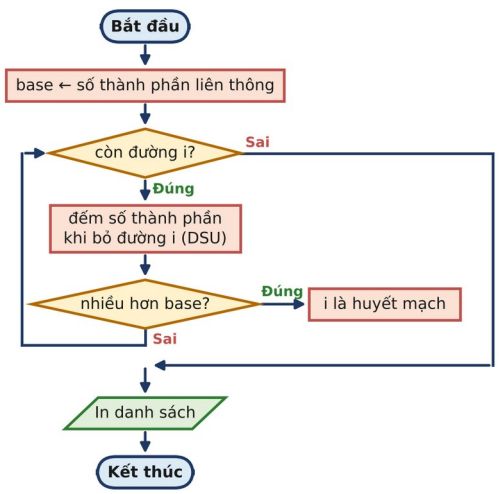
Con đường huyết mạch chính là **cầu** của đồ thị vô hướng (Tập 1, Bài 13). Điểm khác: có thể có **nhiều con đường nối cùng một cặp nút**; hai con đường song song không bao giờ là cầu.



Hình 13.1.1. *Ví dụ: đường 4 và 7 là huyết mạch; đường 5 và 6 song song.*

2. Cách ngây thơ: Chặn thử từng con đường

Đếm số thành phần liên thông (DSU). Với mỗi con đường i, đếm lại khi bỏ i; tăng lên thì i là huyết mạch.



Hình 13.1.2. *Sơ đồ khối của cách chặn thử.*

Cách ngây thơ: chặn thử từng con đường

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<int> U, V, p;
int root(int x) { while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; } return x; }

int comps(int skip) {                          // số thành phần khi bỏ con đường skip
    p.resize(n + 1); iota(p.begin(), p.end(), 0);
    int c = n;
    for (int i = 1; i <= m; i++) if (i != skip) {
        int a = root(U[i]), b = root(V[i]);
        if (a != b) { p[a] = b; c--; }
    }
    return c;
}

int main() {
    if (fopen("HUYETMACH.INP", "r")) {
        freopen("HUYETMACH.INP", "r", stdin);
        freopen("HUYETMACH.OUT", "w", stdout);
    }
    scanf("%d %d", &n, &m);
    U.resize(m + 1); V.resize(m + 1);
    for (int i = 1; i <= m; i++) scanf("%d %d", &U[i], &V[i]);
    int base = comps(0);
    vector<int> br;
    for (int i = 1; i <= m; i++) if (comps(i) > base) br.push_back(i);   // chặn thử từng đường
    printf("%d\n", (int)br.size());
    for (size_t i = 0; i < br.size(); i++) printf("%d%c", br[i], i + 1 == br.size() ? '\n' : ' ');
    return 0;
}
```

Độ phức tạp: $O(m \cdot (n + m))$.

3. Bế tắc

- **Subtask 1** ($n, m \le 1000$): **30 điểm**.
- **Subtask 2, 3**: $2 \cdot 10^5$ lần dựng DSU: quá thời gian.

Thuật toán num/low trong nhiều tài liệu viết "bỏ qua đỉnh cha" (`if (v == parent) continue`). Với đồ thị có cạnh song song, cách viết này bỏ qua **cả** con đường song song thứ hai, nên coi đường 5 là cầu. Chỉ đúng khi không có đường song song: **30 điểm** (chỉ Subtask 2).

ĐIỂM BẾ TẮC

Cạnh song song là một "đường vòng" độ dài 2. Phải bỏ qua đúng **con đường** vừa đi tới, không phải mọi con đường dẫn về đỉnh cha.

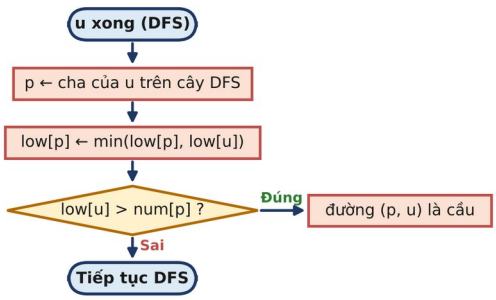
4. Phá bế tắc

Lưu số hiệu con đường cùng với đỉnh kề. Khi DFS từ u, bỏ qua đúng con đường pe[u] đã dùng để tới u. Con đường song song thứ hai có số hiệu khác, nên được coi là **cạnh ngược**, cập nhật low.

Đường (p, u) trên cây DFS là cầu khi và chỉ khi **low[u] > num[p]**: từ cây con gốc u không có cạnh ngược nào "trèo" lên tới p hoặc cao hơn.

Chạy tay ví dụ (DFS từ 1):

Đỉnh	Tới bằng	num	low	Kết luận khi xong
1		1	1	
2	đường 1	2	1	
3	đường 2	3	1 (đường 3 về 1)	
4	đường 4	4	4	low[4] = 4 > num[3] = 3: **đường 4 là cầu**
5	đường 5	5	4 (đường 6 về 4)	low[5] = 4 = num[4]: không
6	đường 7	6	6	low[6] = 6 > num[5] = 5: **đường 7 là cầu**



Hình 13.1.3. *Xử lí khi DFS rời khỏi đỉnh u.*

DFS KHÔNG ĐỆ QUY

Đồ thị là một đường thẳng 10^5 đỉnh thì DFS đệ quy sâu 10^5 tầng. Lời giải dùng ngăn xếp tự quản lí: st chứa đường đi từ gốc, it[u] là vị trí cạnh tiếp theo cần xét của u. Khi u hết cạnh, lấy u ra và cập nhật low của đỉnh cha (đỉnh mới ở đỉnh ngăn xếp).

5. Cách tối ưu

HUYETMACH.CPP: num/low không đệ quy, bỏ qua theo số hiệu cạnh

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("HUYETMACH.INP", "r")) {
        freopen("HUYETMACH.INP", "r", stdin);
        freopen("HUYETMACH.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<vector<pair<int,int>>> g(n + 1);     // (đỉnh kề, số hiệu đường)
    for (int i = 1; i <= m; i++) {
        int u, v; scanf("%d %d", &u, &v);
        g[u].push_back({v, i}); g[v].push_back({u, i});
    }
    vector<int> num(n + 1, 0), low(n + 1, 0), pe(n + 1, 0), it(n + 1, 0),
br;
    int timer = 0;
    for (int s = 1; s <= n; s++) if (!num[s]) {
        // DFS không đệ quy: st giữ đường đi từ gốc tới đỉnh đang xét
        vector<int> st = {s};
        num[s] = low[s] = ++timer;
        while (!st.empty()) {
            int u = st.back();
            if (it[u] < (int)g[u].size()) {
                auto [v, id] = g[u][it[u]++];
                if (id == pe[u]) continue;      // bỏ đúng CON ĐƯỜNG vừa đi
                    tới, không bỏ đỉnh cha
                if (num[v]) low[u] = min(low[u], num[v]);
                else { num[v] = low[v] = ++timer; pe[v] = id;
st.push_back(v);
                    }
            } else {                            // u xong: báo kết quả cho
cha
                st.pop_back();
                if (!st.empty()) {
                    int p = st.back();
                    low[p] = min(low[p], low[u]);
                    if (low[u] > num[p]) br.push_back(pe[u]);   // cây con
của
                        u không "trèo" qua được
                }
            }
        }
    }
    sort(br.begin(), br.end());
    printf("%d\n", (int)br.size());
    for (size_t i = 0; i < br.size(); i++) printf("%d%c", br[i], i + 1 ==
        br.size() ? '\n' : ' ');
    return 0;
}
```

Độ phức tạp: O(n + m).

6. Bẫy thường gặp

- **Bỏ qua theo đỉnh cha** khi có cạnh song song.
- **Quên đồ thị không liên thông:** phải DFS từ mọi đỉnh chưa thăm.

- **Dùng low[u] ≥ num[p]** (điều kiện của khớp) cho cầu.
- **In số hiệu theo thứ tự tìm được**, quên sắp xếp tăng dần.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
Chặn thử từng con đường	30	0	0	**30**
num/low, bỏ qua đỉnh cha	0	30	0	**30**
num/low, bỏ qua cạnh vừa đi tới	30	30	40	**100**

Chấm trên 9 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

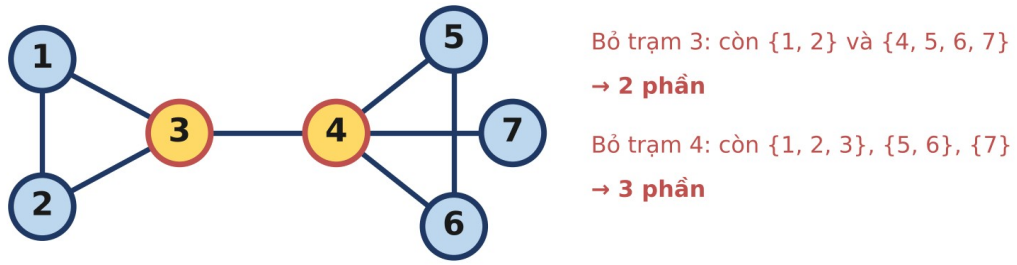
BÀI 13.2 TRẠM BIẾN ÁP

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
BIENAP.INP	BIENAP.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Tập 1, Bài 13 · Đề bài: Tập 2, trang 96

1. Hiểu bài toán

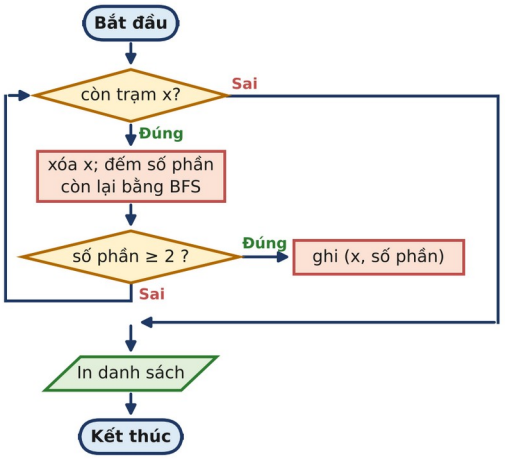
Trạm có sự cố làm lưới bị chia cắt chính là **khớp** (đỉnh khớp). Ở đây cần thêm **số phần** sau khi bỏ khớp đó.



Hình 13.2.1. *Ví dụ: trạm 3 chia lưới thành 2 phần, trạm 4 thành 3 phần.*

2. Cách ngây thơ: Cho từng trạm hỏng rồi đếm

Với mỗi trạm x, bỏ x đi và đếm số thành phần liên thông của phần còn lại bằng BFS.



Hình 13.2.2. *Sơ đồ khối của cách thử từng trạm.*

Cách ngây thơ: thử từng trạm

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("BIENAP.INP", "r")) {
        freopen("BIENAP.INP", "r", stdin);
        freopen("BIENAP.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<vector<int>> g(n + 1);
    for (int i = 0; i < m; i++) { int u, v; scanf("%d %d", &u, &v);
        g[u].push_back(v); g[v].push_back(u); }
    vector<int> mark(n + 1, 0);
    vector<pair<int,int>> ans;
    for (int x = 1; x <= n; x++) {              // cho trạm x hỏng, đếm số
        phần bằng BFS
        mark[x] = x;
        int parts = 0;
        for (int s = 1; s <= n; s++) if (mark[s] != x) {
            parts++;
            queue<int> q; q.push(s); mark[s] = x;
            while (!q.empty()) {
                int u = q.front(); q.pop();
                for (int v : g[u]) if (mark[v] != x) { mark[v] = x;
q.push(v);
                    }
            }
        }
        if (parts >= 2) ans.push_back({x, parts});
    }
    printf("%d\n", (int)ans.size());
    for (auto [v, c] : ans) printf("%d %d\n", v, c);
    return 0;
}
```

Độ phức tạp: $O(n \cdot (n + m))$.

3. Bế tắc

- **Subtask 1** ($n, m \le 1000$): **30 điểm**.
- **Subtask 2**: 10^5 lần BFS trên $3 \cdot 10^5$ phần tử: quá thời gian.

Dùng num/low nhưng áp dụng cùng một công thức "số con bị tách + 1" cho **cả gốc**: gốc không có "phần phía trên", nên gốc có một con sẽ bị báo nhầm là chia lưới thành 2 phần. Kết quả: **0 điểm**.

ĐIỂM BẾ TẮC

Cần đếm, chỉ bằng một lần DFS, khi bỏ u thì các con của u trên cây DFS rơi vào bao nhiêu phần khác nhau.

4. Phá bế tắc

Xét đỉnh u và một con c của nó trên cây DFS. Khi u hỏng, cây con gốc c **bị tách riêng** khi và chỉ khi không có cạnh ngược nào từ cây con đó lên **trên** u: **low[c] ≥ num[u]**. Gọi cnt[u] là số con như vậy.

- u **không phải gốc**: mỗi con bị tách là một phần; các con còn lại cùng **phần phía trên** (chứa cha của u) tạo thêm 1 phần. Số phần = cnt[u] + 1.
- u **là gốc**: không có phần phía trên; mọi con của gốc đều có low ≥ num[gốc] = 1. Số phần = cnt[gốc] = số con.

Chạy tay ví dụ (DFS 1 → 2 → 3 → 4 → 5 → 6, rồi 4 → 7):

Đỉnh	num	low	Con c có low[c] ≥ num	Số phần khi hỏng
1 (gốc)	1	1	2 (con duy nhất)	cnt = 1: không chia cắt
2	2	1	không có	1
3	3	1	4 (low 4 ≥ 3)	1 + 1 = **2**
4	4	4	5 (low 4 ≥ 4), 7 (low 7 ≥ 4)	2 + 1 = **3**
5	5	4	không có (low[6] = 4 < 5)	1
6, 7	6, 7	4, 7	là lá	1

Đúng với đáp án: trạm 3 (2 phần) và trạm 4 (3 phần).

5. Cách tối ưu

BIENAP.CPP: num/low không đệ quy, đếm số con bị tách

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("BIENAP.INP", "r")) {
        freopen("BIENAP.INP", "r", stdin);
        freopen("BIENAP.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<vector<pair<int,int>>> g(n + 1);
    for (int i = 1; i <= m; i++) {
        int u, v; scanf("%d %d", &u, &v);
        g[u].push_back({v, i}); g[v].push_back({u, i});
    }
    // cnt[u] = số con c của u trên cây DFS có low[c] >= num[u]
    vector<int> num(n + 1, 0), low(n + 1, 0), pe(n + 1, 0), it(n + 1, 0),
        cnt(n + 1, 0);
    int timer = 0;
    vector<int> st = {1};
    num[1] = low[1] = ++timer;
    while (!st.empty()) {
        int u = st.back();
        if (it[u] < (int)g[u].size()) {
            auto [v, id] = g[u][it[u]++];
            if (id == pe[u]) continue;
            if (num[v]) low[u] = min(low[u], num[v]);
            else { num[v] = low[v] = ++timer; pe[v] = id; st.push_back(v); }
        } else {
            st.pop_back();
            if (!st.empty()) {
                int p = st.back();
                low[p] = min(low[p], low[u]);
                if (low[u] >= num[p]) cnt[p]++;  // cây con của u bị tách
khỏi
                    phần còn lại khi p hỏng
            }
        }
    }
    vector<pair<int,int>> ans;
    for (int v = 1; v <= n; v++) {
        int parts = (v == 1) ? cnt[v] : cnt[v] + 1;   // đỉnh không phải
gốc:
            thêm "phần phía trên"
        if (parts >= 2) ans.push_back({v, parts});
    }
    printf("%d\n", (int)ans.size());
    for (auto [v, c] : ans) printf("%d %d\n", v, c);
    return 0;
}
```

Độ phức tạp: O(n + m).

6. Bẫy thường gặp

- **Xử lí gốc như mọi đỉnh khác.**
- **Đếm số con có low[c] ≥ num[u] nhưng quên +1** cho phần phía trên.

- **DFS đệ quy** trên lưới dạng đường thẳng 10^5 trạm.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (70)	Tổng
Thử từng trạm, BFS đếm phần	30	0	**30**
num/low, quên trường hợp gốc	0	0	**0**
num/low, xử lí riêng gốc	30	70	**100**

Chấm trên 6 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

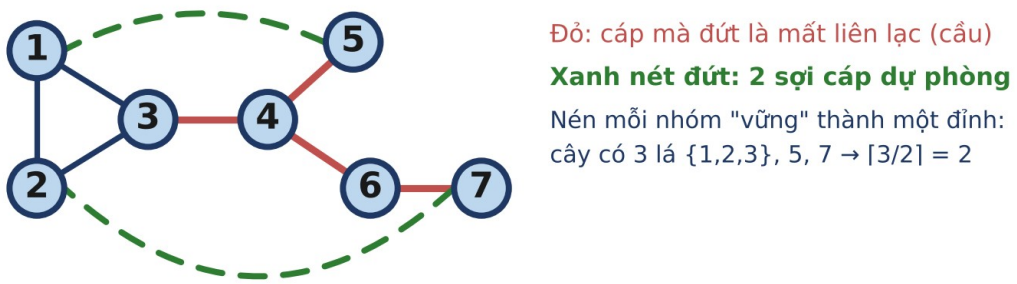
BÀI 13.3 CÁP DỰ PHÒNG

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
CAPDUPHONG.INP	CAPDUPHONG.OUT	1 giây	256 MB	★★★ NÂNG CAO

Liên hệ: Tập 1, Bài 13 · Đề bài: Tập 2, trang 98

1. Hiểu bài toán

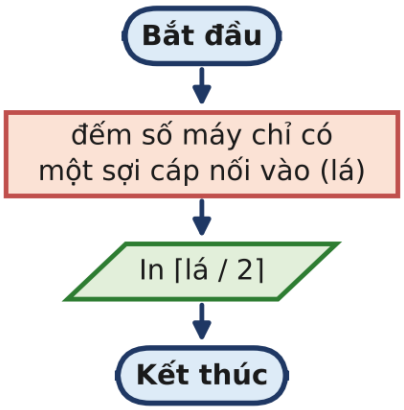
Mạng chịu được một sự cố khi và chỉ khi **không có cầu**. Cần thêm ít cáp nhất để mọi cầu biến mất. Một sợi cáp mới (a, b) "cứu" mọi cầu nằm trên đường đi từ a tới b.



Hình 13.3.1. *Ví dụ: bốn cầu, nén vòng {1, 2, 3} được cây có 3 lá.*

2. Cách ngây thơ: Coi mạng như một cây

Nếu mạng là một cây (Subtask 1), mọi cạnh đều là cầu. Mỗi lá phải được nối thêm ít nhất một sợi cáp, và một sợi cáp phủ được nhiều nhất hai lá: cần ⌈L / 2⌉ sợi với L là số lá (và cận này đạt được).



Hình 13.3.2. *Sơ đồ khối của cách đếm lá trên cây.*

Cách ngây thơ: đếm lá của đồ thị ban đầu

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("CAPDUPHONG.INP", "r")) {
        freopen("CAPDUPHONG.INP", "r", stdin);
        freopen("CAPDUPHONG.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<int> deg(n + 1, 0);
    for (int i = 0; i < m; i++) { int u, v; scanf("%d %d", &u, &v); deg[u]++;
        deg[v]++; }
    int leaves = 0;                              // coi mạng như một cây: đếm lá
    for (int v = 1; v <= n; v++) if (deg[v] == 1) leaves++;
    printf("%d\n", n == 1 ? 0 : (leaves + 1) / 2);
    return 0;
}
```

Độ phức tạp: O(n + m).

3. Bế tắc

- **Subtask 1** (cây): **20 điểm**.
- **Subtask 2, 3**: có vòng; đỉnh bậc 1 của đồ thị không còn là "lá" đúng nghĩa. Ở ví dụ, đồ thị chỉ có 2 đỉnh bậc 1 (5 và 7) nhưng đáp số là 2, không phải 1.

Tìm cầu bằng cách xóa thử từng cạnh rồi nén lại thì đúng, nhưng O(m · (n + m)): **30 điểm**.

ĐIỂM BẾ TẮC

Công thức ⌈L / 2⌉ chỉ dùng được trên một **cây**. Cần biến đồ thị thành một cây mà vẫn giữ nguyên các cầu.

4. Phá bế tắc

1) Tìm mọi cầu bằng num/low (như bài HUYETMACH).

2) **Nén** mỗi thành phần còn lại sau khi bỏ hết cầu (thành phần liên thông 2-cạnh) thành một đỉnh: dùng DSU gộp hai đầu của mọi cạnh **không phải** cầu.

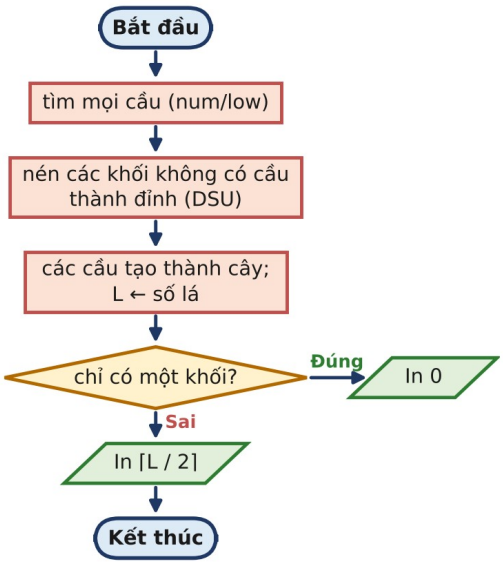
3) Các cầu nối các đỉnh nén thành một **cây** (gọi là cây cầu). Đếm số lá L = số đỉnh nén có đúng một cầu đi vào.

4) Nếu cây chỉ có một đỉnh (không có cầu), đáp số 0; ngược lại đáp số ⌈L / 2⌉.

Đỉnh nén	Các cầu nối vào	Bậc trên cây cầu
{1, 2, 3}	3 – 4	1: **lá**
{4}	3 – 4, 4 – 5, 4 – 6	3
{5}	4 – 5	1: **lá**

Đỉnh nén	Các cầu nối vào	Bậc trên cây cầu
{6}	4 – 6, 6 – 7	2
{7}	6 – 7	1: **lá**

L = 3, đáp số ⌈3 / 2⌉ = **2**.



Hình 13.3.3. *Sơ đồ khối của lời giải.*

VÌ SAO ⌈L / 2⌉ SỢI LÀ ĐỦ?

Đánh số các lá theo thứ tự DFS trên cây cầu: $\ell_1, \ell_2, ..., \ell_L$. Nối ℓ_i với $\ell_{i+L/2}$ (khi L lẻ, nối thêm lá cuối với một lá bất kì). Mỗi cạnh của cây chia các lá thành hai phần, mỗi phần là một **đoạn liên tiếp** trong thứ tự DFS (tính vòng quanh), và cách nối "cách nhau L/2" luôn có một sợi cáp đi qua hai phần. Vậy mọi cạnh của cây đều nằm trên một vòng mới.

5. Cách tối ưu

CAPDUPHONG.CPP: tìm cầu, nén bằng DSU, đếm lá

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> p;
int root(int x) { while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; } return x; }

int main() {
    if (fopen("CAPDUPHONG.INP", "r")) {
        freopen("CAPDUPHONG.INP", "r", stdin);
        freopen("CAPDUPHONG.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<vector<pair<int,int>>> g(n + 1);
    vector<int> U(m + 1), V(m + 1);
    for (int i = 1; i <= m; i++) {
        scanf("%d %d", &U[i], &V[i]);
        g[U[i]].push_back({V[i], i}); g[V[i]].push_back({U[i], i});
    }
    // Bước 1: tìm mọi cầu (num/low, không đệ quy)
    vector<int> num(n + 1, 0), low(n + 1, 0), pe(n + 1, 0), it(n + 1, 0);
    vector<char> isb(m + 1, 0);
    int timer = 0;
    vector<int> st = {1};
    num[1] = low[1] = ++timer;
    while (!st.empty()) {
        int u = st.back();
        if (it[u] < (int)g[u].size()) {
            auto [v, id] = g[u][it[u]++];
            if (id == pe[u]) continue;
            if (num[v]) low[u] = min(low[u], num[v]);
            else { num[v] = low[v] = ++timer; pe[v] = id; st.push_back(v); }
        } else {
            st.pop_back();
            if (!st.empty()) {
                int q = st.back();
                low[q] = min(low[q], low[u]);
                if (low[u] > num[q]) isb[pe[u]] = 1;
            }
        }
    }
    // Bước 2: nén mỗi "khối không cầu" thành một đỉnh (DSU trên các cạnh không phải cầu)
    p.resize(n + 1); iota(p.begin(), p.end(), 0);
    for (int i = 1; i <= m; i++) if (!isb[i]) p[root(U[i])] = root(V[i]);
    // Bước 3: các cầu tạo thành một cây; đếm lá của cây đó
    vector<int> deg(n + 1, 0);
    int comps = 0, leaves = 0;
    for (int v = 1; v <= n; v++) if (root(v) == v) comps++;
    for (int i = 1; i <= m; i++) if (isb[i]) { deg[root(U[i])]++; deg[root(V[i])]++; }
    for (int v = 1; v <= n; v++) if (root(v) == v && deg[v] == 1) leaves++;
    printf("%d\n", comps == 1 ? 0 : (leaves + 1) / 2);
    return 0;
}
```

Độ phức tạp: $O((n + m) \cdot \alpha(n))$.

6. Bẫy thường gặp

- **Đếm lá trên đồ thị ban đầu** thay vì trên cây cầu.
- **Quên trường hợp không có cầu:** cả mạng nén thành một đỉnh, đáp số là 0.
- **Bỏ qua cạnh song song** khi tìm cầu (như bài HUYETMACH).

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (20)	ST2 (30)	ST3 (50)	Tổng
Đếm lá của đồ thị ban đầu	20	0	0	**20**
Tìm cầu bằng cách xóa thử	0	30	0	**30**
num/low + nén + đếm lá	20	30	50	**100**

Chấm trên 9 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

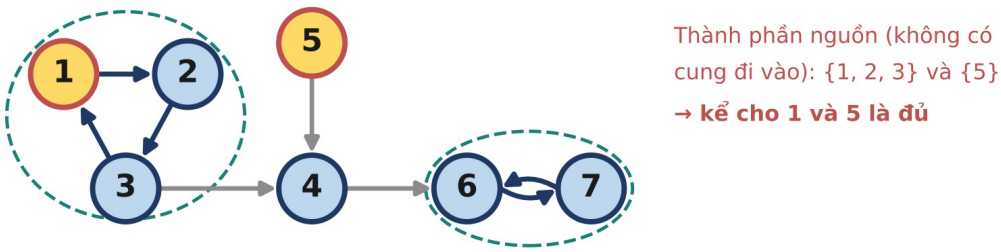
BÀI 14.1 LAN TRUYỀN TIN ĐỒN

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
TINDON.INP	TINDON.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Tập 1, Bài 14 · Đề bài: Tập 2, trang 100

1. Hiểu bài toán

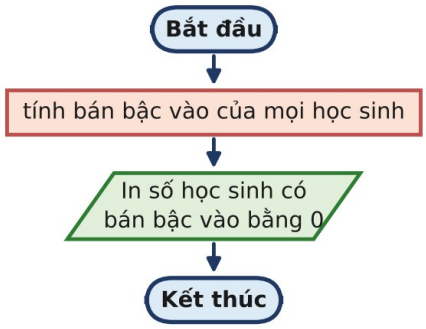
Báo tin cho một học sinh thì cả **thành phần liên thông mạnh** chứa bạn đó đều biết, rồi tin lan sang các thành phần "phía sau". Nén mỗi thành phần thành một đỉnh, ta được một DAG (Tập 1, Bài 14).



Hình 14.1.1. *Ví dụ: bốn thành phần {1, 2, 3}, {4}, {5}, {6, 7}.*

2. Cách ngây thơ: Đếm học sinh không ai kể cho

Một học sinh không có ai kể cho (bán bậc vào 0) thì bắt buộc phải được báo trực tiếp. Trên đồ thị không có chu trình (Subtask 1), báo cho đúng những bạn đó là đủ: từ mỗi đỉnh, đi ngược mãi các cung sẽ dừng ở một đỉnh có bán bậc vào 0.



Hình 14.1.2. *Sơ đồ khối của cách đếm bán bậc vào 0.*

Cách ngây thơ: đếm học sinh có bán bậc vào 0

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("TINDON.INP", "r")) {
        freopen("TINDON.INP", "r", stdin);
        freopen("TINDON.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<int> in(n + 1, 0);
    for (int i = 0; i < m; i++) { int a, b; scanf("%d %d", &a, &b); in[b]++;
}
    int cnt = 0;                                    // đếm những người không ai
kể
        cho
    for (int v = 1; v <= n; v++) if (in[v] == 0) cnt++;
    printf("%d\n", cnt);
    return 0;
}
```

Độ phức tạp: O(n + m).

3. Bế tắc

- **Subtask 1** (không có chu trình): **30 điểm**.
- **Subtask 2, 3**: một nhóm bạn kể tin vòng quanh cho nhau mà không ai bên ngoài kể cho họ (như nhóm {1, 2, 3} ở ví dụ) cần một người được báo, nhưng không ai trong nhóm có bán bậc vào 0. Ví dụ ra 1 thay vì 2.

Nén thành phần đúng, nhưng **nhầm chiều**: đếm các thành phần không có cung đi **ra** (những nhóm "cuối dòng" mà tin phải tới). Đó là số nhóm cần **nhận** tin, không phải số nhóm cần **được báo**: **0 điểm**.

ĐIỂM BẾ TẮC

"Bán bậc vào 0" chỉ đúng trên DAG. Cần biến đồ thị có chu trình thành DAG mà không làm thay đổi đáp số.

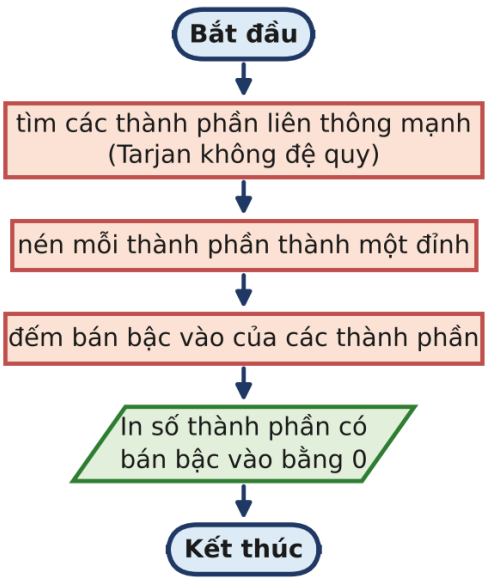
4. Phá bế tắc

Nén mỗi thành phần liên thông mạnh thành một đỉnh. Trên DAG mới, câu trả lời là số đỉnh có bán bậc vào 0 (lập luận của Subtask 1). Một thành phần có bán bậc vào 0 khi không có cung nào từ thành phần **khác** đi vào nó.

Thành phần	Cung đi vào từ thành phần khác	Phải báo trực tiếp?
{1, 2, 3}	không có	**có**
{4}	3 → 4, 5 → 4	không
{5}	không có	**có**

Thành phần	Cung đi vào từ thành phần khác	Phải báo trực tiếp?
{6, 7}	4 → 6	không

Đáp số **2**: báo cho một bạn trong {1, 2, 3} và cho bạn 5.



Hình 14.1.3. *Sơ đồ khối của lời giải.*

TARJAN KHÔNG ĐỆ QUY

Ngăn xếp cs mô phỏng ngăn xếp gọi hàm; ngăn xếp S là ngăn xếp của thuật toán Tarjan. Khi u hết cạnh: lấy u khỏi cs, cập nhật low của cha, và nếu low[u] = num[u] thì lấy khỏi S mọi đỉnh tới u: đó là một thành phần. Cũng có thể dùng Kosaraju với hai lần DFS không đệ quy.

5. Cách tối ưu

TINDON.CPP: Tarjan không đệ quy, đếm thành phần nguồn

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("TINDON.INP", "r")) {
        freopen("TINDON.INP", "r", stdin);
        freopen("TINDON.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<vector<int>> g(n + 1);
    vector<pair<int,int>> e(m);
    for (auto &x : e) { scanf("%d %d", &x.first, &x.second);
        g[x.first].push_back(x.second); }
    // Tarjan không đệ quy: comp[v] = số hiệu thành phần liên thông mạnh của v
    vector<int> num(n + 1, 0), low(n + 1, 0), comp(n + 1, -1), it(n + 1, 0),
        S, cs;
    vector<char> onS(n + 1, 0);
    int timer = 0, c = 0;
    for (int s = 1; s <= n; s++) if (!num[s]) {
        cs.push_back(s); num[s] = low[s] = ++timer; S.push_back(s); onS[s] =
            1;
        while (!cs.empty()) {
            int u = cs.back();
            if (it[u] < (int)g[u].size()) {
                int v = g[u][it[u]++];
                if (!num[v]) { num[v] = low[v] = ++timer; S.push_back(v);
                    onS[v] = 1; cs.push_back(v); }
                else if (onS[v]) low[u] = min(low[u], num[v]);
            } else {
                cs.pop_back();
                if (!cs.empty()) low[cs.back()] = min(low[cs.back()],
low[u]);
                if (low[u] == num[u]) {         // u là "gốc" của một thành
                    phần: lấy cả thành phần ra
                    int v;
                    do { v = S.back(); S.pop_back(); onS[v] = 0; comp[v] =
c;
                        } while (v != u);
                    c++;
                }
            }
        }
    }
    // đếm các thành phần không có cung đi vào từ thành phần khác
    vector<int> in(c, 0);
    for (auto &x : e) if (comp[x.first] != comp[x.second])
        in[comp[x.second]]++;
    printf("%d\n", (int)count(in.begin(), in.end(), 0));
    return 0;
}
```

Độ phức tạp: $O(n + m)$.

6. Bẫy thường gặp

- **Đếm học sinh có bán bậc vào 0** khi có chu trình.

- **Đếm cung trong cùng thành phần** vào bán bậc vào.
- **Tarjan đệ quy** khi máy chấm có ngăn xếp nhỏ (1 MB như nhiều máy chấm chạy Windows): vòng dài 10^5 đỉnh sẽ làm tràn ngăn xếp. Lời giải mẫu dùng bản không đệ quy cho an toàn.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
Đếm học sinh có bán bậc vào 0	30	0	0	**30**
Nén, đếm thành phần không có cung ra	0	0	0	**0**
Nén, đếm thành phần không có cung vào	30	30	40	**100**

Chấm trên 10 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

BÀI 14.2 KHO BÁU THỊ TRẤN

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
KHOBAU.INP	KHOBAU.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Tập 1, Bài 14 · Đề bài: Tập 2, trang 102

1. Hiểu bài toán

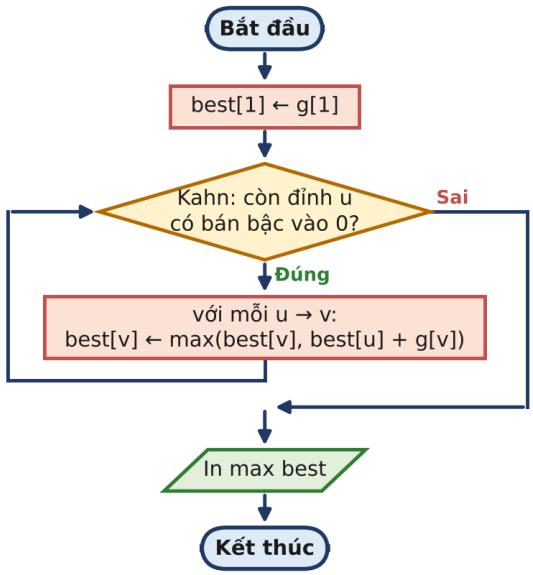
Đã vào một thành phần liên thông mạnh thì đi được tới **mọi** địa điểm trong đó rồi quay ra: nhặt hết vàng của cả thành phần. Nén mỗi thành phần thành một đỉnh có trọng số w = tổng vàng, bài toán trở thành **đường đi có tổng trọng số lớn nhất** trên DAG, xuất phát từ thành phần chứa 1.



Hình 14.2.1. *Ví dụ: vòng {1, 2, 3} (9 đồng) rồi sang 6 (7 đồng): 16.*

2. Cách ngây thơ: QHĐ theo thứ tự tô pô trên đồ thị gốc

Nếu đồ thị không có chu trình (Subtask 1), mỗi địa điểm thăm nhiều nhất một lần: best[v] = g_v + max best[u] trên các cung u → v, tính theo thứ tự tô pô (Kahn).



Hình 14.2.2. *Sơ đồ khối của QHĐ trên DAG.*

Cách ngây thơ: QHĐ theo Kahn trên đồ thị gốc

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

int main() {
    if (fopen("KHOBAU.INP", "r")) {
        freopen("KHOBAU.INP", "r", stdin);
        freopen("KHOBAU.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<ll> a(n + 1);
    for (int i = 1; i <= n; i++) scanf("%lld", &a[i]);
    vector<vector<int>> g(n + 1);
    vector<int> in(n + 1, 0);
    for (int i = 0; i < m; i++) { int u, v; scanf("%d %d", &u, &v);
        g[u].push_back(v); in[v]++; }
    // quy hoạch động theo thứ tự tô pô (Kahn), coi như đồ thị không có chu
        trình
    vector<ll> best(n + 1, -1);
    best[1] = a[1];
    queue<int> q;
    for (int v = 1; v <= n; v++) if (!in[v]) q.push(v);
    ll ans = a[1];
    while (!q.empty()) {
        int u = q.front(); q.pop();
        if (best[u] >= 0) ans = max(ans, best[u]);
        for (int v : g[u]) {
            if (best[u] >= 0) best[v] = max(best[v], best[u] + a[v]);
            if (--in[v] == 0) q.push(v);
        }
    }
    printf("%lld\n", ans);
    return 0;
}
```

Độ phức tạp: O(n + m).

3. Bế tắc

- **Subtask 1** (không có chu trình): **30 điểm**.
- **Subtask 2**: các đỉnh trên chu trình không bao giờ có bán bậc vào 0, Kahn bỏ sót cả vùng đó.

Một lời giải đúng thuật toán nhưng lưu tổng vàng bằng `int`: tổng có thể tới $10^5 \cdot 10^9 = 10^{14}$. Tràn số ngay cả ở Subtask 1: **0 điểm**.

ĐIỂM BẾ TẮC

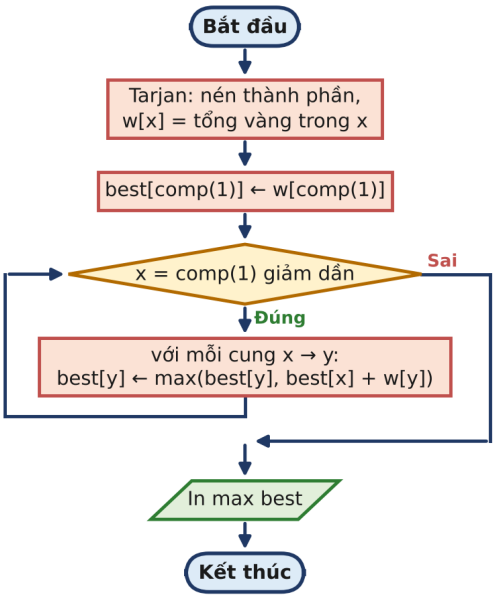
Đi vòng thì được nhặt nhiều địa điểm, nhưng không được tính một địa điểm hai lần. Cần một cách "gói" cả vòng lại thành một khối.

4. Phá bế tắc

Tìm các thành phần bằng Tarjan. Thuật toán Tarjan đánh số thành phần theo **thứ tự tô pô ngược**: mọi cung giữa hai thành phần đều đi từ số **lớn** sang số **nhỏ**. Vậy chỉ cần duyệt x từ comp(1) giảm dần về 0 là đã theo đúng thứ tự tô pô:

- best[comp(1)] = w[comp(1)], các thành phần khác chưa tới được (−1).
- Với mỗi x đã tới được và mỗi cung x → y: best[y] = max(best[y], best[x] + w[y]).
- Đáp số là max best[x].

Thành phần	w	Tới từ	best
A = {1, 2, 3}	5 + 1 + 3 = 9	xuất phát	9
B = {4, 5}	4 + 2 = 6	A	9 + 6 = 15
C = {6}	7	A	9 + 7 = **16**



Hình 14.2.3. *Sơ đồ khối của QHĐ trên đồ thị nén.*

5. Cách tối ưu

KHOBAU.CPP: Tarjan không đệ quy + QHĐ trên đồ thị nén

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

int main() {
    if (fopen("KHOBAU.INP", "r")) {
        freopen("KHOBAU.INP", "r", stdin);
        freopen("KHOBAU.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<ll> a(n + 1);
    for (int i = 1; i <= n; i++) scanf("%lld", &a[i]);
    vector<vector<int>> g(n + 1);
    vector<pair<int,int>> e(m);
    for (auto &x : e) { scanf("%d %d", &x.first, &x.second);
        g[x.first].push_back(x.second); }
    // Tarjan không đệ quy (như bài TINDON)
    vector<int> num(n + 1, 0), low(n + 1, 0), comp(n + 1, -1), it(n + 1, 0),
        S, cs;
    vector<char> onS(n + 1, 0);
    int timer = 0, c = 0;
    for (int s = 1; s <= n; s++) if (!num[s]) {
        cs.push_back(s); num[s] = low[s] = ++timer; S.push_back(s); onS[s] =
            1;
        while (!cs.empty()) {
            int u = cs.back();
            if (it[u] < (int)g[u].size()) {
                int v = g[u][it[u]++];
                if (!num[v]) { num[v] = low[v] = ++timer; S.push_back(v);
                    onS[v] = 1; cs.push_back(v); }
                else if (onS[v]) low[u] = min(low[u], num[v]);
            } else {
                cs.pop_back();
                if (!cs.empty()) low[cs.back()] = min(low[cs.back()],
low[u]);
                if (low[u] == num[u]) {
                    int v;
                    do { v = S.back(); S.pop_back(); onS[v] = 0; comp[v] =
c;

                        } while (v != u);
                    c++;
                }
            }
        }
    }
    // nén: mỗi thành phần là một "siêu địa điểm" có w = tổng vàng
    vector<ll> w(c, 0);
    for (int v = 1; v <= n; v++) w[comp[v]] += a[v];
    vector<vector<int>> h(c);
    for (auto &x : e) if (comp[x.first] != comp[x.second])
        h[comp[x.first]].push_back(comp[x.second]);
    // Tarjan đánh số theo thứ tự tô pô NGƯỢC: cung luôn đi từ số lớn sang
số
        nhỏ
    vector<ll> best(c, -1);                    // best[x]: vàng nhiều nhất
        khi tới thành phần x
    int s = comp[1];
    best[s] = w[s];
    ll ans = 0;
```

```
    for (int x = s; x >= 0; x--) if (best[x] >= 0) {
        ans = max(ans, best[x]);
        for (int y : h[x]) best[y] = max(best[y], best[x] + w[y]);
    }
    printf("%lld\n", ans);
    return 0;
}
```

Độ phức tạp: O(n + m).

6. Bẫy thường gặp

- **Tràn số:** dùng long long cho w và best.
- **QHĐ từ mọi thành phần**, quên rằng phải xuất phát từ 1.
- **Duyệt thành phần theo chiều tăng** với Tarjan (ngược thứ tự tô pô).
- **Cộng w[x] hai lần** (một lần khi tới x, một lần khi rời x).

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (70)	Tổng
QHĐ trên đồ thị gốc (Kahn)	30	0	**30**
Nén + QHĐ, dùng int	0	0	**0**
Nén + QHĐ, dùng long long	30	70	**100**

Chấm trên 6 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

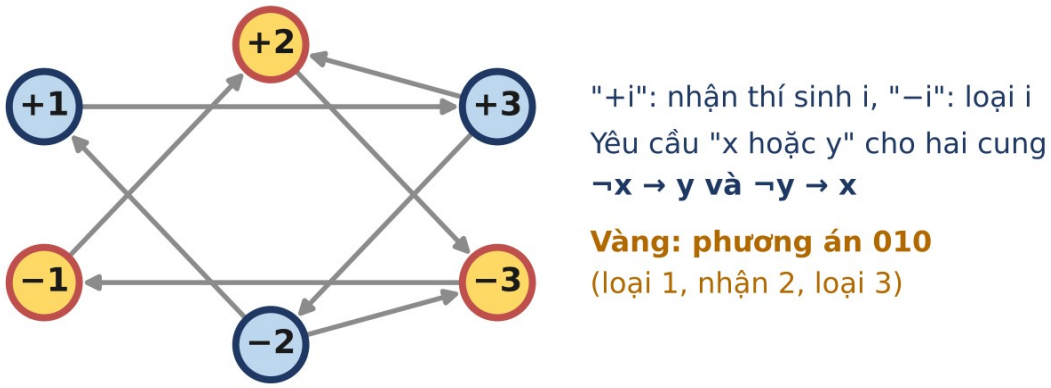
BÀI 14.3 HỘI ĐỒNG TUYỂN CHỌN

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
TUYENCHON.INP	TUYENCHON.OUT	1 giây	256 MB	★★★ NÂNG CAO

Liên hệ: Tập 1, Bài 14 · Đề bài: Tập 2, trang 104

1. Hiểu bài toán

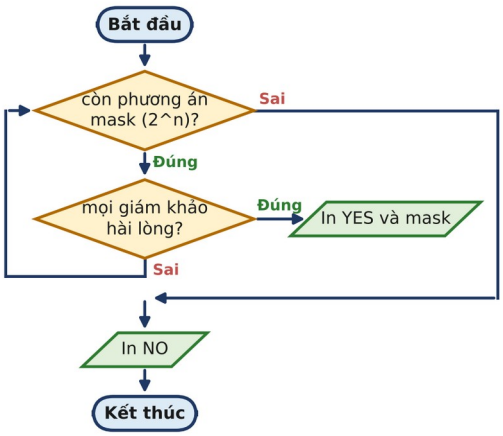
Mỗi thí sinh i ứng với một biến đúng/sai: x_i = "nhận i". Mỗi giám khảo là một mệnh đề (a ∨ b) với a, b là x_i hoặc $\neg x_i$. Đây chính là bài toán **2-SAT** (Tập 1, Bài 14).



Hình 14.3.1. *Đồ thị kéo theo của ví dụ 1.*

2. Cách ngây thơ: Thử mọi phương án

Với n ≤ 20 có $2^{20} \approx 10^6$ phương án. Biểu diễn phương án bằng một mặt nạ bit và kiểm tra lần lượt các giám khảo, dừng ngay khi có người không hài lòng.



Hình 14.3.2. *Sơ đồ khối của cách thử mọi phương án.*

Cách ngây thơ: thử mọi mặt nạ

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("TUYENCHON.INP", "r")) {
        freopen("TUYENCHON.INP", "r", stdin);
        freopen("TUYENCHON.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<int> X(m), Y(m);
    for (int i = 0; i < m; i++) scanf("%d %d", &X[i], &Y[i]);
    auto sat = [](int mask, int x) {            // mong muốn x có được thực hiện không?
        bool take = mask >> (abs(x) - 1) & 1;
        return x > 0 ? take : !take;
    };
    for (int mask = 0; mask < (1 << n); mask++) {   // thử mọi phương án nhận/loại
        bool ok = true;
        for (int i = 0; i < m && ok; i++) ok = sat(mask, X[i]) || sat(mask, Y[i]);
        if (ok) {
            printf("YES\n");
            for (int i = 0; i < n; i++) putchar('0' + (mask >> i & 1));
            printf("\n");
            return 0;
        }
    }
    printf("NO\n");
    return 0;
}
```

Độ phức tạp: $O(2^n \cdot m)$ trong trường hợp xấu nhất.

3. Bế tắc

- **Subtask 1** ($n \le 20$): **20 điểm**.
- **Subtask 2**: 2^{100000} phương án.

Cài đúng 2-SAT nhưng dựng phương án **ngược chiều** (chọn khẳng định có số thành phần **lớn** hơn) thì vẫn in YES đúng lúc, nhưng phương án thường không thỏa mãn: **0 điểm**.

> **ĐIỂM BẾ TẮC**
>
> Phải "lan truyền" hệ quả của từng lựa chọn: chọn loại thí sinh này thì bắt buộc nhận thí sinh kia... Cần một cấu trúc ghi lại mọi hệ quả cùng lúc.

4. Phá bế tắc

Dựng **đồ thị kéo theo** 2n đỉnh (x_i và $\neg x_i$). Mệnh đề ($a \vee b$) tương đương "nếu không a thì b" và "nếu không b thì a": thêm hai cung $\neg \mathbf{a} \to \mathbf{b}$ và $\neg \mathbf{b} \to \mathbf{a}$.

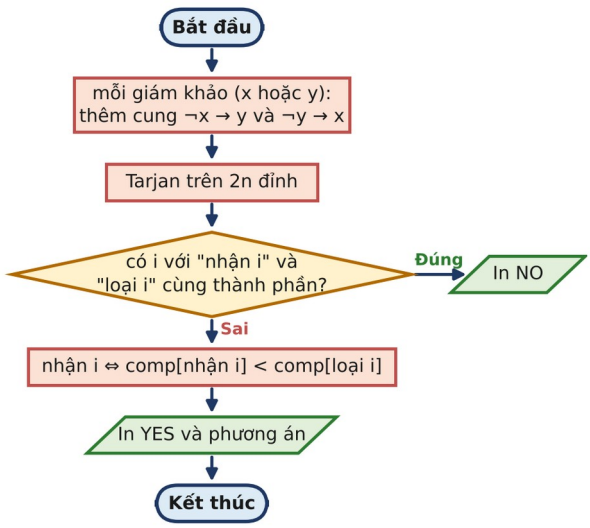
- Nếu x_i và $\neg x_i$ cùng một thành phần liên thông mạnh thì x_i kéo theo $\neg x_i$ và ngược lại: **vô nghiệm**.

- Ngược lại luôn có nghiệm: với Tarjan (đánh số theo thứ tự tô pô ngược), chọn x_i đúng khi **comp[x_i] < comp[$\neg x_i$]**, tức là x_i nằm "sau" $\neg x_i$ trong thứ tự tô pô.

Ví dụ 1: (1 ∨ 2), (¬1 ∨ 3), (¬2 ∨ ¬3), (2 ∨ ¬3) cho 8 cung:

Giám khảo	Cung thứ nhất	Cung thứ hai
1 ∨ 2	¬1 → 2	¬2 → 1
¬1 ∨ 3	1 → 3	¬3 → ¬1
¬2 ∨ ¬3	2 → ¬3	3 → ¬2
2 ∨ ¬3	¬2 → ¬3	3 → 2

Từ 3 có 3 → 2 → ¬3: chọn "nhận 3" sẽ kéo theo "loại 3", nên 3 bị loại. Rồi ¬3 → ¬1 (loại 1), ¬1 → 2 (nhận 2). Phương án **010**.



Hình 14.3.3. *Sơ đồ khối của lời giải 2-SAT.*

VÌ SAO CHỌN THEO THỨ TỰ TÔ PÔ LÀ ĐÚNG?

Giả sử cách chọn làm hỏng một mệnh đề (a ∨ b): cả a và b đều sai, tức ¬a và ¬b đúng. Theo cách chọn, ¬a đứng sau a và ¬b đứng sau b trong thứ tự tô pô. Nhưng có cung ¬a → b nên b đứng sau ¬a, và cung ¬b → a nên a đứng sau ¬b. Ghép lại: a < ¬a ≤ b < ¬b ≤ a, vô lí. (Ở đây "≤" là thứ tự tô pô của các thành phần.)

5. Cách tối ưu

TUYENCHON.CPP: 2-SAT bằng Tarjan không đệ quy

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("TUYENCHON.INP", "r")) {
        freopen("TUYENCHON.INP", "r", stdin);
        freopen("TUYENCHON.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    // đỉnh 2i: "nhận i", đỉnh 2i+1: "loại i"; phủ định của đỉnh x là x ^ 1
    auto node = [](int x) { return x > 0 ? 2 * x : 2 * (-x) + 1; };
    int N = 2 * n + 2;
    vector<vector<int>> g(N);
    for (int i = 0; i < m; i++) {
        int x, y; scanf("%d %d", &x, &y);
        int a = node(x), b = node(y);
        g[a ^ 1].push_back(b);                  // (a hoặc b): không a thì phải b
        g[b ^ 1].push_back(a);                  //             không b thì phải a
    }
    // Tarjan không đệ quy
    vector<int> num(N, 0), low(N, 0), comp(N, -1), it(N, 0), S, cs;
    vector<char> onS(N, 0);
    int timer = 0, c = 0;
    for (int s = 2; s < N; s++) if (!num[s]) {
        cs.push_back(s); num[s] = low[s] = ++timer; S.push_back(s); onS[s] = 1;
        while (!cs.empty()) {
            int u = cs.back();
            if (it[u] < (int)g[u].size()) {
                int v = g[u][it[u]++];
                if (!num[v]) { num[v] = low[v] = ++timer; S.push_back(v); onS[v] = 1; cs.push_back(v); }
                else if (onS[v]) low[u] = min(low[u], num[v]);
            } else {
                cs.pop_back();
                if (!cs.empty()) low[cs.back()] = min(low[cs.back()], low[u]);
                if (low[u] == num[u]) {
                    int v;
                    do { v = S.back(); S.pop_back(); onS[v] = 0; comp[v] = c;
                        } while (v != u);
                    c++;
                }
            }
        }
    }
    string s(n, '0');
    for (int i = 1; i <= n; i++) {
        if (comp[2 * i] == comp[2 * i + 1]) { printf("NO\n"); return 0; }
        // Tarjan đánh số theo thứ tự tô pô ngược: chọn khẳng định nằm "sau" (số nhỏ hơn)
        s[i - 1] = comp[2 * i] < comp[2 * i + 1] ? '1' : '0';
    }
    printf("YES\n%s\n", s.c_str());
    return 0;
}
```

Độ phức tạp: O(n + m).

6. Bẫy thường gặp

- **Chọn ngược chiều** (đặc biệt khi đổi từ Tarjan sang Kosaraju: Kosaraju đánh số theo thứ tự tô pô **xuôi**).
- **Chỉ thêm một cung** cho mỗi mệnh đề.
- **Mệnh đề (x ∨ x)** (cả hai mong muốn giống nhau): vẫn đúng với cách dựng trên, nó tạo cung ¬x → x, buộc x đúng.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (20)	ST2 (80)	Tổng
Thử mọi phương án	20	0	**20**
2-SAT, dựng phương án ngược chiều	0	0	**0**
2-SAT bằng Tarjan	20	80	**100**

Chấm trên 6 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

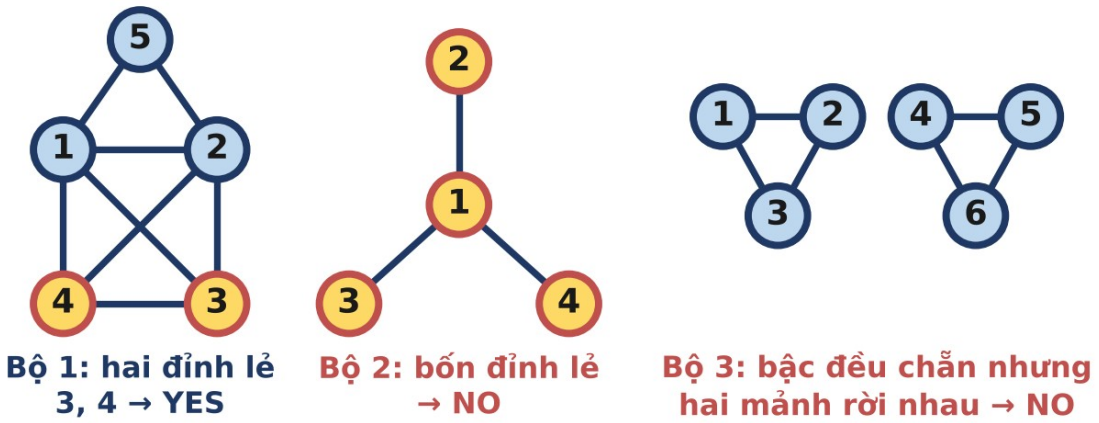
BÀI 15.1 NÉT VẼ

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
NETVE.INP	NETVE.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Tập 1, Bài 15 · Đề bài: Tập 2, trang 106

1. Hiểu bài toán

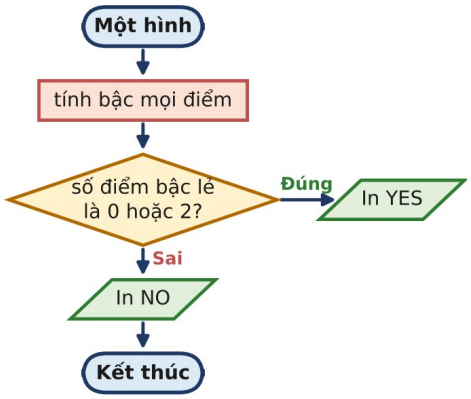
Vẽ một nét là đi qua mọi cạnh đúng một lần: **đường đi Euler** (Tập 1, Bài 15). Có hai điều kiện: số đỉnh bậc lẻ là 0 hoặc 2, và mọi đỉnh **có cạnh** cùng nằm trong một thành phần liên thông. Điểm không có đoạn nào thì bỏ qua.



Hình 15.1.1. *Ba hình của ví dụ: vẽ được, bốn điểm lẻ, hai mảnh rời.*

2. Cách ngây thơ: Chỉ đếm điểm bậc lẻ

Ở Subtask 1, hình liên thông và mọi điểm đều có đoạn: chỉ cần đếm số điểm bậc lẻ.



Hình 15.1.2. *Sơ đồ khối của cách chỉ đếm bậc.*

Cách ngây thơ: chỉ đếm điểm bậc lẻ

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("NETVE.INP", "r")) {
        freopen("NETVE.INP", "r", stdin);
        freopen("NETVE.OUT", "w", stdout);
    }
    int T;
    scanf("%d", &T);
    while (T--) {
        int n, m;
        scanf("%d %d", &n, &m);
        vector<int> deg(n + 1, 0);
        for (int i = 0; i < m; i++) { int u, v; scanf("%d %d", &u, &v);
            deg[u]++; deg[v]++; }
        int odd = 0;                              // chỉ đếm điểm bậc lẻ, quên
            kiểm tra liên thông
        for (int v = 1; v <= n; v++) odd += deg[v] % 2;
        puts(odd == 0 || odd == 2 ? "YES" : "NO");
    }
    return 0;
}
```

Độ phức tạp: O(n + m) cho mỗi hình.

3. Bế tắc

- **Subtask 1**: **30 điểm**.
- **Subtask 2**: hình 3 của ví dụ (hai tam giác rời nhau) có mọi điểm bậc chẵn nhưng không vẽ được bằng một nét.

Thêm kiểm tra liên thông nhưng tính bậc của vòng nhỏ (u, u) là **1** thay vì 2: một hình có vòng nhỏ bị coi là có điểm bậc lẻ. Được **30 điểm**.

ĐIỂM BẾ TẮC

Điều kiện về bậc là cần nhưng chưa đủ; điều kiện liên thông chỉ áp dụng cho các điểm **có đoạn**, và vòng nhỏ đóng góp 2 vào bậc.

4. Phá bế tắc

1) Đọc từng đoạn (u, v): deg[u]++, deg[v]++ (vòng nhỏ u = v làm deg[u] tăng 2, đúng như "đi vào rồi đi ra"), và gộp u, v trong DSU.

2) Duyệt các điểm có deg > 0: đếm số điểm bậc lẻ, và kiểm tra chúng có cùng gốc DSU không.

3) Vẽ được khi và chỉ khi liên thông và số điểm lẻ bằng 0 hoặc 2.

Hình	Bậc các điểm	Liên thông?	Kết luận
1	1: 4, 2: 4, **3: 3, 4: 3**, 5: 2	có	**YES**
2	**1: 3, 2: 1, 3: 1, 4: 1**	có	**NO**

Hình	Bậc các điểm	Liên thông?	Kết luận
3	mọi điểm bậc 2	**không**	**NO**

5. Cách tối ưu

NETVE.CPP: đếm bậc lẻ + DSU kiểm tra liên thông

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> p;
int root(int x) { while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; } return x; }

int main() {
    if (fopen("NETVE.INP", "r")) {
        freopen("NETVE.INP", "r", stdin);
        freopen("NETVE.OUT", "w", stdout);
    }
    int T;
    scanf("%d", &T);
    while (T--) {
        int n, m;
        scanf("%d %d", &n, &m);
        p.resize(n + 1); iota(p.begin(), p.end(), 0);
        vector<int> deg(n + 1, 0);
        for (int i = 0; i < m; i++) {
            int u, v; scanf("%d %d", &u, &v);
            deg[u]++; deg[v]++;                 // vòng nhỏ u = v: bậc của u tăng 2
            p[root(u)] = root(v);
        }
        int odd = 0, r = -1;
        bool ok = true;
        for (int v = 1; v <= n; v++) if (deg[v]) {   // chỉ xét các điểm có đoạn nối vào
            odd += deg[v] % 2;
            if (r < 0) r = root(v);
            else if (root(v) != r) ok = false;   // các đoạn nằm trên hai mảnh rời nhau
        }
        puts(ok && (odd == 0 || odd == 2) ? "YES" : "NO");
    }
    return 0;
}
```

Độ phức tạp: $O((n + m) \cdot \alpha(n))$ cho mỗi hình.

6. Bẫy thường gặp

- **Kiểm tra liên thông trên mọi điểm**, kể cả điểm không có đoạn nào.
- **Vòng nhỏ chỉ cộng 1 vào bậc.**
- **Quên khởi tạo lại** mảng bậc và DSU giữa các hình.
- **Cho rằng "số điểm lẻ = 1"** có thể xảy ra: tổng bậc luôn chẵn, nên số điểm lẻ luôn chẵn.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (70)	Tổng
Chỉ đếm điểm bậc lẻ	30	0	**30**
Có kiểm tra liên thông, vòng nhỏ cộng 1	30	0	**30**
Định lí Euler đầy đủ	30	70	**100**

Chấm trên 6 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

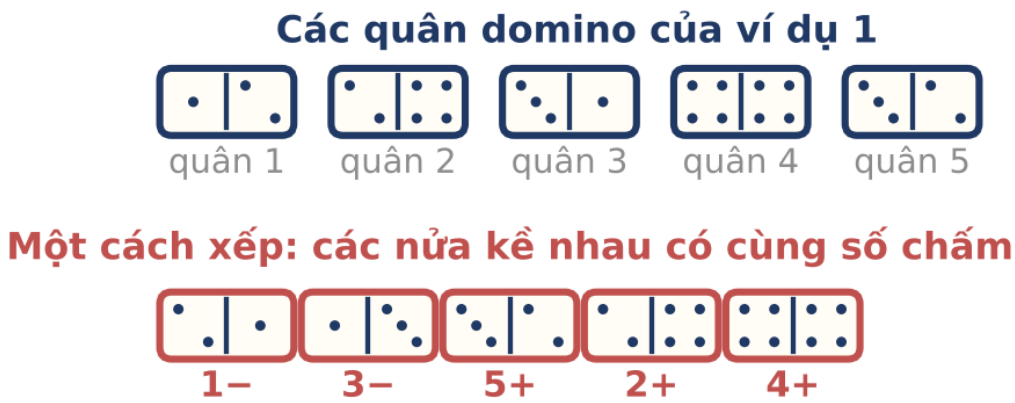
BÀI 15.2 GHÉP DOMINO

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
DOMINO.INP	DOMINO.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Tập 1, Bài 15 · Đề bài: Tập 2, trang 109

1. Hiểu bài toán

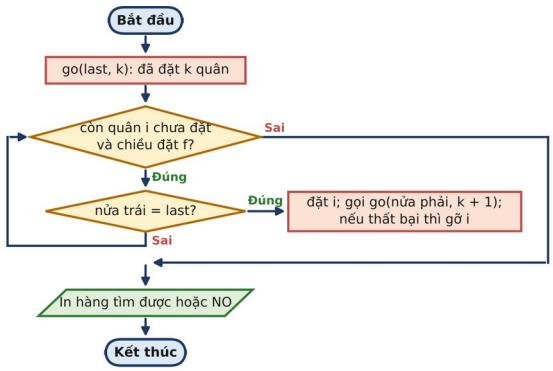
Coi mỗi số chấm 0..6 là một **đỉnh**, mỗi quân domino a|b là một **cạnh** nối a với b (quân a|a là vòng nhỏ). Một hàng domino hợp lệ chính là một **đường đi Euler** trên đồ thị 7 đỉnh này: quân đứng sau bắt đầu ở đúng đỉnh mà quân đứng trước kết thúc.



Hình 15.2.1. *Ví dụ 1: đồ thị 7 đỉnh và đường Euler 2 → 1 → 3 → 2 → 4 → 4.*

2. Cách ngây thơ: Quay lui đặt từng quân

Thử đặt lần lượt từng quân chưa dùng (theo hai chiều) vào cuối hàng, quay lui khi không đặt tiếp được.



Hình 15.2.2. *Sơ đồ khối của cách quay lui.*

Cách ngây thơ: quay lui

```cpp
#include <bits/stdc++.h>
using namespace std;

int n;
vector<int> A, B, ord, sg;
vector<char> used;

bool go(int last, int k) {                    // đã đặt k quân, nửa phải của
    quân cuối là last
    if (k == n) return true;
    for (int i = 1; i <= n; i++) if (!used[i])
        for (int f = 0; f < 2; f++) {         // f = 0: đặt +, f = 1: xoay
            ngược
            int l = f ? B[i] : A[i], r = f ? A[i] : B[i];
            if (k > 0 && l != last) continue;
            used[i] = 1; ord[k] = i; sg[k] = f;
            if (go(r, k + 1)) return true;
            used[i] = 0;
        }
    return false;
}

int main() {
    if (fopen("DOMINO.INP", "r")) {
        freopen("DOMINO.INP", "r", stdin);
        freopen("DOMINO.OUT", "w", stdout);
    }
    scanf("%d", &n);
    A.resize(n + 1); B.resize(n + 1); ord.resize(n); sg.resize(n);
        used.assign(n + 1, 0);
    for (int i = 1; i <= n; i++) scanf("%d %d", &A[i], &B[i]);
    if (!go(-1, 0)) { puts("NO"); return 0; }
    for (int k = 0; k < n; k++) printf("%d %c\n", ord[k], sg[k] ? '-' :
'+');
    return 0;
}
```

Độ phức tạp: $O(n! \cdot 2^n)$ trong trường hợp xấu nhất.

3. Bế tắc

- **Subtask 1** ($n \le 8$): **30 điểm**.
- **Subtask 2**: $n = 10^5$, quay lui không thể.

Cài Hierholzer nhưng **quên kiểm tra** đường tìm được có dùng đủ n quân không: nếu các quân tạo thành hai nhóm rời nhau (ví dụ 2: 0|0 và 1|1), chương trình in một hàng thiếu quân. Kết quả: **0 điểm**.

> **ĐIỂM BẾ TẮC**
>
> Quay lui đặt **quân** (cạnh) theo kiểu "đường đi Hamilton" là bài toán khó. Phải nhìn mỗi quân là một **cạnh** để dùng được lí thuyết Euler.

4. Phá bế tắc

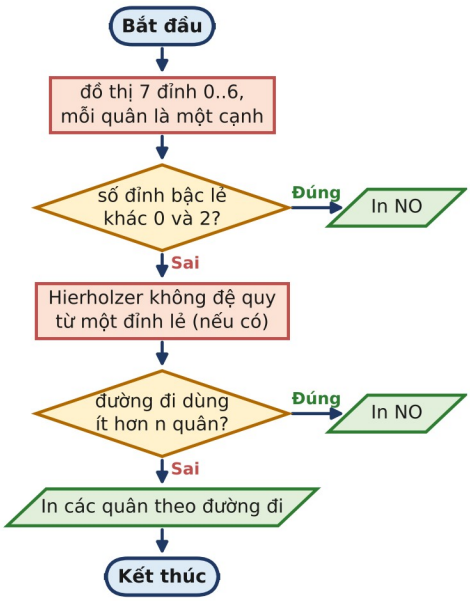
1) Đếm bậc 7 đỉnh. Số đỉnh bậc lẻ khác 0 và 2: in NO.

2) Bắt đầu từ một đỉnh lẻ (nếu có), hoặc từ một đầu của quân 1.

3) Chạy **Hierholzer không đệ quy**: ngăn xếp chứa (đỉnh, quân dùng để tới đỉnh đó). Đỉnh trên cùng còn quân chưa dùng thì đi tiếp; hết quân thì đẩy vào kết quả.

4) Kết quả ít hơn n + 1 phần tử: các quân không liên thông, in NO. Ngược lại đảo ngược kết quả, in từng quân kèm dấu + hoặc − tùy chiều đi.

Đỉnh	1	2	3	4	Khác
Bậc (ví dụ 1)	2	**3**	2	**3**	0
Ghi chú		lẻ: xuất phát		lẻ: kết thúc	



Hình 15.2.3. *Sơ đồ khối của lời giải.*

5. Cách tối ưu

DOMINO.CPP: Hierholzer không đệ quy trên đồ thị 7 đỉnh

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DOMINO.INP", "r")) {
        freopen("DOMINO.INP", "r", stdin);
        freopen("DOMINO.OUT", "w", stdout);
    }
    int n;
    scanf("%d", &n);
    // đồ thị 7 đỉnh (số chấm 0..6), quân i là cạnh A[i] - B[i]
    vector<int> A(n + 1), B(n + 1), deg(7, 0);
    vector<vector<int>> g(7);
    for (int i = 1; i <= n; i++) {
        scanf("%d %d", &A[i], &B[i]);
        g[A[i]].push_back(i); g[B[i]].push_back(i);
        deg[A[i]]++; deg[B[i]]++;
    }
    int s = -1, odd = 0;
    for (int v = 0; v < 7; v++) if (deg[v] % 2) { odd++; if (s < 0) s = v; }
    if (odd != 0 && odd != 2) { puts("NO"); return 0; }
    if (s < 0) s = A[1];                        // không có đỉnh lẻ: bắt đầu ở
        đâu cũng được
    // Hierholzer không đệ quy; st giữ (đỉnh, quân dùng để tới đỉnh đó)
    vector<char> used(n + 1, 0);
    vector<int> it(7, 0);
    vector<pair<int,int>> st = {{s, 0}}, path;
    while (!st.empty()) {
        int u = st.back().first;
        while (it[u] < (int)g[u].size() && used[g[u][it[u]]]) it[u]++;
        if (it[u] == (int)g[u].size()) { path.push_back(st.back());
            st.pop_back(); }
        else {
            int id = g[u][it[u]++];
            used[id] = 1;
            st.push_back({A[id] ^ B[id] ^ u, id});   // đầu kia của quân id
        }
    }
    if ((int)path.size() != n + 1) { puts("NO"); return 0; }   // còn quân
        chưa dùng: không liên thông
    reverse(path.begin(), path.end());
    for (int i = 1; i <= n; i++) {
        int from = path[i - 1].first, id = path[i].second;
        printf("%d %c\n", id, A[id] == from ? '+' : '-');
    }
    return 0;
}
```

Độ phức tạp: O(n).

6. Bẫy thường gặp

- **Không kiểm tra tính liên thông** (số quân trong đường đi).
- **Đánh dấu đỉnh** thay vì đánh dấu **quân**: các quân giống hệt nhau phải được dùng riêng.
- **Bỏ qua quân a|a** khi tính bậc (vòng nhỏ cộng 2).

- **Hierholzer đệ quy:** đường đi dài 10^5 cạnh.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (70)	Tổng
Quay lui	30	0	**30**
Hierholzer, quên kiểm tra liên thông	0	0	**0**
Hierholzer đầy đủ	30	70	**100**

Chấm trên 8 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

BÀI 15.3 MẬT KHẨU KÉT SẮT

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
KETSAT.INP	KETSAT.OUT	1 giây	256 MB	★★★ NÂNG CAO

Liên hệ: Tập 1, Bài 15 · Đề bài: Tập 2, trang 111

1. Hiểu bài toán

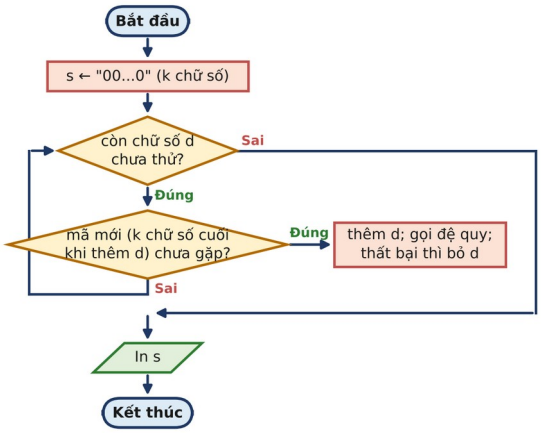
Có b^k mã, mỗi mã phải xuất hiện như một đoạn con. Xâu độ dài L có $L - k + 1$ đoạn con độ dài k, nên $L \geq b^k + k - 1$. Đáp số đạt đúng cận này: mỗi phím bấm sau $k - 1$ phím đầu tạo ra một mã **mới**.



Hình 15.3.1. *Đồ thị de Bruijn với b = 2, k = 3.*

2. Cách ngây thơ: Quay lui từng phím

Bắt đầu bằng k chữ số 0. Thử bấm thêm từng chữ số d sao cho k chữ số cuối là một mã **chưa gặp**; không bấm tiếp được thì quay lui.



Hình 15.3.2. *Sơ đồ khối của cách quay lui.*

Cách ngây thơ: quay lui

```cpp
#include <bits/stdc++.h>
using namespace std;

int b, k, total, P;                              // total = b^k mã; P = b^(k-1)
vector<char> seen;
string s;

bool go(int code, int cnt) {                     // code: k chữ số cuối; cnt: số mã đã gặp
    if (cnt == total) return true;
    for (int d = 0; d < b; d++) {                // thử bấm thêm chữ số d
        int nc = (code % P) * b + d;
        if (seen[nc]) continue;
        seen[nc] = 1; s.push_back('0' + d);
        if (go(nc, cnt + 1)) return true;
        seen[nc] = 0; s.pop_back();              // quay lui
    }
    return false;
}

int main() {
    if (fopen("KETSAT.INP", "r")) {
        freopen("KETSAT.INP", "r", stdin);
        freopen("KETSAT.OUT", "w", stdout);
    }
    scanf("%d %d", &b, &k);
    total = 1;
    for (int i = 0; i < k; i++) total *= b;
    P = total / b;
    seen.assign(total, 0);
    s = string(k, '0'); seen[0] = 1;             // bắt đầu bằng mã 00...0
    go(0, 1);
    printf("%d\n%s\n", (int)s.size(), s.c_str());
    return 0;
}
```

Độ phức tạp: Hàm mũ trong trường hợp xấu nhất; độ sâu đệ quy b^k.

3. Bế tắc

- **Subtask 1, 2**: điểm thực tế **20** và **30** (tổng **50 điểm**).
- **Subtask 3**: $b^k = 10^6$, đệ quy sâu 10^6 tầng: tràn ngăn xếp.

Hierholzer **đệ quy** trên đồ thị de Bruijn là lời giải đúng, nhưng chu trình có 10^6 cạnh và độ sâu đệ quy cũng tới 10^6: **50 điểm**.

ĐIỂM BẾ TẮC

Quay lui đang đi tìm đường **Hamilton** qua các mã (mỗi mã là một đỉnh). Cần đổi góc nhìn: mỗi mã là một **cạnh**, để bài toán trở thành chu trình Euler.

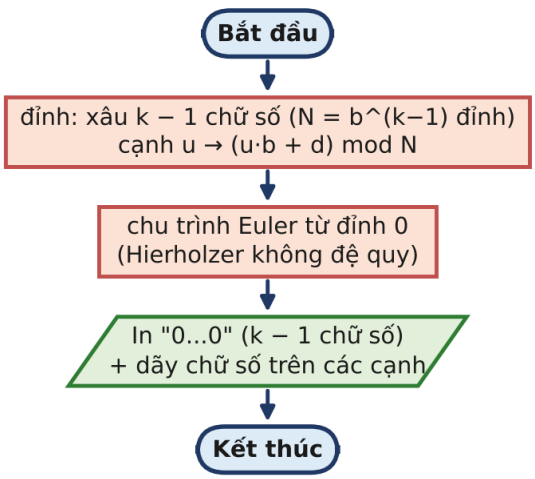
4. Phá bế tắc

Đồ thị **de Bruijn**: đỉnh là các xâu k − 1 chữ số ($N = b^{k-1}$ đỉnh, viết như một số hệ b). Từ đỉnh u, bấm chữ số d đi tới đỉnh $(u \cdot b + d) \bmod N$; cạnh này ứng với mã k chữ số "u nối d". Mỗi đỉnh có đúng b cạnh ra và b cạnh vào, đồ thị liên thông mạnh: **luôn có chu trình Euler**.

Đi theo chu trình Euler từ đỉnh 00...0: xâu kết quả là k − 1 chữ số 0, tiếp theo là chữ số trên từng cạnh. Mỗi mã xuất hiện đúng một lần, độ dài $b^k + k - 1$.

Cạnh thứ	Đi từ đỉnh	Bấm	Mã mới
bắt đầu	00		
1	00	0	000
2	00	1	001
3	01	0	010
4	10	1	101
5	01	1	011
6	11	1	111
7	11	0	110
8	10	0	100

Ví dụ 1 (b = 2, k = 3) cho xâu **0001011100**, độ dài $2^3 + 2 = 10$.



Hình 15.3.3. *Sơ đồ khối của lời giải.*

TRƯỜNG HỢP k = 1

$N = b^0 = 1$: chỉ có một đỉnh, mọi cạnh là vòng nhỏ. Thuật toán vẫn chạy đúng và in b chữ số 0, 1, ..., b − 1 (theo một thứ tự nào đó).

5. Cách tối ưu

KETSAT.CPP: chu trình Euler trên đồ thị de Bruijn, không đệ quy

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("KETSAT.INP", "r")) {
        freopen("KETSAT.INP", "r", stdin);
        freopen("KETSAT.OUT", "w", stdout);
    }
    int b, k;
    scanf("%d %d", &b, &k);
    // đồ thị de Bruijn: đỉnh = xâu k-1 chữ số (viết như số hệ b), N = b^(k-1)
        đỉnh;
    // từ đỉnh u, bấm chữ số d đi tới đỉnh (u*b + d) mod N. Mỗi cạnh là một mã
        k chữ số.
    int N = 1;
    for (int i = 0; i < k - 1; i++) N *= b;
    vector<int> ptr(N, 0);                      // chữ số tiếp theo chưa dùng
        ở mỗi đỉnh
    vector<pair<int,int>> st = {{0, -1}};       // (đỉnh, chữ số vừa bấm để tới)
    string out;
    while (!st.empty()) {                       // Hierholzer không đệ quy
        auto [u, c] = st.back();
        if (ptr[u] < b) {
            int d = ptr[u]++;
            st.push_back({(int)(((long long)u * b + d) % N), d});
        } else {
            st.pop_back();
            if (c >= 0) out.push_back('0' + c);
        }
    }
    reverse(out.begin(), out.end());
    string ans = string(k - 1, '0') + out;      // bắt đầu ở đỉnh 00...0
    printf("%d\n%s\n", (int)ans.size(), ans.c_str());
    return 0;
}
```

Độ phức tạp: $O(b^k)$.

6. Bẫy thường gặp

- **Đệ quy sâu 10^6 tầng.**
- **Quên k − 1 chữ số đầu** của xâu.
- **Tính $N = b^{k-1}$ bằng pow** (số thực): dùng vòng nhân số nguyên.
- **b = 1:** chỉ có một mã 00...0, đáp số là k chữ số 0.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (20)	ST2 (30)	ST3 (50)	Tổng
Quay lui từng phím	20	30	0	**50**

Thuật toán	ST1 (20)	ST2 (30)	ST3 (50)	Tổng
Hierholzer đệ quy	20	30	0	**50**
Hierholzer không đệ quy	20	30	50	**100**

Chấm trên 11 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

CHƯƠNG 5. GHÉP CẶP VÀ LUỒNG

*Chương 5 gồm những bài toán "phân phối": xếp người vào chỗ, đặt quân lên bảng, chia nước qua đường ống. Cách ngây thơ thường là **tham lam** (ai đến trước được chọn trước) hoặc **vét cạn**. Tham lam sai vì một lựa chọn sớm có thể chặn mất nhiều lựa chọn tốt hơn về sau; lời giải đúng cho phép **đổi ý** thông qua đường tăng (cặp ghép) hoặc cung ngược (luồng). Nhiều bài còn đòi chỉ ra **cách làm**, không chỉ con số: phải đọc thêm thông tin từ cặp ghép, từ đồ thị thặng dư.*

Bài	Tên bài	Mức	Cách ngây thơ (điểm)	Chìa khóa của lời giải
16.1	**XẾP PHÒNG THI**	★	Vét cạn có cắt nhánh (30)	Cặp ghép cực đại trên đồ thị hai phía bằng đường tăng (Kuhn).
16.2	**ĐẶT DOMINO**	★★	Số ô trống chia 2 (20)	Tô bàn cờ: domino = cạnh nối ô trắng với ô đen; cặp ghép cực đại.
16.3	**CAMERA GIÁM SÁT**	★★★	Thử mọi tập hàng (20)	Định lí König: phủ đỉnh nhỏ nhất = cặp ghép cực đại; dựng phủ từ tập Z.
17.1	**ĐƯỜNG ỐNG**	★	In các ống bão hòa (40)	Dinic; S = các trạm tới được từ 1 trên đồ thị thặng dư.
17.2	**LỘ TRÌNH RỜI NHAU**	★★	Tìm lộ trình rồi xóa (0)	Mỗi đường dung lượng 1, luồng cực đại; lần theo luồng và cắt bỏ vòng.
17.3	**PHÂN CÔNG TRỰC NHẬT**	★★★	Vét cạn, quay lui theo ngày (30)	Mạng luồng s → học sinh (a_i) → ngày (1) → t (b_j); đủ luồng thì có lịch.

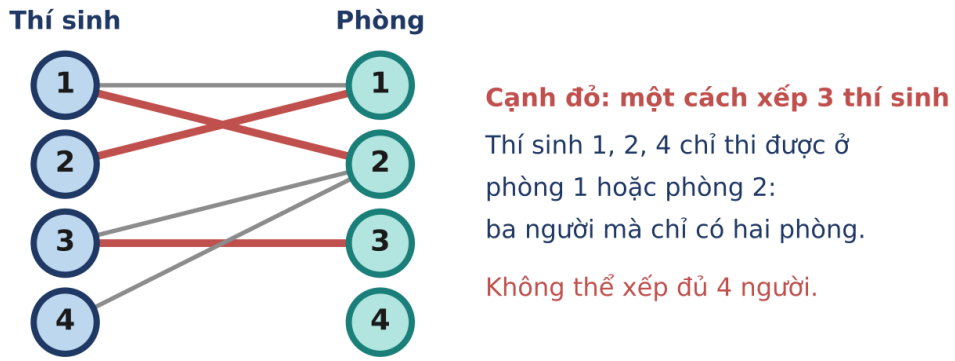
BÀI 16.1 XẾP PHÒNG THI

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
XEPPHONG.INP	XEPPHONG.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Tập 1, Bài 16 · Đề bài: Tập 2, trang 114

1. Hiểu bài toán

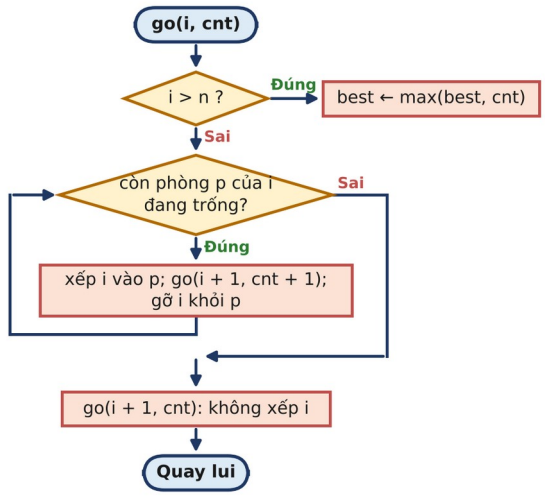
Bên trái là thí sinh, bên phải là phòng; thí sinh i nối với các phòng trong danh sách của mình. Mỗi cách xếp là một **cặp ghép**; cần cặp ghép **cực đại** (Tập 1, Bài 16).



Hình 16.1.1. *Ví dụ: xếp được 3 thí sinh.*

2. Cách ngây thơ: Vét cạn cách xếp

Lần lượt xét từng thí sinh: hoặc xếp vào một phòng còn trống trong danh sách, hoặc không xếp. Cắt nhánh khi có xếp hết số thí sinh còn lại cũng không vượt kỉ lục.



Hình 16.1.2. *Sơ đồ khối của cách vét cạn.*

Cách ngây thơ: vét cạn có cắt nhánh

```
#include <bits/stdc++.h>
using namespace std;

int n, m, best = 0;
vector<vector<int>> g;
vector<char> busy;

void go(int i, int cnt) {                       // xét thí sinh i, đã xếp được
    cnt người
    if (cnt + (n - i + 1) <= best) return;      // có xếp hết số còn lại cũng
        không hơn
    if (i > n) { best = cnt; return; }
    for (int p : g[i]) if (!busy[p]) {          // xếp i vào phòng p
        busy[p] = 1; go(i + 1, cnt + 1); busy[p] = 0;
    }
    go(i + 1, cnt);                             // hoặc không xếp i
}

int main() {
    if (fopen("XEPPHONG.INP", "r")) {
        freopen("XEPPHONG.INP", "r", stdin);
        freopen("XEPPHONG.OUT", "w", stdout);
    }
    scanf("%d %d", &n, &m);
    g.resize(n + 1); busy.assign(m + 1, 0);
    for (int i = 1; i <= n; i++) { int k; scanf("%d", &k); g[i].resize(k); for
        (int &x : g[i]) scanf("%d", &x); }
    go(1, 0);
    printf("%d\n", best);
    return 0;
}
```

Độ phức tạp: Hàm mũ: khoảng $(k + 1)^n$.

3. Bế tắc

- **Subtask 1** (n, m ≤ 10): **30 điểm**.
- **Subtask 2**: n = 1000, không thể.

Tham lam "mỗi thí sinh lấy phòng trống đầu tiên trong danh sách" chạy rất nhanh nhưng sai. Ở ví dụ: thí sinh 1 lấy phòng 1, thí sinh 2 (chỉ thi được phòng 1) hết chỗ, thí sinh 3 lấy phòng 2, thí sinh 4 hết chỗ: chỉ 2 người. Kết quả: **0 điểm**.

> **ĐIỂM BẾ TẮC**
>
> Tham lam không biết **đổi ý**: thí sinh 1 có thể nhường phòng 1 cho thí sinh 2 rồi sang phòng 2. Cần một cách "dời chỗ dây chuyền" có hệ thống.

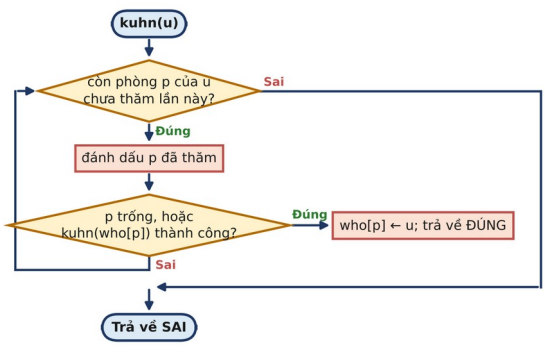
4. Phá bế tắc

Đường tăng: khi xếp thí sinh u, thử từng phòng p của u. Nếu p trống, xếp ngay. Nếu p đã có người w, thử "đẩy" w sang một phòng khác của w (đệ quy). Mỗi lần xếp thêm được một người, cặp ghép tăng 1. Mảng vis đánh dấu phòng đã thử trong **lần tìm hiện tại** để không đi vòng.

Chạy tay ví dụ (thí sinh 1: {1, 2}, 2: {1}, 3: {2, 3}, 4: {2}):

Xếp	Đường tăng tìm được	Phòng 1, 2, 3
1	phòng 1 trống	1, –, –
2	phòng 1 (của 1) → 1 dời sang phòng 2	2, 1, –
3	phòng 2 (của 1) → 1 thử phòng 1 (của 2) → 2 hết đường; thử phòng 3: trống	2, 1, 3
4	phòng 2 (của 1) → 1 thử phòng 1 (của 2) → 2 hết đường: **thất bại**	2, 1, 3

Xếp được **3** thí sinh.



Hình 16.1.3. Sơ đồ khối của hàm kuhn(u).

ĐỊNH LÍ BERGE

Một cặp ghép là cực đại khi và chỉ khi không còn đường tăng. Thuật toán Kuhn thử tìm đường tăng từ từng đỉnh trái đúng một lần; có thể chứng minh rằng nếu không tìm được đường tăng từ u lúc này thì về sau cũng không cần thử lại u.

5. Cách tối ưu

XEPPHONG.CPP: thuật toán Kuhn

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, m, stamp = 0;
vector<vector<int>> g;                          // g[i]: các phòng thí sinh i
    thi được
vector<int> who, vis;                           // who[p]: thí sinh đang ở
    phòng p (0: trống)

bool kuhn(int u) {                              // tìm đường tăng xuất phát từ
    thí sinh u
    for (int p : g[u]) if (vis[p] != stamp) {
        vis[p] = stamp;
        if (!who[p] || kuhn(who[p])) { who[p] = u; return true; }   // phòng
            trống, hoặc người cũ đổi được phòng
    }
    return false;
}

int main() {
    if (fopen("XEPPHONG.INP", "r")) {
        freopen("XEPPHONG.INP", "r", stdin);
        freopen("XEPPHONG.OUT", "w", stdout);
    }
    scanf("%d %d", &n, &m);
    g.resize(n + 1); who.assign(m + 1, 0); vis.assign(m + 1, 0);
    for (int i = 1; i <= n; i++) {
        int k; scanf("%d", &k);
        g[i].resize(k);
        for (int &x : g[i]) scanf("%d", &x);
    }
    int ans = 0;
    for (int i = 1; i <= n; i++) { stamp++; if (kuhn(i)) ans++; }
    printf("%d\n", ans);
    return 0;
}
```

Độ phức tạp: $O(n \cdot \Sigma k_i) = 1000 \cdot 10^5 = 10^8$ trong trường hợp xấu nhất; thực tế nhanh hơn nhiều.

6. Bẫy thường gặp

- **Tham lam** theo thứ tự đọc vào.
- **Quên đặt lại vis** cho mỗi thí sinh (dùng biến đếm stamp để khỏi xóa mảng).
- **Đánh dấu thí sinh** thay vì đánh dấu phòng trong vis.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (70)	Tổng
Vét cạn có cắt nhánh	30	0	**30**
Tham lam: phòng trống đầu tiên	0	0	**0**
Kuhn (đường tăng)	30	70	**100**

Chấm trên 7 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

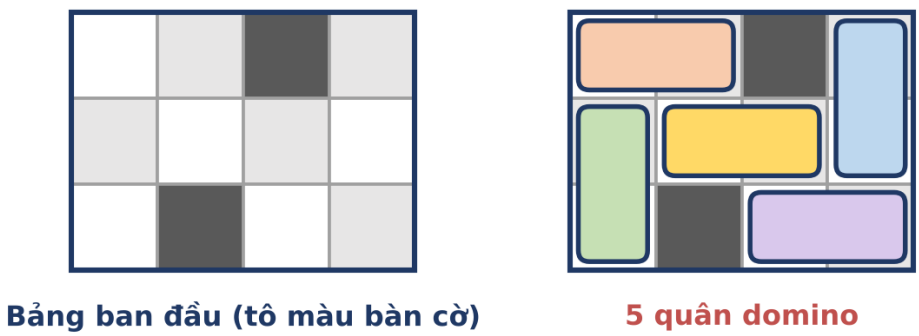
BÀI 16.2 ĐẶT DOMINO

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
DATDOMINO.INP	DATDOMINO.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Tập 1, Bài 16 · Đề bài: Tập 2, trang 116

1. Hiểu bài toán

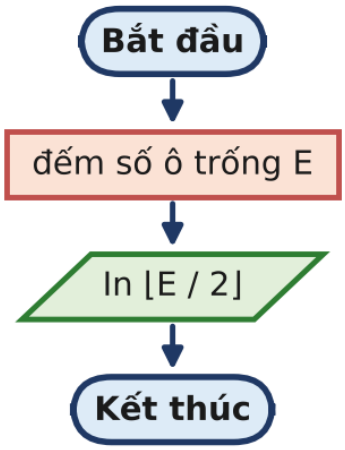
Tô bảng như bàn cờ: ô (i, j) trắng nếu i + j chẵn, đen nếu lẻ. Hai ô kề cạnh luôn **khác màu**, nên mỗi quân domino phủ đúng một ô trắng và một ô đen. Đặt domino không chồng nhau chính là chọn một **cặp ghép** giữa ô trắng trống và ô đen trống kề cạnh.



Hình 16.2.1. *Ví dụ: 10 ô trống, đặt được 5 quân.*

2. Cách ngây thơ: Chia đôi số ô trống

Khi không có ô cấm (Subtask 1), luôn phủ được ⌊R·C / 2⌋ quân: xếp ngang từng hàng, hàng lẻ ô thì dùng thêm quân dọc. Vậy chỉ cần đếm ô trống chia 2.



Hình 16.2.2. Sơ đồ khối của công thức.

Cách ngây thơ: số ô trống chia 2

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DATDOMINO.INP", "r")) {
        freopen("DATDOMINO.INP", "r", stdin);
        freopen("DATDOMINO.OUT", "w", stdout);
    }
    int R, C;
    cin >> R >> C;
    int empty = 0;
    for (int i = 0; i < R; i++) { string s; cin >> s; empty +=
        count(s.begin(), s.end(), '.'); }
    printf("%d\n", empty / 2);                  // đúng khi không có ô cấm,
        nói chung thì không
    return 0;
}
```

Độ phức tạp: $O(R \cdot C)$.

3. Bế tắc

- **Subtask 1** (không có ô cấm): **20 điểm**.
- **Subtask 2, 3**: sai. Bảng 2×2 có hai ô cấm chéo nhau còn 2 ô trống nhưng không kề nhau: đáp số 0, công thức cho 1.

Với bảng hẹp có thể dùng **QHĐ trạng thái theo hồ sơ** (broken profile): duyệt từng ô, mặt nạ C bit ghi ô nào đã bị quân trước chiếm. Đúng, nhưng 2^C trạng thái chỉ dùng được khi cạnh ngắn của bảng nhỏ: **30 điểm**.

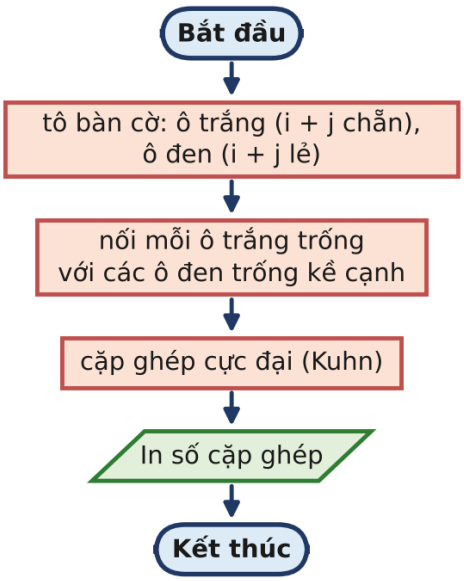
ĐIỂM BẾ TẮC

Bảng 100 × 100 có tới 10^4 ô; mọi cách "nhớ trạng thái của cả một hàng" đều quá lớn. Cần một mô hình không phụ thuộc vào hình dạng bảng.

4. Phá bế tắc

Dựng đồ thị hai phía: bên trái các ô trắng trống, bên phải các ô đen trống, cạnh nối hai ô kề cạnh. Đáp số là **kích thước cặp ghép cực đại**. Đồ thị có tới 5000 đỉnh mỗi bên và khoảng $2 \cdot 10^4$ cạnh (mỗi ô có tối đa 4 ô kề), Kuhn chạy rất nhanh.

Bảng	Ô trắng / ô đen trống	Cặp ghép cực đại
Ví dụ (3 × 4)	5 / 5	**5**
2 × 2, cấm (1, 2) và (2, 1)	2 / 0	**0**
3 × 3 không cấm	5 / 4	**4**



Hình 16.2.3. *Sơ đồ khối của lời giải.*

MỞ RỘNG: HOPCROFT–KARP

Khi đồ thị lớn hơn nữa (10^5 đỉnh), dùng Hopcroft–Karp: mỗi pha BFS chia tầng rồi tìm nhiều đường tăng ngắn nhất rời nhau bằng DFS; độ phức tạp $O(E \cdot \sqrt{V})$.

5. Cách tối ưu

DATDOMINO.CPP: tô bàn cờ + Kuhn

```cpp
#include <bits/stdc++.h>
using namespace std;

int R, C, stamp = 0;
vector<vector<int>> g;                          // g[ô trắng] = các ô đen trống kề cạnh
vector<int> mt, vis;                            // mt[ô đen] = ô trắng đang ghép với nó (-1: chưa)

bool kuhn(int u) {
    for (int v : g[u]) if (vis[v] != stamp) {
        vis[v] = stamp;
        if (mt[v] < 0 || kuhn(mt[v])) { mt[v] = u; return true; }
    }
    return false;
}

int main() {
    if (fopen("DATDOMINO.INP", "r")) {
        freopen("DATDOMINO.INP", "r", stdin);
        freopen("DATDOMINO.OUT", "w", stdout);
    }
    cin >> R >> C;
    vector<string> a(R);
    for (auto &s : a) cin >> s;
    int N = R * C;
    g.resize(N); mt.assign(N, -1); vis.assign(N, 0);
    const int dx[4] = {-1, 1, 0, 0}, dy[4] = {0, 0, -1, 1};
    // tô bàn cờ: ô (i, j) "trắng" nếu i + j chẵn; mỗi quân domino phủ một ô trắng và một ô đen
    for (int i = 0; i < R; i++) for (int j = 0; j < C; j++)
        if (a[i][j] == '.' && (i + j) % 2 == 0)
            for (int d = 0; d < 4; d++) {
                int x = i + dx[d], y = j + dy[d];
                if (x >= 0 && x < R && y >= 0 && y < C && a[x][y] == '.')
g[i
                    * C + j].push_back(x * C + y);
            }
    int ans = 0;
    for (int u = 0; u < N; u++) if (!g[u].empty()) { stamp++; if (kuhn(u)) ans++; }
    printf("%d\n", ans);
    return 0;
}
```

Độ phức tạp: O(V · E) trên lí thuyết, với V ≈ 5000, E ≈ $2 \cdot 10^4$; thực tế chỉ vài phần trăm giây.

6. Bẫy thường gặp

- **Công thức "số ô trống chia 2"** khi có ô cấm.
- **Nối cả hai chiều** (ô trắng – ô đen và ô đen – ô trắng) rồi chạy Kuhn từ mọi ô: đếm mỗi quân hai lần.
- **Đánh số ô** i · C + j nhưng cấp phát mảng R · R.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (20)	ST2 (30)	ST3 (50)	Tổng
Số ô trống chia 2	20	0	0	**20**
QHĐ hồ sơ trên mặt nạ bit	0	30	0	**30**
Tô bàn cờ + cặp ghép	20	30	50	**100**

Chấm trên 9 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

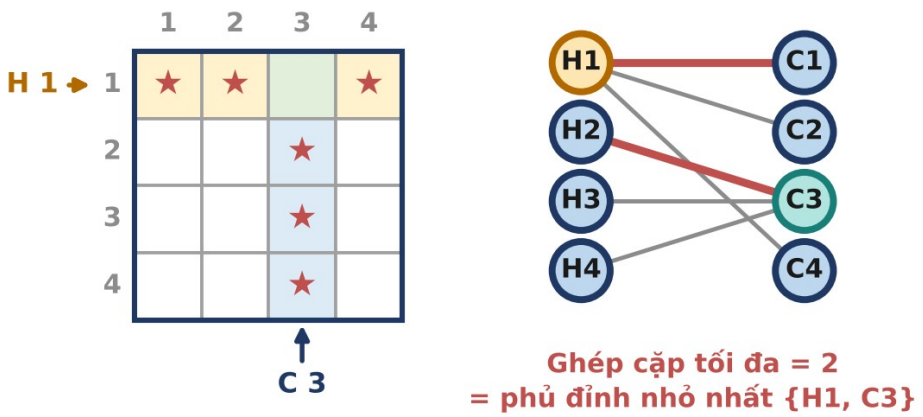
BÀI 16.3 CAMERA GIÁM SÁT

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
CAMERA.INP	CAMERA.OUT	1 giây	256 MB	★★★ NÂNG CAO

Liên hệ: Tập 1, Bài 16 · Đề bài: Tập 2, trang 118

1. Hiểu bài toán

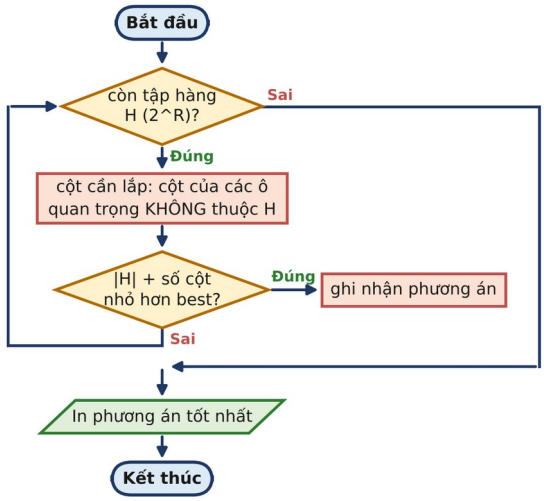
Đồ thị hai phía: bên trái R hàng, bên phải C cột; mỗi ô quan trọng (r, c) là một **cạnh** nối hàng r với cột c. Lắp camera ở hàng r giám sát mọi cạnh tại r. Cần chọn ít đỉnh nhất sao cho mọi cạnh có ít nhất một đầu được chọn: **phủ đỉnh nhỏ nhất**.



Hình 16.3.1. *Ví dụ: hàng 1 và cột 3 là đủ.*

2. Cách ngây thơ: Thử mọi tập hàng

Thử mọi tập hàng H (2^R tập). Mọi ô quan trọng không thuộc hàng nào trong H bắt buộc phải được giám sát bằng cột của nó: số camera = |H| + số cột như vậy.



Hình 16.3.2. *Sơ đồ khối của cách thử mọi tập hàng.*

Cách ngây thơ: thử mọi tập hàng

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("CAMERA.INP", "r")) {
        freopen("CAMERA.INP", "r", stdin);
        freopen("CAMERA.OUT", "w", stdout);
    }
    int R, C, K;
    scanf("%d %d %d", &R, &C, &K);
    vector<int> rr(K), cc(K);
    for (int i = 0; i < K; i++) scanf("%d %d", &rr[i], &cc[i]);
    int best = INT_MAX; long long bestMask = 0;
    for (long long mask = 0; mask < (1LL << R); mask++) {   // chọn tập hàng;
        cột: lấy hết những cột còn thiếu
        set<int> cols;
        for (int i = 0; i < K; i++) if (!(mask >> (rr[i] - 1) & 1))
            cols.insert(cc[i]);
        int t = __builtin_popcountll(mask) + cols.size();
        if (t < best) { best = t; bestMask = mask; }
    }
    set<int> cols;
    for (int i = 0; i < K; i++) if (!(bestMask >> (rr[i] - 1) & 1))
        cols.insert(cc[i]);
    printf("%d\n", best);
    for (int r = 1; r <= R; r++) if (bestMask >> (r - 1) & 1) printf("H %d\n",
        r);
    for (int c : cols) printf("C %d\n", c);
    return 0;
}
```

Độ phức tạp: $O(2^R \cdot K \cdot \log K)$.

3. Bế tắc

- **Subtask 1** (R, C ≤ 8): **20 điểm**.
- **Subtask 2, 3**: 2^{1000} tập hàng.

Biết định lí König thì in đúng **số** camera bằng kích thước cặp ghép cực đại. Nhưng nhiều bạn in luôn "các hàng đã được ghép" làm cách lắp: ở ví dụ đó là hàng 1 và hàng 2, bỏ sót ô (3, 3) và (4, 3). Cách này chỉ đúng dòng đầu: **30 điểm**.

> **ĐIỂM BẾ TẮC**
>
> Cặp ghép cho biết **bao nhiêu** camera, nhưng chưa cho biết **lắp ở đâu**. Cần một quy tắc chọn, với mỗi cạnh ghép, đúng một đầu của nó.

4. Phá bế tắc

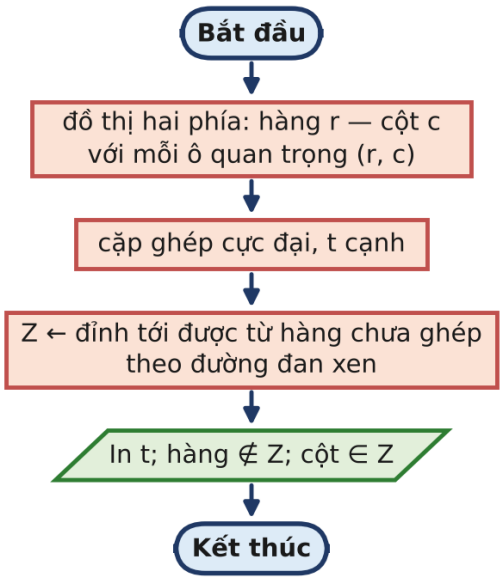
1) Tìm cặp ghép cực đại (Kuhn), kích thước t.

2) Tìm tập **Z**: các đỉnh tới được từ một **hàng chưa ghép** bằng đường đan xen (từ hàng sang cột theo cạnh bất kì, từ cột về hàng theo cạnh **ghép**).

3) Phủ đỉnh nhỏ nhất: **các hàng KHÔNG thuộc Z** và **các cột THUỘC Z**. Tập này có đúng t đỉnh: mỗi cạnh ghép đóng góp đúng một đầu.

Chạy tay ví dụ:

Bước	Kết quả	Ghi chú
Cặp ghép	hàng 1 – cột 1, hàng 2 – cột 3	t = 2
Hàng chưa ghép	hàng 3, hàng 4	đưa vào Z
Từ hàng 3, 4	cột 3	cột 3 ghép với hàng 2: thêm hàng 2
Từ hàng 2	cột 3 (đã có)	Z = {H2, H3, H4, C3}
Phủ	**H 1** (không thuộc Z), **C 3** (thuộc Z)	2 camera



Hình 16.3.3. *Sơ đồ khối của lời giải.*

VÌ SAO TẬP NÀY LÀ PHỦ?

Xét một cạnh (r, c). Nếu r ∉ Z thì r được chọn. Nếu r ∈ Z: khi r là hàng chưa ghép, mọi cạnh từ r đều đưa c vào Z; khi r được ghép, r vào Z qua chính cột ghép với nó, và từ r lại đi tiếp được sang c. Vậy c ∈ Z, c được chọn. Mỗi hàng ∉ Z đều đã ghép (hàng chưa ghép thuộc Z) và mỗi cột ∈ Z đều đã ghép (nếu không, có đường tăng); hai đầu của một cạnh ghép không thể cùng được chọn. Do đó số đỉnh đúng bằng t.

5. Cách tối ưu

CAMERA.CPP: Kuhn + dựng phủ đỉnh theo König

```cpp
#include <bits/stdc++.h>
using namespace std;

int R, C, stamp = 0;
vector<vector<int>> g;                          // g[hàng r] = các cột c có ô
    quan trọng (r, c)
vector<int> mc, mr, vis;                        // mc[c]: hàng ghép với cột c;
    mr[r]: cột ghép với hàng r

bool kuhn(int r) {
    for (int c : g[r]) if (vis[c] != stamp) {
        vis[c] = stamp;
        if (!mc[c] || kuhn(mc[c])) { mc[c] = r; mr[r] = c; return true; }
    }
    return false;
}

int main() {
    if (fopen("CAMERA.INP", "r")) {
        freopen("CAMERA.INP", "r", stdin);
        freopen("CAMERA.OUT", "w", stdout);
    }
    int K;
    scanf("%d %d %d", &R, &C, &K);
    g.resize(R + 1); mc.assign(C + 1, 0); mr.assign(R + 1, 0); vis.assign(C +
        1, 0);
    for (int i = 0; i < K; i++) { int r, c; scanf("%d %d", &r, &c);
        g[r].push_back(c); }
    int t = 0;
    for (int r = 1; r <= R; r++) { stamp++; if (kuhn(r)) t++; }
    // König: Z = các đỉnh tới được từ hàng CHƯA ghép theo đường đan xen
    // (hàng -> cột bằng cạnh bất kì, cột -> hàng bằng cạnh ghép).
    // Phủ đỉnh nhỏ nhất = (hàng KHÔNG thuộc Z) + (cột THUỘC Z).
    vector<char> zr(R + 1, 0), zc(C + 1, 0);
    queue<int> q;
    for (int r = 1; r <= R; r++) if (!mr[r]) { zr[r] = 1; q.push(r); }
    while (!q.empty()) {
        int r = q.front(); q.pop();
        for (int c : g[r]) if (!zc[c]) {
            zc[c] = 1;
            int r2 = mc[c];
            if (r2 && !zr[r2]) { zr[r2] = 1; q.push(r2); }
        }
    }
    printf("%d\n", t);
    for (int r = 1; r <= R; r++) if (!zr[r]) printf("H %d\n", r);
    for (int c = 1; c <= C; c++) if (zc[c]) printf("C %d\n", c);
    return 0;
}
```

Độ phức tạp: $O(R \cdot K)$ cho Kuhn, $O(R + C + K)$ cho bước dựng phủ.

6. Bẫy thường gặp

- **In các hàng đã ghép** làm cách lắp.

- **Bắt đầu đường đan xen từ cột chưa ghép** nhưng vẫn dùng công thức "hàng $\notin Z$, cột $\in Z$" (phải đổi vai trò tương ứng).
- **In một hàng hai lần** (khi vừa $\notin Z$ vừa được thêm theo cách khác).

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (20)	ST2 (30)	ST3 (50)	Tổng
Thử mọi tập hàng	20	0	0	**20**
Đúng số camera, in các hàng đã ghép	0	30	0	**30**
Kuhn + định lí König	20	30	50	**100**

Chấm trên 9 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

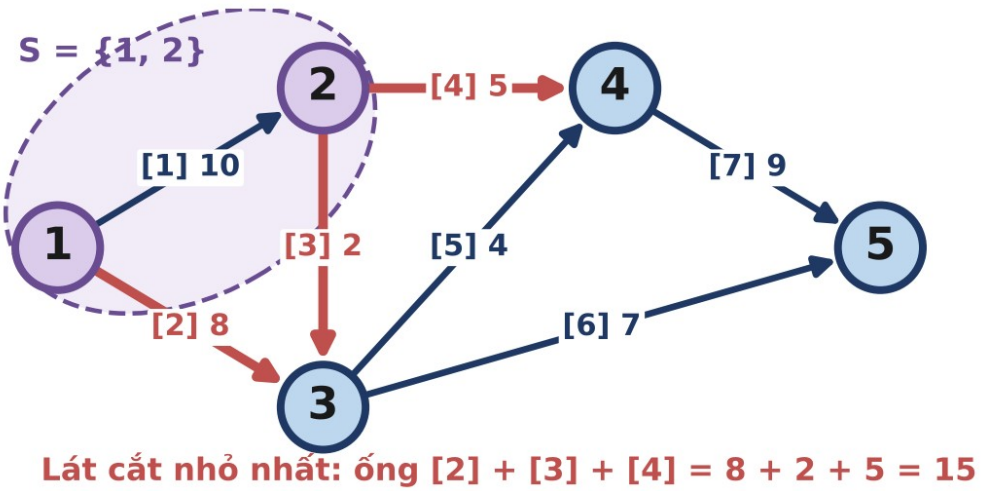
BÀI 17.1 ĐƯỜNG ỐNG

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
DUONGONG.INP	DUONGONG.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Tập 1, Bài 17 · Đề bài: Tập 2, trang 120

1. Hiểu bài toán

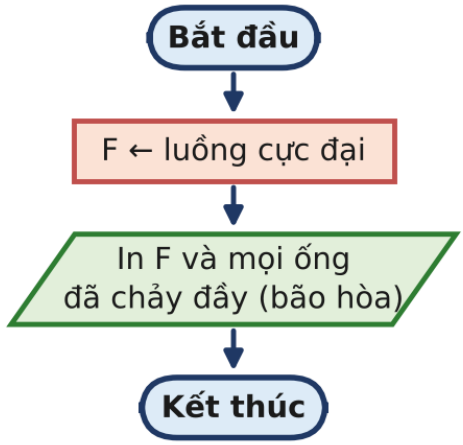
Phần đầu là **luồng cực đại** (Tập 1, Bài 17). Phần sau là **lát cắt nhỏ nhất**: theo định lí luồng cực đại – lát cắt nhỏ nhất, dung lượng nhỏ nhất bằng đúng F. Có thể có nhiều lát cắt cùng dung lượng F; đề bài chọn lát cắt có S **ít trạm nhất**.



Hình 17.1.1. *Ví dụ: F = 15, lát cắt S = {1, 2}.*

2. Cách ngây thơ: In các ống chảy đầy

Tính F bằng Dinic, rồi in mọi ống mà luồng chảy đầy (dung lượng thặng dư bằng 0), vì "ống nút cổ chai thì phải chảy đầy".



Hình 17.1.2. *Sơ đồ khối của cách in ống bão hòa.*

Cách ngây thơ: in các ống bão hòa

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

struct Dinic {                                  // luồng cực đại, thuật toán Dinic
    struct E { int to; ll cap; };
    vector<E> e;
    vector<vector<int>> g;
    vector<int> lv, it;
    Dinic(int n) : g(n + 1), lv(n + 1), it(n + 1) {}
    int add(int u, int v, ll c) {               // cung u -> v và cung ngược v
        -> u (dung lượng 0)
        e.push_back({v, c}); g[u].push_back(e.size() - 1);
        e.push_back({u, 0}); g[v].push_back(e.size() - 1);
        return e.size() - 2;
    }
    bool bfs(int s, int t) {                    // chia tầng theo khoảng cách
        trên đồ thị thặng dư
        fill(lv.begin(), lv.end(), -1); lv[s] = 0;
        queue<int> q; q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int id : g[u]) if (e[id].cap > 0 && lv[e[id].to] < 0) {
                lv[e[id].to] = lv[u] + 1; q.push(e[id].to); }
        }
        return lv[t] >= 0;
    }
    ll dfs(int u, int t, ll f) {                // đẩy luồng theo các cung đi
        lên đúng một tầng
        if (u == t) return f;
        for (int &i = it[u]; i < (int)g[u].size(); i++) {
            int id = g[u][i], v = e[id].to;
            if (e[id].cap > 0 && lv[v] == lv[u] + 1) {
                ll d = dfs(v, t, min(f, e[id].cap));
                if (d > 0) { e[id].cap -= d; e[id ^ 1].cap += d; return d; }
            }
        }
        return 0;
    }
    ll maxflow(int s, int t) {
        ll F = 0;
        while (bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            while (ll f = dfs(s, t, LLONG_MAX)) F += f;
        }
        return F;
    }
};

int main() {
    if (fopen("DUONGONG.INP", "r")) {
        freopen("DUONGONG.INP", "r", stdin);
        freopen("DUONGONG.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    Dinic D(n);
    vector<int> U(m + 1), V(m + 1), id(m + 1);
```

```cpp
    for (int i = 1; i <= m; i++) { ll c; scanf("%d %d %lld", &U[i], &V[i],
        &c); id[i] = D.add(U[i], V[i], c); }
    ll F = D.maxflow(1, n);
    vector<int> cut;                          // SAI: in mọi ống đã chảy
đầy
    for (int i = 1; i <= m; i++) if (D.e[id[i]].cap == 0) cut.push_back(i);
    printf("%lld\n%d\n", F, (int)cut.size());
    for (size_t i = 0; i < cut.size(); i++) printf("%d%c", cut[i], i + 1 ==
        cut.size() ? '\n' : ' ');
    return 0;
}
```

Độ phức tạp: $O(n^2 \cdot m)$ cho Dinic.

3. Bế tắc

- **Subtask 1** (chỉ cần F): **40 điểm**.
- **Subtask 2**: mọi ống trên lát cắt đều chảy đầy, nhưng điều ngược lại không đúng: một ống ở "giữa" mạng cũng có thể chảy đầy mà không thuộc lát cắt nào.

Một cách khác: lấy T = các trạm còn **tới được n** trên đồ thị thặng dư, S là phần còn lại. Đó cũng là một lát cắt nhỏ nhất, nhưng là lát cắt có S **nhiều** trạm nhất: **40 điểm**.

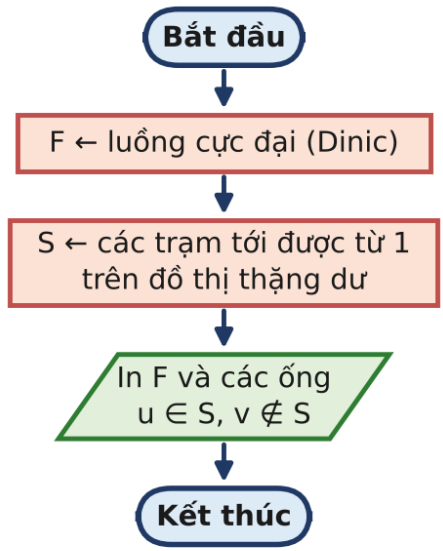
> **ĐIỂM BẾ TẮC**
>
> Biết giá trị F, nhưng phải tìm đúng **tập** S nhỏ nhất có dung lượng F.

4. Phá bế tắc

Sau khi chạy xong luồng cực đại, xét **đồ thị thặng dư**. Gọi S là tập các trạm **tới được từ 1** bằng các cung còn dung lượng thặng dư dương.

- $n \notin S$ (nếu không sẽ còn đường tăng), nên (S, phần còn lại) là một lát cắt.
- Mọi ống từ S ra ngoài đều chảy đầy, mọi ống từ ngoài vào S đều không có luồng: dung lượng lát cắt bằng đúng F, nên đây là lát cắt nhỏ nhất.
- Mọi lát cắt nhỏ nhất (S', T') đều có $S \subseteq S'$: một trạm tới được từ 1 trên đồ thị thặng dư không thể nằm ở phía T'. Vậy S là lát cắt có **ít trạm nhất**.

Ống	Từ S = {1, 2} sang	Dung lượng
[2] 1 → 3	trạm 3	8
[3] 2 → 3	trạm 3	2
[4] 2 → 4	trạm 4	5
Tổng		**15** = F



Hình 17.1.3. *Sơ đồ khối của lời giải.*

5. Cách tối ưu

DUONGONG.CPP: Dinic + lát cắt từ đồ thị thặng dư

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

struct Dinic {                                  // luồng cực đại, thuật toán Dinic
    struct E { int to; ll cap; };
    vector<E> e;
    vector<vector<int>> g;
    vector<int> lv, it;
    Dinic(int n) : g(n + 1), lv(n + 1), it(n + 1) {}
    int add(int u, int v, ll c) {               // cung u -> v và cung ngược v
        -> u (dung lượng 0)
        e.push_back({v, c}); g[u].push_back(e.size() - 1);
        e.push_back({u, 0}); g[v].push_back(e.size() - 1);
        return e.size() - 2;
    }
    bool bfs(int s, int t) {                    // chia tầng theo khoảng cách
        trên đồ thị thặng dư
        fill(lv.begin(), lv.end(), -1); lv[s] = 0;
        queue<int> q; q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int id : g[u]) if (e[id].cap > 0 && lv[e[id].to] < 0) {
                lv[e[id].to] = lv[u] + 1; q.push(e[id].to); }
        }
        return lv[t] >= 0;
    }
    ll dfs(int u, int t, ll f) {                // đẩy luồng theo các cung đi
        lên đúng một tầng
        if (u == t) return f;
        for (int &i = it[u]; i < (int)g[u].size(); i++) {
            int id = g[u][i], v = e[id].to;
            if (e[id].cap > 0 && lv[v] == lv[u] + 1) {
                ll d = dfs(v, t, min(f, e[id].cap));
                if (d > 0) { e[id].cap -= d; e[id ^ 1].cap += d; return d; }
            }
        }
        return 0;
    }
    ll maxflow(int s, int t) {
        ll F = 0;
        while (bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            while (ll f = dfs(s, t, LLONG_MAX)) F += f;
        }
        return F;
    }
};

int main() {
    if (fopen("DUONGONG.INP", "r")) {
        freopen("DUONGONG.INP", "r", stdin);
        freopen("DUONGONG.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
```

```
    Dinic D(n);
    vector<int> U(m + 1), V(m + 1);
    for (int i = 1; i <= m; i++) { ll c; scanf("%d %d %lld", &U[i], &V[i],
        &c); D.add(U[i], V[i], c); }
    ll F = D.maxflow(1, n);
    // S = các trạm còn tới được từ 1 trên đồ thị thặng dư: lát cắt nhỏ nhất
        có S ít trạm nhất
    vector<char> S(n + 1, 0);
    S[1] = 1;
    vector<int> st = {1};
    while (!st.empty()) {
        int u = st.back(); st.pop_back();
        for (int id : D.g[u]) if (D.e[id].cap > 0 && !S[D.e[id].to]) {
            S[D.e[id].to] = 1; st.push_back(D.e[id].to); }
    }
    vector<int> cut;
    for (int i = 1; i <= m; i++) if (S[U[i]] && !S[V[i]]) cut.push_back(i);
    printf("%lld\n%d\n", F, (int)cut.size());
    for (size_t i = 0; i < cut.size(); i++) printf("%d%c", cut[i], i + 1 ==
        cut.size() ? '\n' : ' ');
    return 0;
}
```

Độ phức tạp: $O(n^2 \cdot m)$ trên lí thuyết; với n = 500, m = 5000 Dinic chạy rất nhanh.

6. Bẫy thường gặp

- **In các ống bão hòa.**
- **Lấy S theo phía của n** (được lát cắt có S lớn nhất).
- **Dùng int** cho luồng: F có thể tới $5000 \cdot 10^9$.
- **Xét cả ống đi từ T vào S**: chúng không thuộc lát cắt.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (40)	ST2 (60)	Tổng
In các ống bão hòa	40	0	**40**
S = phần không tới được n	40	0	**40**
S = phần tới được từ 1 trên đồ thị thặng dư	40	60	**100**

Chấm trên 6 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

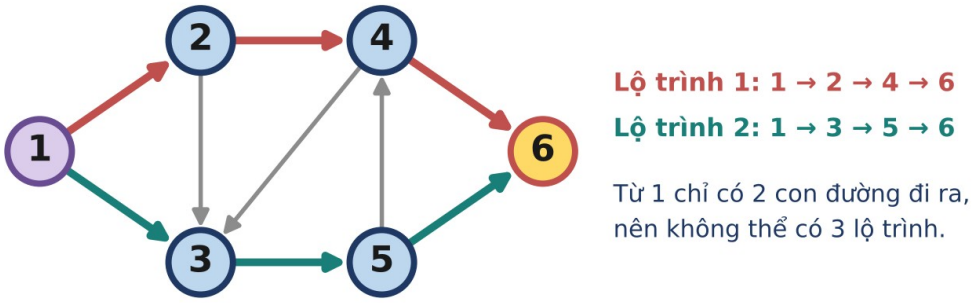
BÀI 17.2 LỘ TRÌNH RỜI NHAU

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
ROINHAU.INP	ROINHAU.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Tập 1, Bài 17 · Đề bài: Tập 2, trang 123

1. Hiểu bài toán

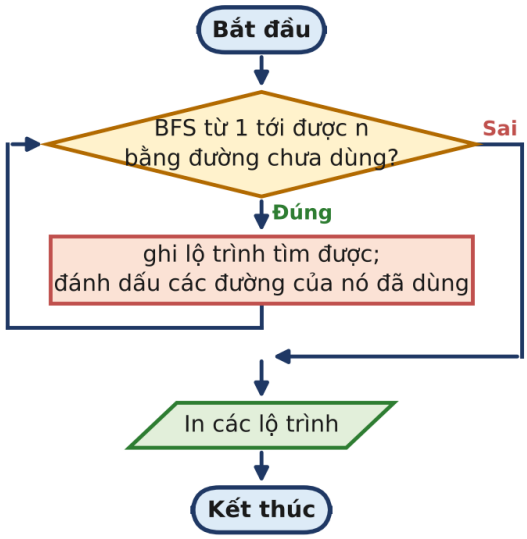
Mỗi con đường chỉ dùng cho một lộ trình: đặt dung lượng 1. Số lộ trình không chung đường nhiều nhất bằng **luồng cực đại** từ 1 tới n (định lí Menger). Phần khó là **in ra** các lộ trình, mỗi lộ trình không đi qua một nút hai lần.



Hình 17.2.1. *Ví dụ: hai lộ trình không chung con đường nào.*

2. Cách ngây thơ: Tìm lộ trình rồi xóa

Lặp: tìm một lộ trình ngắn nhất từ 1 tới n bằng BFS trên các con đường chưa dùng, ghi lại, đánh dấu các con đường của nó đã dùng.



Hình 17.2.2. *Sơ đồ khối của cách tham lam.*

Cách ngây thơ: tìm lộ trình rồi xóa

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("ROINHAU.INP", "r")) {
        freopen("ROINHAU.INP", "r", stdin);
        freopen("ROINHAU.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
    vector<vector<pair<int,int>>> g(n + 1);     // (nút kề, số hiệu đường)
    for (int i = 1; i <= m; i++) { int u, v; scanf("%d %d", &u, &v);
        g[u].push_back({v, i}); }
    vector<char> used(m + 1, 0);
    vector<vector<int>> res;
    while (true) {                              // tìm một lộ trình ngắn
nhất
        bằng BFS, rồi xóa các đường của nó
        vector<int> pe(n + 1, -1), pv(n + 1, 0);
        pe[1] = 0;
        queue<int> q; q.push(1);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto [v, id] : g[u]) if (!used[id] && pe[v] < 0) { pe[v] =
                id; pv[v] = u; q.push(v); }
        }
        if (pe[n] < 0) break;
        vector<int> path;
        for (int v = n; v != 1; v = pv[v]) { path.push_back(v); used[pe[v]]
=
            1; }
        path.push_back(1);
        reverse(path.begin(), path.end());
        res.push_back(path);
    }
    printf("%d\n", (int)res.size());
    for (auto &p : res) {
        printf("%d", (int)p.size());
        for (int x : p) printf(" %d", x);
        printf("\n");
    }
    return 0;
}
```

Độ phức tạp: $O(k \cdot (n + m))$.

3. Bế tắc

Tham lam sai: một lộ trình chọn sớm có thể chiếm con đường mà hai lộ trình khác cần. Ví dụ đồ thị $1 \to 2, 1 \to 3, 2 \to 4, 3 \to 4$ ($n = 4$) có cả hai lộ trình $1 \to 2 \to 4$ và $1 \to 3 \to 4$; nếu lộ trình đầu tiên chọn $1 \to 2 \to 3 \to 4$ thì chỉ còn đường $1 \to 3$ và $2 \to 4$ không nối được với nhau. Kết quả: **0 điểm**.

Dùng luồng là đúng hướng, nhưng nhiều bạn dựng mạng bằng **ma trận** hay **tập** các cặp (u, v), vô tình **gộp các đường song song** làm một cung dung lượng 1. Ba con đường song song từ 1 tới x chỉ còn cho một lộ trình: **0 điểm**.

ĐIỂM BẾ TẮC

Tham lam không biết "trả lại" con đường đã chọn. Cần cung ngược của luồng, giữ nguyên từng con đường song song; rồi cần biến luồng thành các lộ trình **đơn**.

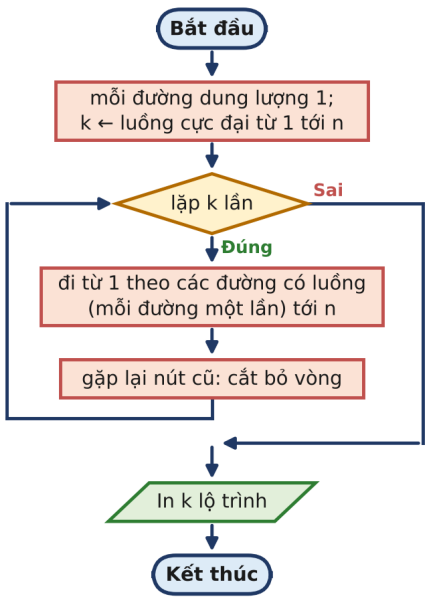
4. Phá bế tắc

1) Mỗi con đường u → v là một cung dung lượng 1 (đường song song là các cung riêng). Chạy Dinic từ 1 tới n: k = luồng cực đại.

2) Con đường có luồng là con đường mà cung của nó đã hết dung lượng. Lưu out[u] = danh sách các con đường có luồng đi ra từ u.

3) Lặp k lần: đi từ 1, mỗi bước lấy con đường có luồng **tiếp theo chưa dùng** ở nút hiện tại (con trỏ ptr[u]), cho tới khi tới n. Bảo toàn luồng đảm bảo luôn đi tiếp được.

4) Nếu quay lại một nút đã có trên lộ trình hiện tại, **cắt bỏ** cả đoạn vòng (các con đường trong vòng vẫn bị coi là đã dùng, không sao vì ta chỉ cần lộ trình không chung đường).



Hình 17.2.3. *Sơ đồ khối của lời giải.*

VÌ SAO LUỒNG CÓ THỂ CÓ VÒNG?

Luồng cực đại không nhất thiết là "đẹp": thuật toán có thể đẩy một đơn vị luồng đi vòng quanh một chu trình (ví dụ 3 → 5 → 4 → 3 ở ví dụ) mà giá trị luồng không đổi. Khi lần theo luồng, gặp vòng đó thì phải cắt.

5. Cách tối ưu

ROINHAU.CPP: Dinic + lần theo luồng, cắt vòng

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

struct Dinic {                                  // luồng cực đại, thuật toán Dinic
    struct E { int to; ll cap; };
    vector<E> e;
    vector<vector<int>> g;
    vector<int> lv, it;
    Dinic(int n) : g(n + 1), lv(n + 1), it(n + 1) {}
    int add(int u, int v, ll c) {               // cung u -> v và cung ngược v
        -> u (dung lượng 0)
        e.push_back({v, c}); g[u].push_back(e.size() - 1);
        e.push_back({u, 0}); g[v].push_back(e.size() - 1);
        return e.size() - 2;
    }
    bool bfs(int s, int t) {                    // chia tầng theo khoảng cách
        trên đồ thị thặng dư
        fill(lv.begin(), lv.end(), -1); lv[s] = 0;
        queue<int> q; q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int id : g[u]) if (e[id].cap > 0 && lv[e[id].to] < 0) {
                lv[e[id].to] = lv[u] + 1; q.push(e[id].to); }
        }
        return lv[t] >= 0;
    }
    ll dfs(int u, int t, ll f) {                // đẩy luồng theo các cung đi
        lên đúng một tầng
        if (u == t) return f;
        for (int &i = it[u]; i < (int)g[u].size(); i++) {
            int id = g[u][i], v = e[id].to;
            if (e[id].cap > 0 && lv[v] == lv[u] + 1) {
                ll d = dfs(v, t, min(f, e[id].cap));
                if (d > 0) { e[id].cap -= d; e[id ^ 1].cap += d; return d; }
            }
        }
        return 0;
    }
    ll maxflow(int s, int t) {
        ll F = 0;
        while (bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            while (ll f = dfs(s, t, LLONG_MAX)) F += f;
        }
        return F;
    }
};

int main() {
    if (fopen("ROINHAU.INP", "r")) {
        freopen("ROINHAU.INP", "r", stdin);
        freopen("ROINHAU.OUT", "w", stdout);
    }
    int n, m;
    scanf("%d %d", &n, &m);
```

```
    Dinic D(n);
    vector<int> U(m + 1), V(m + 1), id(m + 1);
    for (int i = 1; i <= m; i++) { scanf("%d %d", &U[i], &V[i]); id[i] =
        D.add(U[i], V[i], 1); }   // mỗi đường: dung lượng 1
    int k = D.maxflow(1, n);
    vector<vector<int>> out(n + 1);             // out[u]: các con đường có
        luồng đi ra từ u
    for (int i = 1; i <= m; i++) if (D.e[id[i]].cap == 0)
        out[U[i]].push_back(i);
    vector<int> ptr(n + 1, 0), pos(n + 1, -1);
    printf("%d\n", k);
    for (int r = 0; r < k; r++) {               // lần theo luồng từ 1 tới n
        vector<int> path = {1};
        pos[1] = 0;
        int u = 1;
        while (u != n) {
            int v = V[out[u][ptr[u]++]];        // dùng con đường có luồng
                tiếp theo, mỗi đường một lần
            if (pos[v] >= 0) {                  // quay lại nút đã đi qua:
cắt
                bỏ cả vòng
            while ((int)path.size() > pos[v] + 1) { pos[path.back()] = -
1;
                path.pop_back(); }
            } else { pos[v] = path.size(); path.push_back(v); }
            u = v;
        }
        printf("%d", (int)path.size());
        for (int x : path) { printf(" %d", x); pos[x] = -1; }
        printf("\n");
    }
    return 0;
}
```

Độ phức tạp: O(m · √m) cho Dinic với dung lượng 1; lần theo luồng O(m).

6. Bẫy thường gặp

- **Tham lam tìm lộ trình rồi xóa.**
- **Gộp đường song song** thành một cung dung lượng 1 (mất lộ trình).
- **Không cắt vòng** khi lần theo luồng.
- **Dùng lại một con đường** ở hai lộ trình (quên con trỏ ptr).

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (40)	ST2 (60)	Tổng
Tìm lộ trình rồi xóa	0	0	**0**
Luồng, gộp đường song song	0	0	**0**
Luồng + lần theo, cắt vòng	40	60	**100**

Chấm trên 6 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

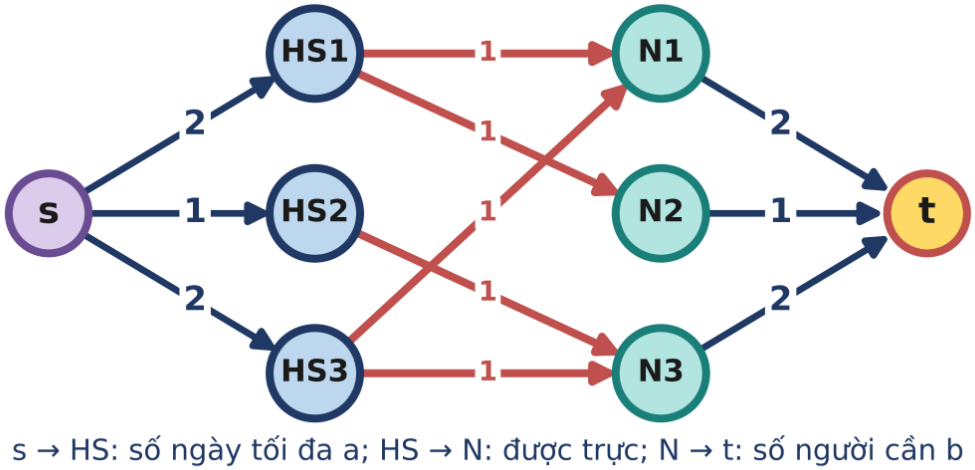
BÀI 17.3 PHÂN CÔNG TRỰC NHẬT

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
TRUCNHAT.INP	TRUCNHAT.OUT	1 giây	256 MB	★★★ NÂNG CAO

Liên hệ: Tập 1, Bài 17 · Đề bài: Tập 2, trang 125

1. Hiểu bài toán

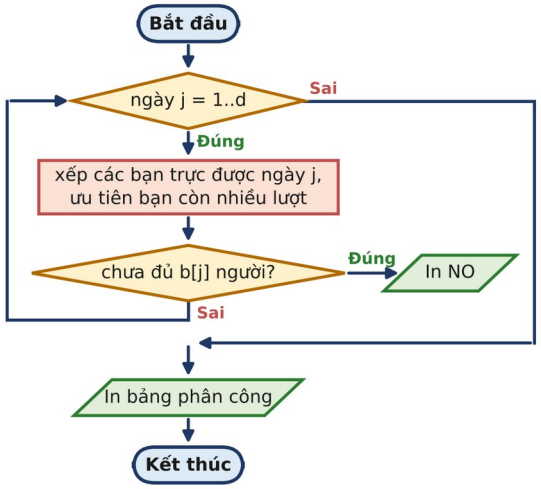
Mỗi học sinh trực nhiều nhất a_i ngày, mỗi ngày cần đúng b_j bạn, mỗi bạn trực một ngày nhiều nhất một lần. Đây là bài toán **phân phối có giới hạn hai phía**, mô hình hóa bằng luồng.



Hình 17.3.1. *Mạng luồng của ví dụ 1.*

2. Cách ngây thơ: Tham lam theo từng ngày

Xếp lần lượt từng ngày; mỗi ngày ưu tiên những bạn còn nhiều lượt trực nhất.



Hình 17.3.2. *Sơ đồ khối của cách tham lam.*

Cách ngây thơ: tham lam theo từng ngày

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("TRUCNHAT.INP", "r")) {
        freopen("TRUCNHAT.INP", "r", stdin);
        freopen("TRUCNHAT.OUT", "w", stdout);
    }
    int n, d;
    scanf("%d %d", &n, &d);
    vector<int> a(n + 1), b(d + 1);
    for (int i = 1; i <= n; i++) scanf("%d", &a[i]);
    for (int j = 1; j <= d; j++) scanf("%d", &b[j]);
    vector<vector<int>> can(d + 1);             // can[j]: những học sinh trực
        được ngày j
    for (int i = 1; i <= n; i++) { int k; scanf("%d", &k); while (k--) { int
        j; scanf("%d", &j); can[j].push_back(i); } }
    vector<vector<int>> day(d + 1);
    for (int j = 1; j <= d; j++) {              // mỗi ngày chọn những bạn còn
        "nhiều lượt" nhất
        sort(can[j].begin(), can[j].end(), [&](int x, int y) { return a[x] >
            a[y]; });
        for (int i : can[j]) if ((int)day[j].size() < b[j] && a[i] > 0) {
            day[j].push_back(i); a[i]--; }
        if ((int)day[j].size() < b[j]) { printf("NO\n"); return 0; }
    }
    printf("YES\n");
    for (int j = 1; j <= d; j++) {
        for (int i : day[j]) printf("%d ", i);
        printf("\n");
    }
    return 0;
}
```

Độ phức tạp: $O(d \cdot n \log n)$.

3. Bế tắc

Tham lam sai. Ví dụ: bạn 1 trực được ngày 1 và 2, bạn 2 chỉ trực được ngày 1, mỗi bạn trực 1 ngày, mỗi ngày cần 1 bạn. Ngày 1 tham lam có thể chọn bạn 1, đến ngày 2 không còn ai: in NO, trong khi xếp bạn 2 ngày 1, bạn 1 ngày 2 là được. Kết quả: **30 điểm**.

Vét cạn (xếp từng ngày, thử mọi tổ hợp b_j bạn, quay lui) đúng nhưng hàm mũ: **30 điểm**.

> **ĐIỂM BẾ TẮC**
>
> Có ba loại giới hạn đan vào nhau (mỗi bạn, mỗi ngày, mỗi cặp bạn – ngày). Cần một mô hình xử lí cả ba cùng lúc và cho phép đổi ý.

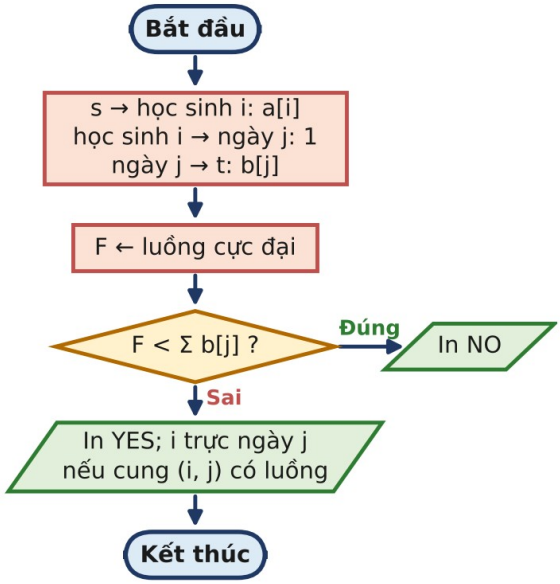
4. Phá bế tắc

Cung	Dung lượng	Ý nghĩa
s → học sinh i	a_i	i trực nhiều nhất a_i ngày
học sinh i → ngày j	1	i trực được ngày j, nhiều nhất một lần
ngày j → t	b_j	ngày j cần b_j bạn

Một lịch trực hợp lệ tương ứng với một luồng có giá trị đúng bằng Σb_j (mọi cung vào t đầy), và ngược lại một luồng nguyên như vậy cho ra lịch trực: học sinh i trực ngày j khi cung i → j có luồng. Vậy: chạy luồng cực đại, nếu $F < \Sigma b_j$ thì in NO.

Ví dụ 1	Luồng	Lịch
Ngày 1 (cần 2)	bạn 1, bạn 3	1 3
Ngày 2 (cần 1)	bạn 1	1
Ngày 3 (cần 2)	bạn 2, bạn 3	2 3
Tổng	$F = 5 = \Sigma b_j$	**YES**

Ví dụ 2: không ai trực được ngày 2 nên cung "ngày 2 → t" không bao giờ có luồng, $F < \Sigma b_j$: **NO**.



Hình 17.3.3. *Sơ đồ khối của lời giải.*

TÍNH NGUYÊN

Mọi dung lượng là số nguyên thì Dinic (cũng như Ford–Fulkerson) cho ra luồng cực đại **nguyên**: mỗi cung học sinh → ngày có luồng 0 hoặc 1, đọc ra được lịch trực.

5. Cách tối ưu

TRUCNHAT.CPP: luồng cực đại trên mạng ba tầng

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

struct Dinic {                                  // luồng cực đại, thuật toán Dinic
    struct E { int to; ll cap; };
    vector<E> e;
    vector<vector<int>> g;
    vector<int> lv, it;
    Dinic(int n) : g(n + 1), lv(n + 1), it(n + 1) {}
    int add(int u, int v, ll c) {               // cung u -> v và cung ngược v
        -> u (dung lượng 0)
        e.push_back({v, c}); g[u].push_back(e.size() - 1);
        e.push_back({u, 0}); g[v].push_back(e.size() - 1);
        return e.size() - 2;
    }
    bool bfs(int s, int t) {                    // chia tầng theo khoảng cách
        trên đồ thị thặng dư
        fill(lv.begin(), lv.end(), -1); lv[s] = 0;
        queue<int> q; q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int id : g[u]) if (e[id].cap > 0 && lv[e[id].to] < 0) {
                lv[e[id].to] = lv[u] + 1; q.push(e[id].to); }
        }
        return lv[t] >= 0;
    }
    ll dfs(int u, int t, ll f) {                // đẩy luồng theo các cung đi
        lên đúng một tầng
        if (u == t) return f;
        for (int &i = it[u]; i < (int)g[u].size(); i++) {
            int id = g[u][i], v = e[id].to;
            if (e[id].cap > 0 && lv[v] == lv[u] + 1) {
                ll d = dfs(v, t, min(f, e[id].cap));
                if (d > 0) { e[id].cap -= d; e[id ^ 1].cap += d; return d; }
            }
        }
        return 0;
    }
    ll maxflow(int s, int t) {
        ll F = 0;
        while (bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            while (ll f = dfs(s, t, LLONG_MAX)) F += f;
        }
        return F;
    }
};

int main() {
    if (fopen("TRUCNHAT.INP", "r")) {
        freopen("TRUCNHAT.INP", "r", stdin);
        freopen("TRUCNHAT.OUT", "w", stdout);
    }
    int n, d;
    scanf("%d %d", &n, &d);
```

```cpp
    // nguồn s -> học sinh i (dung lượng a[i]) -> ngày j (1) -> đích t (b[j])
    int s = n + d + 1, t = n + d + 2;
    Dinic D(t);
    ll need = 0;
    for (int i = 1; i <= n; i++) { int a; scanf("%d", &a); D.add(s, i, a); }
    for (int j = 1; j <= d; j++) { int b; scanf("%d", &b); D.add(n + j, t, b);
        need += b; }
    vector<array<int,3>> e;                    // (học sinh, ngày, số hiệu cung)
    for (int i = 1; i <= n; i++) {
        int k; scanf("%d", &k);
        while (k--) { int j; scanf("%d", &j); e.push_back({i, j, D.add(i, n +
            j, 1)}); }
    }
    if (D.maxflow(s, t) != need) { printf("NO\n"); return 0; }   // không lấp
        đầy mọi ngày
    vector<vector<int>> day(d + 1);
    for (auto &x : e) if (D.e[x[2]].cap == 0) day[x[1]].push_back(x[0]);
    //
        cung có luồng: i trực ngày j
    printf("YES\n");
    for (int j = 1; j <= d; j++) {
        for (size_t i = 0; i < day[j].size(); i++) printf("%d ", day[j][i]);
        printf("\n");
    }
    return 0;
}
```

Độ phức tạp: Mạng có $n + d + 2 \le 402$ đỉnh và tối đa $n \cdot d + n + d \approx 4 \cdot 10^4$ cung; Dinic chạy tức thì.

6. Bẫy thường gặp

- **Tham lam theo ngày.**
- **Đặt dung lượng học sinh → ngày bằng a_i** (một bạn trực một ngày nhiều lần).
- **Kiểm tra $F = \Sigma a_i$** thay vì Σb_j.
- **Quên in dòng trống** cho ngày có $b_j = 0$.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (70)	Tổng
Vét cạn, quay lui theo ngày	30	0	**30**
Tham lam theo từng ngày	30	0	**30**
Luồng cực đại	30	70	**100**

Chấm trên 6 test, giới hạn 1 giây, 256 MB, ngăn xếp 8 MB. Dòng tô xanh là lời giải mẫu.

PHỤ LỤC A. TỔNG HỢP LỜI GIẢI VÀ ĐIỂM SỐ

Bảng dưới đây tóm tắt "chìa khóa" của từng lời giải, cùng điểm của cách ngây thơ và của lời giải mẫu trên bộ test của sách. Khoảng cách giữa hai cột điểm chính là phần thưởng cho việc phá bế tắc.

Chương 1. LÀM QUEN VỚI ĐỒ THỊ

Bài	Tên bài	Chìa khóa của lời giải	Ngây thơ	Lời giải mẫu
1.1	**BẮT TAY**	Định lí bắt tay + điều kiện "người nhiều nhất".	60	**100**
1.2	**NGƯỜI NỔI TIẾNG**	Bán bậc vào/ra sau khi bỏ cung lặp.	30	**100**
1.3	**TAM GIÁC BẠN BÈ**	Hướng cạnh từ đỉnh bậc nhỏ sang bậc lớn: $O(m\sqrt{m})$.	50	**100**
2.1	**CHUYỂN ĐỔI**	Duyệt ma trận đúng thứ tự, chỉ lấy $i < j$.	0	**100**
2.2	**BẠN CHUNG**	Duyệt danh sách ngắn hơn + ghi nhớ cặp đã hỏi.	30	**100**
2.3	**BẢN ĐỒ KHO BÁU**	Mỗi sự kiện chỉ ảnh hưởng tới 5 ô.	60	**100**
3.1	**ĐẾM ĐẢO**	Loang bằng ngăn xếp tự quản lí.	40	**100**
3.2	**CÂY HAY KHÔNG**	Đáp số $= (m - (n - k)) + (k - 1)$.	30	**100**
3.3	**MẠNG LƯỚI DỰ PHÒNG**	Kích thước thành phần; nối hai nhóm lớn nhất.	30	**100**
4.1	**MÃ ĐI TUẦN**	BFS một lần từ ô xuất phát.	60	**100**
4.2	**CHẠY TRỐN ĐÁM CHÁY**	BFS nhiều nguồn cho lửa, BFS cho An với điều kiện "<".	30	**100**
4.3	**CHIA ĐỘI**	Tô hai màu + tổng tập con bằng bitset.	20	**100**

Chương 2. ĐƯỜNG ĐI NGẮN NHẤT

Bài	Tên bài	Chìa khóa của lời giải	Ngây thơ	Lời giải mẫu
5.1	**ĐƯỜNG TỚI TRƯỜNG**	Dijkstra với hàng đợi ưu tiên và long long.	40	**100**
5.2	**NÚT GIAO QUAN TRỌNG**	Dijkstra hai đầu + quét các "mốc" d_1.	30	**100**
5.3	**XE BUÝT ĐÊM**	Dijkstra theo thời điểm tới; tính chuyến kế tiếp bằng công thức.	30	**100**
6.1	**ĐỔI TIỀN**	Trọng số $-\log r$, chu trình âm với mọi đỉnh là nguồn.	30	**100**
6.2	**CHUYẾN BAY GIÁ RẺ**	Bellman-Ford đúng k lượt, mỗi lượt từ bản sao.	30	**100**
6.3	**LỊCH THI ĐẤU**	Hệ ràng buộc hiệu: đường đi dài nhất, phát hiện chu trình dương.	40	**100**
7.1	**THỦ ĐÔ MỚI**	Floyd với k ở vòng ngoài cùng.	30	**100**
7.2	**XẾP HẠNG KÌ THỦ**	Bao đóng bắc cầu bằng bitset.	80	**100**
7.3	**XE TẢI QUÁ KHỔ**	Floyd với (max, min).	30	**100**

Bài	Tên bài	Chìa khóa của lời giải	Ngây thơ	Lời giải mẫu
8.1	**ĐÈN GIAO THÔNG**	BFS trên trạng thái (nút, t mod P).	60	**100**
8.2	**PHÁ TƯỜNG**	BFS trên trạng thái (ô, số tường đã phá).	30	**100**
8.3	**ROBOT XOAY**	BFS 0-1 trên trạng thái (ô, hướng).	30	**100**

Chương 3. CÂY VÀ CÂY KHUNG

Bài	Tên bài	Chìa khóa của lời giải	Ngây thơ	Lời giải mẫu
9.1	**MẠNG BẠN BÈ**	DSU nén đường đi + gộp theo kích thước, duy trì cnt và best.	40	**100**
9.2	**PHÁ CẦU**	Đi ngược thời gian: phá cầu thành xây cầu.	30	**100**
9.3	**HAI PHE**	DSU có chẵn lẻ (hoặc DSU trên 2n đỉnh).	30	**100**
10.1	**NỐI MẠNG TRƯỜNG HỌC**	Gộp cáp có sẵn vào DSU rồi chạy Kruskal.	30	**100**
10.2	**ĐƯỜNG ÍT DỐC**	Cây khung nhỏ nhất + cạnh lớn nhất trên đường đi (nhảy nhị phân).	30	**100**
10.3	**GIẾNG VÀ ỐNG**	Đỉnh ảo "nguồn nước" + Kruskal.	20	**100**
11.1	**HỌ HÀNG**	LCA nhảy nhị phân, dựng bảng bằng BFS.	40	**100**
11.2	**CÁP QUANG YẾU NHẤT**	Nhảy nhị phân kèm min trên đoạn.	30	**100**
11.3	**NHÂN VIÊN CẤP K**	Trả lời ngoại tuyến: DFS với ngăn xếp đường đi.	20	**100**

Chương 4. CẤU TRÚC CỦA ĐỒ THỊ

Bài	Tên bài	Chìa khóa của lời giải	Ngây thơ	Lời giải mẫu
12.1	**XẾP HẠNG GIẢI ĐẤU**	Kahn: thiếu đỉnh thì có vòng; hàng đợi từng có ≥ 2 đỉnh thì nhiều đáp án.	30	**100**
12.2	**DỰ ÁN XÂY NHÀ**	Hai lượt QHĐ trên thứ tự tô pô: sớm nhất (xuôi) và muộn nhất (ngược).	40	**100**
12.3	**ĐẾM LỘ TRÌNH**	Chỉ giữ các nút vừa tới được từ 1 vừa tới được n; có vòng trong đó thì vô số.	30	**100**
13.1	**ĐƯỜNG HUYẾT MẠCH**	num/low, bỏ qua đúng CẠNH vừa đi tới (không bỏ đỉnh cha).	30	**100**
13.2	**TRẠM BIẾN ÁP**	Khớp: số phần = (số con c có low[c] ≥ num[u]) + 1, riêng gốc thì không cộng 1.	30	**100**
13.3	**CÁP DỰ PHÒNG**	Nén khối không cầu → cây cầu; đáp số ⌈số lá / 2⌉.	20	**100**
14.1	**LAN TRUYỀN TIN ĐỒN**	Nén thành phần liên thông mạnh; đếm thành phần có bán bậc vào 0.	30	**100**
14.2	**KHO BÁU THỊ TRẤN**	Nén SCC (vào thành phần là nhặt hết), QHĐ đường dài nhất trên DAG.	30	**100**
14.3	**HỘI ĐỒNG TUYỂN CHỌN**	2-SAT: cung ¬x → y, ¬y → x; Tarjan; chọn khẳng định có số thành phần nhỏ hơn.	20	**100**

Bài	Tên bài	Chìa khóa của lời giải	Ngây thơ	Lời giải mẫu
15.1	**NÉT VẼ**	Định lí Euler: số điểm bậc lẻ là 0 hoặc 2, và các đoạn liên thông với nhau.	30	**100**
15.2	**GHÉP DOMINO**	7 đỉnh, mỗi quân là một cạnh: đường Euler bằng Hierholzer không đệ quy.	30	**100**
15.3	**MẬT KHẨU KÉT SẮT**	Đồ thị de Bruijn: đỉnh là xâu $k-1$ chữ số, cạnh là mã; chu trình Euler.	50	**100**

Chương 5. GHÉP CẶP VÀ LUỒNG

Bài	Tên bài	Chìa khóa của lời giải	Ngây thơ	Lời giải mẫu
16.1	**XẾP PHÒNG THI**	Cặp ghép cực đại trên đồ thị hai phía bằng đường tăng (Kuhn).	30	**100**
16.2	**ĐẶT DOMINO**	Tô bàn cờ: domino = cạnh nối ô trắng với ô đen; cặp ghép cực đại.	20	**100**
16.3	**CAMERA GIÁM SÁT**	Định lí König: phủ đỉnh nhỏ nhất = cặp ghép cực đại; dựng phủ từ tập Z.	20	**100**
17.1	**ĐƯỜNG ỐNG**	Dinic; S = các trạm tới được từ 1 trên đồ thị thặng dư.	40	**100**
17.2	**LỘ TRÌNH RỜI NHAU**	Mỗi đường dung lượng 1, luồng cực đại; lần theo luồng và cắt bỏ vòng.	0	**100**
17.3	**PHÂN CÔNG TRỰC NHẬT**	Mạng luồng s → học sinh (a_i) → ngày (1) → t (b_j); đủ luồng thì có lịch.	30	**100**

PHỤ LỤC B. THƯ VIỆN MÃ NGUỒN HAY DÙNG

Những đoạn mã dưới đây xuất hiện lặp lại trong nhiều lời giải của sách. Hãy tự gõ lại chúng cho tới khi viết được không cần nhìn, không sai một kí tự: trong phòng thi, chúng là "viên gạch" để xây những lời giải lớn hơn.

DSU: nén đường đi (không đệ quy) + gộp theo kích thước

```
struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n + 1), sz(n + 1, 1) { iota(p.begin(), p.end(), 0); }
    int root(int x) {                          // nén đường đi kiểu "nhảy hai bậc"
        while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; }
        return x;
    }
    bool unite(int a, int b) {                 // trả về false nếu a, b đã cùng nhóm
        a = root(a); b = root(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);         // treo cây nhỏ vào cây lớn
        p[b] = a; sz[a] += sz[b];
        return true;
    }
};
```

BFS trên lưới bằng dx, dy

```
const int dx[4] = {-1, 1, 0, 0}, dy[4] = {0, 0, -1, 1};

// khoảng cách từ ô (sx, sy) tới mọi ô trên lưới R x C; '#' là ô cấm, -1 là không tới được
vector<vector<int>> bfsGrid(const vector<string> &a, int sx, int sy) {
    int R = a.size(), C = a[0].size();
    vector<vector<int>> d(R, vector<int>(C, -1));
    queue<pair<int,int>> q;
    d[sx][sy] = 0; q.push({sx, sy});
    while (!q.empty()) {
        auto [x, y] = q.front(); q.pop();
        for (int k = 0; k < 4; k++) {
            int u = x + dx[k], v = y + dy[k];
            if (u < 0 || u >= R || v < 0 || v >= C || a[u][v] == '#' ||
                d[u][v] >= 0) continue;
            d[u][v] = d[x][y] + 1; q.push({u, v});
        }
    }
    return d;
}
```

Dijkstra với hàng đợi ưu tiên, bỏ qua bản cũ

```cpp
typedef long long ll;
const ll INF = LLONG_MAX / 4;

// g[u] = danh sách (v, w); trả về khoảng cách ngắn nhất từ s
vector<ll> dijkstra(const vector<vector<pair<int,int>>> &g, int s) {
    vector<ll> d(g.size(), INF);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
    d[s] = 0; pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du != d[u]) continue;               // bản cũ: bỏ qua
        for (auto [v, w] : g[u])
            if (d[u] + w < d[v]) { d[v] = d[u] + w; pq.push({d[v], v}); }
    }
    return d;
}
```

Tarjan tìm thành phần liên thông mạnh, không đệ quy

```cpp
// g: đồ thị có hướng, đỉnh 1..n. comp[v] = số hiệu thành phần, đánh số theo
// thứ tự tô pô NGƯỢC (cung giữa hai thành phần đi từ số lớn sang số nhỏ).
Trả
    về số thành phần.
int tarjan(int n, const vector<vector<int>> &g, vector<int> &comp) {
    vector<int> num(n + 1, 0), low(n + 1, 0), it(n + 1, 0), S, cs;
    vector<char> onS(n + 1, 0);
    comp.assign(n + 1, -1);
    int timer = 0, c = 0;
    for (int s = 1; s <= n; s++) if (!num[s]) {
        cs.push_back(s); num[s] = low[s] = ++timer; S.push_back(s); onS[s] =
            1;
        while (!cs.empty()) {
            int u = cs.back();
            if (it[u] < (int)g[u].size()) {
                int v = g[u][it[u]++];
                if (!num[v]) { num[v] = low[v] = ++timer; S.push_back(v);
                    onS[v] = 1; cs.push_back(v); }
                else if (onS[v]) low[u] = min(low[u], num[v]);
            } else {
                cs.pop_back();
                if (!cs.empty()) low[cs.back()] = min(low[cs.back()],
low[u]);
                if (low[u] == num[u]) {
                    int v;
                    do { v = S.back(); S.pop_back(); onS[v] = 0; comp[v] =
c;
                        } while (v != u);
                    c++;
                }
            }
        }
    }
    return c;
}
```

num/low tìm cầu và khớp, không đệ quy, đúng với cạnh lặp

```cpp
// g[u] = danh sách (v, số hiệu cạnh); đỉnh 1..n, cạnh 1..m, cho phép cạnh song song.
// isBridge[id] = cạnh id là cầu; cut[u] = số phần còn lại khi bỏ đỉnh u (1 nếu u không là khớp)
void lowlink(int n, int m, const vector<vector<pair<int,int>>> &g,
    vector<char> &isBridge, vector<int> &cut) {
    vector<int> num(n + 1, 0), low(n + 1, 0), pe(n + 1, 0), it(n + 1, 0),
        cnt(n + 1, 0);
    isBridge.assign(m + 1, 0); cut.assign(n + 1, 1);
    int timer = 0;
    for (int s = 1; s <= n; s++) if (!num[s]) {
        vector<int> st = {s};
        num[s] = low[s] = ++timer;
        while (!st.empty()) {
            int u = st.back();
            if (it[u] < (int)g[u].size()) {
                auto [v, id] = g[u][it[u]++];
                if (id == pe[u]) continue;      // bỏ đúng cạnh vừa đi tới
                if (num[v]) low[u] = min(low[u], num[v]);
                else { num[v] = low[v] = ++timer; pe[v] = id;
st.push_back(v);
                    }
            } else {
                st.pop_back();
                if (st.empty()) break;
                int p = st.back();
                low[p] = min(low[p], low[u]);
                if (low[u] > num[p]) isBridge[pe[u]] = 1;
                if (low[u] >= num[p]) cnt[p]++;
            }
        }
        cnt[s]--;                               // gốc: không có "phần phía trên", bù trước cho +1 dưới đây
    }
    for (int v = 1; v <= n; v++) cut[v] = max(1, cnt[v] + 1);
}
```

Luồng cực đại Dinic

```cpp
typedef long long ll;

struct Dinic {                                  // luồng cực đại, thuật toán Dinic
    struct E { int to; ll cap; };
    vector<E> e;
    vector<vector<int>> g;
    vector<int> lv, it;
    Dinic(int n) : g(n + 1), lv(n + 1), it(n + 1) {}
    int add(int u, int v, ll c) {               // cung u -> v và cung ngược v
        -> u (dung lượng 0)
        e.push_back({v, c}); g[u].push_back(e.size() - 1);
        e.push_back({u, 0}); g[v].push_back(e.size() - 1);
        return e.size() - 2;
    }
    bool bfs(int s, int t) {                    // chia tầng theo khoảng cách
        trên đồ thị thặng dư
        fill(lv.begin(), lv.end(), -1); lv[s] = 0;
        queue<int> q; q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int id : g[u]) if (e[id].cap > 0 && lv[e[id].to] < 0) {
                lv[e[id].to] = lv[u] + 1; q.push(e[id].to); }
        }
        return lv[t] >= 0;
    }
    ll dfs(int u, int t, ll f) {                // đẩy luồng theo các cung đi
        lên đúng một tầng
        if (u == t) return f;
        for (int &i = it[u]; i < (int)g[u].size(); i++) {
            int id = g[u][i], v = e[id].to;
            if (e[id].cap > 0 && lv[v] == lv[u] + 1) {
                ll d = dfs(v, t, min(f, e[id].cap));
                if (d > 0) { e[id].cap -= d; e[id ^ 1].cap += d; return d; }
            }
        }
        return 0;
    }
    ll maxflow(int s, int t) {
        ll F = 0;
        while (bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            while (ll f = dfs(s, t, LLONG_MAX)) F += f;
        }
        return F;
    }
};
```